



EPOKA UNIVERSITY

Faculty of Architecture and Engineering

Software Engineering - Department of Computer Engineering

Course: *Software Modeling and Design*

Project Title: *Airport Management System*

Academic Year 2025-2026

Semester: *Spring*

Submission Date:*03/06/2026*

Group Members: *Ajla Alcani , Alvi Elezi , Ameo Lulaj , Andri Muçllari , Ani Velo , Betül Kaan , Eneida Balliu, Albina Blloshmi*

Lecturers:

M.Sc. Ari Gjerazi

M.Sc. Edlira Cani

Project: Airport Management System

1. Project Topic

The project focuses on the design and documentation of an **Airport Management System (AMS)**. This system represents an integrated software platform intended to support and coordinate the primary operational activities of an airport. Airports involve complex interactions between airlines, passengers, ground staff, security authorities, and administrative departments. Managing these operations manually or through isolated systems leads to inefficiencies, delays, and data inconsistencies.

The Airport Management System aims to centralize airport operations into a single digital environment where data can be managed, accessed, and updated in real time. The system will support airport staff in handling flights, passengers, aircraft, scheduling, security monitoring, and operational reporting.

This project will focus on designing the structure, behavior, and documentation of such a system using software engineering modeling techniques. The final deliverable will be a comprehensive system design document that reflects industry-level software documentation standards.

2. Description of the Project

2.1 Aim of the System

The primary aim of the Airport Management System is to **improve the efficiency, coordination, and reliability of airport operations** by integrating multiple operational processes into a centralized information system.

Airports handle thousands of passengers and numerous flights daily, requiring precise coordination between airlines, airport staff, security personnel, and administrative departments. The proposed system aims to simplify and automate many of these processes while ensuring that accurate information is available to all relevant stakeholders.

The system will provide tools for flight scheduling, passenger processing, resource allocation, and operational monitoring. By digitizing these processes, the system will reduce manual errors, improve communication between departments, and enable better decision-making through centralized data management.

In addition, the system will support real-time updates for flight statuses, passenger check-ins, gate assignments, and airport resource management. This allows airport authorities to respond quickly to operational changes such as delays, cancellations, or security issues.

2.2 User Levels

The Airport Management System will support multiple types of users, each with different access permissions and responsibilities. These user levels ensure that sensitive information and operational controls are only accessible to authorized personnel.

System Administrator

The System Administrator manages the overall configuration and maintenance of the system. This user has the highest level of access and is responsible for creating user accounts, assigning roles, maintaining system security, and monitoring system performance.

Administrators ensure that the system operates correctly and that all operational data remains secure and consistent.

Airport Operations Staff

Airport operations staff are responsible for managing daily airport activities. Their responsibilities include updating flight schedules, assigning gates, coordinating aircraft arrivals and departures, and monitoring operational status.

These users interact heavily with the system to ensure smooth airport operations and accurate flight information.

Airline Staff

Airline personnel use the system to manage flights associated with their airline. Their tasks include updating passenger lists, handling check-in processes, updating flight information, and coordinating boarding activities.

They require access to flight-related information and passenger data relevant to their airline operations.

Security Personnel

Security staff use the system to monitor passenger screening, security alerts, and restricted areas. Their role focuses on maintaining airport safety and responding to potential security issues.

The system may provide tools for tracking incidents and verifying passenger and staff credentials.

Passengers (Limited Access)

Passengers interact with the system indirectly through information services such as flight status displays, check-in kiosks, or mobile applications. Their access level is limited to viewing flight information, check-in confirmation, and possibly boarding details.

2.3 General Requirements of the System

The Airport Management System must fulfill several key functional and operational requirements to support airport activities effectively.

Flight Management

The system must allow airport staff and airline personnel to create, update, and manage flight schedules. It should support recording flight numbers, airlines, destinations, arrival and departure times, and current flight status.

Real-time updates should be possible to reflect delays, cancellations, or gate changes.

Passenger Management

The system should support the management of passenger information associated with each flight. This includes storing passenger details, managing check-in processes, and tracking boarding status.

Passenger data must be handled securely while allowing authorized airline staff to access necessary information.

Gate and Resource Allocation

The system must support the allocation of airport resources such as gates, terminals, and boarding areas. Airport staff should be able to assign gates to flights and modify assignments when necessary.

Efficient resource allocation helps reduce traffic and improves airport flow.

Security Monitoring

The system should assist security personnel by providing access to relevant security alerts and monitoring information. It should allow authorized users to record incidents and track security-related events.

Security controls must ensure that only authorized personnel can access sensitive information.

Reporting and Monitoring

The system should provide reporting capabilities that allow airport management to analyze operational data. Reports may include flight statistics, passenger volumes, delays, and system usage.

These reports help airport administrators make informed operational and strategic decisions.

User Authentication and Access Control

The system must implement secure authentication mechanisms to verify user identities. Each user role should have defined permissions to ensure that users can only access functions relevant to their responsibilities.

This requirement is essential for maintaining both system security and operational integrity.

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for the Airport Management System (AMS). The system is designed to support airport operations and improve the interaction between passengers, airport staff, and administrators.

This document specifies the system modules, actors, use cases, functional requirements, and non-functional requirements that describe how the system should operate.

The document serves as the foundation for later stages of the software engineering process, including system design, modeling, and development.

1.2 System Overview

Airports are complex environments where many operations must be coordinated simultaneously. These operations include passenger services, flight scheduling, baggage handling, security control, staff management, financial transactions, and airport resource management.

The Airport Management System (AMS) is an integrated software platform that manages these operations through a centralized digital system.

The system helps:

- Passengers access travel services
- Airport staff manage operations
- Administrators monitor system activities

The system improves efficiency, accuracy, and the overall airport experience.

1.3 System Scope

The Airport Management System (AMS) is designed to support and manage major airport operations through a centralized software platform. The system integrates multiple operational modules including passenger services, flight operations, baggage management, airport resources, security, finance, human resources, transportation services, and system administration. The system allows passengers to perform online check-in, track baggage, and receive flight notifications. Airport staff can manage operational activities such as assigning gates, handling baggage, and monitoring airport facilities. The system aims to improve operational efficiency, data accuracy, communication between departments, and the overall passenger experience at the airport.

1.4 Assumptions

- ✓ Users have access to an internet connection in order to interact with the system.
- ✓ Passengers must create user accounts before booking tickets or performing online check-in.
- ✓ Airport staff are properly trained to operate the system modules.
- ✓ The system will be accessed primarily through web-based interfaces.
- ✓ The system database will store passenger, flight, baggage, and operational information.

1.5 Constraints

- ✓ The system must comply with aviation security and data protection regulations.
- ✓ The system must operate using a centralized relational database.
- ✓ The system must be accessible through modern web browsers.
- ✓ The system must ensure data privacy and protection of passenger information.
- ✓ The system must integrate with existing airport information systems where required.

1.6. Future Enhancements

Future versions of the Airport Management System may include additional advanced features such as:

- ✓ Mobile application support for passengers and airport staff.
- ✓ Biometric passenger identification systems.
- ✓ AI-based flight delay prediction.
- ✓ Automated baggage tracking using IoT technology.
- ✓ Real-time airport navigation for passengers.
- ✓ Integration with airline loyalty and reward programs.

2. System Users (Actors)

The system supports multiple types of users who interact with it in different ways.

Passenger - Passengers are the primary users of the system. They interact with the system to manage travel-related services such as checking flight status, performing check-in, and tracking baggage.

Visitor - Visitors are users who access the system. They may want to check flight information or explore airport facilities.

Airport Staff - Airport staff handle operational tasks such as baggage processing, gate assignment, and facility management.

Security Officer - Security officers ensure airport safety by verifying passenger identities, monitoring security checkpoints, and managing restricted areas.

Customer Support Agent - Customer support agents assist passengers with inquiries, complaints, and service requests.

Finance Staff - Finance staff manage financial transactions, billing processes, and financial reporting related to airport services.

Human Resource Staff - Human resource staff manage employee records, staff scheduling, and role assignments.

System Administrator - System administrators manage the overall system operation including user accounts, system permissions, and maintenance.

3. Major System Modules – User Characteristics

The Airport Management System is divided into several functional modules. Each module manages a specific group of airport operations.

3.1 Passenger Service Management Module

This module manages services directly related to passengers.

Main Responsibilities

- Online check-in
- Boarding pass generation
- Flight notifications
- Passenger support

Actors

- Passenger
- Customer Support Agent

Example Use Cases

- Perform Online Check-In
- Download Boarding Pass
- Receive Notifications

3.2 Flight Operations Management Module

This module manages flight scheduling and flight operations.

Main Responsibilities

- Flight status updates
- Gate assignment
- Flight delay management

Actors

- Airport Staff

Example Use Cases

- Update Flight Status
- Assign Gate
- Manage Delayed Flights

3.3 Baggage Management Module

This module manages passenger baggage tracking and handling.

Main Responsibilities

- Baggage registration
- Baggage tracking
- Baggage loading
- Lost baggage reporting
- Baggage claim management

Actors

- Airport Staff
- Passenger

Example Use Cases

- Register Baggage
- Track Baggage
- Report Lost Baggage
- Update Baggage Location

3.4 Security Management Module

This module ensures airport safety and security.

Main Responsibilities

- Identity verification
- Security checkpoint monitoring
- Security incident reporting
- Restricted area control

Actors

- Security Officer

Example Use Cases

- Verify Passenger Identity
- Monitor Security Checkpoints
- Record Security Incidents

3.5 Airport Resource Management Module

This module manages airport infrastructure and resources.

Main Responsibilities

- Gate allocation
- Runway scheduling
- Terminal management
- Equipment monitoring

Actors

- Airport Staff

Example Use Cases

- Assign Gates
- Schedule Runway Usage
- Monitor Terminal Capacity

3.6 Finance and Billing Module

This module manages financial transactions related to airport services.

Main Responsibilities

- Service billing
- Financial reporting
- Transaction tracking

Actors

- Passenger
- Finance Staff

Example Use Cases

- Process Ticket Payment
- Record Payment Transactions
- Generate Financial Reports

3.7 Human Resource Management Module

This module manages airport employees and staff operations.

Main Responsibilities

- Employee records
- Staff scheduling
- Role assignment
- Staff management

Actors

- HR Staff
- System Administrator

Example Use Cases

- Register Employee
- Assign Staff Roles
- Manage Staff Schedules

3.8 Customer Support Module

This module handles passenger support services.

Main Responsibilities

- Inquiry management
- Complaint management
- Assistance services
- Feedback handling

Actors

- Passenger
- Customer Support Agent

Example Use Cases

- Submit Complaint
- Request Assistance
- Track Support Requests

3.9 Transportation and Parking Module

This module manages airport transportation services.

Main Responsibilities

- Parking management
- Taxi services
- Shuttle services
- Transportation information

Actors

- Passenger
- Visitor

Example Use Cases

- View Parking Availability
- Book Parking Space
- View Transportation Options

3.10 System Administration Module

This module manages the system infrastructure.

Main Responsibilities

- User account management
- System monitoring
- Permission control
- System maintenance

Actors

- System Administrator

Example Use Cases

- Manage User Accounts
- Monitor System Activity
- Update System Settings

4. Requirements

4.1 Functional Requirements organized by System Modules

4.1.1 Passenger Service Management

These requirements describe the services that the system provides directly to passengers.

1. Passenger views flight schedules
2. Passenger uploads travel documents
3. Passenger performs online check-in
4. Passenger downloads boarding pass
5. Passenger checks assigned gate information
6. Passenger tracks baggage status
7. Passenger receives flight notifications

4.1.2 Flight Operations Management

These requirements manage flight scheduling and airline operations.

8. Airport staff updates flight status
9. Airport staff manages flight delays
10. Airport staff updates departure times
11. Airport staff updates arrival times
12. Airport staff coordinates boarding processes
13. Airport staff monitors flight operations

4.1.3 Baggage Management System

These requirements handle passenger baggage processing and tracking.

14. Airport staff registers passenger baggage
15. Airport staff tracks baggage movement
16. Airport staff records baggage loading

17. Airport staff updates baggage location
18. Airport staff reports lost baggage
19. Airport staff manages baggage claims
20. Passenger reports delayed baggage
21. Airport staff schedules baggage delivery

4.1.4 Security Management System

These requirements support airport security and safety operations.

22. Security officer verifies passenger identity
23. Security officer monitors security checkpoints
24. Security officer records security incidents
25. Security officer manages restricted area access
26. Security officer verifies boarding pass at security checkpoints
27. Security officer manages emergency security alerts

4.1.5 Airport Resource Management

These requirements manage airport infrastructure and operational resources.

28. Airport staff assigns gates to flights
29. Airport staff schedules runway usage
30. Airport staff monitors terminal occupancy
31. Airport staff manages airport facility resources

4.1.6 Customer Support Management

These requirements handle passenger assistance and customer service operations.

32. Customer support agent responds to inquiries
33. Customer support agent tracks service requests

4.1.7 Finance and Billing Management

These requirements manage financial transactions and billing processes.

- 34. Finance staff generates billing reports
- 35. Finance staff processes service charges
- 36. Finance staff generates financial transaction reports

4.1.8 Human Resource Management

These requirements support airport staff management.

- 37. HR staff registers employee records
- 38. HR staff manages employee schedules
- 39. HR staff assigns staff roles

4.1.9 System Administration

These requirements manage the system infrastructure and user control.

- 40. System administrator manages user accounts
- 41. System administrator updates airport information
- 42. System administrator monitors system usage
- 43. System administrator manages system permissions
- 44. System administrator performs system maintenance
- 45. System administrator generates system activity reports

4.2 Non-Functional Requirements (50)

Non-functional requirements describe how the system (Airport Management System) should perform.

4.2.1 Product Requirements

Product requirements describe how the system behaves and its quality attributes (usability, performance, security, availability).

4.2.1.1 Usability Requirements

1. The system user interface must comply with WCAG 2.1 Level AA accessibility standards to accommodate users with disabilities.
2. The system must allow easy navigation.
3. Users must be able to complete core tasks (e.g., checking flight status, finding a gate) within a maximum of 3 clicks from the homepage.
4. The system must provide clear instructions.
5. The system must display helpful error messages.
6. The system must provide user help documentation.
7. The interface must be visually consistent.
8. The system must be accessible to first-time users.

4.2.1.2 Performance Requirements

9. The system must load pages within three seconds.
10. The system must process searches within two seconds.
11. The system must support at least 1000 concurrent users.
12. Payment processing must complete within five seconds.
13. The system must support at least 1500 simultaneous users during peak travel periods.
14. The system must optimize database queries.
15. The system must maintain stable performance under high workload conditions.
16. The system must process requests without noticeable delays.

4.2.1.3 Availability

17. The system must operate 24/7.
18. System downtime must not exceed 1% of total operational time.

19. Maintenance must occur during low-traffic hours.

4.2.1.4 Security

- 20. The system must require user authentication.
- 21. Passwords must be encrypted.
- 22. The system must protect personal data.
- 23. The system must prevent unauthorized access.
- 24. Sessions must expire after inactivity.
- 25. Login attempts must be logged.
- 26. Data transmission must use secure communication protocols.
- 27. Sensitive information must be protected.
- 28. The system must implement Role-Based Access Control (RBAC) to ensure staff can only access modules relevant to their job title.

4.2.1.5 Portability

- 29. The web application must be fully functional on the latest two major versions of Google Chrome, Mozilla Firefox, Apple Safari, and Microsoft Edge.
- 30. The mobile web interface must support responsive design optimized for iOS 15+ and Android 12+.
- 31. The system must operate smoothly on-screen resolutions ranging from 360x800 pixels (mobile) to 1920x1080 pixels (desktop).

4.2.2 Organizational Requirements

These requirements relate to how the system is developed, maintained, and structured.

Maintainability

- 32. The system must have modular code.
- 33. The system must be well documented.
- 34. Updates must be easy to apply.
- 35. Bugs must be easy to identify and fix.
- 36. Developers must be able to add new features easily.

Reliability

- 37. The system must maintain at least 99% system uptime.
- 38. The system must recover from minor failures.
- 39. The system must maintain data integrity during all transactions.
- 40. The system must prevent data corruption.
- 41. The system must perform automated daily incremental backups and weekly full backups, retaining data for a minimum of 30 days.
- 42. System errors must be logged.
- 43. The system must support future system expansion.
- 44. The system must handle large volumes of operational and passenger data efficiently

Data Management

- 45. User data must be stored securely.
- 46. Flight data must remain accurate.
- 47. The system must maintain data consistency across all system modules.

4.2.3 External Requirements

These requirements concern interaction with external environments (devices, systems, platforms).

Compatibility

- 48. The system must comply with international and national government laws.
- 49. The system must coordinate with other airport systems and air traffic control.
- 50. The system must comply with Airport Collaborative Decision Making (A-CDM) standards.

5. User Scenarios / Use Cases

Use Case 1: Passenger Performs Online Check-in

Actor: Passenger

Description:

A passenger verifies their travel documents and generates a boarding pass for an upcoming scheduled flight.

Steps:

1. Passenger logs into the system.
2. Passenger inputs their flight reference number.
3. Passenger verifies travel document information
4. Passenger confirms check-in

Outcome:

The system generates a boarding pass and updates the passenger's check-in status.

6. Conclusion

The Airport Management System integrates multiple airport operations into a unified digital platform. The requirements described in this document define the system's expected behavior, operational capabilities, and quality attributes.

These requirements will guide future development phases including system modeling, architectural design, database design, and implementation.

Airport Management System

Passenger Service Management Use Cases Documentation

Prepared by:

Betül Kaan

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *Viewing Flight Schedules*
- ✓ *Uploading Flight Documents*
- ✓ *Online Check-In*
- ✓ *Boarding Pass Download*
- ✓ *Checking Gate Information*
- ✓ *Tracking Baggage Status*
- ✓ *Receiving Flight Notification*
- ✓ *Each use case is defined with:*

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

UC Name	<i>UC-01: View Flight Schedules</i>
Summary	<i>A passenger views available flight schedules including departure/arrival times and flight status.</i>
Dependency	<i>None</i>
Actors	<i>Primary: Passenger</i>
Preconditions	<i>The passenger has access to the system (web). The system has up-to-date flight data.</i>
Description of the Main Sequence	<i>Step 1: Passenger opens the system and navigates to the flight schedules page.</i> <i>Step 2: Passenger enters search criteria (date, destination, or flight number).</i> <i>Step 3: System retrieves and displays matching flight schedules.</i> <i>Step 4: Passenger views the flight details.</i>
Description of the Alternative Sequence	<i>Step 1: Passenger enters invalid or incomplete search criteria.</i> <i>Step 2: System displays an error message prompting the passenger to correct the input.</i> <i>Step 3: Passenger re-enters valid criteria and the system displays results.</i>
Non functional requirements	<i>The system must process the search and return results within 2 seconds. The interface must be accessible and usable on both desktop and mobile devices.</i>
Postconditions	<i>Flight schedule information is displayed to the passenger.</i>

UC Name	<i>UC-02: Upload Travel Documents</i>
Summary	<i>A passenger uploads required travel documents (e.g., passport, visa) to the system for verification purposes.</i>
Dependency	<i>UC-01 (Passenger may need to view flight info before uploading documents)</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has an active booking reference. The passenger has digital copies of their documents.</i>

Description of the Main Sequence	Step 1: Passenger logs into the system and navigates to the document upload section. As Step 2: Passenger selects the document type (e.g., passport, visa). Step 3: Passenger uploads the document file. Step 4: System validates the file format and size. Step 5: System confirms successful upload and stores the document securely.
Description of the Alternative Sequence	Step 1: Passenger uploads a file in an unsupported format or exceeding the size limit. Step 2: System displays an error message specifying the accepted formats and size. Step 3: Passenger uploads a valid file and the system confirms the upload.
Non functional requirements	Uploaded documents must be encrypted and stored securely. The system must protect personal data in compliance with data protection regulations.
Postconditions	Travel documents are stored in the system and linked to the passenger's profile.

UC Name	UC-03: Perform Online Check-In
Summary	A passenger completes the check-in process online through the airport management system prior to their flight.
Dependency	UC-02: Upload Travel Documents
Actors	Primary: Passenger
Preconditions	The passenger has a valid booking reference. The check-in window for the flight is open. Travel documents have been uploaded.
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the online check-in section. Step 2: Passenger enters their booking reference or flight details. Step 3: System retrieves and displays the passenger's flight information. Step 4: Passenger confirms personal and travel details. Step 5: System processes the check-in and confirms the passenger's checked-in status.
Description of the Alternative Sequence	Step 1: Passenger attempts check-in outside the allowed time window. Step 2: System displays a message indicating that check-in is not

	yet open or has closed. Step 3: Passenger is redirected to contact customer support or the airline directly.
Non functional requirements	The check-in process must be completable within 3 clicks from the homepage. Page load times must not exceed 3 seconds.
Postconditions	The passenger is marked as checked in and a boarding pass becomes available for download.

UC Name	UC-04: Download Boarding Pass
Summary	A passenger downloads their boarding pass after completing online check-in.
Dependency	UC-03: Perform Online Check-In
Actors	Primary: Passenger
Preconditions	The passenger has successfully completed online check-in. The system has generated a boarding pass for the passenger.
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the boarding pass section. Step 2: System retrieves the passenger's boarding pass. Step 3: Passenger selects the download option. Step 4: System generates and downloads the boarding pass as a PDF or image file.
Description of the Alternative Sequence	Step 1: Passenger attempts to download a boarding pass before completing check-in. Step 2: System displays a message indicating that check-in must be completed first. Step 3: Passenger is redirected to the check-in page (UC-03).
Non functional requirements	The boarding pass must be generated and ready for download within 3 seconds. The downloaded file must be compatible with standard mobile and desktop devices.
Postconditions	The passenger has a downloaded boarding pass ready for use at the airport.

UC Name	UC-05: Check Assigned Gate Information
----------------	--

Summary	<i>A passenger checks which gate their flight is departing from.</i>
Dependency	<i>UC-01: View Flight Schedules</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has a valid flight on the day of travel. A gate has been assigned to the flight by airport staff.</i>
Description of the Main Sequence	Step 1: Passenger logs in or accesses the public flight information page. Step 2: Passenger searches for their flight by number or destination. Step 3: System retrieves and displays the assigned gate information. Step 4: Passenger views the gate number and terminal details.
Description of the Alternative Sequence	Step 1: <i>Passenger searches for a flight that has not yet been assigned a gate.</i> Step 2: <i>System displays a message indicating the gate assignment is pending.</i> Step 3: <i>Passenger is advised to check back later or enable flight notifications (UC-07).</i>
Non functional requirements	<i>Gate information must be current and updated in real time. The system must display results within 2 seconds of the search.</i>
Postconditions	<i>The passenger is informed of their assigned departure gate.</i>

UC Name	<i>UC-06: Track Baggage Status</i>
Summary	<i>A passenger tracks the current status and location of their checked baggage.</i>
Dependency	<i>UC-03: Perform Online Check-In</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has checked in and their baggage has been registered in the system by airport staff.</i>
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the baggage tracking section. Step 2: Passenger enters their booking reference or baggage tag number. Step 3: System retrieves and displays the current baggage status and location. Step 4: Passenger reviews the baggage tracking information.

Description of the Alternative Sequence	<i>Step 1: Passenger enters an invalid or unrecognized baggage tag number. Step 2: System displays an error message and prompts the passenger to re-enter or contact support. Step 3: Passenger contacts customer support (links to UC in Customer Support module).</i>
Non functional requirements	<i>Baggage status must reflect real-time updates. The system must handle simultaneous tracking requests from multiple passengers without performance degradation.</i>
Postconditions	<i>The passenger is informed of the current status and location of their baggage.</i>

UC Name	<i>UC-07: Receive Flight Notifications</i>
Summary	<i>A passenger opts in to receive real-time notifications about their flight, including delays, gate changes, and boarding calls.</i>
Dependency	<i>UC-01: View Flight Schedules</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has a valid flight in the system. The passenger has provided a valid contact method (email or phone number).</i>
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the notifications settings section. Step 2: Passenger selects their flight and preferred notification method (email or SMS). Step 3: Passenger confirms notification preferences. Step 4: System registers the passenger for flight alerts. Step 5: System sends real-time notifications when flight status changes (e.g., delay, gate change, boarding).
Description of the Alternative Sequence	Step 1: Passenger provides an invalid email address or phone number. Step 2: System displays a validation error and prompts the passenger to correct the contact information. Step 3: Passenger re-enters valid contact details and confirms.
Non functional requirements	<i>Notifications must be delivered within a reasonable time of a status change. The system must operate 24/7 to ensure passengers receive timely alerts at any hour.</i>
Postconditions	<i>The passenger receives notifications for any updates related to their flight.</i>

Airport Management System
Flight Operations Management Use Cases Documentation

Prepared by:

Albina Blloshmi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of Baggage Management Use Cases for the Airport Management System.

It includes:

- ✓ Baggage check-in and registration*
- ✓ Baggage tracking and real-time status updates*
- ✓ Handling lost and delayed baggage*
- ✓ Baggage delivery to passengers*
- ✓ Baggage routing and transfer between flights*
- ✓ Security screening of baggage*
- ✓ Each use case is defined with:*
 - ✓ Actors*
 - ✓ Preconditions*
 - ✓ Main and Alternative Flows*
 - ✓ Non-functional Requirements*
 - ✓ Postconditions*

The goal of this document is to clearly model and describe baggage management processes involving both passengers and airport staff in a professional and structured way.

UC Name	UC-08: Claim Baggage at Destination
Summary	A passenger collects their baggage upon arrival at the destination airport.
Dependency	UC-01: Check-In Baggage (external dependency from earlier module)
Actors	Primary: Passenger Airport Staff
Preconditions	Flight has arrived; Baggage has been delivered to the carousel.
Description of the Main	Step 1: Passenger goes to the baggage claim area. Step 2: System displays carousel number. Step 3: Passenger identifies their baggage. Step 4: Passenger collects baggage. <i>Step 2: Passenger enters search criteria (date, destination, or flight number).</i> <i>Step 3: System retrieves and displays matching flight schedules.</i> <i>Step 4: Passenger views the flight details.</i>
Description of the Alternative Sequence	Step 1: Passenger cannot find baggage. Step 2: Passenger reports missing baggage to staff.
Non functional requirements	Clear display systems; Minimal waiting time
Postconditions	Passenger successfully retrieves baggage

UC Name	UC-09: Track Baggage in Real-Time
Summary	Passenger monitors baggage location throughout the journey in real time.
Dependency	UC-08: Claim Baggage at Destination (or None if used independently)
Actors	<i>Primary: Passenger</i>
Preconditions	Baggage has been checked in; Tracking system is active.
Description of the Main Sequence	Step 1: Passenger logs into the system.Step 2: Passenger selects baggage tracking option.Step 3: System retrieves real-time data.Step 4: System displays current baggage location.
Description of the Alternative Sequence	Step 1: Tracking data unavailable.Step 2: System shows last known location. <i>Step 3: Passenger re-enters valid criteria and the system displays results.</i>
Non functional requirements	Real-time updates; High accuracy
Postconditions	Passenger is informed of baggage location

UC Name	UC-10: Report Damaged Baggage
Summary	Passenger reports damage to baggage after arrival.
Dependency	UC-08: Claim Baggage at Destination
Actors	Primary: Passenger; Secondary:Airport Staff

Preconditions	Passenger has received baggage
Description of the Main Sequence	Step 1: Passenger inspects baggage.Step 2: Passenger reports damage via system or desk.Step 3: System records complaint.Step 4: Staff verifies damage.Step 5: System processes claim.
Description of the Alternative Sequence	Step 1: Claim rejected due to invalid proof.Step 2: System notifies passenger.
Non functional requirements	Secure claim handling; Quick processing.
Postconditions	Damage report is recorded and processed.

UC Name	UC-11: Handle Delayed Baggage
Summary	System manages cases where baggage is delayed.
Dependency	UC-09: Track Baggage in Real-Time
Actors	Primary: System; Secondary: Staff
Preconditions	Baggage is not delivered on time.
Description of the Main Sequence	Step 1: System detects delay.Step 2: System updates baggage status.Step 3: Passenger is notified.Step 4: Staff arranges delivery.
Description of the Alternative Sequence	Step 1: Baggage cannot be located.Step 2: Lost baggage procedure initiated.

Non functional requirements	Timely notifications; Reliable tracking.
Postconditions	Passenger is informed and baggage is processed for delivery

UC Name	UC-12: Deliver Baggage to Passenger
Summary	Baggage is delivered to the passenger after delay or rerouting.
Dependency	UC-11: Handle Delayed Baggage
Actors	Primary: Staff Secondary: Passenger
Preconditions	Baggage has been located.
Description of the Main Sequence	Step 1: Staff schedules delivery Step 2: System updates delivery status Step 3: Baggage is transported to passenger location Step 4: Passenger confirms receipt
Description of the Alternative Sequence	Step 1: Delivery fails Step 2: System reschedules delivery Step 3: Staff is notified to attempt redelivery
Non functional requirements	Timely delivery; Secure handling.

Postconditions	Passenger receives baggage.
-----------------------	-----------------------------

UC Name	UC-13: Update Baggage Information
Summary	Staff updates baggage details in the system.
Dependency	UC-11: Handle Delayed Baggage
Actors	Primary: Staff
Preconditions	Baggage exists in the system.
Description of the Main Sequence	Step 1: Staff accesses baggage record.Step 2: Staff updates details.Step 3: System saves changes.Step 4: System reflects updated info.
Description of the Alternative Sequence	Step 1: Invalid data entered.Step 2: System displays error message.
Non functional requirements	Data accuracy; Secure access.
Postconditions	Baggage information is updated.

Airport Management System
Flight Operations Management Use Cases Documentation

Prepared by:

Alvi Elezi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a set of Flight Operations Management Use Cases for the Airport Management System.

It includes:

- ✓ *Flight status updates*
- ✓ *Flight delay management*
- ✓ *Departure time updates*
- ✓ *Arrival time updates*
- ✓ *Coordination between boarding processes*
- ✓ *Flight operations monitoring*
- ✓ *Each use case is defined with:*
 - ✓ *Actors*
 - ✓ *Preconditions*
 - ✓ *Main and alternative flows*
 - ✓ *Non-functional requirements*
 - ✓ *Postconditions*

The goal of this document is to model and describe the management of flight operations by airport staff and the behavior of the system in a structured way.

UC Name	<i>FO_01 Update Flight Status</i>
Summary	<i>Allows airport staff to update the current status of a flight (e.g., on time, delayed, boarding, departed).</i>
Dependency	<i>Depends on existing flight records in the system.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Staff member is authenticated</i> · <i>Flight exists in the system</i> · <i>Staff has permission to update flight data</i>
Description of the Main Sequence	● <i>Step 1: Airport staff logs into the system.</i> ● <i>Step 2: Staff accesses the flight operations dashboard.</i> ● <i>Step 3: Staff selects a specific flight.</i> ● <i>Step 4: System displays current flight details.</i> ● <i>Step 5: Staff updates the flight status (e.g., delayed, boarding).</i> ● <i>Step 6: System validates the update.</i> ● <i>Step 7: System saves the updated flight status.</i> ● <i>Step 8: System notifies relevant stakeholders (passengers, displays).</i>

Description of the Alternative Sequence	• <i>Step 1: Staff selects an invalid flight ID.</i> • <i>Step 2: System displays an error message.</i> • <i>Step 3: If system validation fails, update is rejected and staff must re-enter valid data.</i>
Nonfunctional requirements	· <i>Updates must propagate in real time</i> · <i>System response time must be under 2 seconds</i> · <i>Data consistency must be maintained across modules</i>
Postconditions	<i>Flight status is updated and visible across the system.</i>

UC Name	<i>FO_02 Manage Flight Delays</i>
Summary	<i>Allows airport staff to handle flight delays and update delay-related information.</i>
Dependency	<i>Extends “Update Flight Status”.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Flight exists in the system</i> · <i>Staff is authorized</i>

Description of the Main Sequence	• <i>Step 1: Staff logs into the system.</i> • <i>Step 2: Staff selects a flight marked for delay.</i> • <i>Step 3: Staff inputs delay duration and reason.</i> • <i>Step 4: System updates the departure/arrival time.</i> • <i>Step 5: System records delay reason.</i> • <i>Step 6: System sends notifications to passengers.</i>
Description of the Alternative Sequence	• <i>Step 1: Staff enters invalid delay duration.</i> • <i>Step 2: System prompts correction before saving.</i>
Nonfunctional requirements	· <i>Notifications must be delivered instantly</i> · <i>System must handle concurrent updates without conflict</i>
Postconditions	<i>Flight delay is recorded and updated in the system.</i>

UC Name	<i>FO_03 Update Departure Time</i>
Summary	<i>Allows airport staff to modify the scheduled departure time of a flight.</i>
Dependency	<i>Related to the flight scheduling subsystem.</i>

Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Flight exists</i> · <i>Staff has update privileges</i>
Description of the Main Sequence	● <i>Step 1: Staff logs into the system.</i> ● <i>Step 2: Staff navigates to flight scheduling.</i> ● <i>Step 3: Staff selects a flight.</i> ● <i>Step 4: Staff enters new departure time.</i> ● <i>Step 5: System validates time constraints.</i> ● <i>Step 6: System updates the flight schedule.</i> ● <i>Step 7: System notifies passengers and airport displays.</i>
Description of the Alternative Sequence	● <i>Step 1: Entered time conflicts with runway schedule.</i> ● <i>Step 2: System rejects update and suggests correction.</i>
Nonfunctional requirements	· <i>System must validate scheduling conflicts</i> · <i>Updates must reflect immediately</i>
Postconditions	<i>Departure time is updated in the system.</i>

UC Name	<i>FO_04 Update Arrival Time</i>
Summary	<i>Allows airport staff to update the arrival time of a flight.</i>
Dependency	<i>Connected to flight tracking systems.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Flight exists</i> · <i>Staff is authorized</i>
Description of the Main Sequence	● <i>Step 1: Staff logs in.</i> ● <i>Step 2: Staff selects flight record.</i> ● <i>Step 3: Staff enters updated arrival time.</i> ● <i>Step 4: System validates input.</i> ● <i>Step 5: System updates arrival time.</i> ● <i>Step 6: System updates passenger notifications.</i>
Description of the Alternative Sequence	● <i>Step 1: Invalid time entered.</i> ● <i>Step 2: System displays validation error.</i>
Nonfunctional requirements	· <i>High data accuracy required</i> · <i>System must synchronize with external systems if integrated</i>

Postconditions	<i>Arrival time is successfully updated.</i>
----------------	--

UC Name	<i>FO_05 Coordinate Boarding Process</i>
Summary	<i>Coordinate Boarding Process</i>
Dependency	<i>Depends on check-in completion and gate assignment.</i>
Actors	<i>Primary Actor: Airport Staff</i> <i>Secondary Actor: Passenger</i>
Preconditions	· <i>Passengers are checked in</i> · <i>Gate is assigned</i> · <i>Boarding process is initiated</i>
Description of the Main Sequence	● <i>Step 1: Staff accesses boarding control interface.</i> ● <i>Step 2: Staff initiates boarding process.</i> ● <i>Step 3: System displays passenger list.</i> ● <i>Step 4: Passengers present boarding passes.</i> ● <i>Step 5: System scans and validates boarding passes.</i> ● <i>Step 6: System marks passengers as boarded.</i> ● <i>Step 7: Staff monitors boarding progress.</i>

	• <i>Step 8: Boarding is closed when complete.</i>
Description of the Alternative Sequence	• <i>Step 1: Invalid boarding pass scanned.</i> • <i>Step 2: System denies boarding and alerts staff.</i>
Nonfunctional requirements	· <i>Real-time passenger validation</i> · <i>High system availability during peak operations</i>
Postconditions	<i>Passenger boarding is completed and recorded.</i>

UC Name	<i>FO_06 Monitor Flight Operations</i>
Summary	<i>Allows airport staff to monitor real-time flight activities and operational status.</i>

Dependency	<i>Integrates with multiple modules (flight, baggage, resources).</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Staff is logged into system</i> · <i>System is operational</i>
Description of the Main Sequence	● <i>Step 1: Staff logs into dashboard.</i> ● <i>Step 2: System displays all active flights.</i> ● <i>Step 3: Staff views real-time updates (status, delays, gates).</i> ● <i>Step 4: Staff identifies issues or anomalies.</i> ● <i>Step 5: Staff takes corrective actions if necessary.</i>
Description of the Alternative Sequence	● <i>Step 1: System fails to retrieve real-time data.</i> ● <i>Step 2: System displays last known status with warning.</i>
Nonfunctional requirements	· <i>Dashboard must refresh in real time</i> · <i>System must support high data throughput</i>
Postconditions	<i>Flight operations are continuously monitored.</i>

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Eneida Balliu

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *User Account Management*
- ✓ *Airport Information Updates*
- ✓ *System Monitoring and Usage Tracking*
- ✓ *Permission and Access Control*
- ✓ *System Maintenance Operations*
- ✓ *System Activity Reporting*
- ✓ *Each use case is defined with:*

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

UC Name	UC-40 Manage User Accounts
Summary	This use case describes how the system administrator creates, updates, deletes, and manages user accounts within the system.
Dependency	None
Actors	Primary Actor: System Administrator Secondary Actors: None
Preconditions	✓ The system administrator is logged into the system. ✓ The administrator has the necessary permissions to manage user accounts. ✓ The system is operational and connected to the database.
Description of the Main Sequence	Step 1: The system administrator accesses the user account management module. Step 2: The system displays a list of existing user accounts. Step 3: The administrator selects an action (create, update, or delete account). Step 4: If creating a new account, the administrator enters user details (name, role, email, credentials) . Administrator assigns the user to an airport-specific role (for example: Baggage Handler, Finance Staff, Customer Support, Security Officer). Step 5: If updating, the administrator modifies the required user information. Step 6: If deleting, the administrator selects the account to remove. Step 7: The system validates the entered information.

	Step 8: The system saves the changes in the database. Step 9: The system confirms the successful operation.
Description of the Alternative Sequence	Step 1: If the administrator enters invalid or incomplete data, the system displays an error message and requests correction. Step 2: If the selected user account does not exist, the system notifies the administrator. Step 3: If there is a system/database failure, the system displays an error and cancels the operation. Step 4: If the administrator cancels the operation, no changes are saved.
Non functional requirements	✓ The system must ensure secure authentication before allowing access to account management. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must process account updates within acceptable response time. ✓ The system must log all account management activities for auditing purposes. ✓ The system must protect user data and ensure privacy.

Postconditions	✓ User account information is successfully created, updated, or deleted. ✓ The system database is updated accordingly. ✓ All changes are recorded in the system logs.
-----------------------	---

UC Name	UC-41 Update Airport Information
Summary	This use case describes how the system administrator updates airport-related information such as facilities, schedules, and general airport data. Admin updates static airport information, such as terminal maps, shop opening hours, runway maintenance statuses, or general airport announcements.
Dependency	None
Actors	Primary Actor: System Administrator Secondary Actors: None
Preconditions	✓ The system administrator is logged into the system. ✓ The administrator has permission to update airport information. ✓ The system database is available.

Description of the Main Sequence	Step 1: The system administrator accesses the airport information management module. Step 2: The system displays current airport information. Step 3: The administrator selects the information to update. Step 4: The administrator edits the required details (e.g., facilities, schedules, announcements). Step 5: The administrator submits the updated information. Step 6: The system validates the entered data. Step 7: The system saves the updated information in the database. Step 8: The system confirms that the update was successful.
Description of the Alternative Sequence	Step 1: If invalid or incomplete data is entered, the system displays an error message and requests correction. Step 2: If the selected information does not exist, the system notifies the administrator. Step 3: If a system or database error occurs, the system cancels the operation and displays an error message. Step 4: If the administrator cancels the update, no changes are saved.
Non functional requirements	✓ The system must ensure secure access through authentication. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must process updates within acceptable response time. ✓ The system must ensure data accuracy and consistency. ✓ The system must log all updates for auditing purposes.

Postconditions	✓ Airport information is successfully updated in the system. ✓ The database reflects the latest information. ✓ The update activity is recorded in system logs.
-----------------------	--

UC Name	UC-42 Monitor System Usage
Summary	This use case describes how the system administrator monitors system activity, usage statistics, and overall system performance.
Dependency	None
Actors	Primary Actor: System Administrator Secondary Actors: None
Preconditions	✓ The system administrator is logged into the system. ✓ The administrator has permission to access system monitoring tools. ✓ The system is operational and collecting usage data.

Description of the Main Sequence	Step 1: The system administrator accesses the system monitoring module. Step 2: The system displays system usage data (active users, transactions, system load). Step 3: The administrator selects specific metrics or reports to view. Step 4: The system retrieves and displays the requested information. Step 5: The administrator reviews system activity and performance indicators. Step 6: The administrator may filter or refine the displayed data. Step 7: The system updates the displayed information based on the selected filters.
Description of the Alternative Sequence	Step 1: If the system cannot retrieve usage data, an error message is displayed. Step 2: If there is a system failure or delay, the system notifies the administrator. Step 3: If no data is available for the selected criteria, the system informs the administrator. Step 4: If the administrator cancels the operation, the process ends without changes.
Non functional requirements	✓ The system must provide real-time system monitoring data. ✓ The system must ensure secure access through authentication. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must display data within acceptable response time.

	✓ The system must ensure accuracy and reliability of system usage data.
Postconditions	✓ The system administrator has successfully viewed system usage information. ✓ No changes are made to system data. ✓ System monitoring activities may be logged for auditing purposes.

UC Name	UC-43 Manage System Permissions
Summary	This use case describes how the system administrator assigns, updates, and controls user access rights and permissions within the system.
Dependency	UC-40 Manage User Accounts (Can not manage permissions if users do not exist.)
Actors	Primary Actor: System Administrator Secondary Actors: None

<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to manage system roles and access rights. ✓ User accounts already exist in the system.
<i>Description of the Main Sequence</i>	<i>Step 1:</i> The system administrator accesses the permission management module. <i>Step 2:</i> The system displays a list of users and their current roles/permissions. <i>Step 3:</i> The administrator selects a user account. <i>Step 4:</i> The administrator assigns or modifies user roles and permissions. <i>Step 5:</i> The administrator confirms the changes. <i>Step 6:</i> The system validates the permission settings. <i>Step 7:</i> The system updates the permissions in the database. <i>Step 8:</i> The system confirms that the permissions have been successfully updated.
<i>Description of the Alternative Sequence</i>	<i>Step 1:</i> If invalid or conflicting permissions are assigned, the system displays an error message. <i>Step 2:</i> If the selected user does not exist, the system notifies the administrator. <i>Step 3:</i> If a system or database error occurs, the system cancels the operation and displays an error message. <i>Step 4:</i> If the administrator cancels the operation, no changes are saved.

<i>Non functional requirements</i>	✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must ensure secure authentication before permission changes. ✓ The system must log all permission changes for auditing purposes. ✓ The system must ensure data integrity and prevent unauthorized access. ✓ The system must process updates within acceptable response time.
<i>Postconditions</i>	✓ User permissions are successfully updated. ✓ The system database reflects the new access rights. ✓ All changes are recorded in system logs.

<i>UC Name</i>	<i>UC-44</i> Perform System Maintenance
<i>Summary</i>	This use case describes how the system administrator performs maintenance operations including data backup, data recovery, and system configuration updates.
<i>Dependency</i>	None
<i>Actors</i>	<i>Primary Actor:</i> System Administrator <i>Secondary Actors:</i> None

Preconditions	✓ The system administrator is logged into the system. ✓ The administrator has permission to perform maintenance operations. ✓ The system is operational and connected to the database
Description of the Main Sequence	Step 1: The system administrator accesses the system maintenance module. Step 2: The system displays available maintenance operations (data backup, data recovery, configuration updates). Step 3: The administrator selects a maintenance operation. Step 4: • If backup is selected, the system creates a backup of system data. • If recovery is selected, the administrator selects a backup to restore. • If configuration update is selected, the administrator modifies system settings. Step 5: The administrator confirms the operation. Step 6: The system validates the request. Step 7: The system executes the selected operation. Step 8: The system saves changes and updates the database if necessary. Step 9: The system confirms successful completion.
Description of the Alternative Sequence	Step 1: If invalid input or configuration is provided, the system displays an error message. Step 2: If backup or recovery fails, the system notifies the administrator. Step 3: If a system or database error occurs, the operation is cancelled and an error is displayed.

	Step 4: If the administrator cancels the operation, no changes are applied.
Non functional requirements	✓ The system must ensure secure access through authentication. ✓ The system must ensure data integrity during backup and recovery operations. ✓ The system must perform maintenance operations within acceptable response time. ✓ The system must log all maintenance activities for auditing purposes. ✓ The system must ensure system availability and minimize downtime.
Postconditions	✓ Maintenance operations are successfully completed. ✓ System data is backed up, restored, or updated accordingly. ✓ All maintenance activities are recorded in system logs.

UC Name	UC-45 Generate System Activity Reports
Summary	This use case describes how the system administrator generates reports on system activity, including user actions, system usage, and performance data.

Dependency	UC_42 Monitor System Usage
Actors	Primary Actor: System Administrator Secondary Actors: None
Preconditions	✓ The system administrator is logged into the system. ✓ The administrator has permission to generate reports. ✓ The system contains activity and usage data.
Description of the Main Sequence	Step 1: The system administrator accesses the reporting module. Step 2: The system displays available report types (user activity, system usage, performance reports). Step 3: The administrator selects a report type. Step 4: The administrator defines report criteria (date range, filters). Step 5: The administrator requests report generation. Step 6: The system retrieves relevant data from the database. Step 7: The system generates the report. Step 8: The system displays the report to the administrator. Step 9: The administrator may export or save the report.

Description of the Alternative Sequence	Step 1: If no data is available for the selected criteria, the system notifies the administrator. Step 2: If invalid criteria are entered, the system displays an error message. Step 3: If a system or database error occurs, the report generation fails and an error is displayed. Step 4: If the administrator cancels the operation, no report is generated.
Non functional requirements	✓ The system must ensure secure access through authentication. ✓ The system must generate reports within acceptable response time. ✓ The system must ensure accuracy and reliability of report data. ✓ The system must allow exporting reports in standard formats (for example: PDF, CSV). ✓ The system must log report generation activities
Postconditions	✓ The system activity report is successfully generated. ✓ The administrator can view or export the report. ✓ Report generation activity is recorded in system logs.

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ameo Lulaj

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *Viewing Flight Schedules*
- ✓ *Uploading Flight Documents*
- ✓ *Online Check-In*
- ✓ *Boarding Pass Download*
- ✓ *Checking Gate Information*
- ✓ *Tracking Baggage Status*
- ✓ *Receiving Flight Notification*
- ✓ *Each use case is defined with:*

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

UC-ARM-01: Gate Allocation Management

Summary	This use case describes how the system assigns and manages airport gates for arriving and departing flights, ensuring no scheduling conflicts.
Dependency	Flight schedule data must be available.
Actors	Primary Actor: System Administrator / Airport Staff Secondary Actors: None
Preconditions	✓ The administrator is logged into the system. ✓ Flight schedules are available. ✓ Gates are registered in the system.
Main Flow	1. Admin navigates to Gate Management module. 2. System displays available and occupied gates. 3. Admin selects a flight. 4. System suggests available gates. 5. Admin assigns a gate. 6. System updates gate status.
Alternative Flow	• If no gates are available → system displays alert. • Admin can manually override gate assignment.
Postconditions	✓ Gate is successfully assigned. ✓ System reflects updated gate availability.

UC-ARM-02: Runway Scheduling

Summary	This use case describes how runways are scheduled for aircraft takeoff and landing operations.
Dependency	Flight schedule and runway data must be available.
Actors	Primary Actor: System Administrator / Air Traffic Controller Secondary Actors: None
Preconditions	✓ User is logged into the system. ✓ Runways are registered in the system. ✓ Flight schedule exists.
Main Flow	1. Admin accesses runway scheduling module. 2. System displays runway timetable. 3. Admin selects a flight. 4. System suggests available time slots. 5. Admin confirms scheduling. 6. System updates runway allocation.
Alternative Flow	• Emergency flight → system prioritizes runway allocation. • Conflict detected → system prompts rescheduling.
Postconditions	✓ Runway is allocated without conflict. ✓ Schedule is updated successfully.

UC-ARM-03: Staff Allocation

Summary	This use case describes how airport staff are assigned to operational tasks and shifts.
Dependency	Staff database must be available.

Actors	Primary Actor: System Administrator Secondary Actors: Staff
Preconditions	✓ Admin is logged into the system. ✓ Staff details exist in the system.
Main Flow	1. Admin opens staff management module. 2. System displays staff availability. 3. Admin assigns tasks and shifts. 4. System updates staff schedule. 5. Staff receives notification.
Alternative Flow	• If selected staff is unavailable → system suggests alternatives.
Postconditions	✓ Staff are assigned successfully. ✓ Work schedules are updated.

UC-ARM-04: Equipment Management

Summary	This use case describes how airport equipment and assets are tracked, monitored, and maintained.
Dependency	Equipment database must be available.
Actors	Primary Actor: System Administrator Secondary Actors: Maintenance Staff

Preconditions	✓ Admin is logged into the system. ✓ Equipment is registered in the system.
Main Flow	1. Admin accesses equipment management module. 2. System displays equipment list. 3. Admin updates equipment status (available, in use, maintenance). 4. Admin logs maintenance details. 5. System saves updates.
Alternative Flow	• Equipment failure → system generates maintenance alert.
Postconditions	✓ Equipment status is updated. ✓ Maintenance records are stored.

UC-CSM-01: Ticket Management

f	This use case describes how passengers submit and track support queries using a ticketing system.
Dependency	System must support user input and ticket storage.
Actors	Primary Actor: Passenger Secondary Actors: Support Staff

Preconditions	✓ Passenger has access to the system.
Main Flow	1. Passenger opens support page. 2. Passenger submits query via form. 3. System generates a unique ticket ID. 4. Ticket is assigned to support staff. 5. Staff updates ticket status. 6. Passenger tracks ticket progress.
Alternative Flow	• Invalid input → system prompts user to correct details.
Postconditions	✓ Ticket is successfully created. ✓ Issue is tracked and managed.

UC-CSM-02: Live Chat Support

Summary	This use case describes how passengers receive real-time assistance through chat support.
Dependency	Chat system must be active and connected.
Actors	Primary Actor: Passenger Secondary Actors: Support Agent, Chatbot
Preconditions	✓ User is online and can access the chat feature.

Main Flow	1. Passenger opens chat widget. 2. Chatbot greets the user. 3. Passenger submits a query. 4. Chatbot responds or transfers to a live agent. 5. Agent resolves the issue. 6. Chat session is stored.
Alternative Flow	• Agent unavailable → chatbot handles query. • Complex issue → system creates support ticket automatically.
Postconditions	✓ Query is resolved or escalated. ✓ Chat history is saved.

***Airport Management System
System Administration Use Cases Documentation***

Prepared by:

Andri Mucllari

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

*This document presents a structured set of use cases for **Finance & Billing Management** and **Human Resource Management** in the Airport Management System. It includes:*

- *Passenger Payment Processing*
- *Airline Billing and Invoicing*
- *Financial Transaction Tracking*
- *Employee Record Management*
- *Staff Attendance Management*
- *Payroll Processing*

Each use case is defined with:

- *Actors*
- *Preconditions*
- *Main and Alternative Flows*
- *Non-functional Requirements*
- *Postconditions*

The purpose of this document is to clearly describe financial and human resource related system operations in a structured and professional manner.

UC Name	UC_50 Process Airport Service Payment
Summary	This use case describes how passengers pay for airport services such as parking, lounge access, or baggage handling.
Dependency	None
Actors	Primary Actor: Passenger Secondary Actor: Payment System
Preconditions	- The passenger is present at the airport or using airport services - The system is operational. - Payment service is available
Description of the Main Sequence	Step 1: The passenger selects an airport service. Step 2: The system displays the service fee. Step 3: The passenger provides payment details. Step 4: The system validates the payment information. Step 5: The system processes the payment. Step 6: The system records the transaction. Step 7: The system generates a receipt. Step 8: The system confirms successful payment.
Description of the Alternative Sequence	Step 1: If payment details are invalid, the system displays an error. Step 2: If payment fails, the system notifies the passenger. Step 3: If system error occurs, the transaction is cancelled. Step 4: If the passenger cancels, no payment is processed.
Non functional requirements	- Secure payment processing must be ensured. - Transactions must be processed quickly. - All payments must be logged. - The system must ensure data privacy.
Postconditions	- Payment is successfully completed. - Transaction is stored in the system. - Receipt is generated.

UC Name	UC_51 Generate Airport Service Invoice
Summary	This use case describes how the airport generates invoices for services provided to external parties (aircraft parking, ground handling).
Dependency	None

Actors	Primary Actor: Finance Staff Secondary Actor: Billing System/Database
Preconditions	- The finance staff is logged in - Service usage data is available
Description of the Main Sequence	Step 1: The finance staff selects a service record Step 2: The system retrieves usage details Step 3: The member confirms billing items Step 4: The system calculates total charges Step 5: The system generates the invoice Step 6: The system stores the invoice Step 7: The system confirms successful generation
Description of the Alternative Sequence	Step 1: If data is missing, the system displays an error Step 2: If invalid input is provided, correction is required Step 3: If system error occurs, invoice generation fails
Non functional requirements	- The system must ensure billing accuracy - The system must process invoices efficiently - All invoices must be securely stored
Postconditions	- Invoice is generated successfully - Billing data is saved
UC Name	UC_52 Monitor Financial Transactions
Summary	This use case describes how financial activities within the airport are monitored and reviewed.
Dependency	UC_50 Process Airport Service Payment
Actors	Primary Actor: Finance Staff /System Administrator
Preconditions	- The user is logged into the system - Financial data exists
Description of the Main Sequence	Step 1: The user accesses the financial monitoring module Step 2: The system displays transaction records Step 3: The user filters transactions Step 4: The system updates and displays results Step 5: The user reviews financial data
Description of the Alternative Sequence	Step 1: If no data is available, the system notifies the user Step 2: If system error occurs, data cannot be retrieved
Non functional requirements	- The system must provide accurate data - The system must respond quickly - Access must be secure
Postconditions	Financial data is successfully reviewed

	No system data is changed
--	---

<i>UC Name</i>	<i>UC_53 Manage Airport Staff Records</i>
<i>Summary</i>	This use case describes how HR manages records of airport employees such as security staff, ground crew, and administrative staff.
<i>Dependency</i>	None
<i>Actors</i>	Primary Actor: Human Resource Staff
<i>Preconditions</i>	- Human resource staff is logged in - System database is available
<i>Description of the Main Sequence</i>	Step 1: HR accesses staff management module Step 2: System displays employee records Step 3: HR selects create, update, or delete Step 4: HR enters or edits employee details Step 5: System validates the data Step 6: System saves changes Step 7: System confirms operation
<i>Description of the Alternative Sequence</i>	Step 1: Invalid data triggers error message Step 2: Non-existent record triggers notification Step 3: System failure cancels operation
<i>Non functional requirements</i>	- Employee data must be secure - Role-based access must be enforced - All changes must be logged
<i>Postconditions</i>	- Employee records are updated - Changes are stored in the system

<i>UC Name</i>	<i>UC_54 Manage Staff Attendance</i>
<i>Summary</i>	This use case describes how airport staff attendance is recorded and monitored.
<i>Dependency</i>	UC_53 Manage Airport Staff Records
<i>Actors</i>	Primary Actor: Employee Secondary Actor: Human Resource Staff
<i>Preconditions</i>	- Employee exists in the system - Attendance system is active

<i>Description of the Main Sequence</i>	Step 1: Employee checks in/out Step 2: System records attendance time Step 3: HR accesses attendance records Step 4: System displays attendance logs Step 5: HR reviews attendance
<i>Description of the Alternative Sequence</i>	Step 1: If check-in fails, system shows error Step 2: If attendance data is missing, system notifies HR
<i>Non functional requirements</i>	- System must ensure accurate time tracking - Data must be secure - System must be reliable
<i>Postconditions</i>	- Attendance is recorded - Data is stored for future use

UC Name	UC_55 Process Payroll
<i>Summary</i>	This use case describes how salaries for airport staff are calculated and processed.
<i>Dependency</i>	UC_54 Manage Staff Attendance
<i>Actors</i>	Primary Actor: Human Resource Staff / Finance Staff
<i>Preconditions</i>	- Employee and attendance data are available - Payroll system is accessible
<i>Description of the Main Sequence</i>	Step 1: HR selects payroll period Step 2: System retrieves staff data Step 3: System calculates salaries (including overtime/deductions) Step 4: HR reviews payroll details Step 5: System processes salary payments Step 6: System records payroll information Step 7: System confirms completion
<i>Description of the Alternative Sequence</i>	Step 1: Missing data triggers error Step 2: Calculation error stops process Step 3: System failure cancels payroll
<i>Non functional requirements</i>	- Payroll must be accurate - Data must remain confidential

	• Processing must be efficient •
<i>Postconditions</i>	• Salaries are successfully processed • Payroll records are stored

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ani Velo

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

*This document presents a structured set of **Security Management Use Cases** for the Airport Management System. It includes:*

- Passenger Security Screening
- Restricted Area Access Control
- Restricted Area Access Control
- Surveillance Monitoring
- Emergency Lockdown Procedure

Each use case is defined with:

- Actors
- Preconditions
- Main and Alternative Flows
- Non-functional Requirements
 - Postconditions

The purpose of this document is to clearly describe security management related system operations in a structured and professional manner.

UC Name	UC-SEC-01: Passenger Security Screening UC-SEC-02: Restricted Area Access UC-SEC-06: Log Prohibited Items.
Summary	Manages the secure entry of all individuals into the terminal and restricted zones by verifying identities and documenting seized contraband
Dependency	UC-AMS-01 (Flight Manifest Management): Required to validate passenger boarding passes against current flight data. UC-HR-02 (Staff Records Administration): Required to verify staff credentials and role-based access permissions.
Actors	Primary Actor: Security Officer Secondary Actor: Passenger
Preconditions	❖ Passenger or Staff is present at a security checkpoint with valid physical or digital credentials.
Description of the Main Sequence	Step 1: Hardware sensors capture credential data from the user's pass or ID badge. Step 2: Internal logic validates the data against the Access Control List (ACL). Step 3: A signal is transmitted to the electronic gate to unlock while the Officer's monitor updates to "Proceed". Step 4: The system prompts the Security Officer to record physical search results. Step 5: Upon a "Threat Found" flag, the Seizure Module generates a unique Incident ID and digital receipt. Step 6: The system sets the user's global status to "Cleared" and writes the event to the secure log.

<i>Description of the Alternative Sequence</i>	Invalid Credentials: If data validation fails, the system maintains the hardware lock and logs an "Unauthorized Access" warning. Secondary Search: If a threat is confirmed, the system triggers a mandatory manual search workflow for supervisor review.
<i>Non-functional requirements</i>	➤ Authentication and scanning must conclude in < 1 second. ➤ Contraband logs must be immutable for legal evidence.
<i>Postconditions</i>	➤ User is granted or denied access. ➤ All contraband found is legally documented in the central database.

<i>UC Name</i>	UC-SEC-03: Report Security Incident UC-SEC-04: Monitor Surveillance Feeds UC-SEC-06: Emergency Lockdown
<i>Summary</i>	Provides 24/7 visual oversight and a rapid response framework for logging threats and executing facility-wide lockdowns.
<i>Dependency</i>	UC-SYS-08 (Hardware Node Management): Required for the system to communicate with the Video Management Subsystem (VMS) and camera nodes. UC-SYS-09 (Electronic Door Control): Required for the system to transmit "Lock" signals to physical gate and door hardware.

Actors	Primary Actor: Airport Staff Secondary Actor: System Administrator
Preconditions	❖ Security Personnel are logged into the surveillance dashboard with active camera feeds.
Description of the Main Sequence	Step 1: Video data is continuously streamed from camera nodes to the operator's HD dashboard. Step 2: User-generated "Bookmarks" are received and linked to specific video timestamps for 30-day archiving. Step 3: A new database entry is initialized upon request, assigning a unique tracking number and timestamp. Step 4: A 2-Factor Authentication (2FA) challenge is issued to the Supervisor to authorize emergency state changes. Step 5: A "Global Lock" signal is broadcast to all terminal door controllers and runway gates. Step 6: The PA subsystem is triggered to play automated emergency instructions across all zones.
Description of the Alternative Sequence	Hardware Error: If a camera signal is lost, the system automatically opens a technical maintenance ticket. Authorization Failure: If 2FA verification fails, the system denies the lockdown command and alerts the Lead Administrator.
Non-functional requirements	➤ Lockdown commands must reach all hardware units in < 2 seconds. ➤ 2-Factor Authentication is mandatory for all emergency state changes.
Postconditions	➤ Incident is logged and prioritized ➤ Facility state is transitioned to "Lockdown" if a critical threat is confirmed

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ajla Alcani

Course:

Software Modeling and Design

Date:

20 March 2026

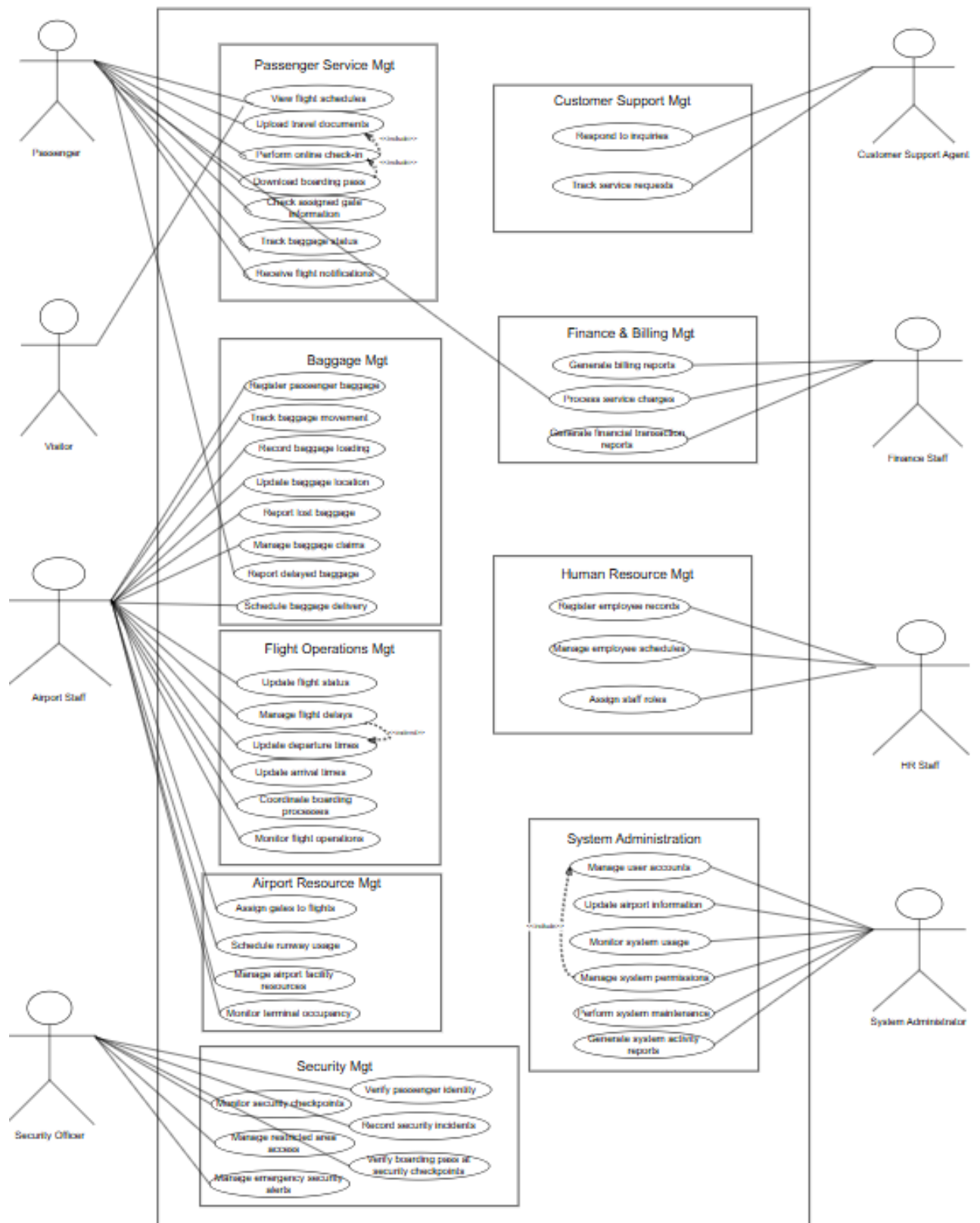
Document Overview

*This document presents the consolidated **Global Use Case Diagram** for the Airport Management System. It includes:*

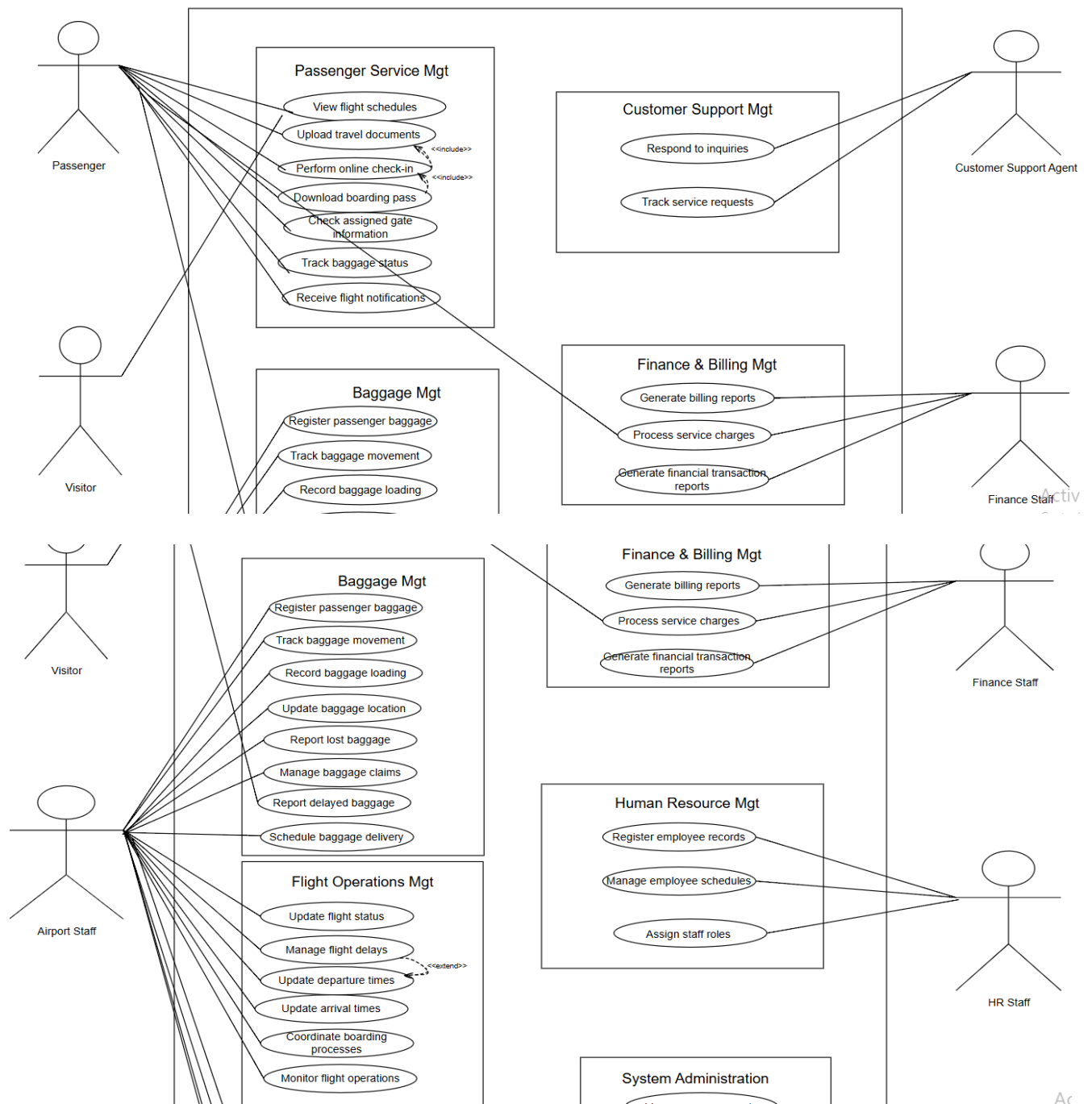
- ✓ Integration of 45 Functional Requirements*
- ✓ Mapping of 8 Primary and Secondary Actors*
- ✓ Cross-Module Logical Dependencies (Includes and Extends)*
- ✓ The diagram is categorized into 9 core system packages:*
 - ✓ Passenger Service Management*
 - ✓ Baggage Management System*
 - ✓ Flight Operations Management*
 - ✓ Security Management System*
 - ✓ System Administration*
 - ✓ Airport Resource Management*
 - ✓ Customer Support Management*
 - ✓ Finance and Billing Management*
 - ✓ Human Resource Management*

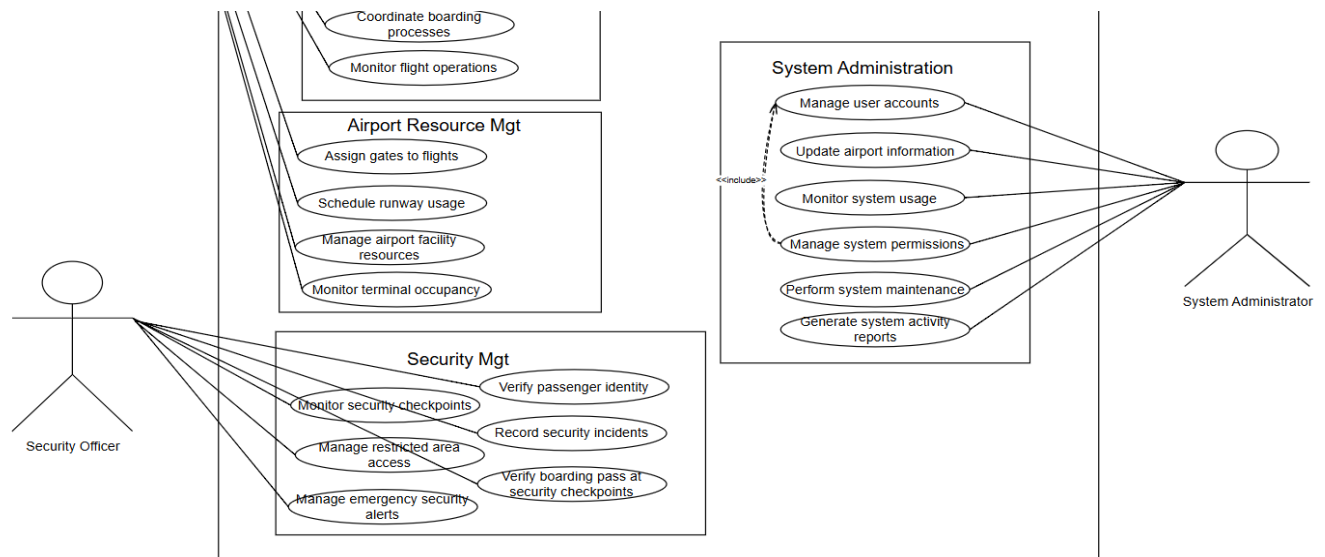
The purpose of this document is to clearly model and describe the complete system architecture and actor interactions in a professional and structured way.

Airport Management System



Airport Management System





Global Use Case Diagram Description

The Global Use Case Diagram serves as the architectural visual blueprint for the Airport Management System (AMS). It maps out the complete scope of the software's functional requirements, illustrating exactly how different users interact with the system's various services.

To effectively manage the complexity of the system's 45 functional requirements, the diagram is structured using several key UML modeling principles:

- Modular Organization (Packages):** Rather than presenting a disorganized web of use cases, the system is divided into nine logical subsystems, represented as UML Packages (e.g., Passenger Service Management, Baggage Management, System Administration). This modularity mirrors professional software architecture and ensures the diagram remains readable and highly organized.
- Role-Based Interactions (Actors):** The diagram identifies eight distinct actors who interact with the system. Solid association lines clearly establish boundaries regarding which actors can trigger which processes. Primary actors (such as Passengers and Airport Staff) drive the core airport operations, while secondary actors (such as HR Staff and System Administrators) manage the back-office and infrastructure capabilities.
- System Logic and Dependencies:** To accurately model the real-world workflows of the airport, the diagram utilizes specific UML dependencies to show how use cases interact with one another:
 - <<include>> Relationships:** These are used to demonstrate mandatory prerequisite actions. For example, the *Perform online check-in* use case explicitly includes *Upload travel documents*, as the system requires the documents before the check-in process can be completed.

- **<<extend>> Relationships:** These indicate optional, conditional, or alternative flows within the system. For instance, *Manage flight delays* is modeled as an extension of *Update flight status*, as a delay is an alternative scenario that extends the base functionality of a standard status update.

Conclusion: Ultimately, this Global Use Case Diagram bridges the gap between the written requirements and the upcoming system design phase. It provides developers and stakeholders with a clear, comprehensive, and logically sound overview of the entire Airport Management System's intended behavior.

Airport Management System

Passenger Service Management Use Cases Documentation

Prepared by:

Betül Kaan

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of System Administration models for the Airport Management System. It includes:

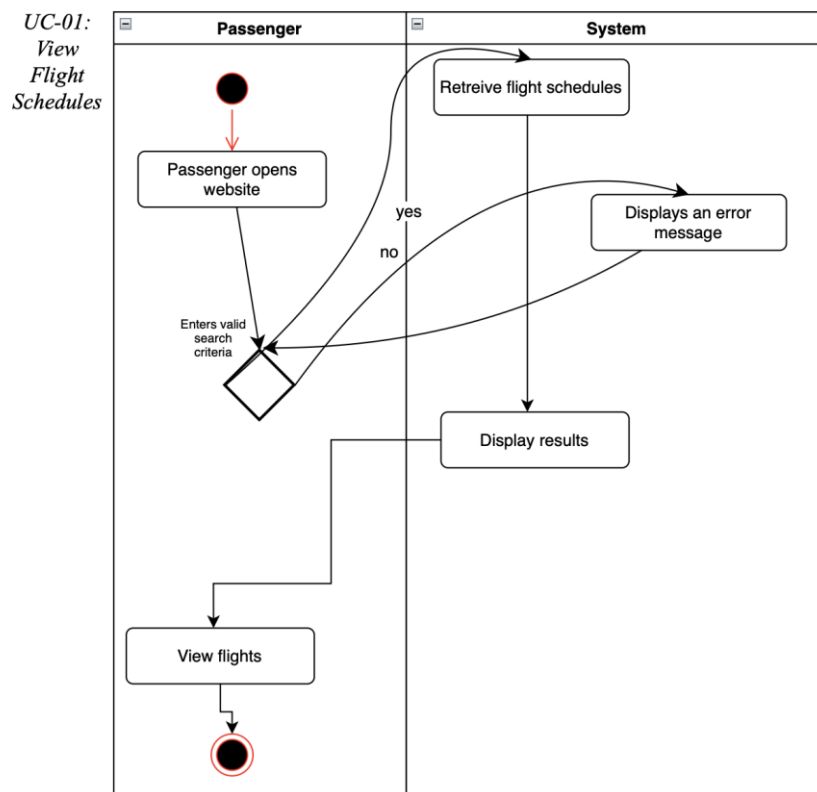
- ✓ *Viewing Flight Schedules (UC-01)*
- ✓ *Uploading Flight Documents (UC-02)*
- ✓ *Online Check-In (UC-03)*
- ✓ *Boarding Pass Download (UC-04)*
- ✓ *Checking Gate Information (UC-05)*
- ✓ *Tracking Baggage Status (UC-06)*
- ✓ *Receiving Flight Notification (UC-07)*

Each use case is defined with:

- 1. Mapped Functional Requirements: Linking diagrams to specific UC IDs.**
- 2. Mapped Alternative Flows: Modeling system behavior during errors or threats.**
- 3. Non-functional Requirements: Defining performance and security constraints.**
- 4. Swimlane Actor Interactions: Visualizing the flow between Passengers, Staff, and the System.**

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

Activity Diagrams:



Core Purpose:

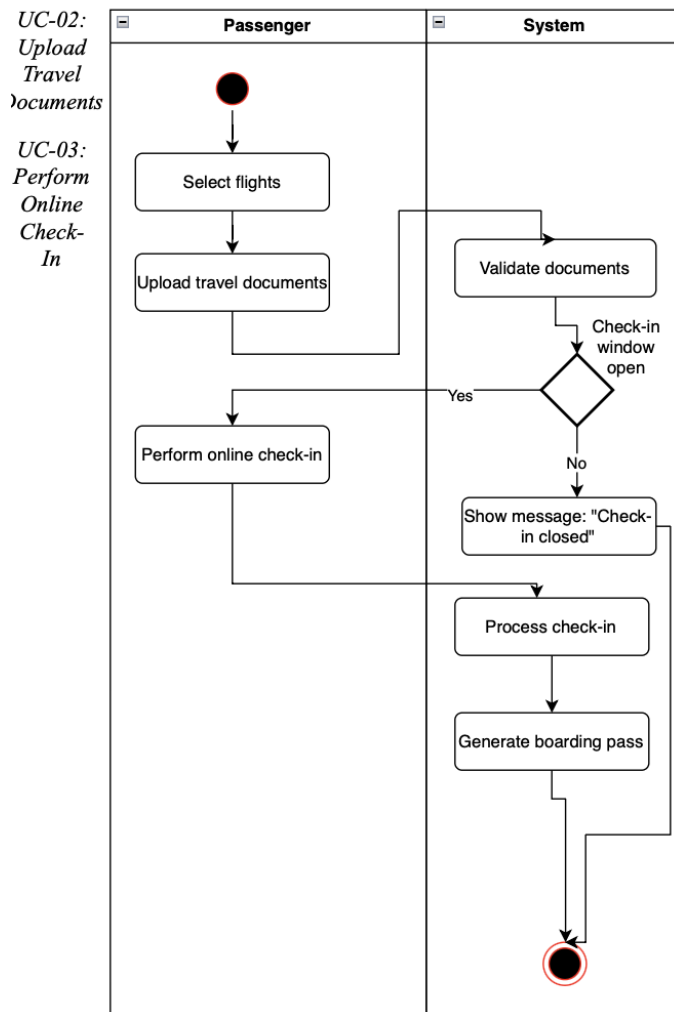
This use case describes how passengers search for and view available flight schedules based on their travel criteria.

Functional Requirements Mapped:

- *Passenger enters search criteria (destination, date, etc.).*
- *System retrieves available flight schedules from the database.*
- *System displays matching flights to the passenger.*
- *Passenger browses and reviews flight options.*

Alternative Flow Mapped:

If no flights match the search criteria, the system displays a “no results found” message and prompts the passenger to modify their search.



Core Purpose:

This use case describes how passengers complete the online check-in process and obtain a boarding pass.

Functional Requirements Mapped:

- Passenger selects a booked flight.
- Passenger uploads required travel documents.
- System validates the submitted documents.
- Passenger initiates online check-in.
- System processes check-in and verifies eligibility.
- System checks whether the check-in window is open.
- System generates a boarding pass upon successful check-in.

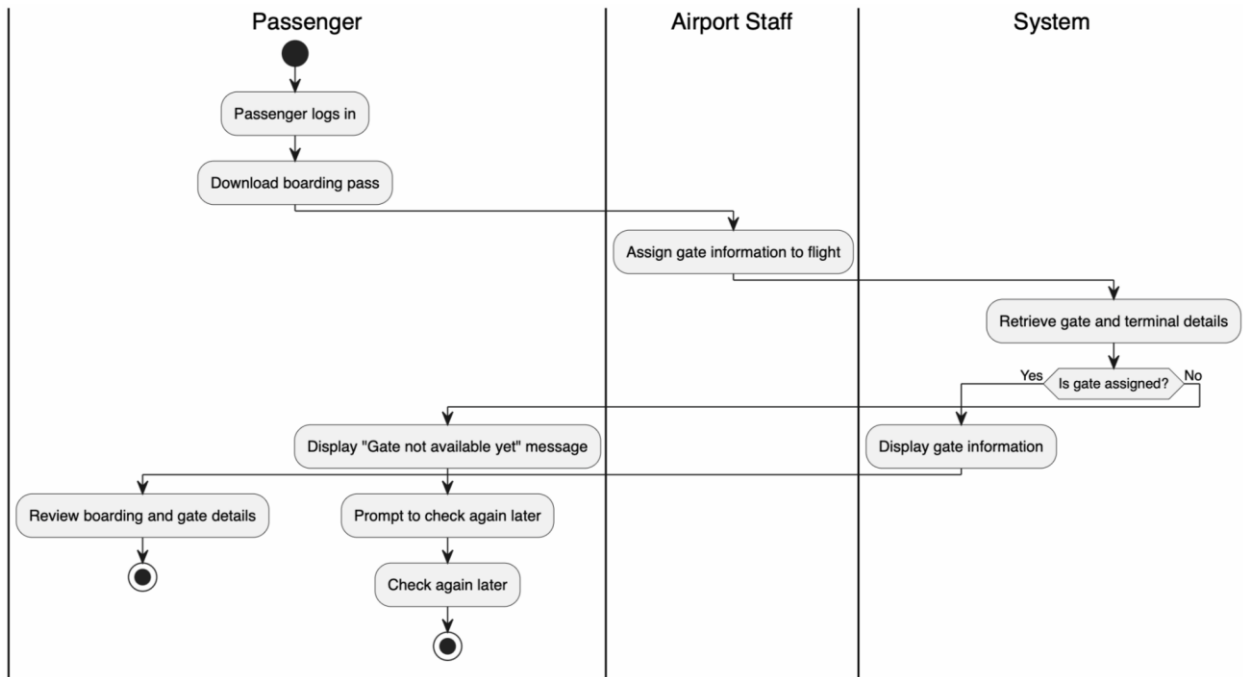
Alternative Flow Mapped:

- If documents are invalid, the system prompts the passenger to re-upload correct documents.

- If the check-in window is closed, the system notifies the passenger that online check-in is unavailable.

UC-04 Download Boarding Pass

UC-05 Check Assigned Gate Information



Core Purpose:

This use case describes how passengers access their boarding pass and retrieve gate and terminal information.

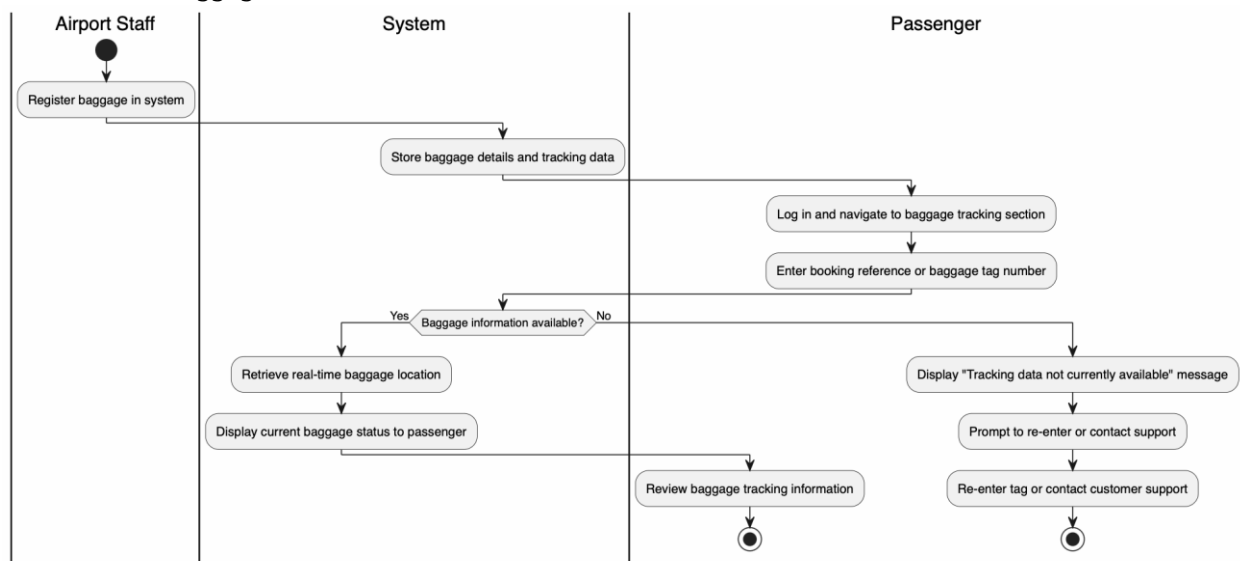
Functional Requirements Mapped:

- Passenger logs into the system.
- Passenger downloads the boarding pass.
- Airport staff assigns gate information to the flight.
- System retrieves gate and terminal details.
- System displays updated gate information to the passenger.
- Passenger reviews boarding and gate details.

Alternative Flow Mapped:

If gate information is not yet assigned, the system displays a “Gate not available yet” message and prompts the passenger to check again later.

UC-06 Track Baggage Status



Core Purpose:

This use case describes how passengers track the status and location of their checked baggage.

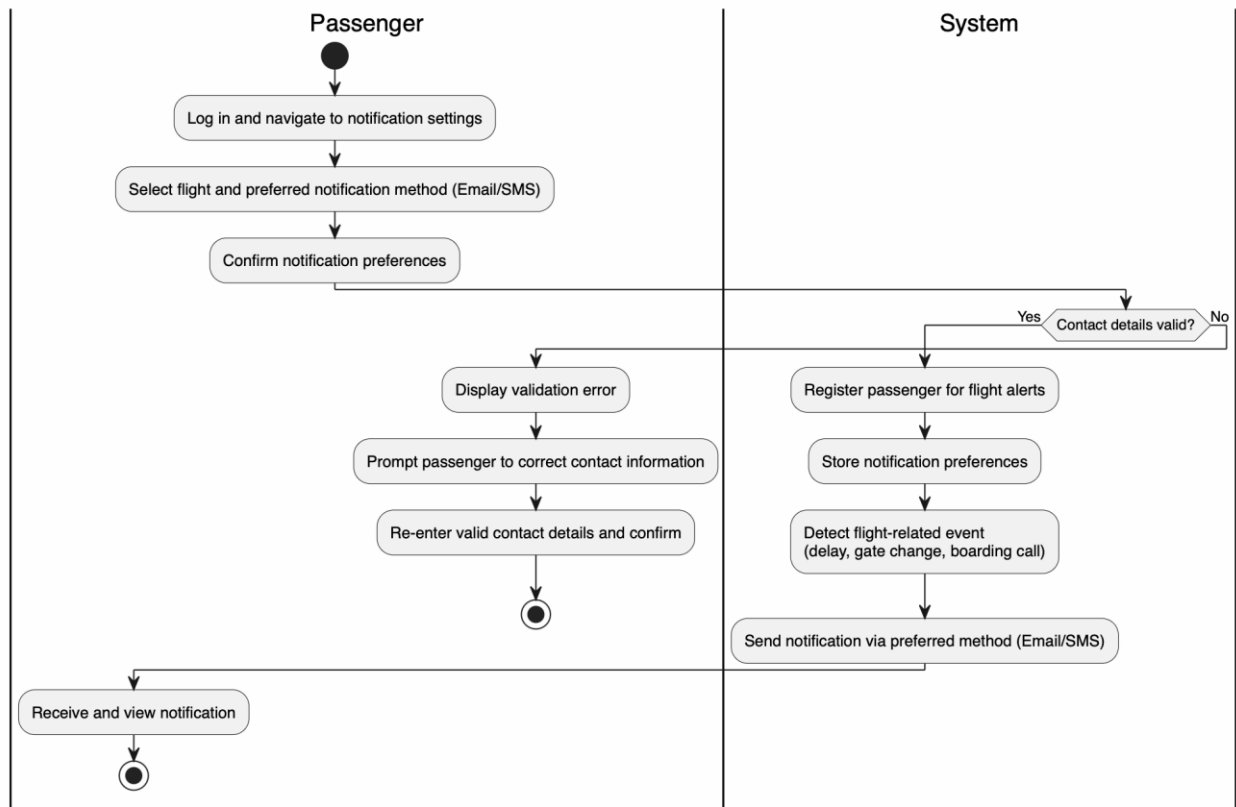
Functional Requirements Mapped:

- Airport staff registers baggage in the system.
- System stores baggage details and tracking data.
- Passenger requests baggage status.
- System retrieves real-time baggage location.
- System displays current baggage status to the passenger.

Alternative Flow Mapped:

If baggage information is unavailable or not yet registered, the system notifies the passenger that tracking data is not currently available.

UC-07 Receive Flight Notifications



Core Purpose:

This use case describes how passengers receive real-time notifications regarding their flight.

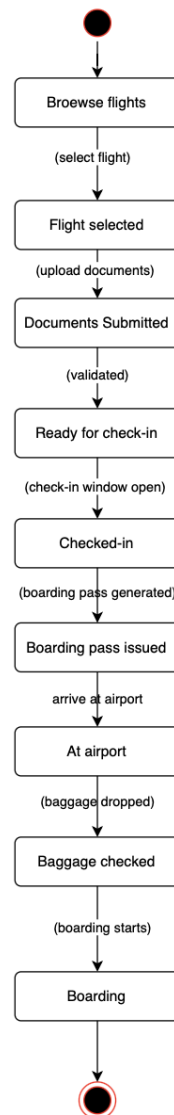
Functional Requirements Mapped:

- Passenger sets notification preferences (e.g., SMS, email).
- System stores passenger preferences.
- System detects flight-related events (delays, gate changes, etc.).
- System sends notifications based on stored preferences.
- Passenger receives and views notifications.

Alternative Flow Mapped:

If notification delivery fails (e.g., invalid contact details), the system prompts the passenger to update their contact information.

State Diagram:



Core Purpose:

This state diagram describes the lifecycle of a passenger's flight journey, from browsing flights to completing the trip, showing how the passenger transitions between different states in the system.

Functional Requirements Mapped:

- *Passenger can browse and select flights.*
- *System transitions passenger from Browsing Flights to Flight Selected upon selection.*
- *Passenger submits required travel documents.*
- *System validates documents and updates state accordingly.*
- *Passenger becomes eligible for check-in once documents are approved.*
- *System verifies check-in window availability.*
- *Passenger completes check-in and receives a boarding pass.*
- *Passenger proceeds to the airport and may check baggage.*

- *Passenger transitions to boarding when boarding begins.*
- *System marks journey as completed after flight departure.*

State Transitions Mapped:

- *Browsing Flights → Flight Selected (flight chosen)*
- *Flight Selected → Documents Submitted (documents uploaded)*
- *Documents Submitted → Ready for Check-in (documents valid)*
- *Ready for Check-in → Checked-in (check-in successful)*
- *Checked-in → Boarding Pass Issued (boarding pass generated)*
- *Boarding Pass Issued → At Airport (passenger arrives)*
- *At Airport → Baggage Checked (baggage registered)*
- *Baggage Checked → Boarding (boarding starts)*
- *Boarding → Flight Completed (journey ends)*

Alternative Flow Mapped:

- *If documents are invalid, the passenger transitions to Documents Rejected and must resubmit valid documents.*
- *If the check-in window is closed, the passenger transitions to Check-in Failed and cannot proceed with online check-in.*
- *If baggage is not checked, the passenger may transition directly from At Airport to Boarding.*

Airport Management System

Prepared by:

Albina Blloshmi

Course:

Software Modeling and Design

Date:

27 March 2026

Baggage Management System – Behavioral Models

✓ *Baggage Check-In & Screening Activity Diagram (UC-08)*

✓ *Baggage Tracking Activity Diagram (UC-09)*

✓ *Baggage Lifecycle State Diagram (UC-08 → UC-13)*

✓ *Each model is defined with:*

- 1. Mapped Functional Requirements*
- 2. Mapped Alternative Flows*
- 3. Non-functional Requirements*
- 4. Actor Interactions*

The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

Core Purpose:

To manage baggage throughout its lifecycle from check-in to final delivery, ensuring accurate tracking, handling of delays and lost baggage, and efficient delivery to passengers.

This use case describes how passengers claim their baggage upon arrival at the destination airport.

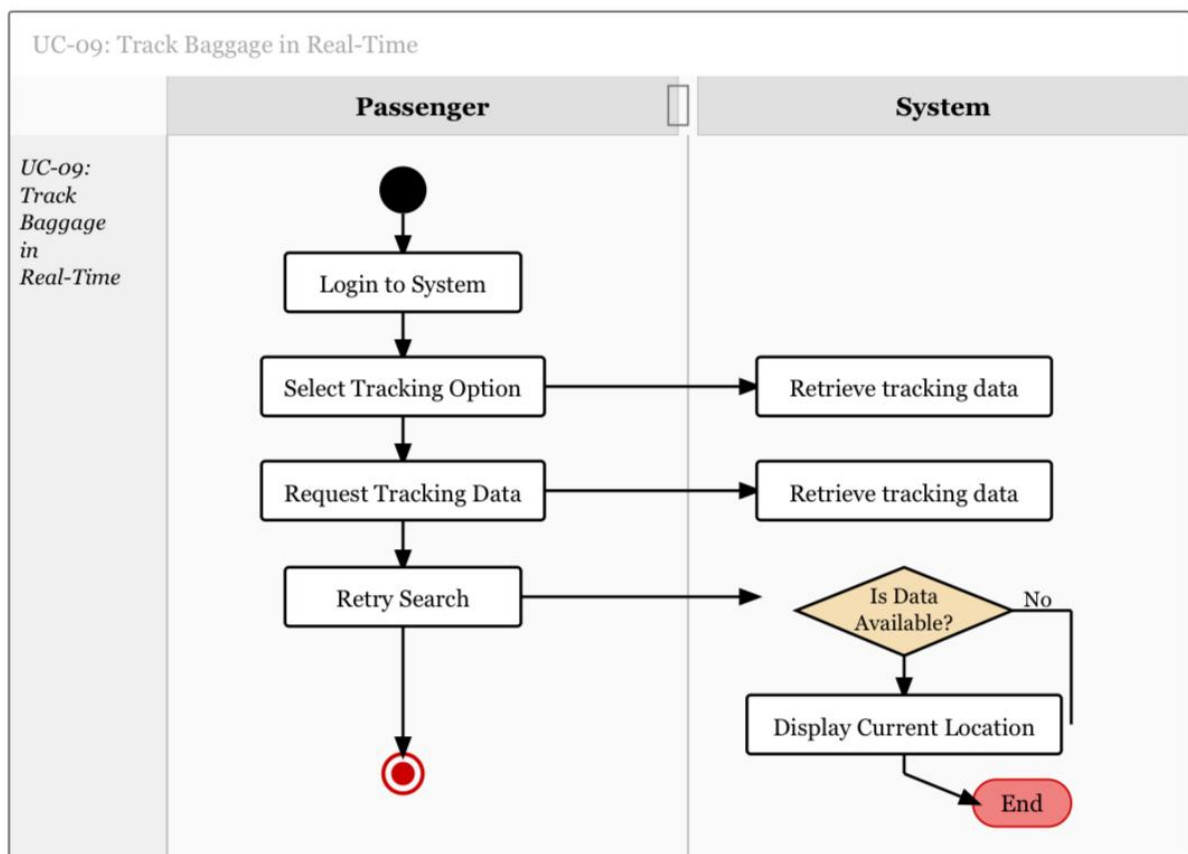
Functional Requirements Mapped:

- Passenger goes to the baggage claim area to pick up their baggage.
- System displays carousel info to the passenger.
- Passenger collects their baggage or reports missing baggage if not found.

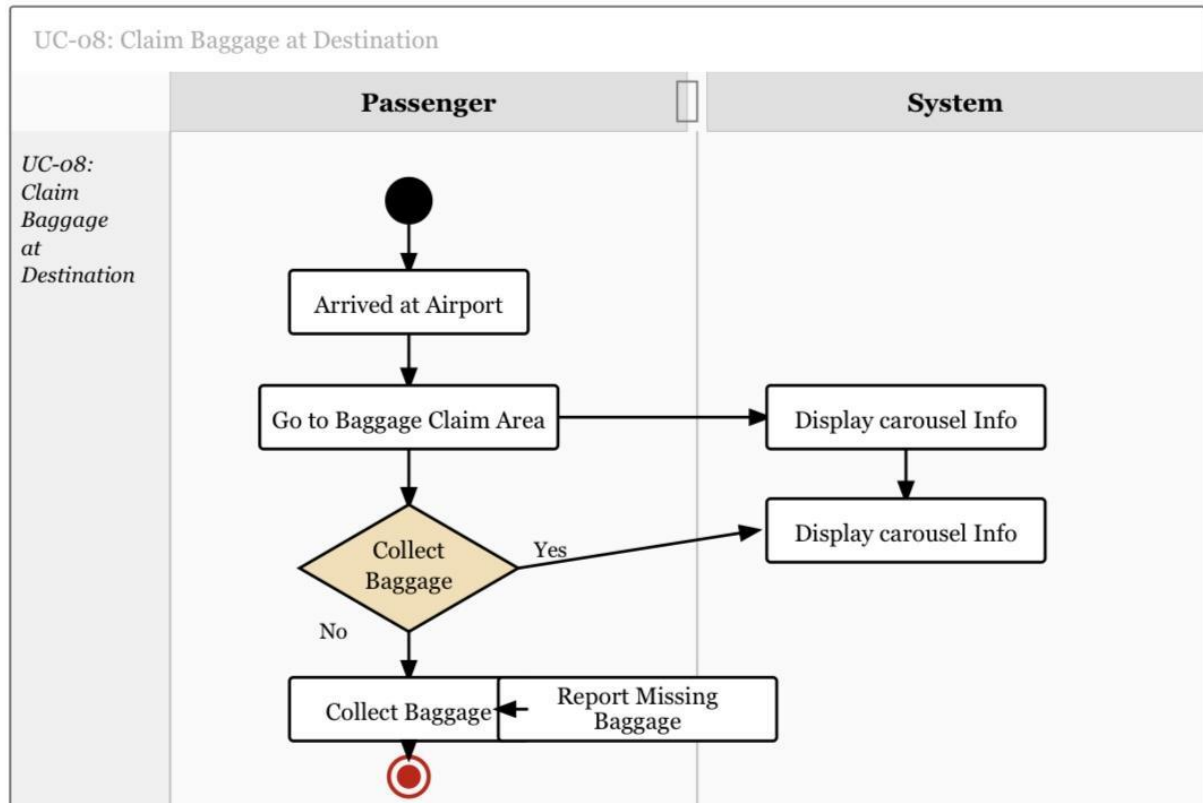
Alternative Flow Mapped:

If the passenger's baggage is not found, the system allows the passenger to report missing baggage to airport staff.

Activity Diagram:



Activity Diagram:



Core Purpose:

Core Purpose:

This use case describes how passengers track the real-time location of their checked baggage during the journey.

Functional Requirements Mapped:

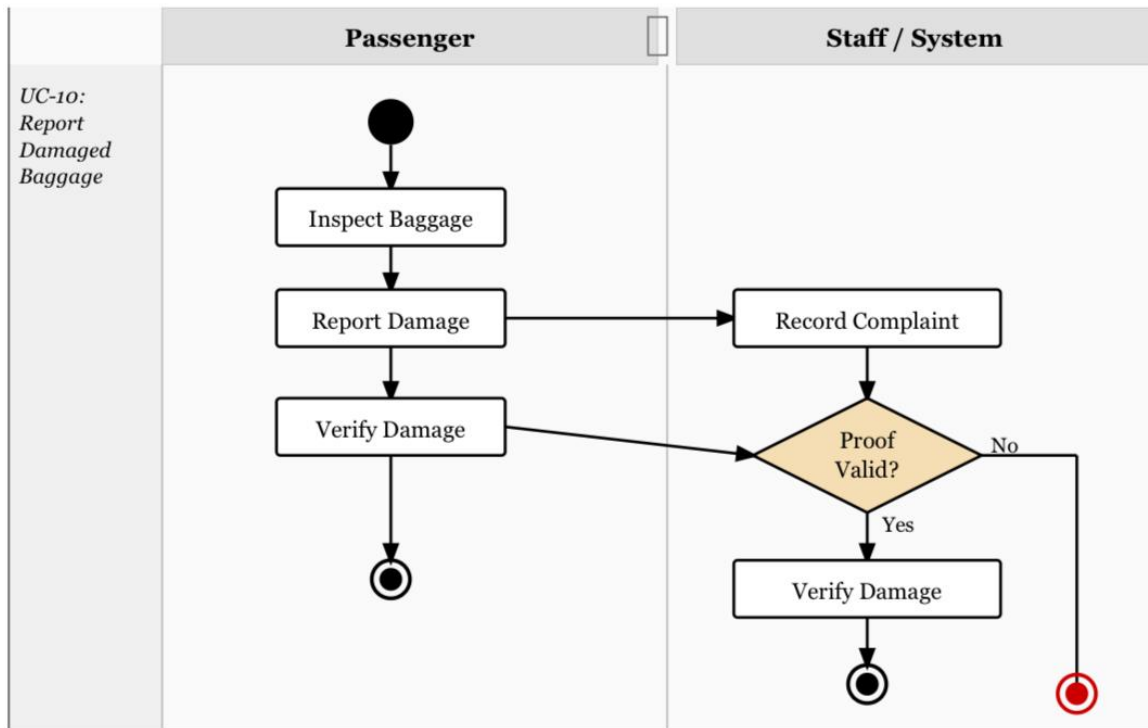
- *Passenger logs into the tracking system.*
- *Passenger selects the tracking option.*
- *System retrieves real-time tracking data and displays the current location of the baggage.*

Alternative Flow Mapped:

If tracking data is not available, the system shows the last known location and prompts the passenger to retry the search.

Activity Diagram:





Core Purpose:

This use case describes how passengers report damage to their baggage upon arrival.

Functional Requirements Mapped:

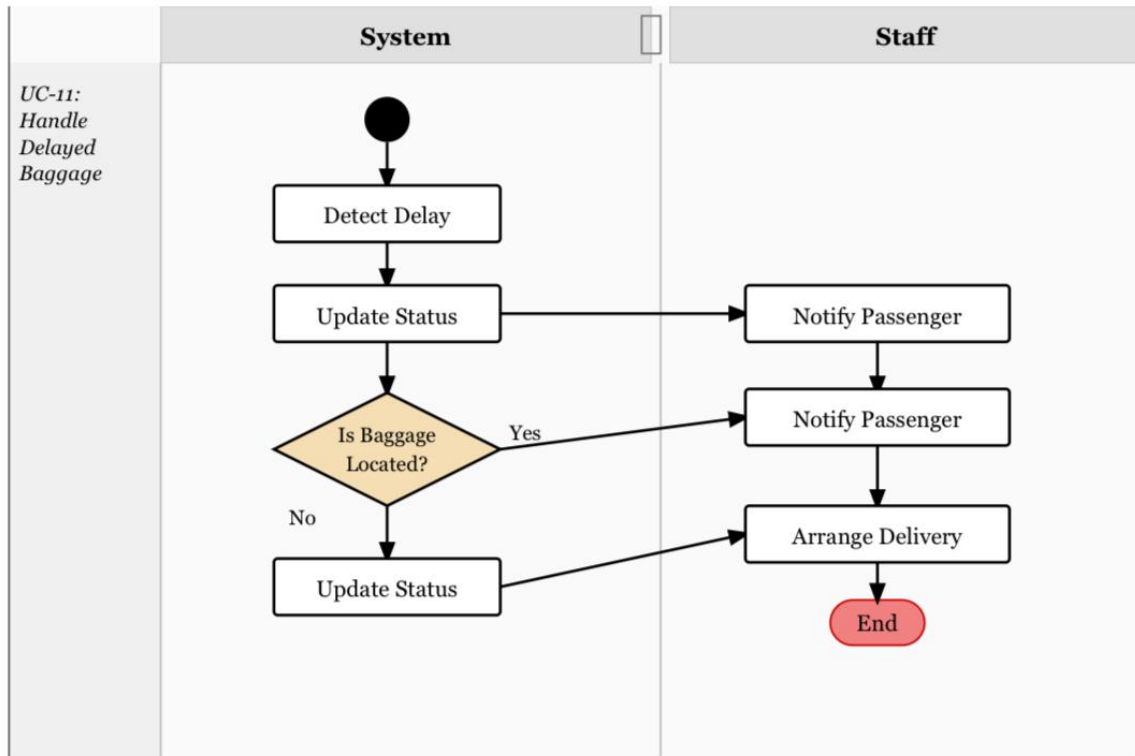
- Passenger inspects their baggage for damage.
- Passenger reports baggage damage via system or counter desk.
- System records the complaint, verifies proof, and processes the damage claim.

Alternative Flow Mapped:

If the proof of damage is invalid, the system rejects the claim and notifies the passenger.

Activity Diagram:

UC-11: Handle Delayed Baggage



Core Purpose:

This use case describes how the system handles cases when a passenger's baggage is delayed.

Functional Requirements Mapped:

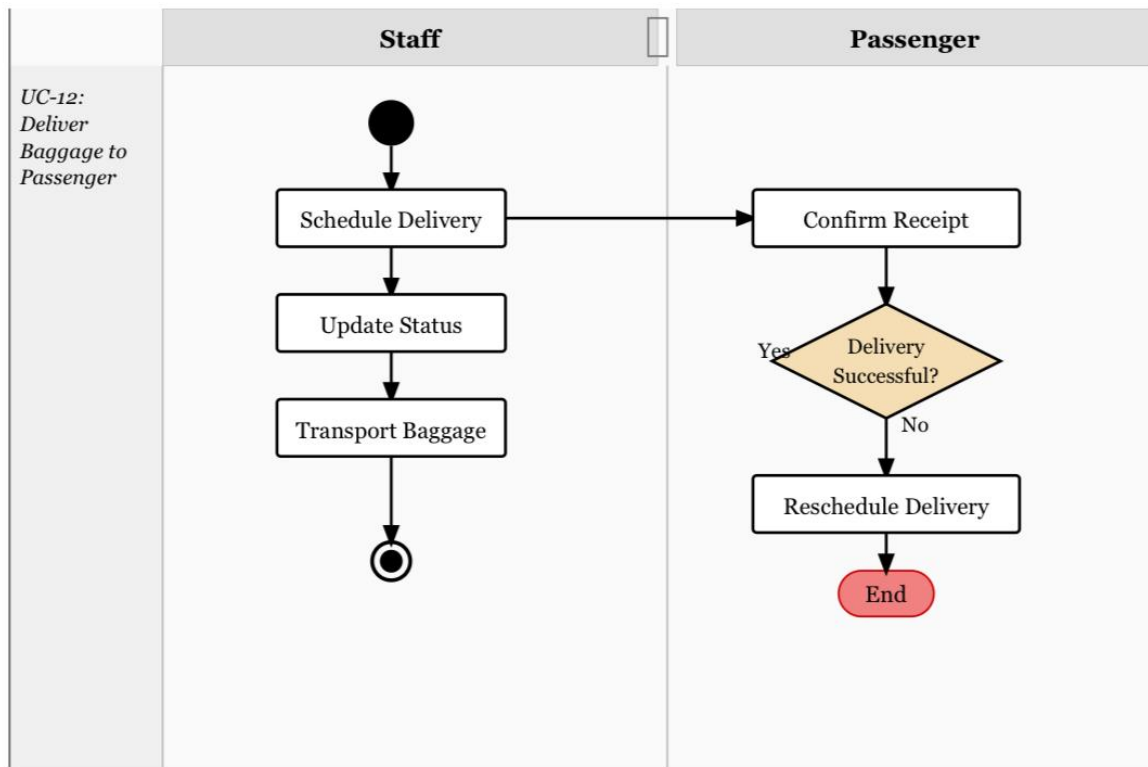
- *System detects when baggage is delayed.*
- *System updates the baggage status to indicate delay.*
- *Passenger is notified and staff arrange for delivery.*

Alternative Flow Mapped:

If baggage cannot be located, the system marks it as lost and initiates the lost baggage procedure.

Activity Diagram:

UC-12: Deliver Baggage to Passenger



Core Purpose:

This use case describes how staff delivers baggage to passengers after a delay or rerouting.

Functional Requirements Mapped:

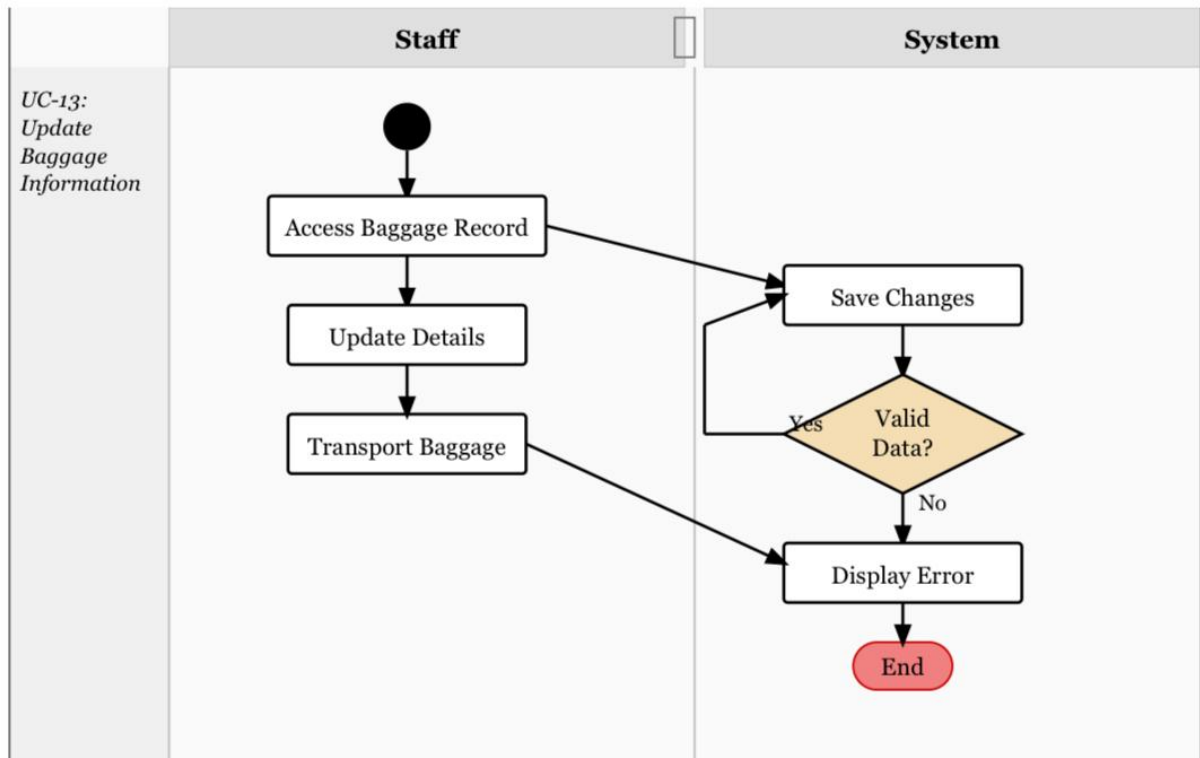
- Staff schedules the delivery of located baggage.
- System updates the delivery status.
- Baggage is transported to the passenger's location.
- Passenger confirms receipt of the baggage.

Alternative Flow Mapped:

If the delivery fails, staff reschedules the delivery and attempts it again.

Activity Diagram:

UC-13: Update Baggage Information



Core Purpose:

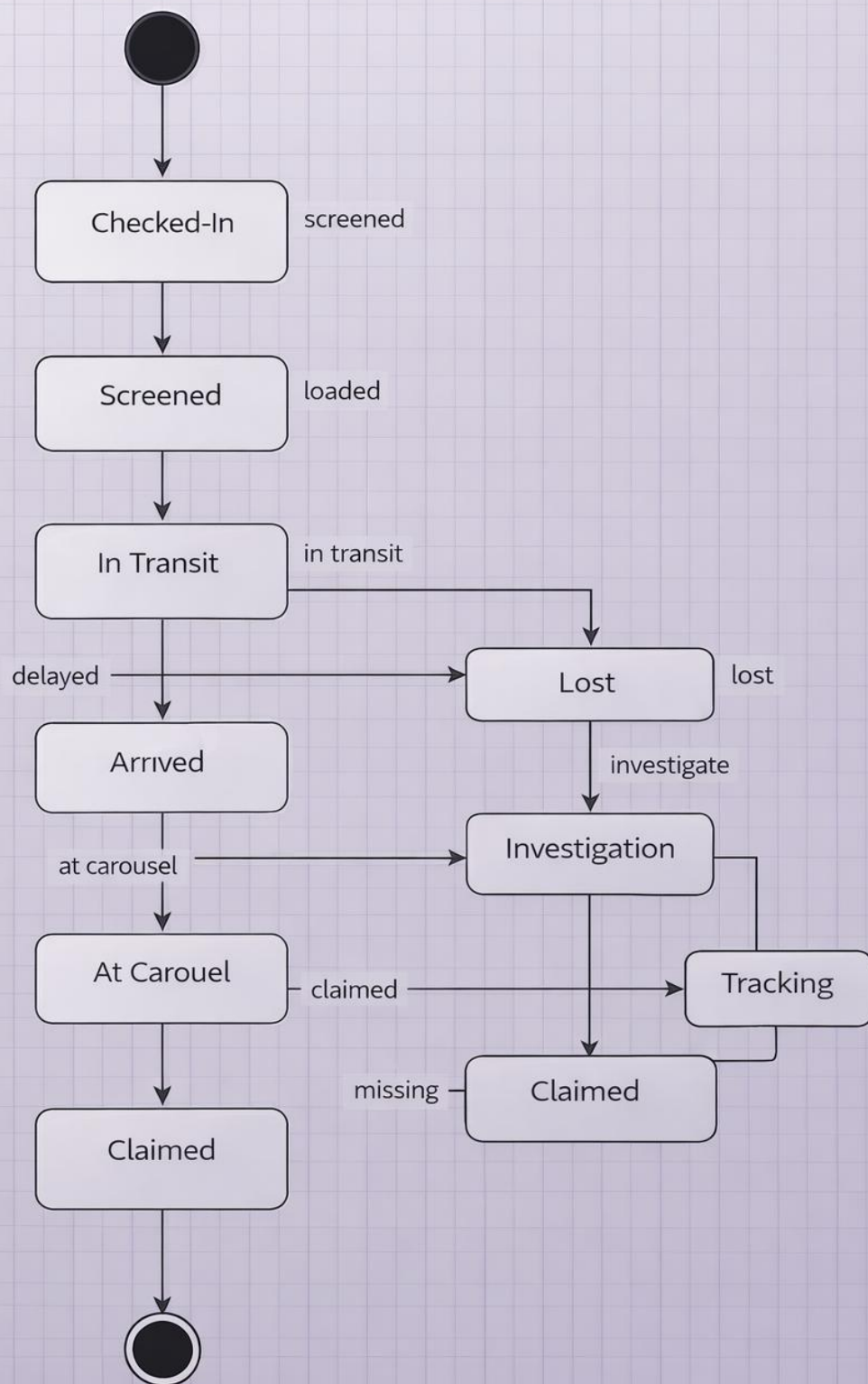
This use case describes how staff updates baggage details within the system.

Functional Requirements Mapped:

- Staff accesses existing baggage record through the system.
- Staff updates the baggage details.
- System saves the changes and reflects the updated information.

Alternative Flow Mapped:

If invalid data is entered, the system displays an error message and halts the update process.



Baggage State Diagram for UC-08 to UC-13

Functional Requirements Mapped:

- ***UC-08 (Baggage Check-In & Screening):***
Register baggage, attach identification, and route it for security screening.
- ***UC-09 (Baggage Tracking):***
Monitor baggage status in real time throughout the journey.
- ***UC-10 (Lost Baggage Handling):***
Report, investigate, and recover lost baggage.
- ***UC-11 (Delayed Baggage Handling):***
Detect delays, notify passengers, and manage delayed baggage.
- ***UC-12 (Baggage Delivery Service):***
Arrange delivery of baggage when not claimed at the airport.
- ***UC-13 (Baggage Status Update):***
Continuously update baggage information at each stage.

Alternative Flow Mapped:

- ***Delayed Baggage (UC-11):***
If baggage is delayed, it is marked as Delayed and tracked until resolved.
- ***Lost Baggage (UC-10):***
If baggage cannot be located, it moves to Investigation and may later be recovered.
- ***Missing at Carousel (UC-12):***
If baggage is not claimed, it is marked as Missing and tracked.
- ***Status Updates (UC-13):***
All status changes are automatically updated in the system.

Airport Management System
Flight Operations Management Use Cases Documentation

Prepared by:

Alvi Elezi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

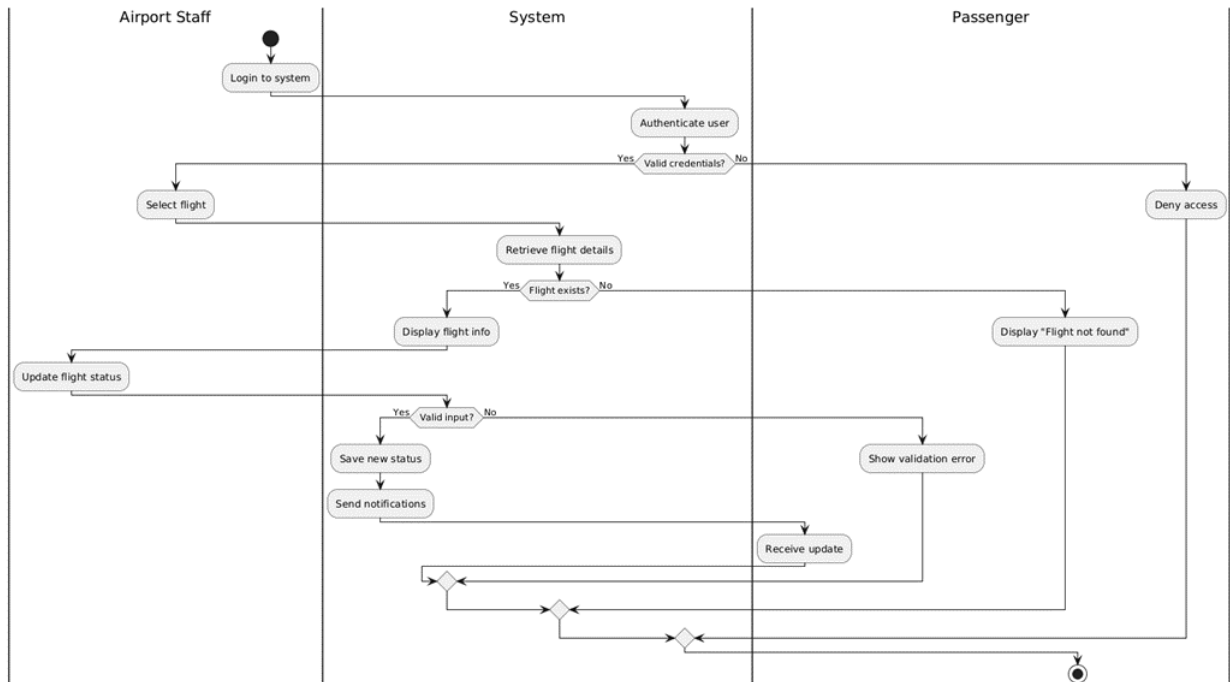
This document presents a set of Flight Operations Management Activity Diagrams for the Airport Management System.

It includes:

- ✓ Flight status updates***
- ✓ Flight delay management***
- ✓ Departure time updates***
- ✓ Arrival time updates***
- ✓ Coordination between boarding processes***
- ✓ Flight operations monitoring***
- ✓ Each use case is defined with:***
 - ✓ Actors***
 - ✓ Preconditions***
 - ✓ Main and alternative flows***
 - ✓ Non-functional requirements***
 - ✓ Postconditions***

The goal of this document is to model and describe the management of flight operations by airport staff and the behavior of the system in a structured way.

Update flight status



Core Purpose: This activity diagram describes how airport staff update the operational status of a flight and notify relevant stakeholders.

Functional Requirements Mapped:

Airport staff updates flight status.

System retrieves and displays flight data.

System validates updates and propagates changes across modules.

System sends notifications to passengers.

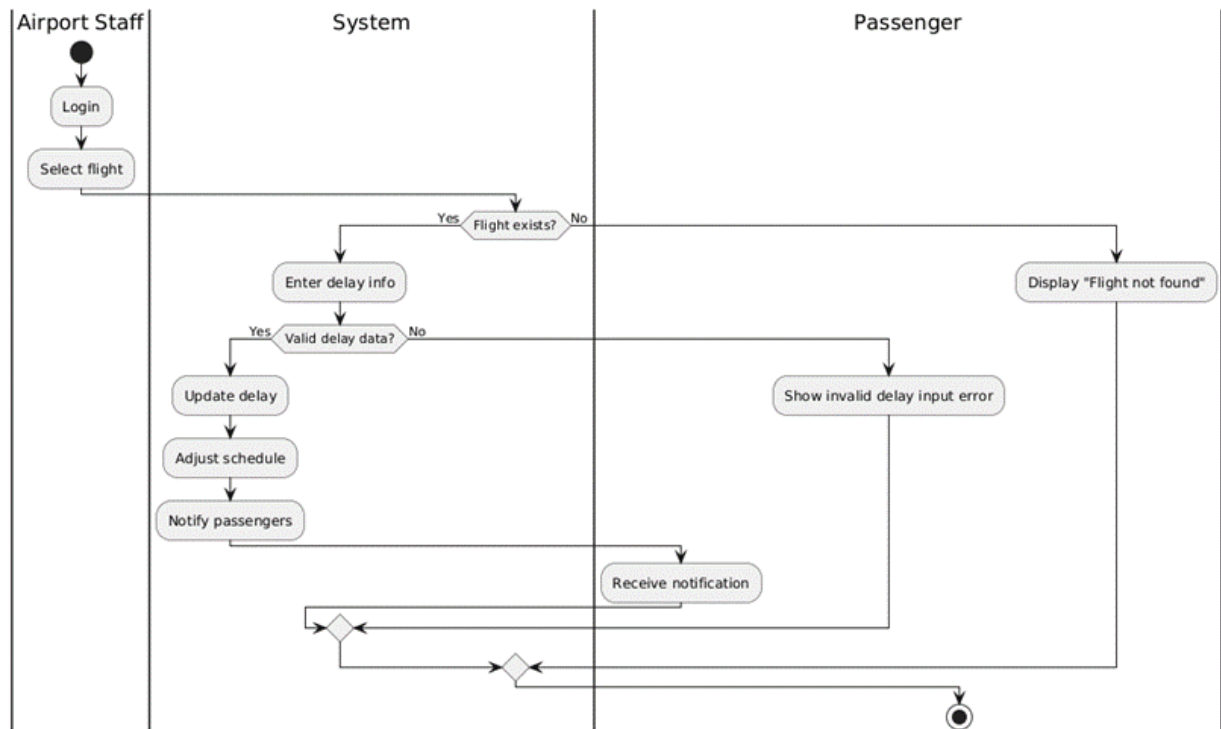
Alternative Flow Mapped:

If login credentials are invalid, access is denied.

If the selected flight does not exist, the system displays an error.

If invalid status data is entered, the system prompts the staff to correct the input.

Manage flight delays



Core Purpose: This activity diagram describes how airport staff manage and record flight delays while ensuring passengers are informed.

Functional Requirements Mapped:

Airport staff manages flight delays.

System updates flight schedule based on delay.

System records delay reason.

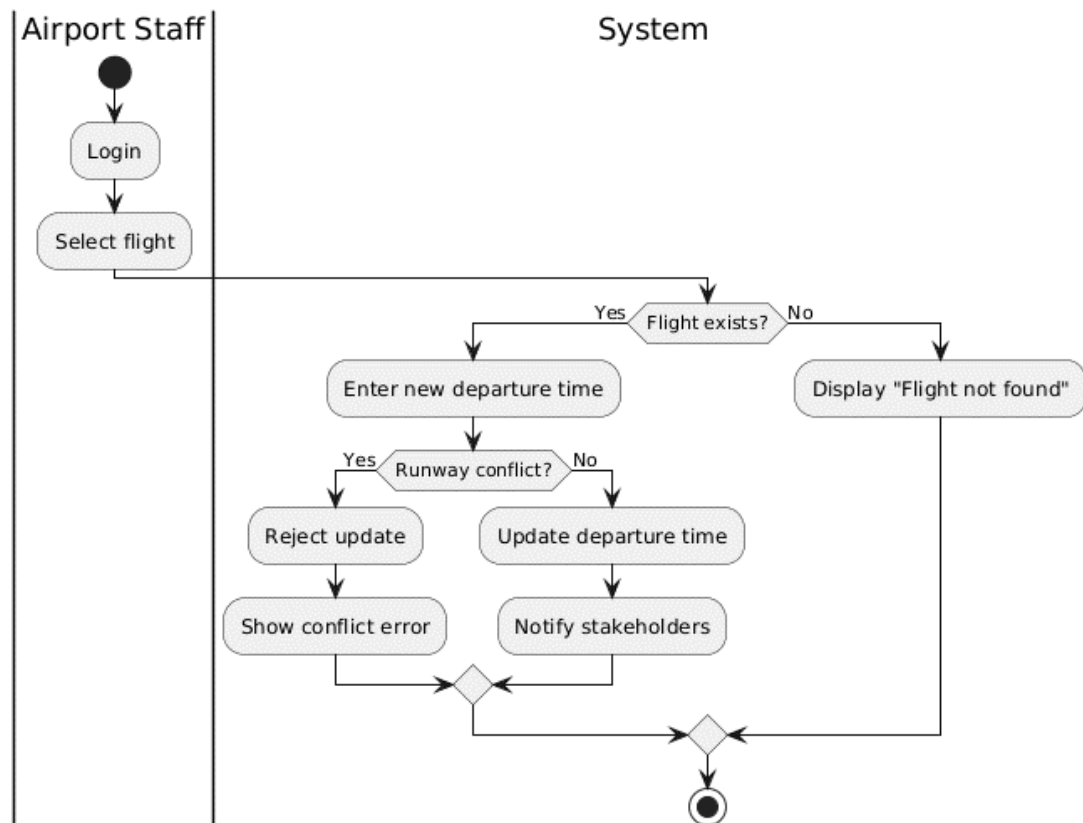
System notifies passengers of delays.

Alternative Flow Mapped:

If the flight does not exist, the system returns an error.

If invalid delay data is entered, the system prompts correction.

Update departure time



Core Purpose: This activity diagram describes how airport staff update the scheduled departure time of a flight.

Functional Requirements Mapped:

Airport staff updates departure times.

System validates scheduling constraints.

System updates flight schedule.

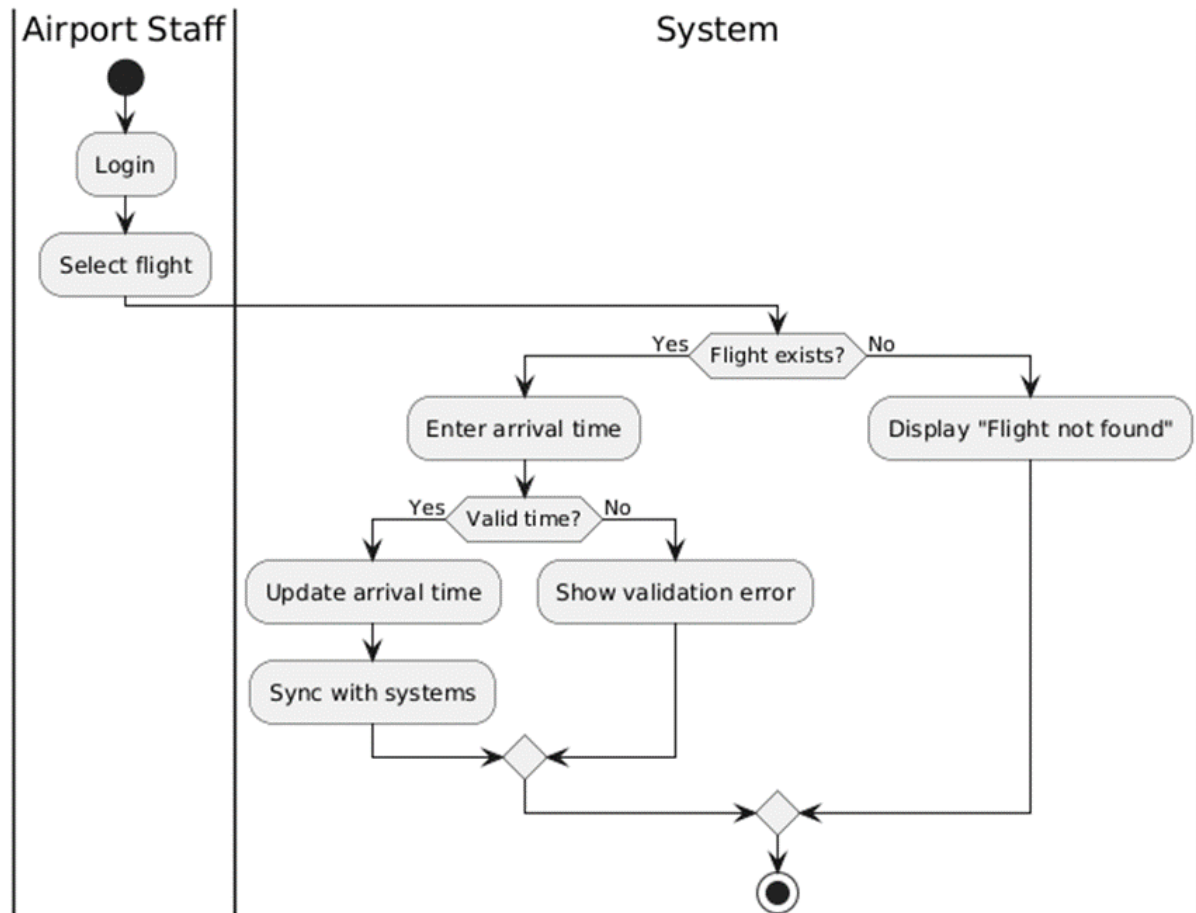
System notifies relevant stakeholders.

Alternative Flow Mapped:

If the flight does not exist, the system displays an error.

If the new time conflicts with runway scheduling, the system rejects the update and prompts correction.

Update arrival time



Core Purpose: This use case describes how airport staff update the arrival time of a flight.

Functional Requirements Mapped:

Airport staff updates arrival times.

System validates arrival data.

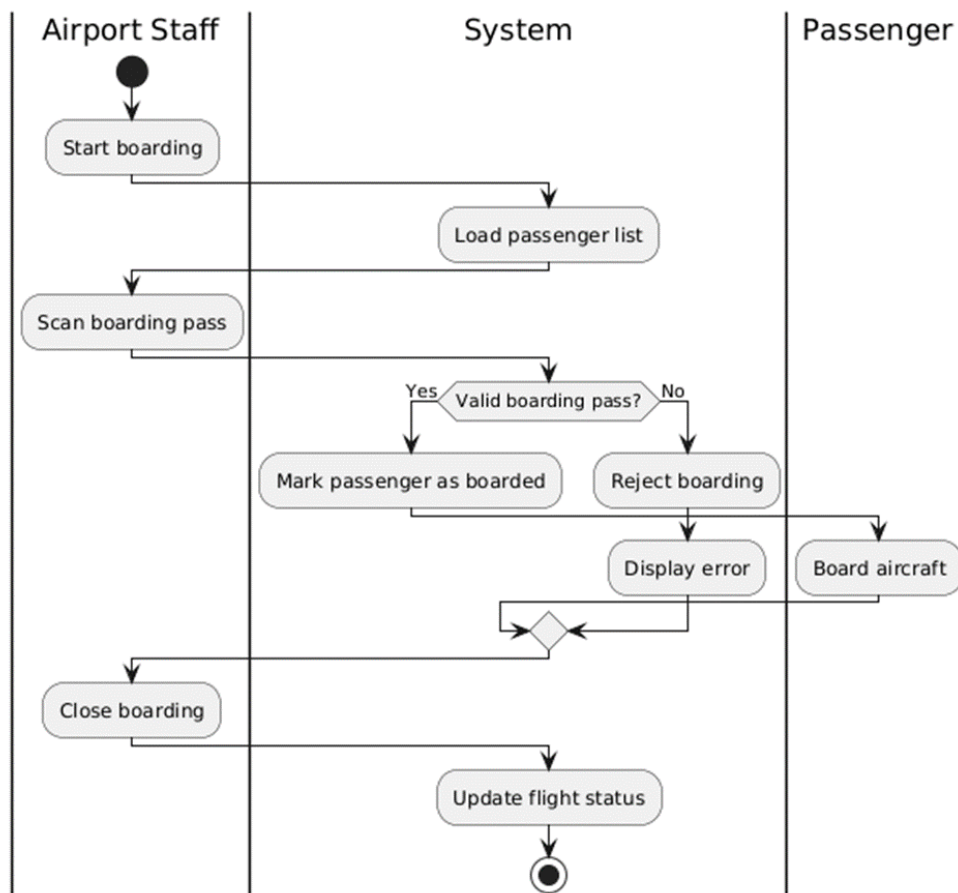
System synchronizes updates with operational displays.

Alternative Flow Mapped:

If the flight does not exist, the system displays an error.

If invalid time data is entered, the system requests correction.

Coordinate Boarding Process



Core Purpose: This use case describes how airport staff manage and control the passenger boarding process.

Functional Requirements Mapped:

Airport staff coordinates boarding processes.

System validates boarding passes.

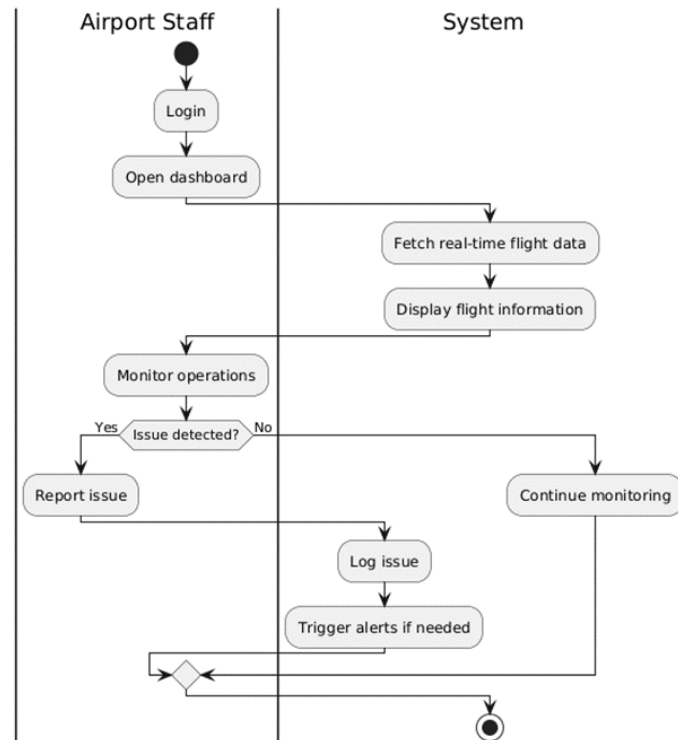
System updates passenger boarding status.

System updates flight status after boarding completion.

Alternative Flow Mapped:

If a boarding pass is invalid, the system denies boarding and alerts staff.

Monitor Flight Operations



Core Purpose: This use case describes how airport staff monitor real-time flight operations and respond to operational issues.

Functional Requirements Mapped:

Airport staff monitors flight operations.

System provides real-time operational data.

System logs operational issues.

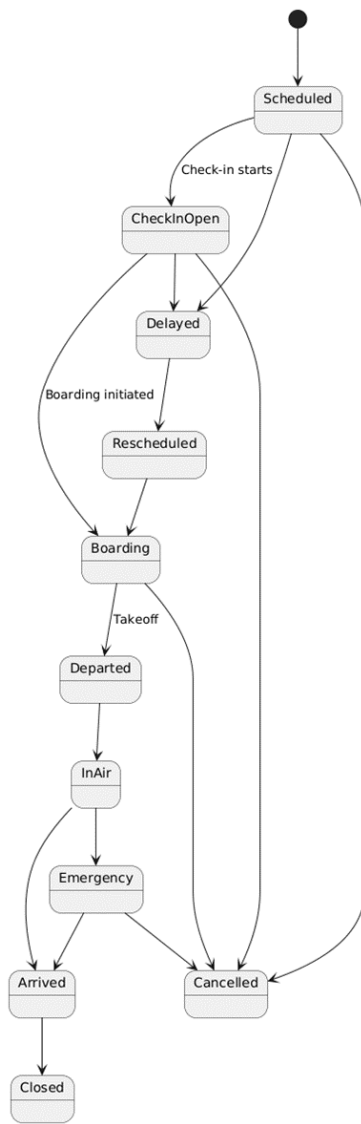
System triggers alerts when necessary.

Alternative Flow Mapped:

If system data retrieval fails, the system displays last known data with a warning.

If no issues are detected, monitoring continues normally.

State Diagram for the whole module



Airport Management System

Security Management Models Documentation

Prepared by:

Ani Velo

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of behavioral models for the Airport Management System, specifically focusing on the Security Management System. It translates the defined functional requirements into visual workflows to ensure terminal safety and operational integrity. It includes:

- ✓ Passenger & Staff Access Activity Diagram (UC-SEC-01, UC-SEC-02)*
- ✓ Contraband & Seizure Management Activity Diagram (UC-SEC-05)*
- ✓ Surveillance & Incident Reporting Activity Diagram (UC-SEC-03, UC-SEC-04)*
- ✓ Emergency Lockdown Activity Diagram (UC-SEC-06)*
- ✓ Security Incident Lifecycle State Diagram*

Each model is defined with:

- 5. Mapped Functional Requirements: Linking diagrams to specific UC IDs.*
- 6. Mapped Alternative Flows: Modeling system behavior during errors or threats.*
- 7. Non-functional Requirements: Defining performance and security constraints.*
- 8. Swimlane Actor Interactions: Visualizing the flow between Passengers, Staff, and the System.*

The goal of this section is to clearly model and describe the security system's behavioral workflows in a professional and structured way, serving as a foundation for the system's implementation phase.

Terminal Access and Screening Operation

(UC-SEC-01, UC-SEC-02, UC-SEC-05)

This table covers the physical movement of people and items.

Core Purpose: To manage the secure entry of all individuals into the airport terminal and restricted zones, ensuring that all identities are verified and prohibited items are seized and logged.

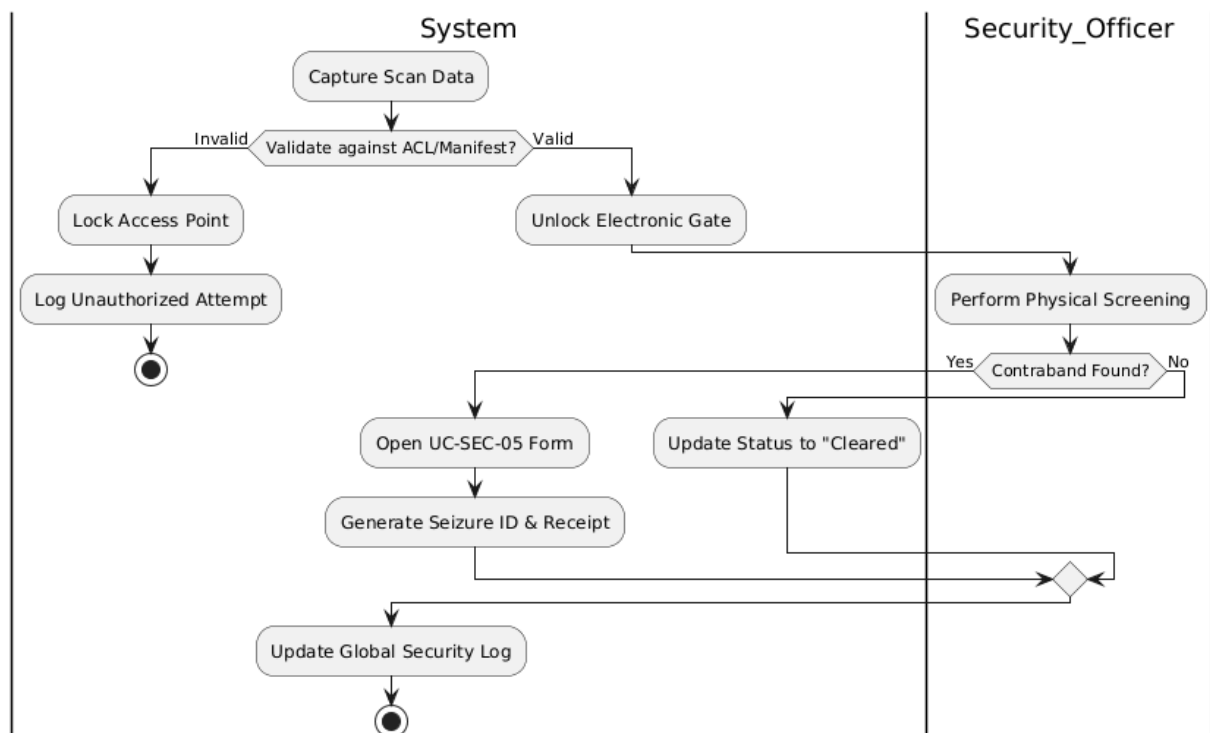
Functional Requirements Mapped:

- *UC-SEC-01:* Passenger Security Screening (Boarding pass & X-ray check).
- *UC-SEC-02:* Restricted Area Access (Staff credential verification).
- *UC-SEC-05:* Log Prohibited Items (Seizure records and receipts).

Alternative Flow Mapped: If credentials are invalid, the system denies entry and logs a warning. If a threat is detected, the system triggers a secondary manual search and logs the event for supervisor review.

Non-functional Requirements:

- *Performance:* Authentication and scanning status must update in < 1 second.
- *Integrity:* Contraband logs must be immutable for legal evidence.



Surveillance, Incident Reporting, and Emergency Response

(UC-SEC-03, UC-SEC-04, UC-SEC-06)

This table covers the digital monitoring and high-level emergency commands.

Core Purpose: To provide 24/7 visual oversight of the airport and establish a rapid response framework for logging incidents and executing facility-wide lockdowns during high-level threats.

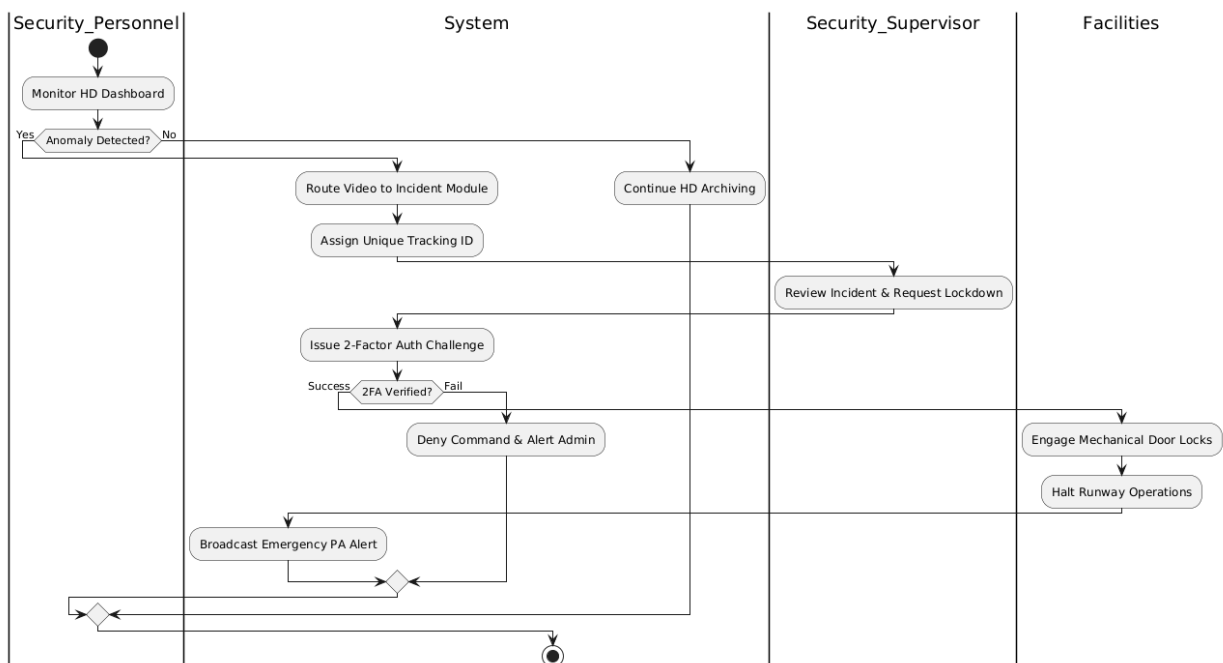
Functional Requirements Mapped:

- *UC-SEC-03:* Report Security Incident (Categorize and prioritize threats).
- *UC-SEC-04:* Monitor Surveillance Feeds (Live HD streaming and 30-day archiving).
- *UC-SEC-06:* Emergency Lockdown (Signal electronic doors and halt flight ops).

Alternative Flow Mapped: If a camera fails, a maintenance ticket is generated. If an incident is "Critical," the system prompts the supervisor to initiate a full lockdown flow.

Non-functional Requirements:

- *Safety:* Lockdown commands must reach hardware in < 2 seconds.
- *Security:* 2-Factor Authentication (2FA) is required for emergency commands



Security Incident Lifecycle
(State Machine)

This table defines the logic for how an incident changes status in the system.

Core Purpose: To track the digital lifecycle and operational status of a security threat from initial discovery through to legal archiving.

State Descriptions:

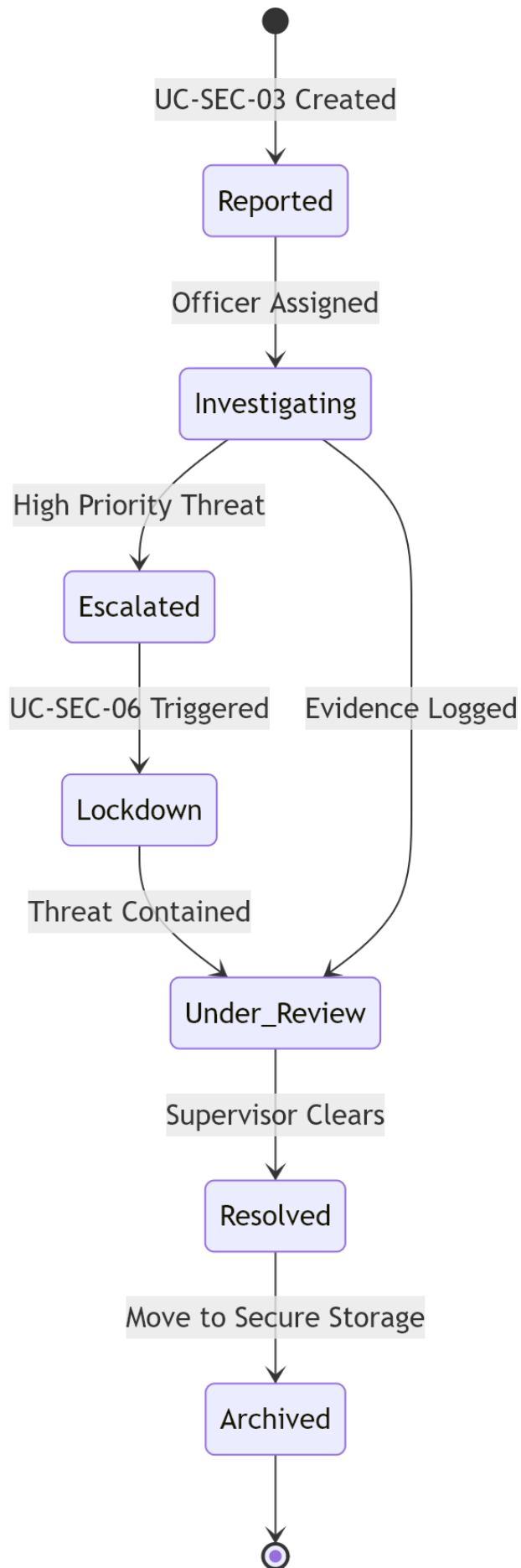
- *Reported*: The initial entry of a threat into the system via UC-SEC-03.
- *Investigating*: An active state where security officers are reviewing surveillance feeds or performing physical checks.
- *Escalated*: A high-alert state triggered when a threat is confirmed as critical.
- *Lockdown*: A restrictive state where UC-SEC-06 is active and facilities are physically secured.
- *Under Review*: The post-incident phase where evidence and logs are verified by a supervisor.
- *Resolved/Archived*: The final state where the threat is closed and data is moved to secure 30-day storage.

Transition Logic & Triggers:

- *Reported* → *Investigating*: Triggered by "Assign Officer" action in the dashboard.
- *Investigating* → *Escalated*: Triggered by "Confirm Threat" button if an anomaly is found.
- *Escalated* → *Lockdown*: Triggered by successful 2-Factor Authentication (2FA) by a supervisor.
- *Under Review* → *Resolved*: Triggered by an administrative digital signature.

Guard Conditions:

- *Authorized* == True: Lockdown cannot be entered without 2FA.
- *All Fields Complete*: An incident cannot move to "Resolved" if the seizure receipt (UC-SEC-05) is missing.



Airport Management System

Airport Resource Management Management Models Documentation

Prepared by:

Ameo Lulaj

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

*This document presents a structured set of behavioral models for the Airport Management System, specifically focusing on **Airport Resource Management and Customer Support Management**. It includes:*

- ✓ Gate Allocation Activity Diagram (UC-ARM-01)*
- ✓ Runway Scheduling Activity Diagram (UC-ARM-02)*
- ✓ Staff Allocation Activity Diagram (UC-ARM-03)*
- ✓ Equipment Management Activity Diagram (UC-ARM-04)*
- ✓ Airport Resource Lifecycle State Diagram (Gate/Asset Lifecycle)*

- ✓ Ticket Management Activity Diagram (UC-CSM-01)*
- ✓ Live Chat Support Activity Diagram (UC-CSM-02)*
- ✓ Support Ticket Lifecycle State Diagram*

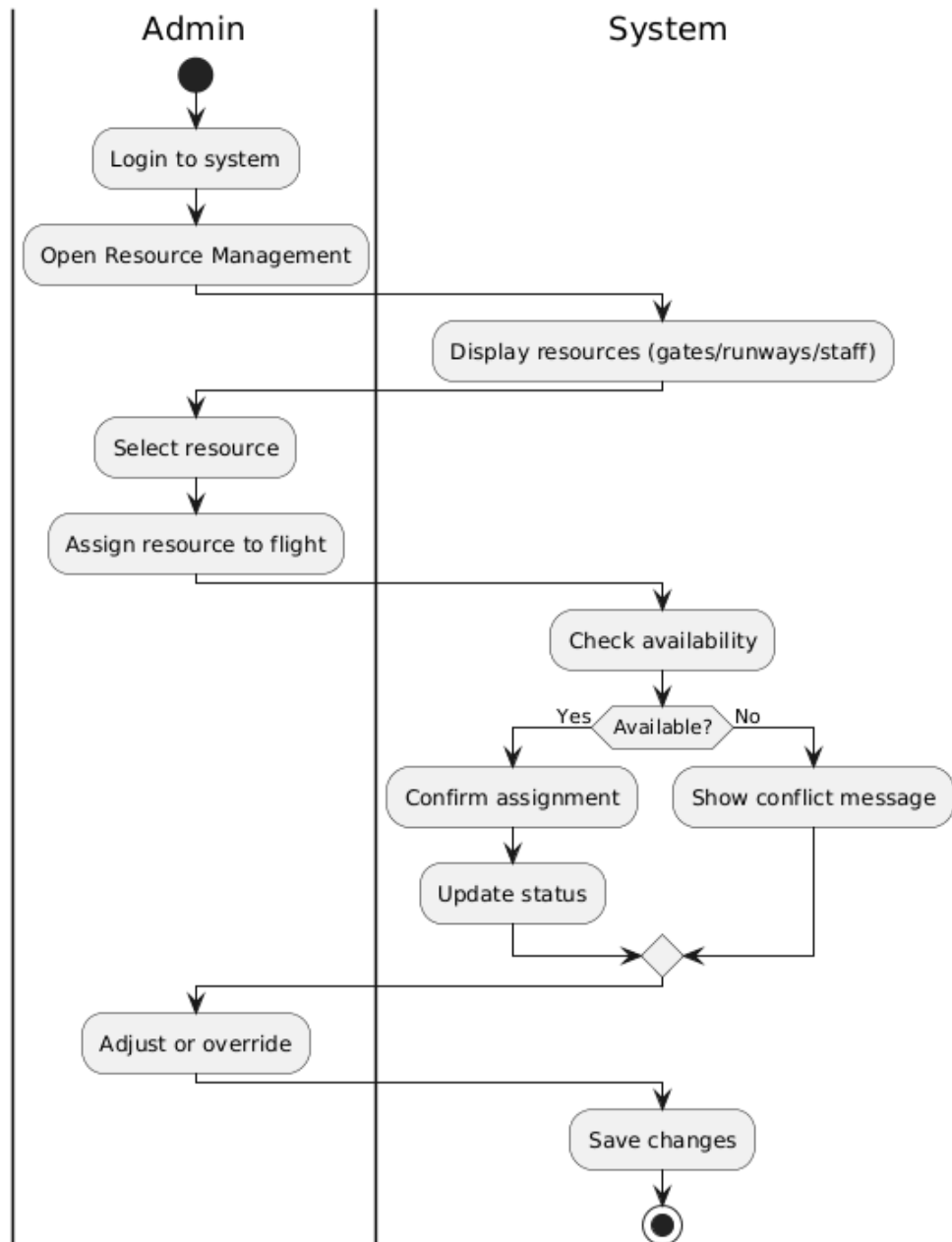
✓ Each model is defined with:

- 1. Mapped Functional Requirements*
- 2. Mapped Alternative Flows*
- 3. Non-functional Requirements*
- 4. Swimlane Actor Interactions*

The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

Core Purpose:

To efficiently manage and allocate airport resources such as gates, runways, staff, and equipment to ensure smooth airport operations, avoid conflicts, and optimize utilization.



ACTIVITY DIAGRAM OF

Functional Requirements Mapped:

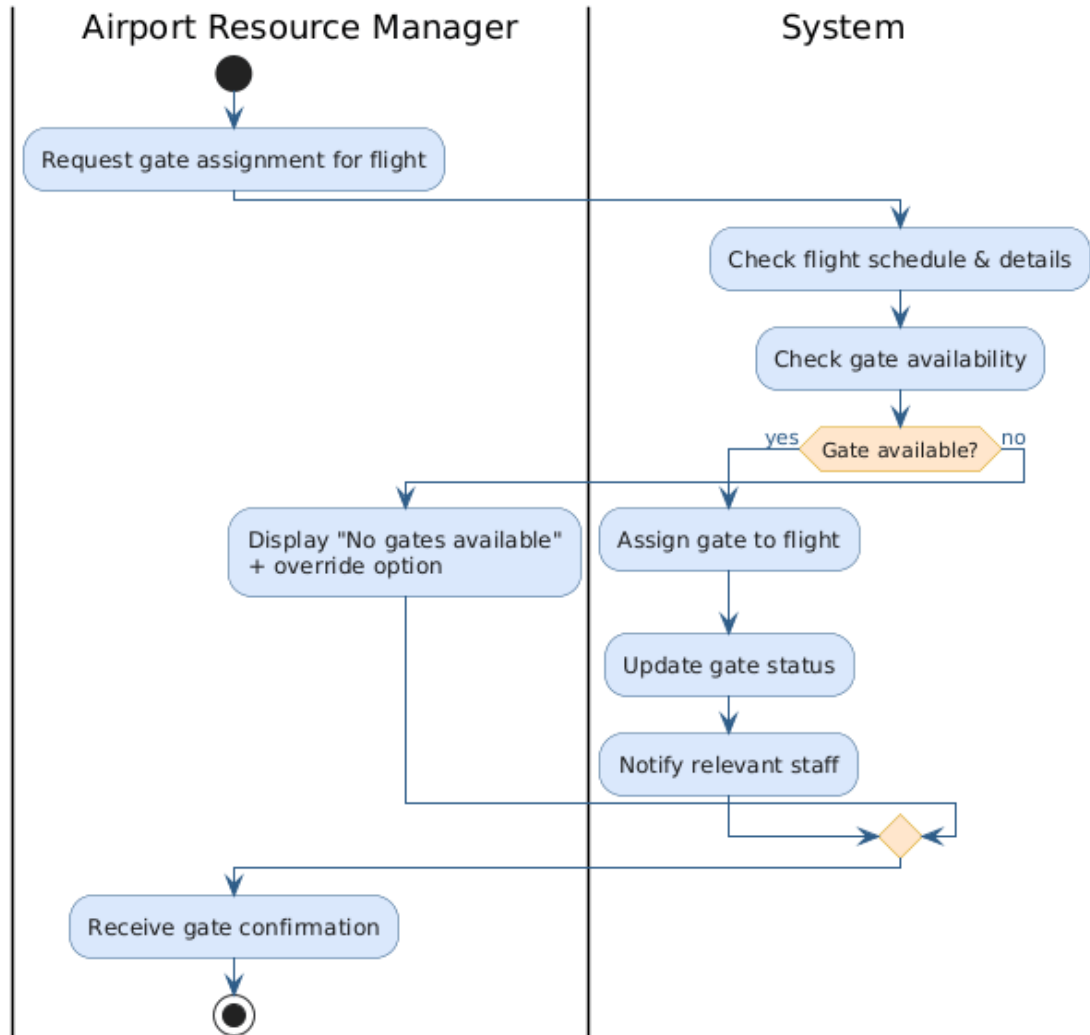
- **UC-ARM-01 (Gate Allocation Management):**
Assign gates to flights, check availability, and prevent scheduling conflicts.
- **UC-ARM-02 (Runway Scheduling):**
Schedule runways for takeoff and landing with conflict detection and priority handling.

- **UC-ARM-03 (Staff Allocation):**
Assign staff to tasks and shifts based on availability.
- **UC-ARM-04 (Equipment Management):**
Track equipment status and manage maintenance activities.

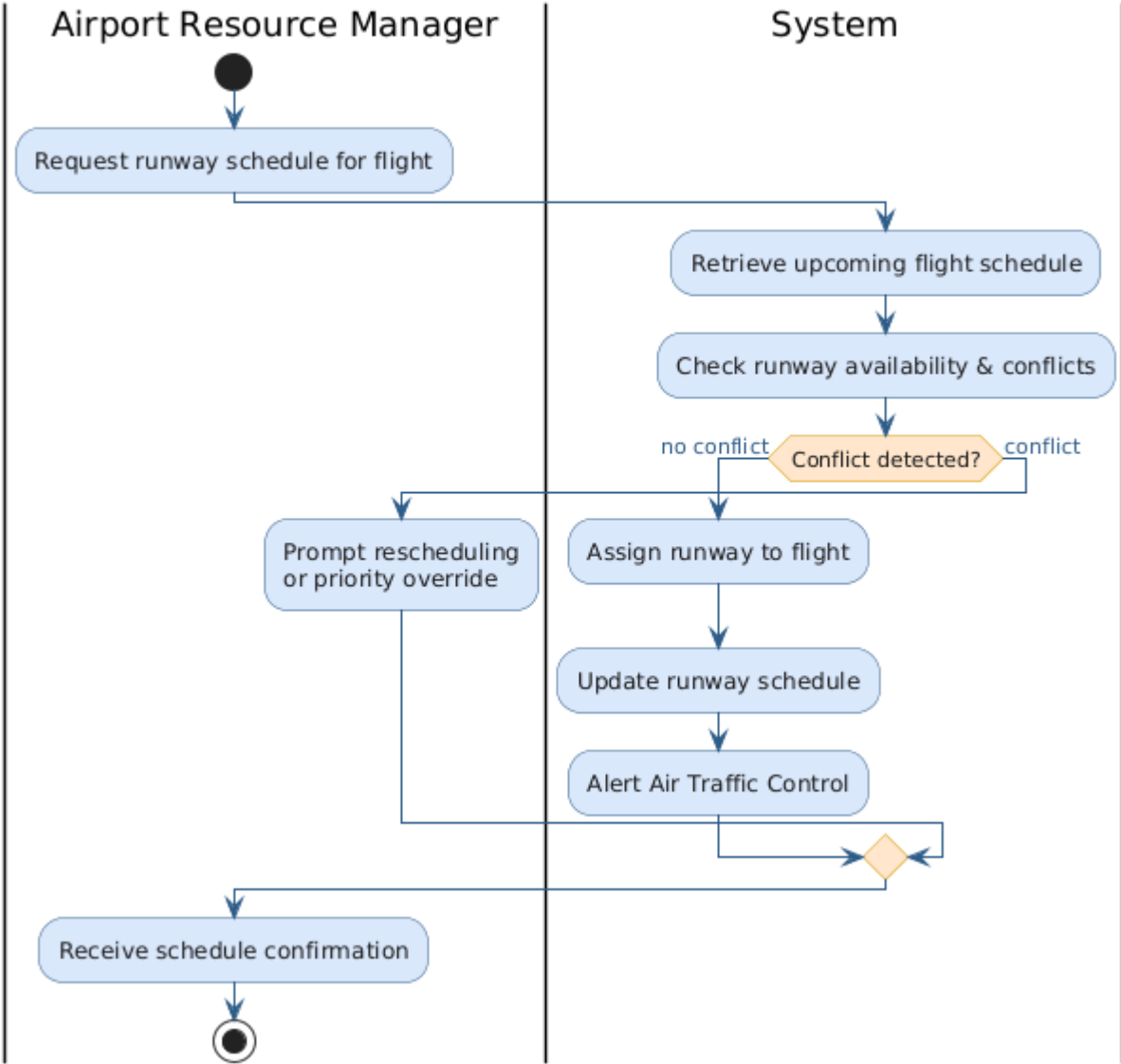
Alternative Flow Mapped:

- **Gate Allocation (UC-ARM-01):**
If no gates are available, the system displays an alert and allows manual override.
- **Runway Scheduling (UC-ARM-02):**
If a scheduling conflict occurs, the system prompts rescheduling. Emergency flights are prioritized.
- **Staff Allocation (UC-ARM-03):**
If selected staff is unavailable, the system suggests alternative staff.
- **Equipment Management (UC-ARM-04):**
If equipment fails, the system triggers a maintenance alert and updates status.

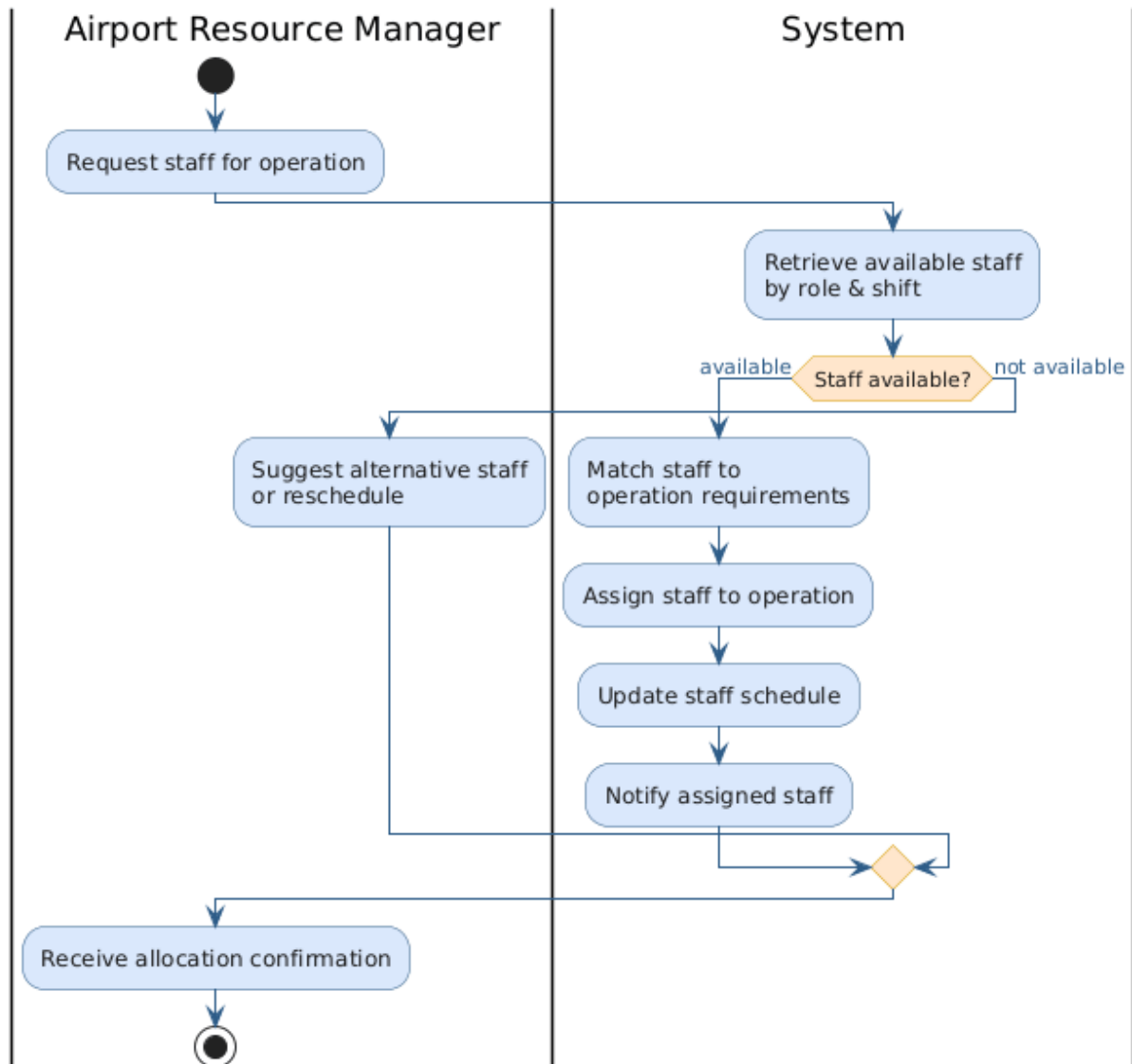
UC-ARM-01: Gate Allocation — Activity Diagram



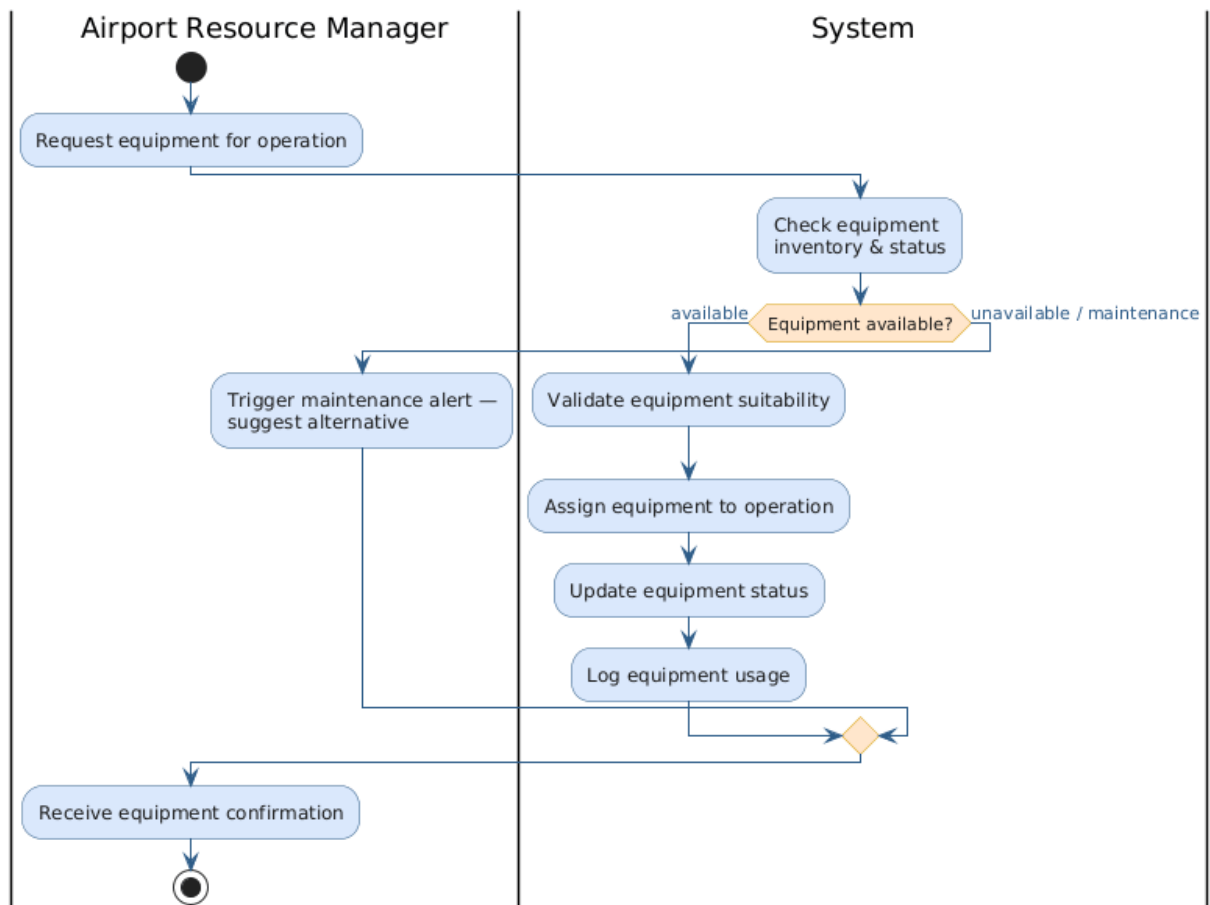
UC-ARM-02: Runway Scheduling — Activity Diagram



UC-ARM-03: Staff Allocation — Activity Diagram



UC-ARM-04: Equipment Management — Activity Diagram



Airport Management System

Finance Office and Human Resources Use Cases Documentation

Prepared by:

Andri Mucllari

Course:

Software Modeling and Design

Date:

27 March 2026

Document Overview

This document presents a structured set of **Finance Office and Human Resources models** for the Airport Management System. It includes:

- ✓ Processing Airport Service Payments (UC_50)
- ✓ Generating Airport Service Invoices (UC_51)
- ✓ Monitoring Financial Transactions (UC_52)
- ✓ Managing Airport Staff Records (UC_53)
- ✓ Managing Staff Attendance (UC_54)
- ✓ Processing Payroll (UC_55)

Each use case is defined with:

- **Mapped Functional Requirements:** Linking system functionality to specific UC IDs.
- **Mapped Alternative Flows:** Modeling system behavior during errors or exceptional conditions.
- **Non-functional Requirements:** Defining performance, security, and reliability constraints.
- **Swimlane Actor Interactions:** Visualizing the interaction between Passengers, Finance Staff, HR Staff, Employees, and the System.

The goal of this document is to clearly model and describe **financial and human resource system operations** in a professional and structured manner.

Processing Airport Service Payments (UC_50)

This use case describes how passengers pay for airport services such as parking, lounge access, or baggage handling through the system.

• **Functional Requirements Mapped:**

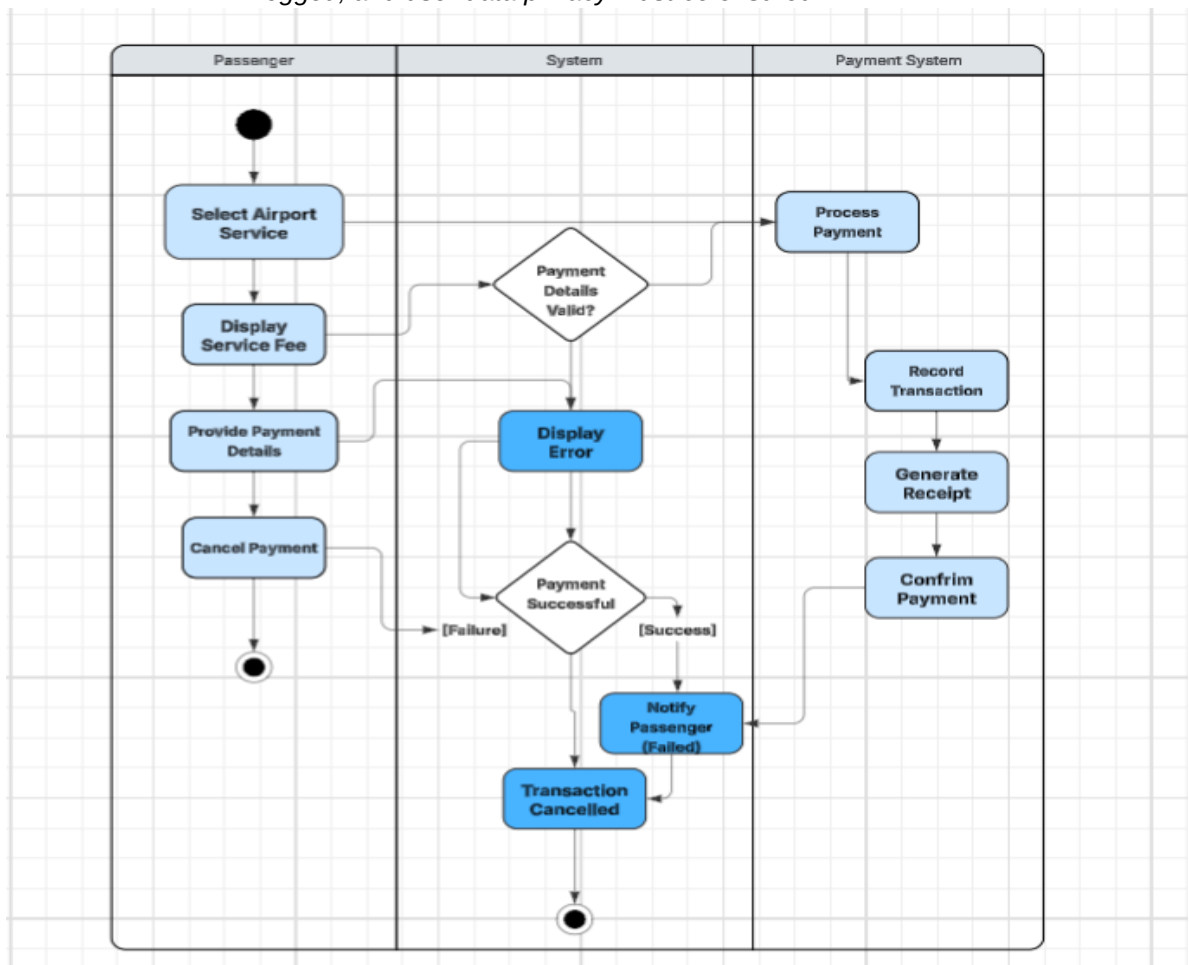
- Passenger selects an airport service.
- System displays the corresponding service fee.
- Passenger provides payment details.
- System validates the payment information.
- System processes the payment through the payment system.
- System records the transaction in the database.
- System generates a receipt.
- System confirms successful payment to the passenger.

• **Alternative Flow Mapped:**

If invalid payment details are entered, the system displays an error message. If the payment fails, the system notifies the passenger. If a system error occurs, the transaction is cancelled. If the passenger cancels the process, no payment is processed.

• **Non-Functional Requirements Mapped:**

Payment processing must be secure, transactions must be completed quickly, all payments must be logged, and user data privacy must be ensured.



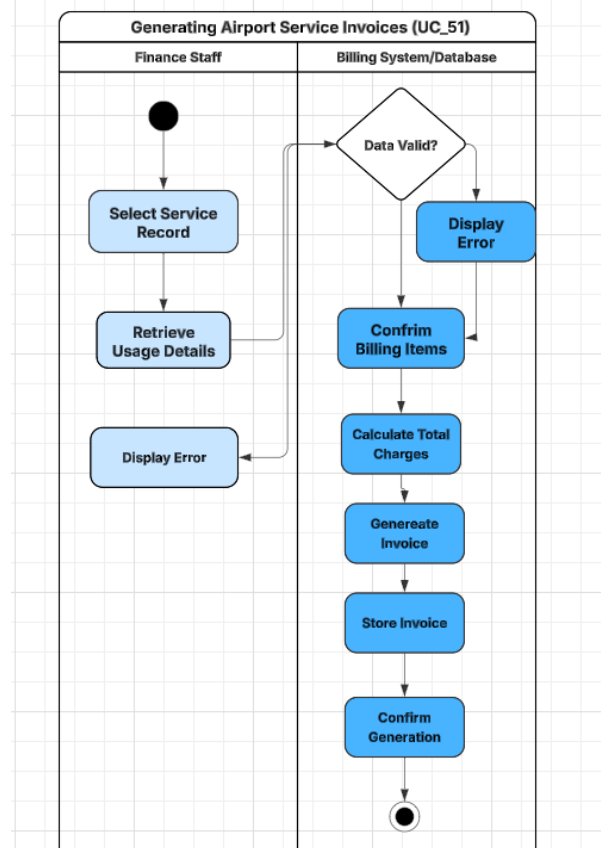
This use case describes how finance staff generate invoices for airport services provided to external parties such as aircraft parking and ground handling.

• **Functional Requirements Mapped:**

- Finance staff accesses the billing or invoice management module.
- System displays available service records.
- Finance staff selects a service record.
- System retrieves and displays usage details.
- Finance staff confirms billing items.
- System calculates total charges.
- System generates the invoice.
- System stores the invoice in the database.
- System confirms successful invoice generation.

• **Alternative Flow Mapped:**

If required data is missing, the system displays an error message. If invalid input is provided, the system prompts for correction.



Monitoring Financial Transactions (UC_52)

This use case describes how financial activities within the airport are monitored and reviewed by authorized users such as finance staff or system administrators.

• **Functional Requirements Mapped:**

- User logs into the system.
- User accesses the financial monitoring module.
- System retrieves and displays transaction records.

- User applies filters to refine transaction data.
- System updates and displays filtered results.
- User reviews the financial data.

• **Alternative Flow Mapped:**

If no financial data is available, the system notifies the user. If a system error occurs, the system is unable to retrieve transaction data and displays an error message.

Managing Airport Staff Records (UC_53)

This use case describes how Human Resource (HR) staff manage records of airport employees, including security staff, ground crew, and administrative personnel.

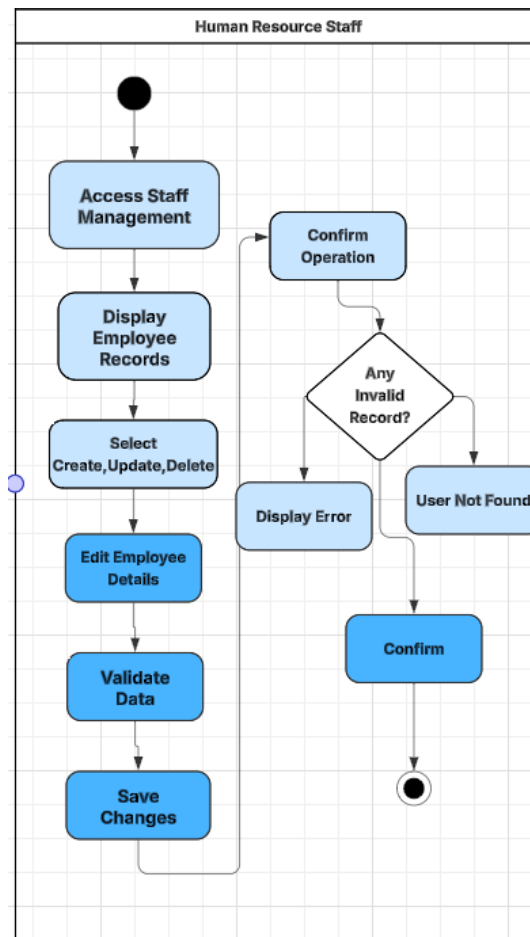
• **Functional Requirements Mapped:**

- HR staff logs into the system.
- HR accesses the staff management module.
- System displays existing employee records.
- HR selects an action (create, update, or delete).
- HR enters new employee details or modifies existing information.
- System validates the entered data.
- System saves changes in the database.

- System confirms successful operation.

• **Alternative Flow Mapped:**

If invalid data is entered, the system displays an error message. If a selected record does not exist, the system notifies the user. If a system failure occurs, the operation is cancelled and no changes are saved.



Managing Staff Attendance (UC_54)

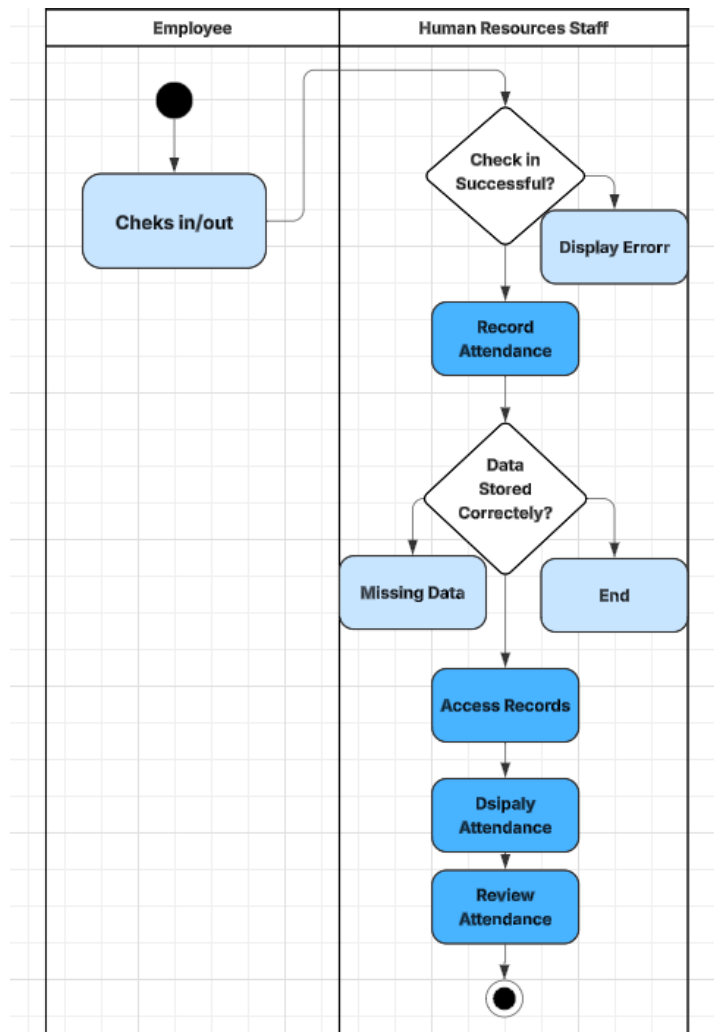
This use case describes how airport staff attendance is recorded and monitored by the system and reviewed by HR staff.

• **Functional Requirements Mapped:**

- Employee checks in or checks out through the system.
- System records the attendance time.
- HR staff accesses the attendance management module.
- System displays attendance records.
- HR reviews attendance logs.

• **Alternative Flow Mapped:**

If the check-in or check-out process fails, the system displays an error message. If attendance data is missing, the system notifies HR staff.



Processing Payroll (UC_55)

This use case describes how salaries for airport staff are calculated and processed based on employee and attendance data.

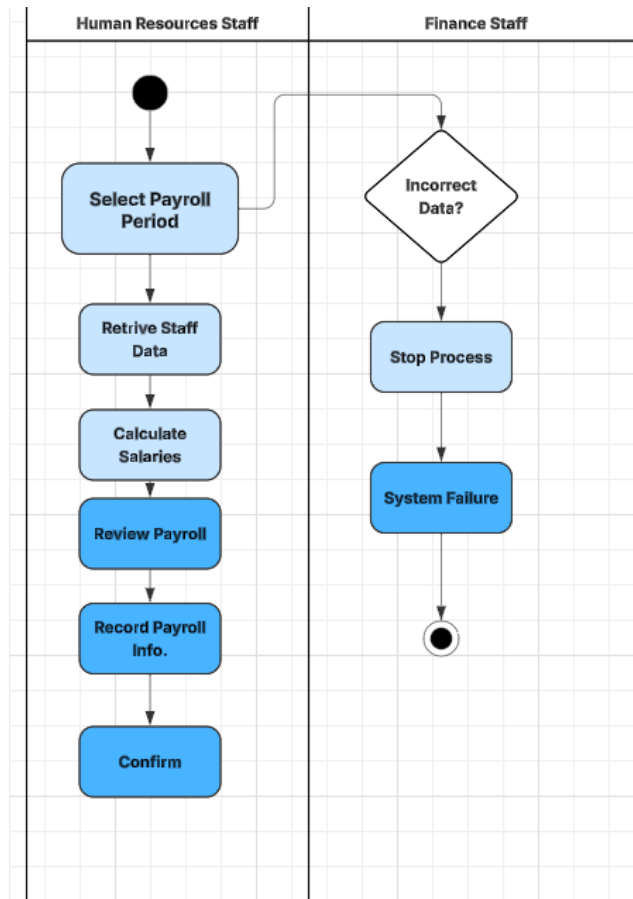
• Functional Requirements Mapped:

- HR or Finance staff accesses the payroll module.
- HR selects the payroll period.
- System retrieves employee and attendance data.
- System calculates salaries, including overtime and deductions.
- HR reviews payroll details.
- System processes salary payments.
- System records payroll information in the database.
- System confirms successful completion of payroll processing.

• Alternative Flow Mapped:

If required data is missing, the system displays an error message. If a calculation

error occurs, the process is stopped. If a system failure occurs, payroll processing is cancelled and no payments are made.



Airport Management System

Customer Support & Human Resource Management Models Documentation

Prepared by:

Ajla Alcani

Course:

Software Modeling and Design

Date:

26 March 2026

Document Overview

This document presents a structured set of behavioral models for the Airport Management System. It includes:

- ✓ *Ticket Management Activity Diagram*
- ✓ *Live Chat Support Activity Diagram*
- ✓ *Support Ticket Lifecycle State Diagram*
- ✓ *Manage Airport Staff Records Activity Diagram*
- ✓ *Manage Staff Attendance Activity Diagram*
- ✓ *Process Payroll Activity Diagram*
- ✓ *Each model is defined with:*
 - 1. Mapped Functional Requirements*
 - 2. Mapped Alternative Flows*
 - 3. Non-functional Requirements*
 - 4. Swimlane Actor Interactions*

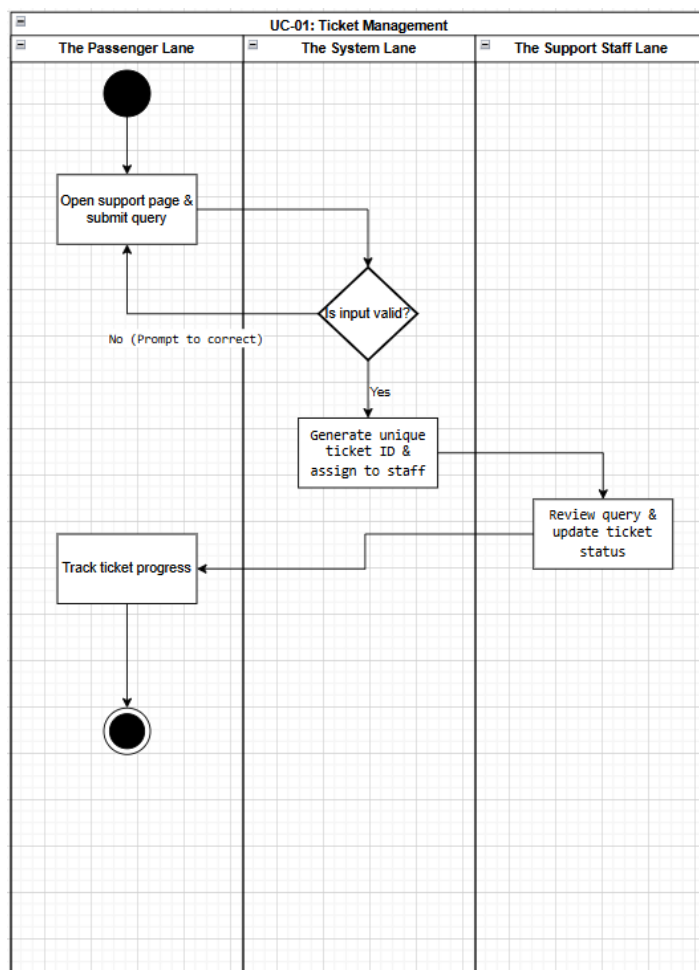
The goal of this document is to clearly model and describe the system's behavioral workflows in a professional and structured way.

The following section presents the visual behavioral models for the Airport Management System (AMS), specifically focusing on the Customer Support Management and Human Resource Management modules. These Activity and State Machine diagrams translate the defined functional and non-functional requirements from the Software Requirements Specification (SRS) into structured workflows. By mapping each diagram directly to its corresponding use case—including both main and alternative flows—this section demonstrates how the system design logically fulfills the expected operational capabilities and serves as a foundation for the implementation phase.

1. Customer Support Management Diagrams

Activity Diagram 1: Ticket Management (UC-CSM-01)

- **Core Purpose:** This use case describes how passengers submit and track support queries using a ticketing system.
- **Functional Requirements Mapped:**
 - Customer support agent tracks service requests.
 - Passenger submits query via form.
 - System generates a unique ticket ID and assigns it to support staff.
 - Staff updates ticket status and passenger tracks ticket progress.
- **Alternative Flow Mapped:** If invalid input is provided, the system prompts the user to correct details.

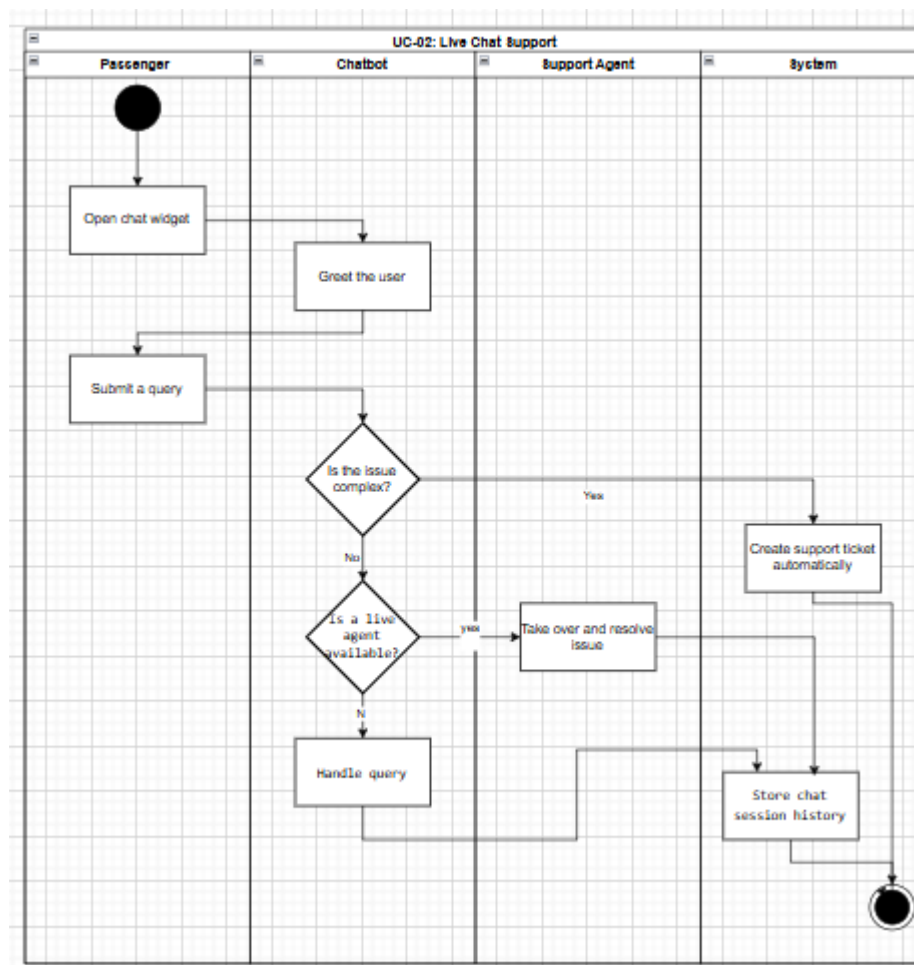


Activity Diagram 2: Live Chat Support (UC-CSM-02)

• *Core Purpose:* This use case describes how passengers receive real-time assistance through chat support.

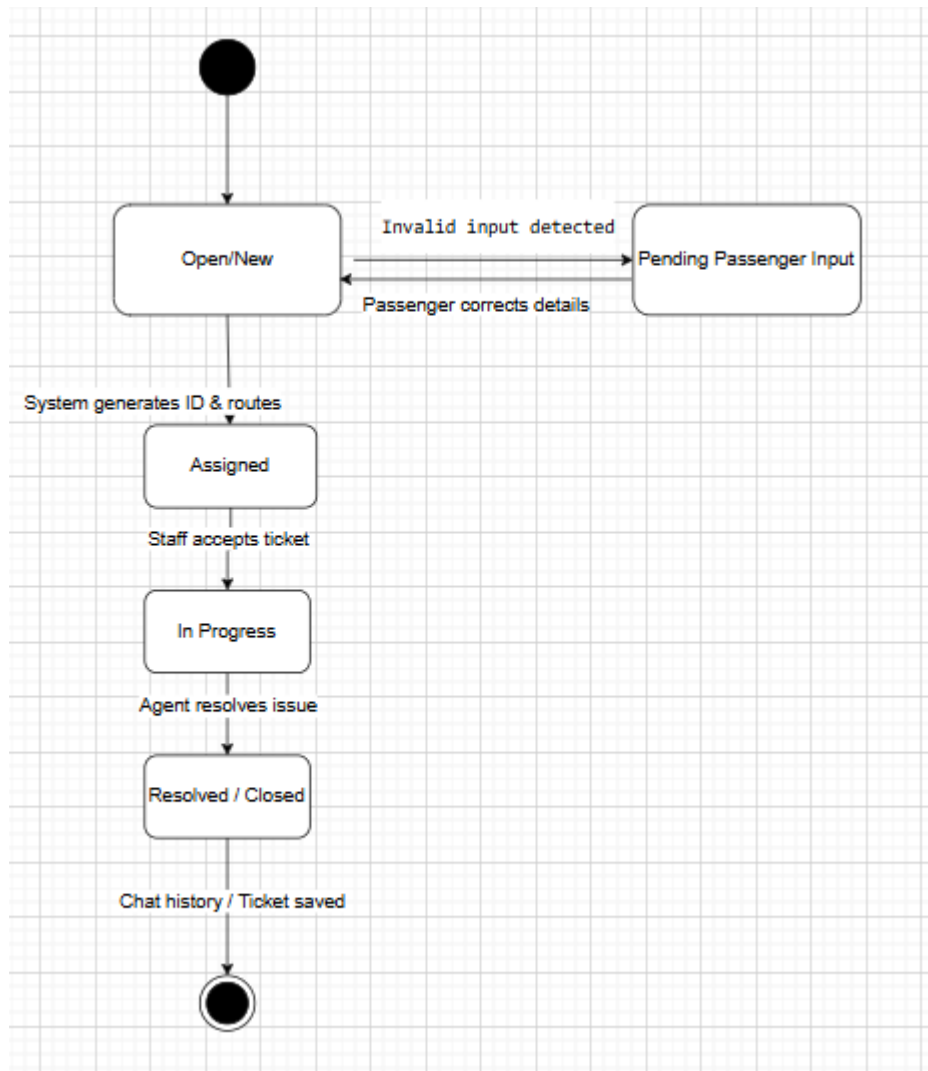
• *Functional Requirements Mapped:*

- Customer support agent responds to inquiries.
- Passenger opens chat widget and submits a query.
- Chatbot greets the user and either responds or transfers to a live agent.
- Agent resolves the issue and the chat session is stored.
- Alternative Flow Mapped: If the issue is complex, the system creates a support ticket automatically.



State Diagram: Support Ticket Lifecycle

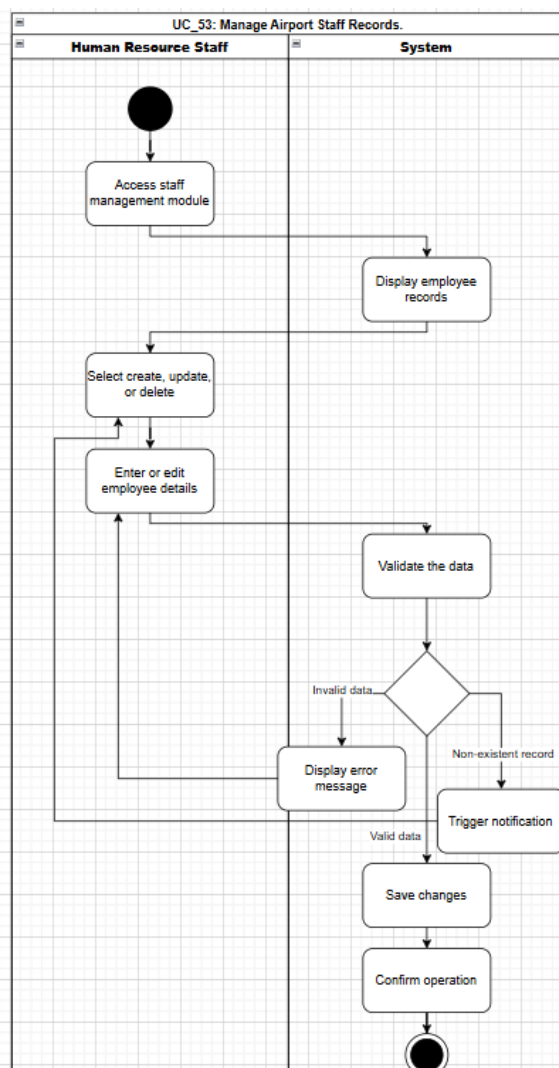
- *Core Purpose:* To model the exact states a ticket passes through from creation to resolution.
- *Requirements Mapped:* Ensures a ticket is successfully created, tracked, and managed, and that the query is either resolved or escalated.



2. Human Resource Management Diagrams

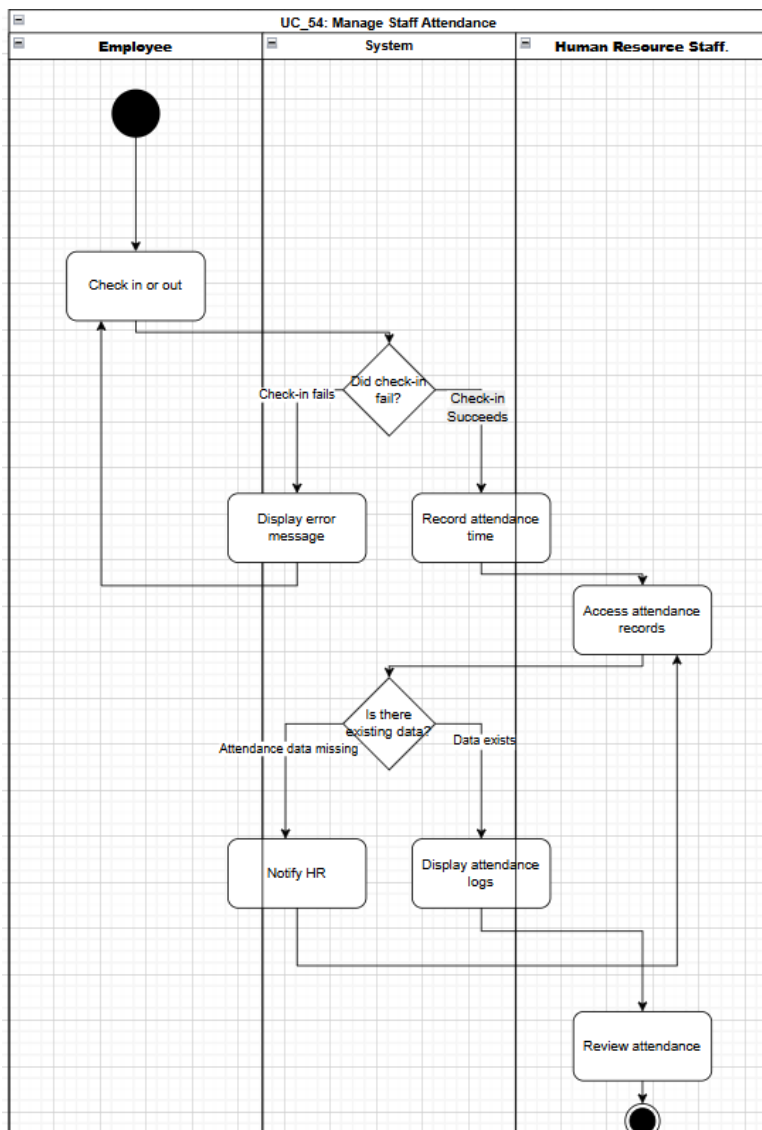
Activity Diagram 3: Manage Airport Staff Records (UC_53)

- **Core Purpose:** This use case describes how HR manages records of airport employees.
- *Functional Requirements Mapped:*
 - HR staff registers employee records and assigns staff roles.
 - HR accesses the module and selects to create, update, or delete.
 - HR enters or edits employee details.
 - System validates the data, saves changes, and confirms the operation.
- *Alternative Flow Mapped:* Invalid data triggers an error message, and a non-existent record triggers a notification.
- *Non-Functional Requirements Mapped:* Employee data must be secure, role-based access must be enforced, and all changes must be logged.



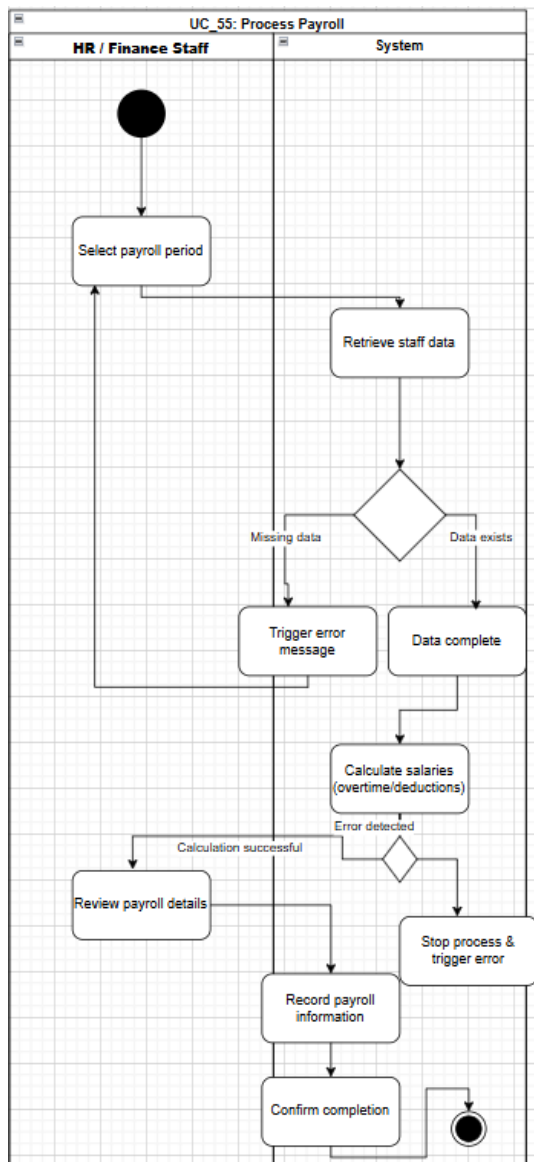
Activity Diagram 4: Manage Staff Attendance (UC_54)

- *Core Purpose:* This use case describes how airport staff attendance is recorded and monitored.
- *Functional Requirements Mapped:*
 - HR staff manages employee schedules.
 - Employee checks in/out and the system records the attendance time.
 - HR accesses the records, the system displays the attendance logs, and HR reviews the attendance.
- *Alternative Flow Mapped:* If check-in fails, the system shows an error; if attendance data is missing, the system notifies HR.
- *Non-Functional Requirements Mapped:* System must ensure accurate time tracking, data must be secure, and the system must be reliable.



Activity Diagram 5: Process Payroll (UC_55)

- *Core Purpose:* This use case describes how salaries for airport staff are calculated and processed.
- *Functional Requirements Mapped:*
 - HR selects the payroll period and the system retrieves staff data.
 - System calculates salaries, including overtime and deductions.
 - HR reviews payroll details.
 - System processes salary payments, records payroll information, and confirms completion.
- *Alternative Flow Mapped:* Missing data triggers an error, and a calculation error stops the process.
- *Non-Functional Requirements Mapped:* Payroll must be accurate, data must remain confidential, and processing must be efficient.



Airport Management System

System Administration Models Documentation

Prepared by:

Eneida Balliu

Course:

Software Modeling and Design

Date:

27 March 2026

Document Overview

This document presents a structured set of behavioral models for the System Administration module of the Airport Management System (AMS). It focuses on modeling the core administrative operations performed by the system administrator through Activity and State Machine diagrams.

The document includes the following system administration models:

- ✓ Manage User Accounts Activity Diagram (UC_40)
- ✓ Update Airport Information Activity Diagram (UC_41)
- ✓ Monitor System Usage Activity Diagram (UC_42)
- ✓ Manage System Permissions Activity Diagram (UC_43)
- ✓ Perform System Maintenance Activity Diagram (UC_44)
- ✓ Generate System Activity Reports Activity Diagram (UC_45)

Model Structure

Each behavioral model is developed based on its corresponding use case and includes:

1. Mapped Functional Requirements
2. Main and Alternative Flows Representation
3. Relevant Non-functional Constraints
4. Swimlane-Based Actor Interactions (System Administrator & System)

Purpose of the Document

The purpose of this document is to provide a clear, structured, and visual representation of the system's administrative workflows. These diagrams translate the functional requirements defined in the Software Requirements Specification (SRS) into logical and easy-to-understand process flows.

Each diagram has been designed following best practices in UML modeling, ensuring:

- simplicity and readability
- minimal and necessary decision points
- correct use of activity flow elements

3. System Administration Diagrams

Activity Diagram 1: Manage User Accounts (UC_40)

- *Core Purpose:*

This use case describes how the system administrator creates, updates, and deletes user accounts within the system.

- *Functional Requirements Mapped:*

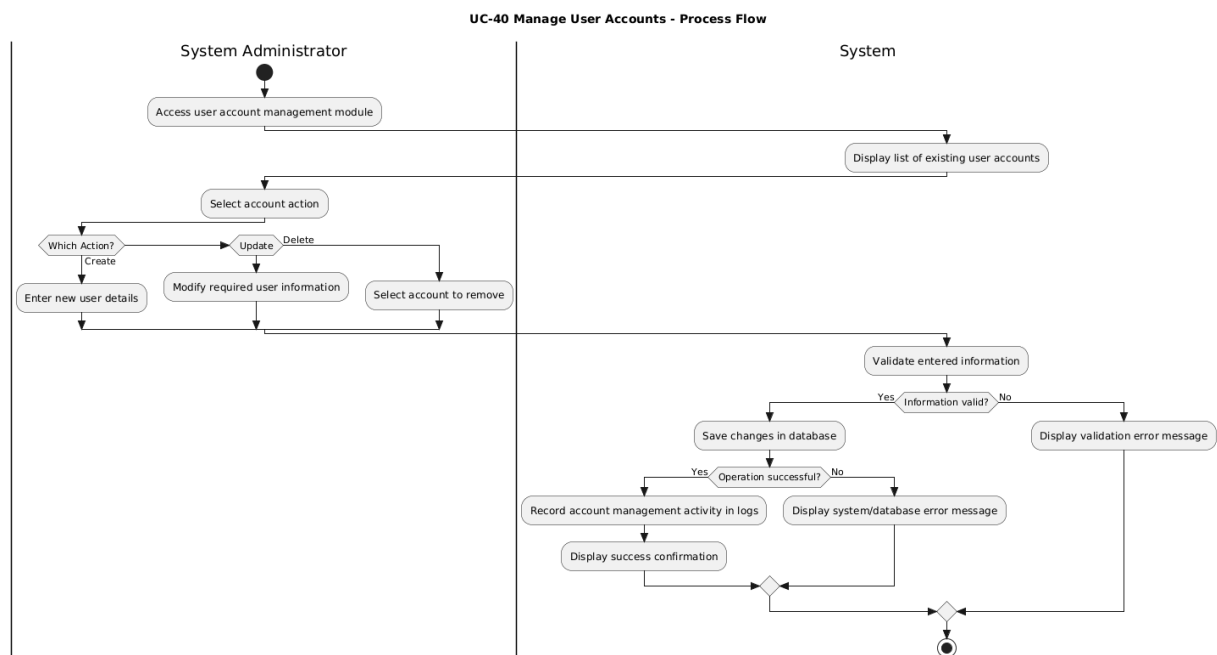
- System administrator accesses the user account management module.
- System displays a list of existing user accounts.
- Administrator selects an action (create, update, or delete).
- Administrator enters new user details, modifies existing information, or selects an account to delete.
- System validates the entered information.
- System saves changes in the database.
- System records account management activities in logs and confirms successful operation.

- *Alternative Flow Mapped:*

If invalid or incomplete data is entered, the system displays a validation error message. If the operation fails due to a system or database error, an error message is displayed. If the selected account does not exist, the system notifies the administrator.

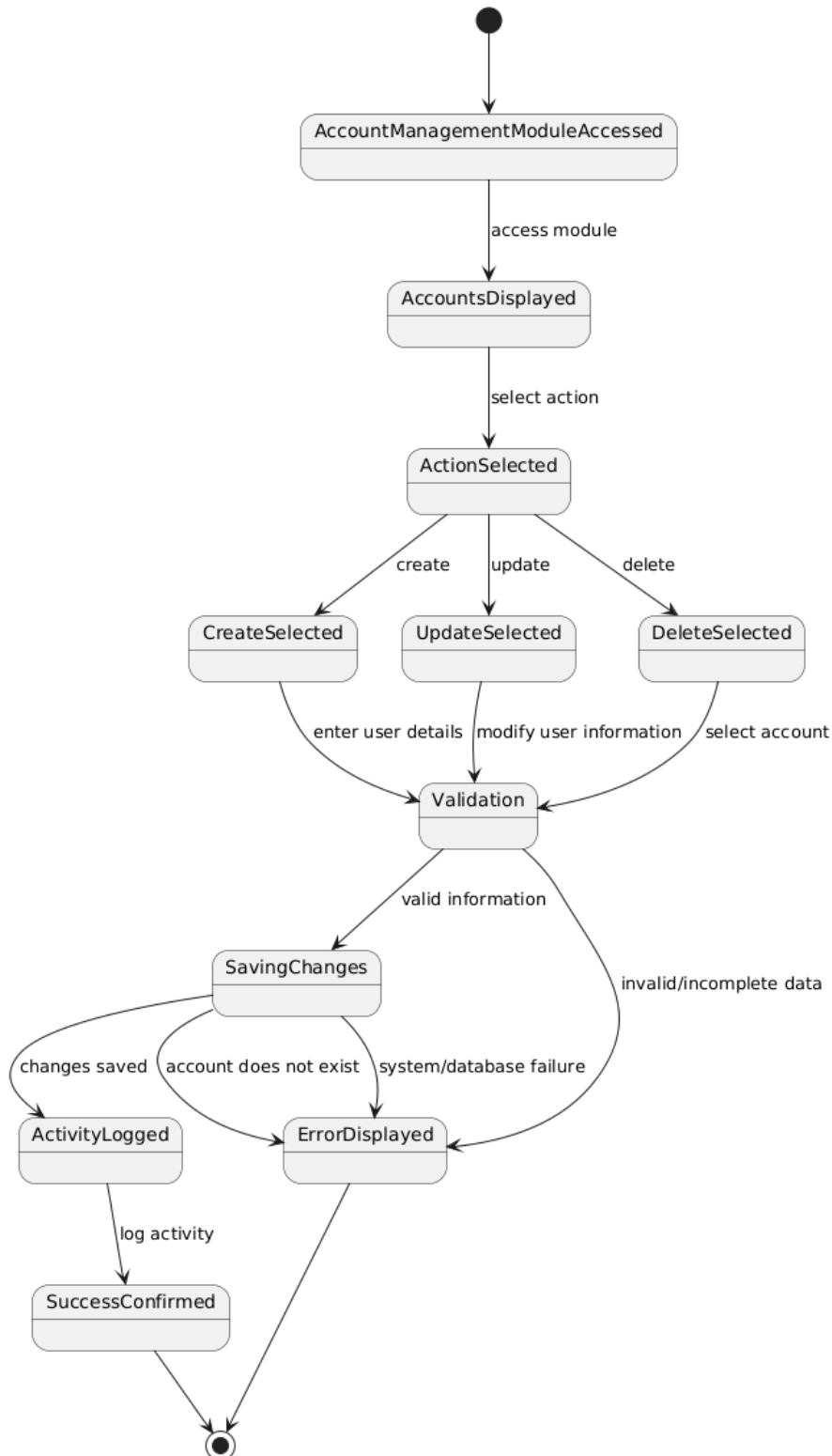
- *Non-Functional Requirements Mapped:*

User data must be securely managed, role-based access control (RBAC) must be enforced, and all account management activities must be logged for auditing purposes.



State Diagram 1: Manage User Accounts (UC_40)

UC_40 Manage User Accounts - State Diagram



Activity Diagram 2: Update Airport Information (UC_41)

• Core Purpose:

This use case describes how the system administrator updates airport-related information within the system.

• *Functional Requirements Mapped:*

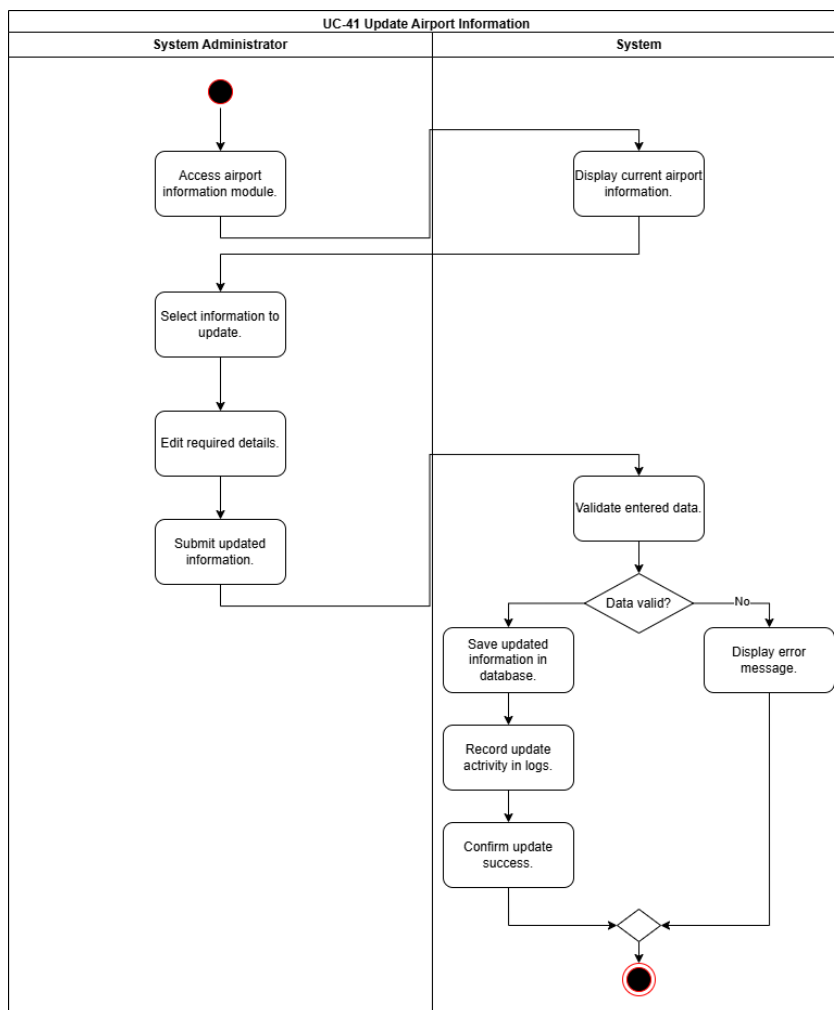
- System administrator accesses the airport information module.
- System displays current airport information.
- Administrator selects and edits the required details.
- Administrator submits updated information.
- System validates the entered data.
- System saves updated information in the database.
- System records update activity in logs and confirms success.

• *Alternative Flow Mapped:*

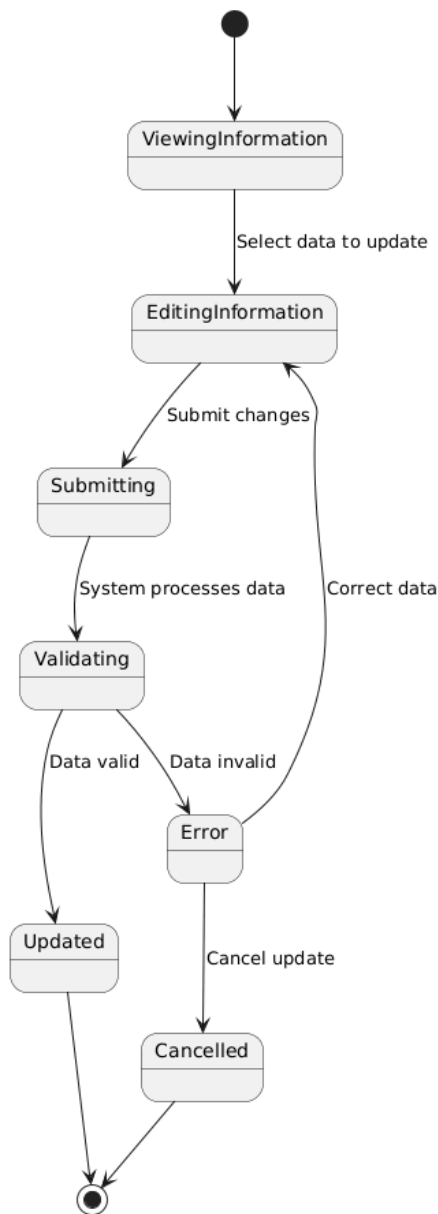
If invalid or incomplete data is entered, the system displays an error message. If the operation fails due to a system or database error, an error is displayed.

• *Non-Functional Requirements Mapped:*

System must ensure secure access, enforce role-based access control (RBAC), maintain data accuracy, and log all update activities.



State Diagram 2: Update Airport Information (UC_41)



Activity Diagram 3: Monitor System Usage (UC_42)

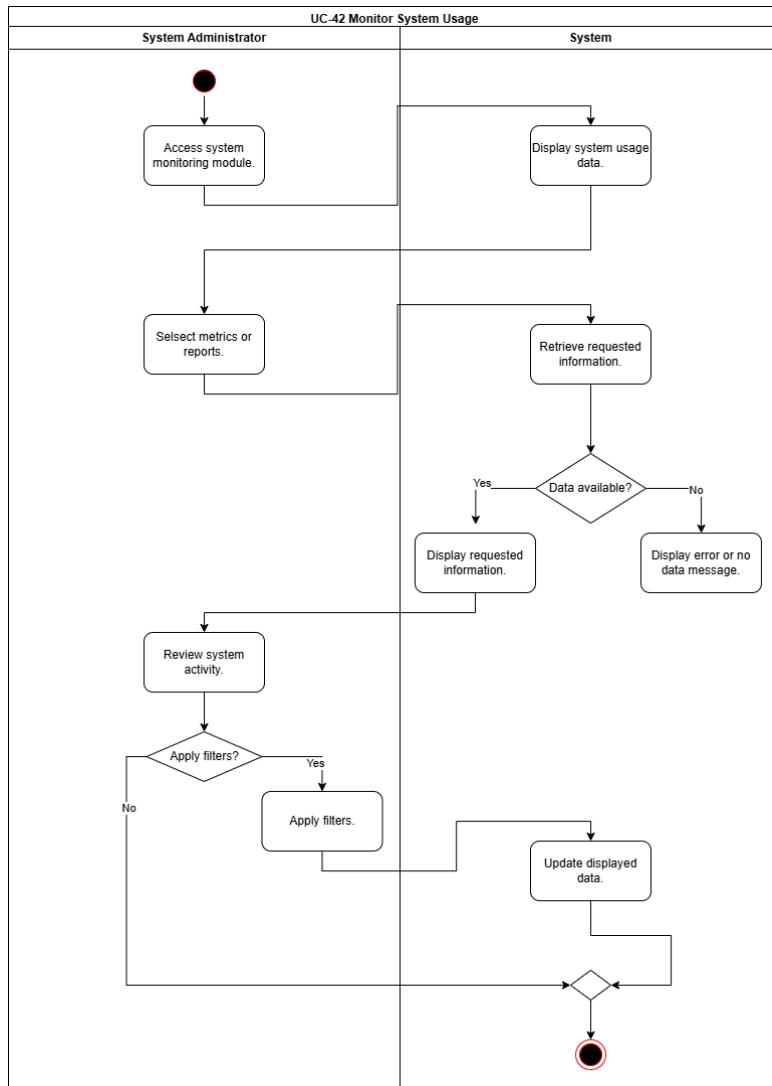
• Core Purpose:

This use case describes how the system administrator monitors system activity, usage statistics, and performance.

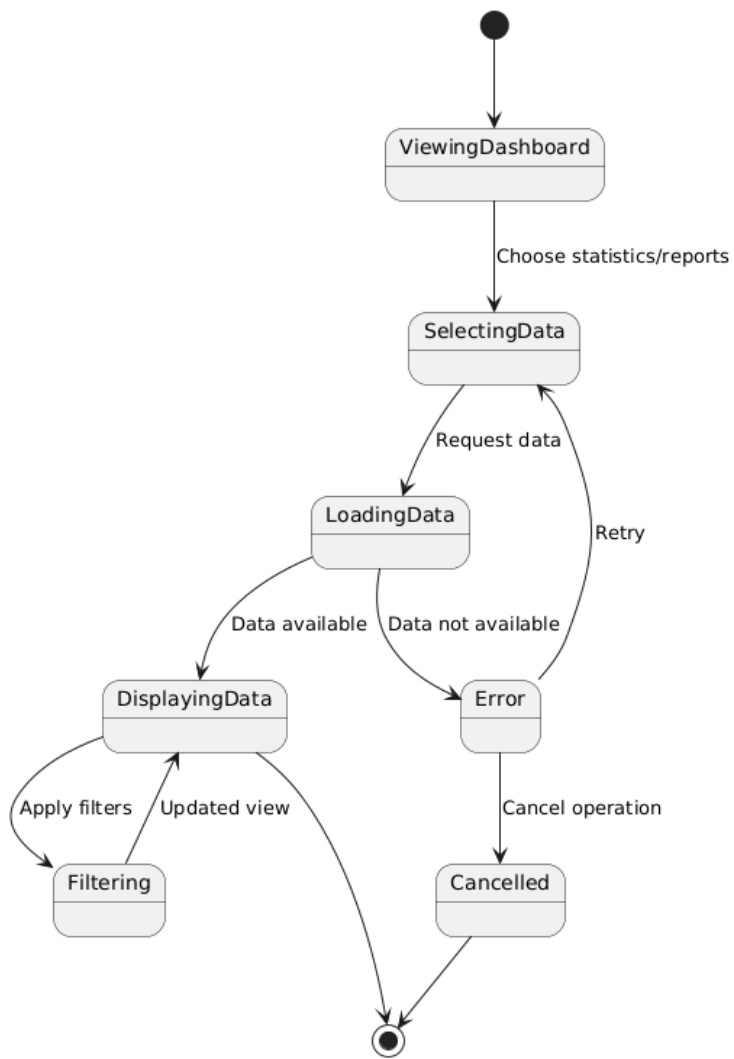
• Functional Requirements Mapped:

- System administrator accesses the system monitoring module.

- System must provide real-time data, ensure secure access, maintain accuracy, and display information within acceptable response time.



State Diagram 4: Manage System Permissions (UC_42)



Activity Diagram 4: Manage System Permissions (UC_43)

- **Core Purpose:**

This use case describes how the system administrator manages user roles and access permissions.

- **Functional Requirements Mapped:**

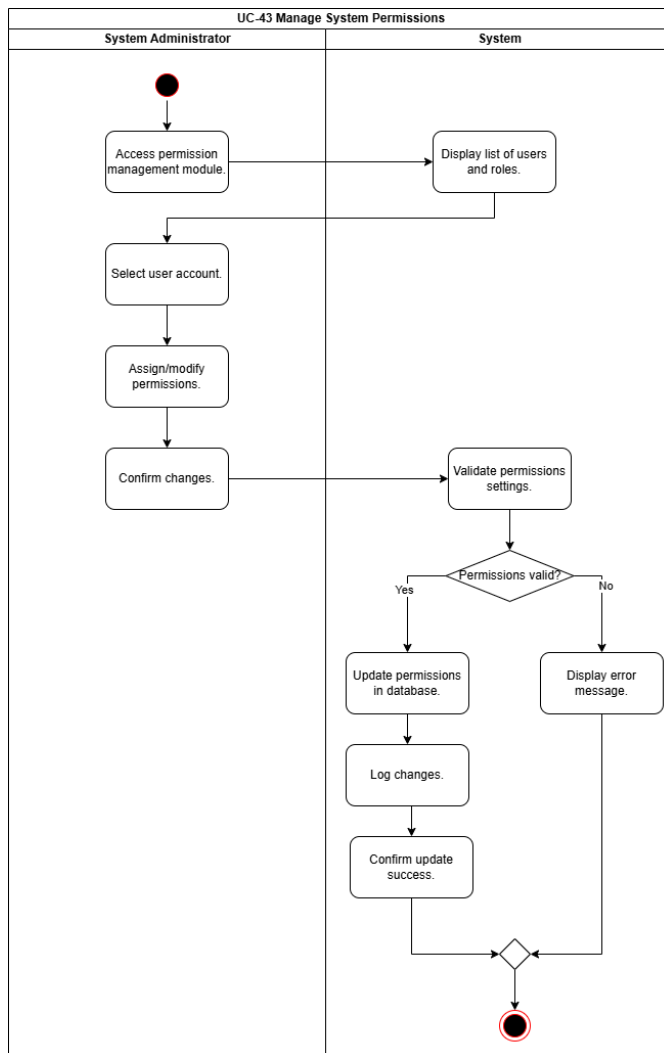
- System administrator accesses the permission management module.
- System displays users and their current permissions.
- Administrator selects a user and modifies roles or permissions.
- Administrator confirms the changes.
- System validates and updates permissions in the database.
- System confirms successful update.

- **Alternative Flow Mapped:**

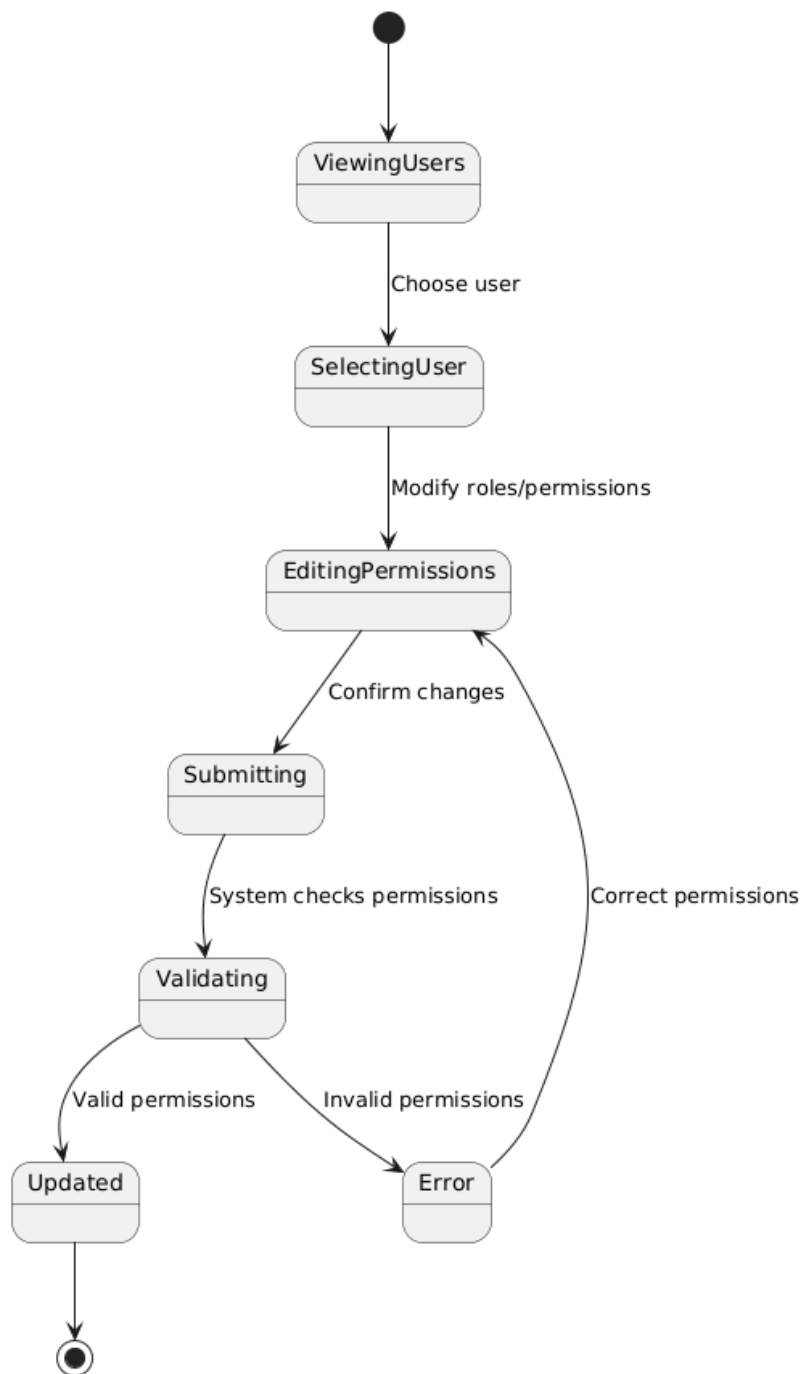
If invalid permissions are assigned or the user does not exist, the system displays an error message.

- **Non-Functional Requirements Mapped:**

System must enforce RBAC, ensure secure access, maintain data integrity, and log all permission changes.



State Diagram 4: Manage System Permissions (UC_43)



Activity Diagram 5: Perform System Maintenance (UC_44)

• **Core Purpose:**

This use case describes how the system administrator performs maintenance operations such as backup, recovery, and configuration updates.

• *Functional Requirements Mapped:*

- System administrator accesses the maintenance module.
- System displays available maintenance operations.
- Administrator selects an operation (backup, recovery, configuration update).
- Administrator confirms the operation.
- System validates and executes the selected operation.
- System saves changes and updates the database if necessary.
- System records maintenance activity and confirms completion.

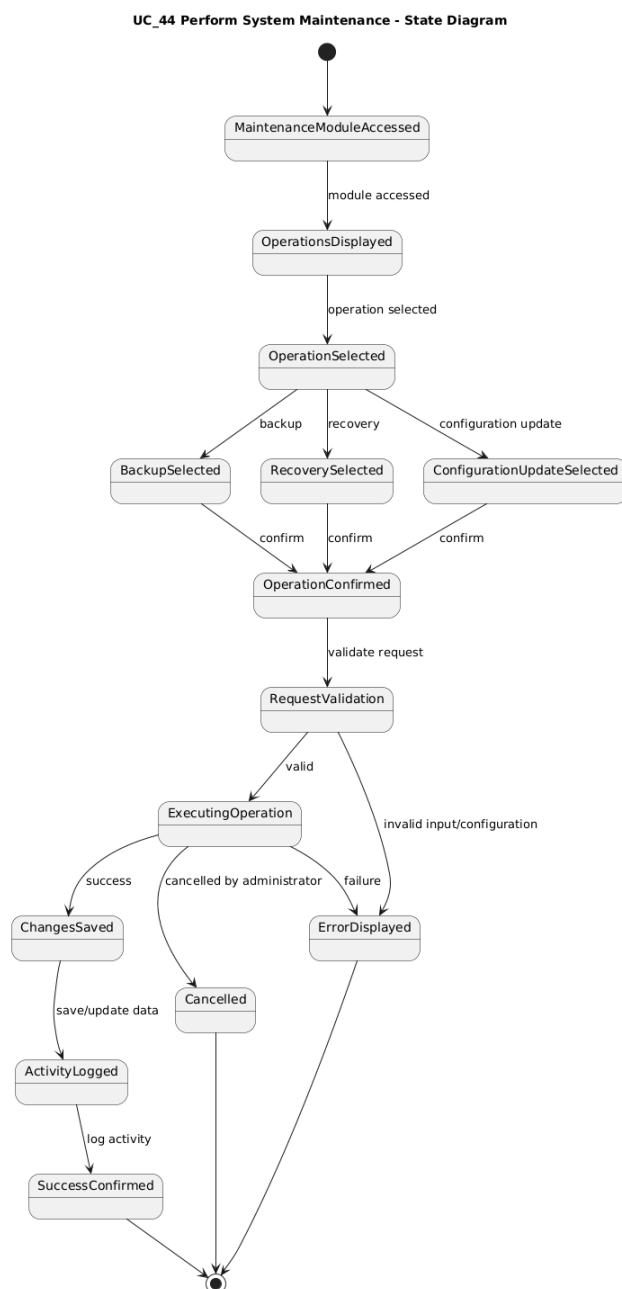
• *Alternative Flow Mapped:*

If invalid input is provided or the operation fails, the system displays an error message.

• *Non-Functional Requirements Mapped:*

System must ensure data integrity, secure access, minimize downtime, and log all maintenance activities

State Diagram 5: Perform System Maintenance (UC_44)



Activity Diagram 6: Generate System Activity Reports (UC_45)

• Core Purpose:

This use case describes how the system administrator generates and views system activity reports.

• Functional Requirements Mapped:

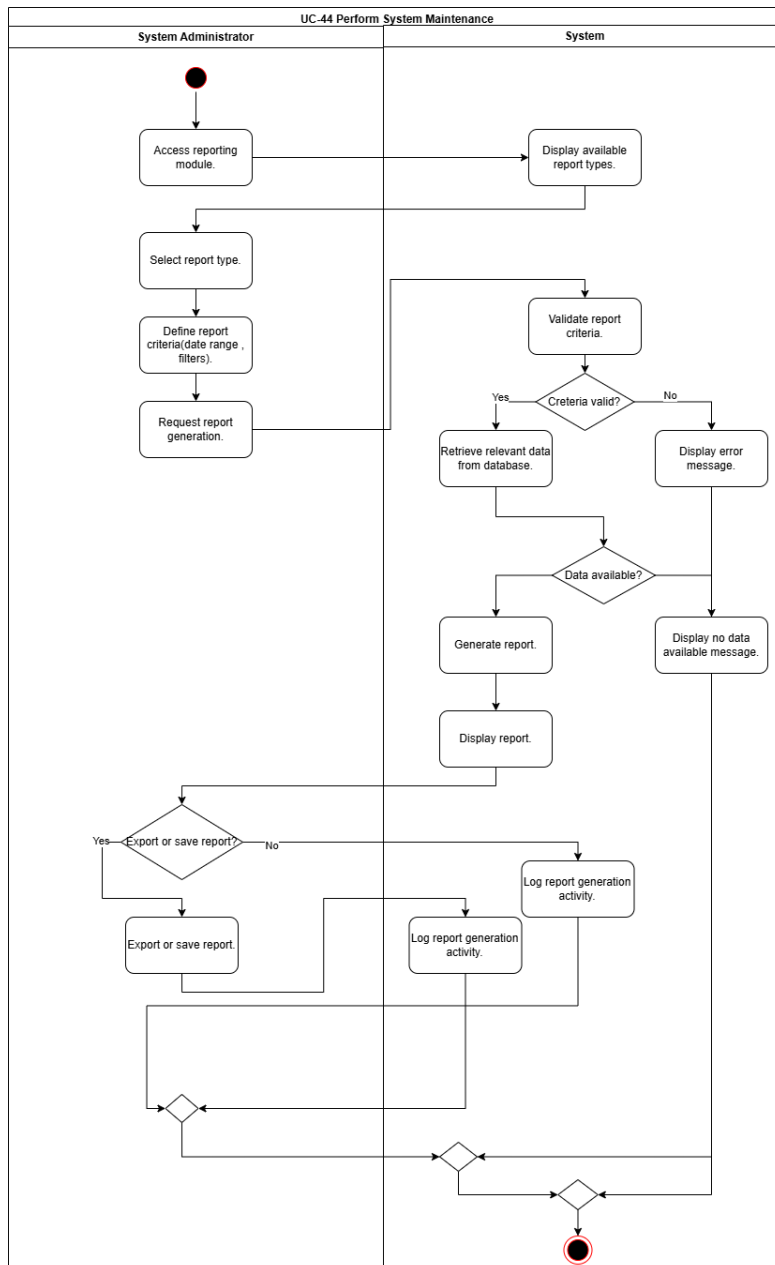
- System administrator accesses the reporting module.
- System displays available report types.
- Administrator selects report type and defines criteria.
- Administrator requests report generation.
- System retrieves data and generates the report.
- System displays the report.
- Administrator may export or save the report.

• Alternative Flow Mapped:

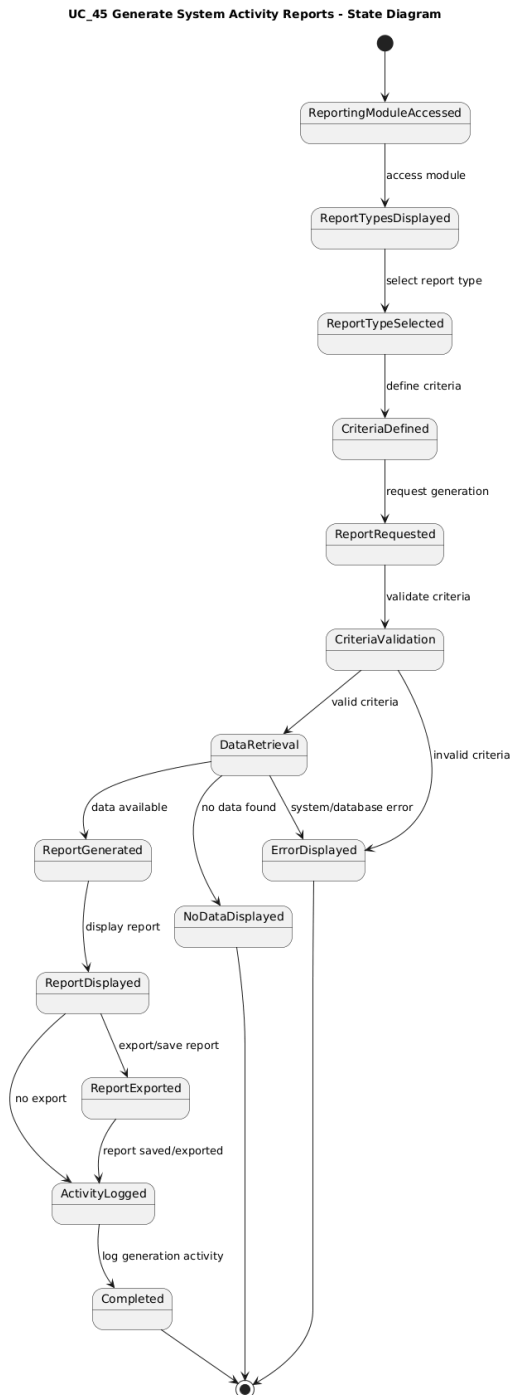
If no data is available or invalid criteria are entered, the system displays an error message.

• Non-Functional Requirements Mapped:

System must ensure report accuracy, secure access, efficient processing, and support exporting in standard formats.



State Diagram 6: Generate System Activity Reports (UC_45)



Airport Management System

Prepared by: Ajla Alcani

Course: Software Modeling and Design

Document Overview:

This document presents the state machine models for the Airport Management System (AMS). It illustrates the exact states, events, and transitions of the system's main objects.

Objective :

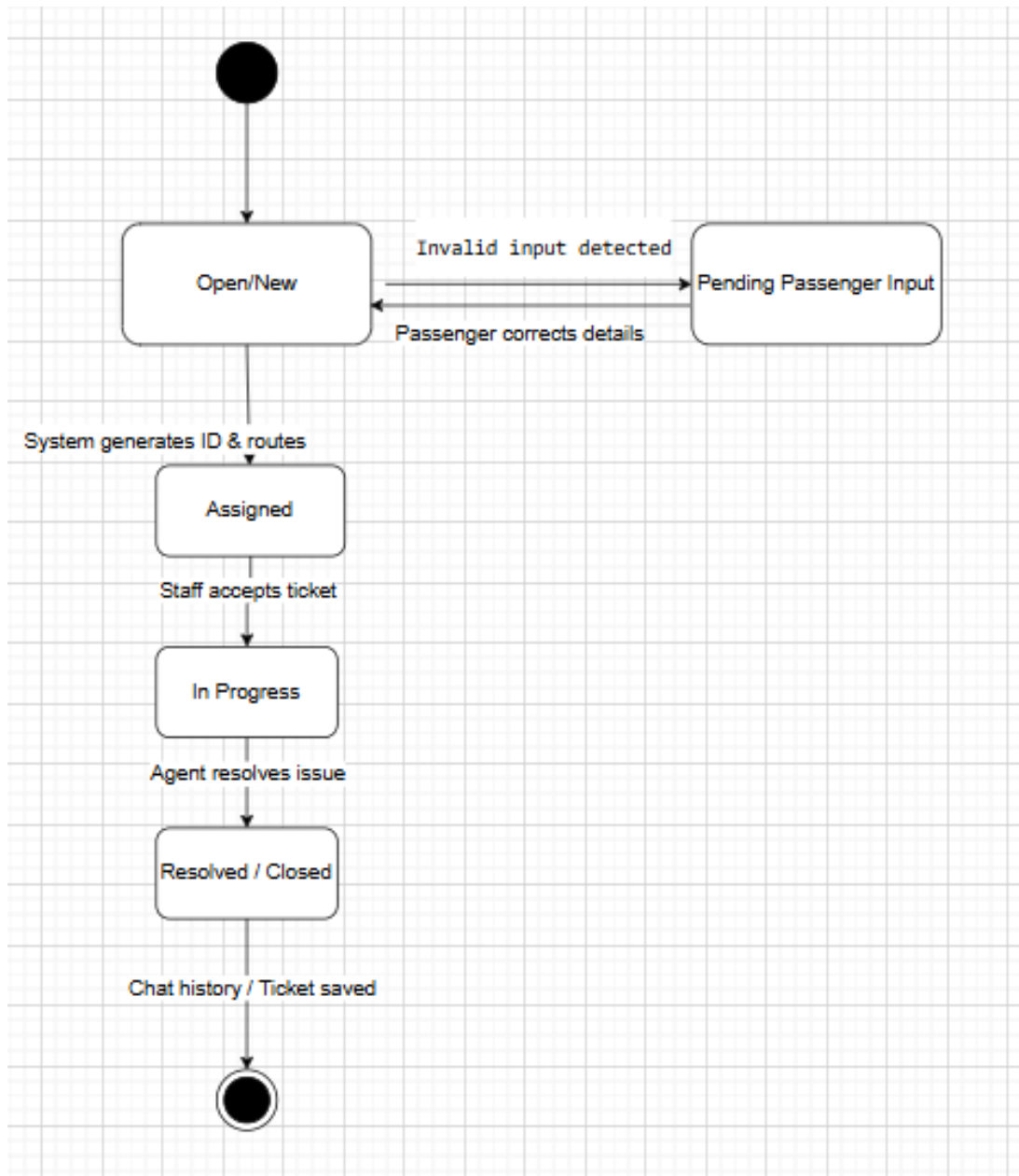
The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements and serves as a foundation for implementation.

State Machine Diagrams

This section defines the dynamic behavior and state transitions for the core system objects managed within the Customer Support and Human Resource modules.

1.1 Support Ticket Lifecycle (Customer Support Module)

- *Core Purpose:* To model the exact states a support ticket passes through from initial creation by a passenger to final resolution by a support agent.
- *Mapped Requirements:* Ensures a ticket is successfully created, tracked, managed, and that the query is either resolved or escalated without data loss.

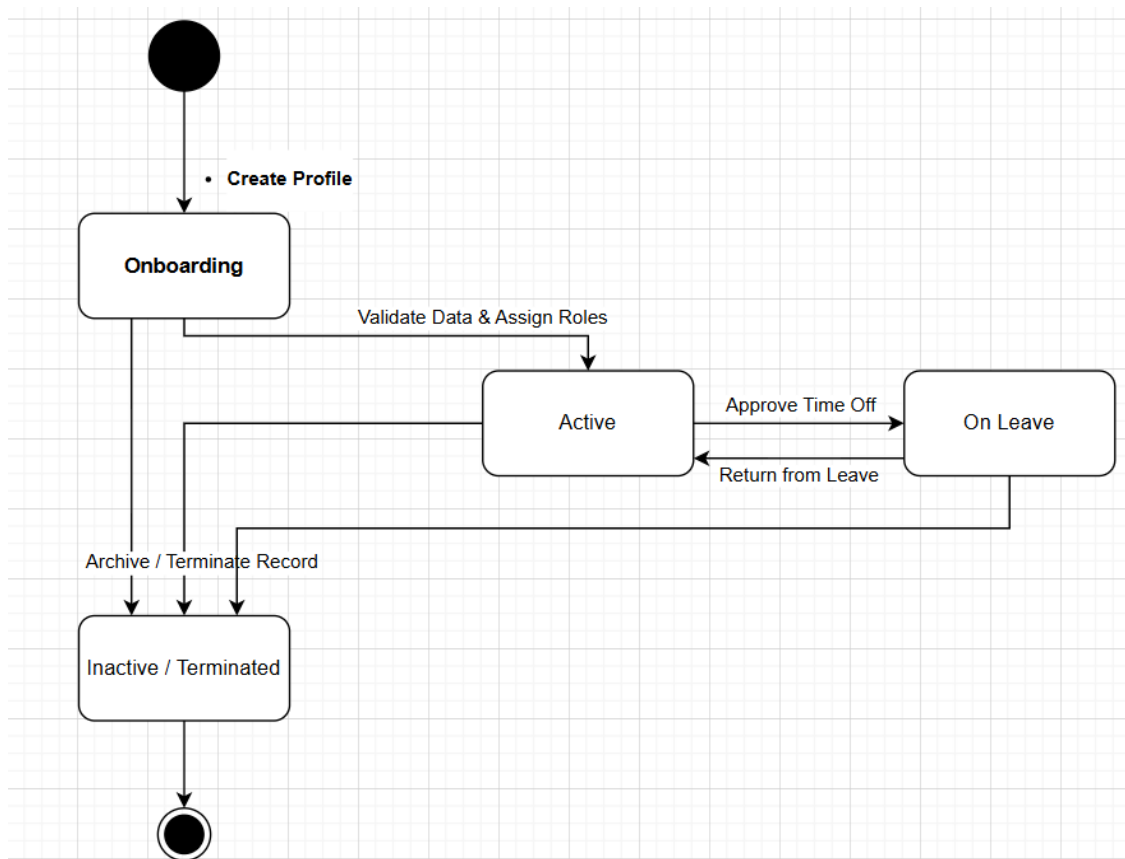


State Descriptions:

- **Open/New:** The ticket has been submitted by the passenger but not yet reviewed.
- **Pending Passenger Input:** The system or agent has detected invalid/missing data and is waiting for the passenger to provide corrections.
- **Assigned:** The system has generated a unique ID and routed the ticket to an available Customer Support Agent.
- **In Progress:** The support staff has accepted the ticket and is actively working on resolving the passenger's issue.
- **Resolved / Closed:** The issue has been fixed, the passenger is notified, and the chat history/ticket data is saved to the system archives.

1.2 Employee Record Lifecycle (Human Resource Module)

- **Core Purpose:** To model the operational states of an airport staff member's profile within the system, from initial hiring to departure.
- **Mapped Requirements:** UC_53 (Manage Airport Staff Records). Ensures HR strictly controls staff access, scheduling eligibility, and payroll status based on their current employment state.



State Descriptions & Transitions:

- **Onboarding (Initial State):** * Trigger: HR staff creates a new employee profile.
 - Description: The record is created, but the employee cannot yet be scheduled for shifts or processed for payroll. Awaiting document validation.
- **Active:** * Trigger: HR validates data and assigns staff roles.
 - Description: The employee is fully registered. They can now use the system to check in/out (UC_54) and are actively included in the payroll process (UC_55).
- **On Leave (Suspended State):** * Trigger: HR updates the schedule to reflect approved time off (vacation, medical leave).
 - Description: The employee remains in the system but is temporarily blocked from being assigned to daily operational shifts.
- **Inactive / Terminated (Final State):** * Trigger: HR deletes or archives the employee record.
 - Description: The employee no longer has system access. Historical payroll and attendance data are retained for auditing, but no new actions can be taken.

Airport Management System

Prepared by: Ameo Lulaj

Course: Software Modeling and Design

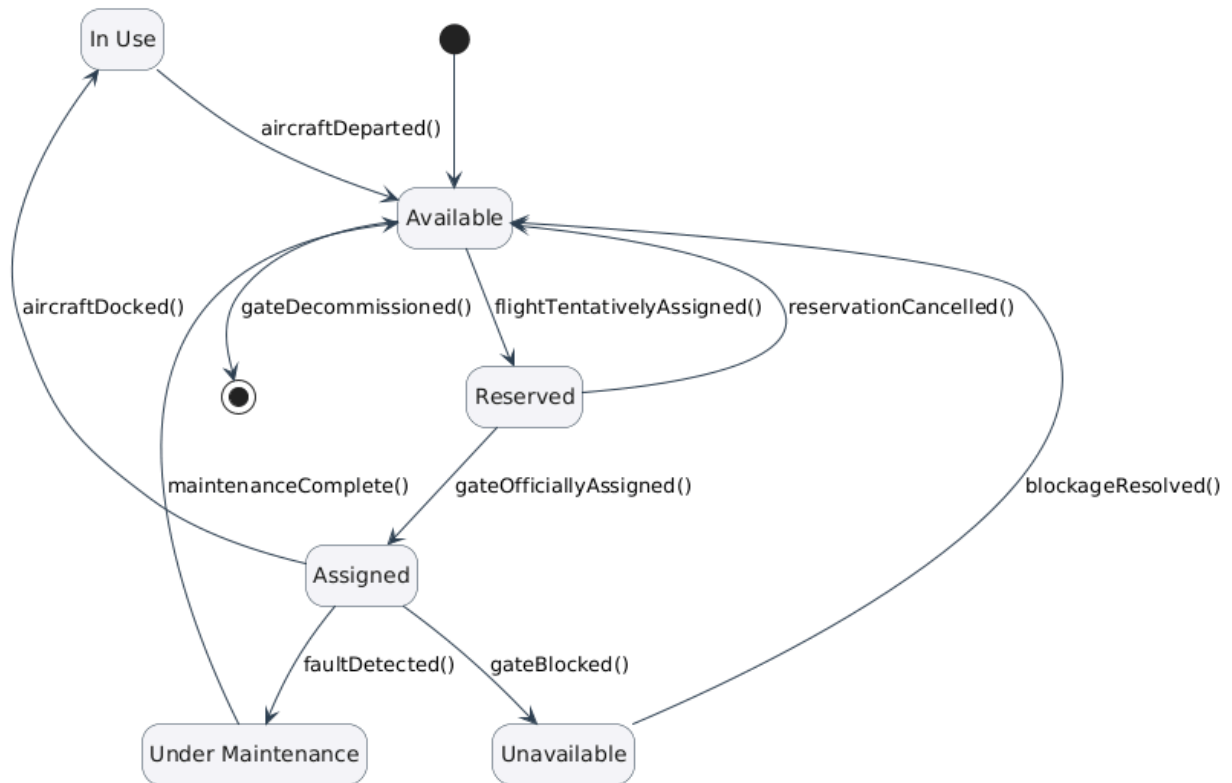
Document Overview:

This document presents the state machine models for the Airport Management System (AMS). It illustrates the exact states, events, and transitions of the system's main objects.

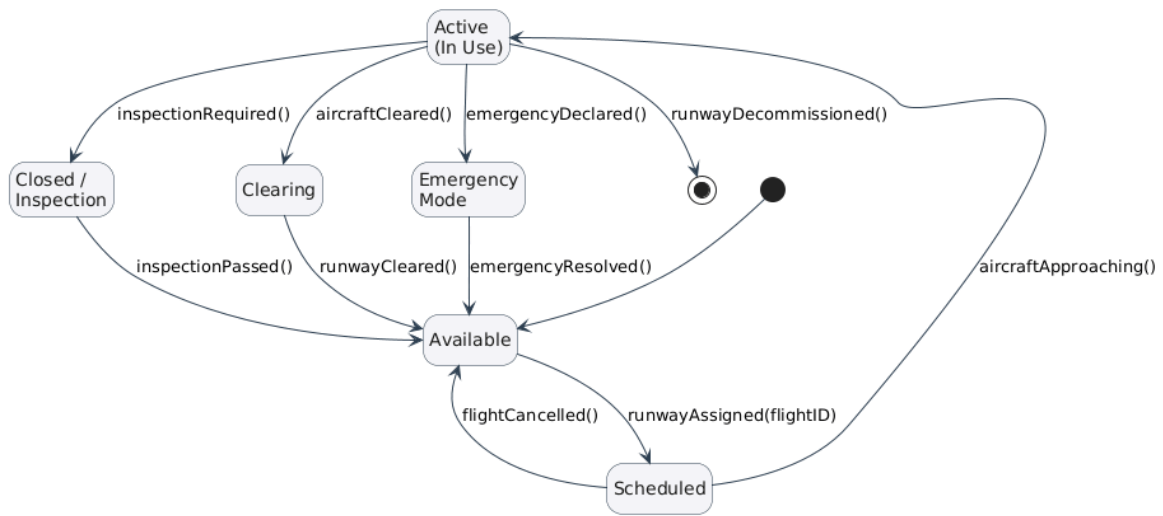
Objective :

The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements and serves as a foundation for implementation.

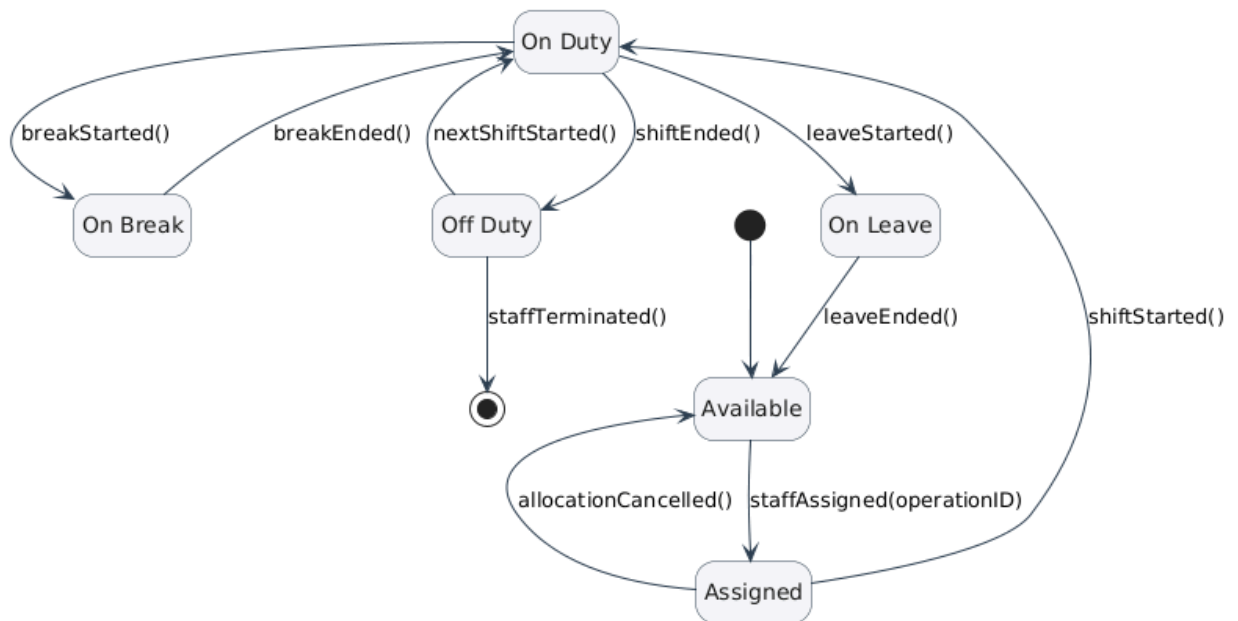
UC-ARM-01: Gate Allocation - State Diagram



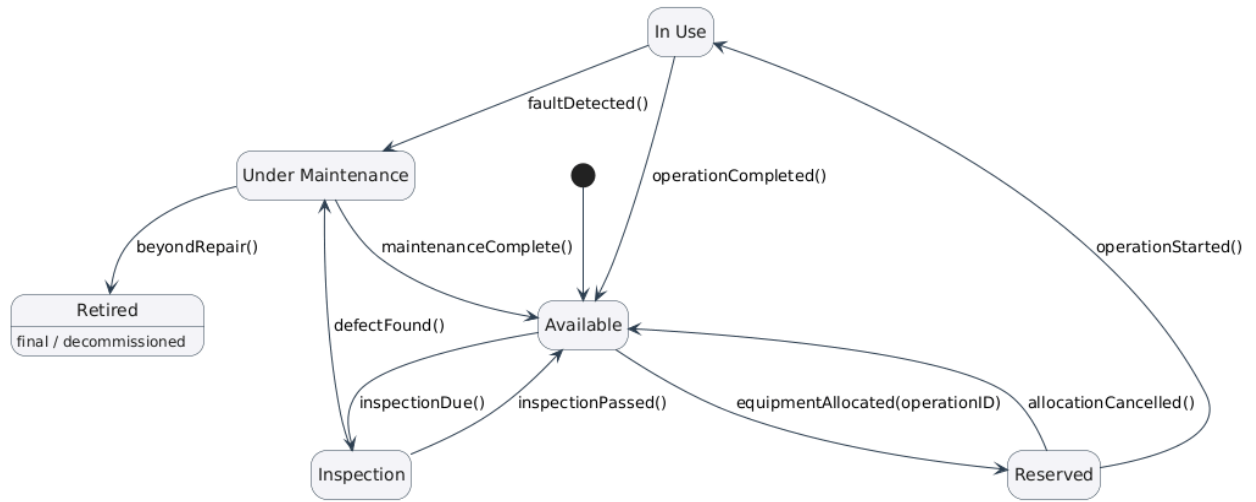
UC-ARM-02: Runway Scheduling - State Diagram



UC-ARM-03: Staff Allocation - State Diagram



UC-ARM-04: Equipment Management - State Diagram



Airport Management System

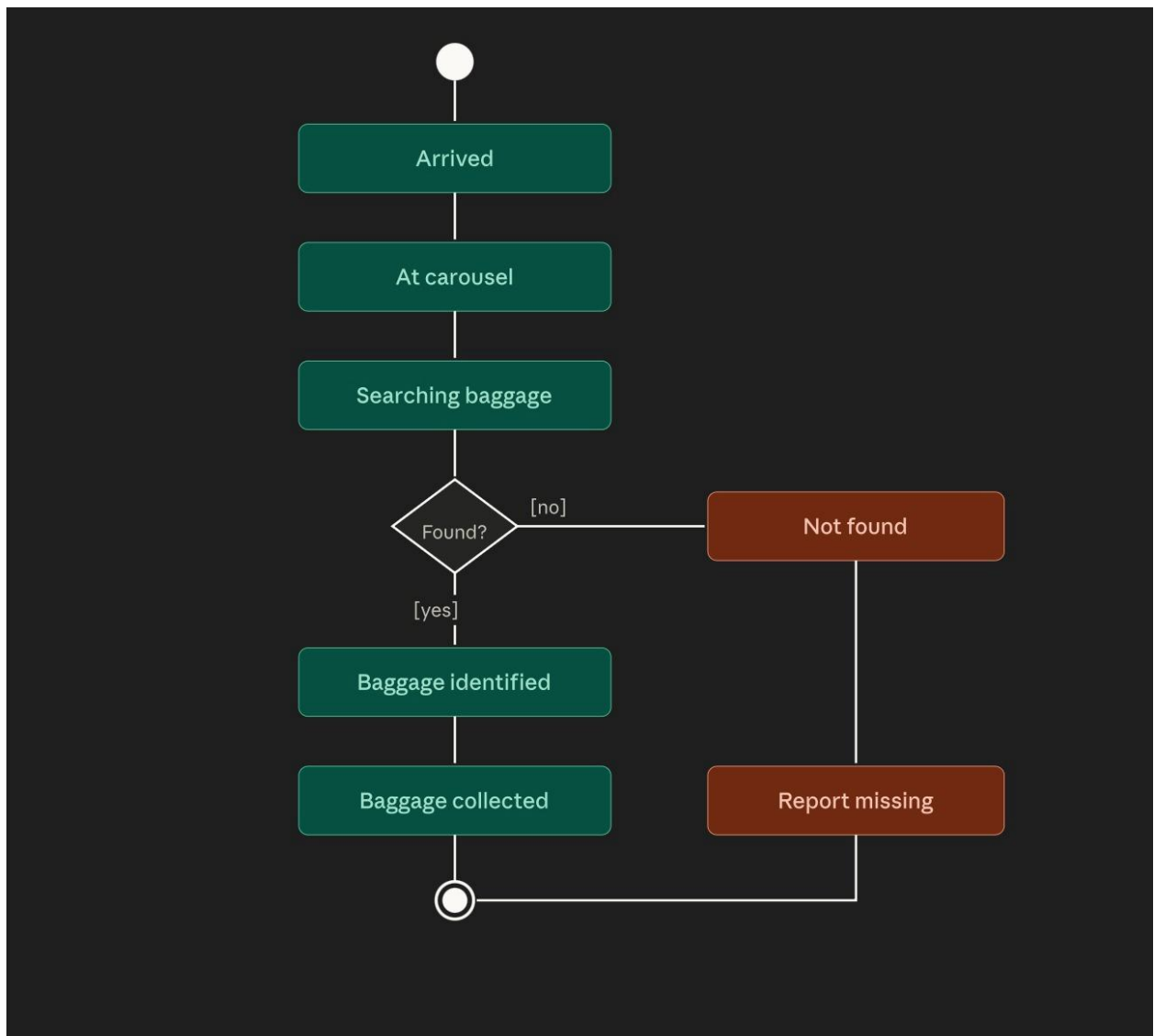
Prepared by: Albina Blloshmi

Course: Software Modeling and Design

UC-08: Claim Baggage at Destination

Core Purpose: To model the process of a passenger retrieving baggage upon arrival at the destination airport, ensuring efficient and accurate baggage claim handling.

Mapped Requirements: UC-08 (Claim Baggage at Destination). Ensures passengers can locate and collect their baggage or report issues if baggage is missing.

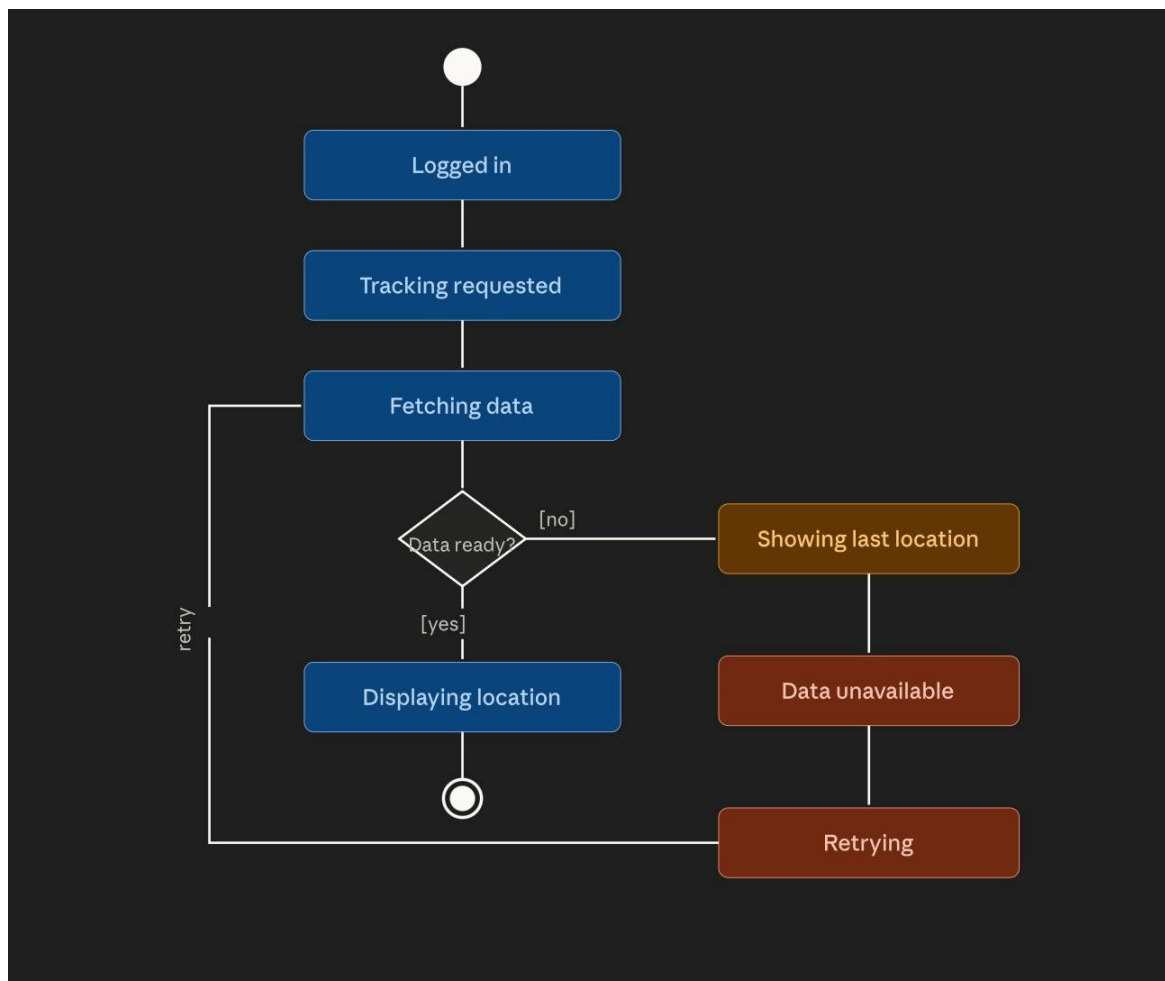


- *Arrived: The flight has landed and baggage is unloaded.*
- *At Carousel: The passenger is at the baggage claim area.*
- *Searching Baggage: The passenger looks for their baggage.*
- *Baggage Identified: The passenger recognizes their baggage.*
- *Baggage Collected: The passenger retrieves their baggage.*
- *Not Found: The passenger cannot locate their baggage.*
- *Report Missing: The passenger reports missing baggage to staff.*

UC-09: Track Baggage in Real-Time

Core Purpose: To model how passengers track the real-time location of their baggage throughout the journey using the system.

Mapped Requirements: UC-09 (Track Baggage in Real-Time). Ensures accurate tracking and visibility of baggage location.

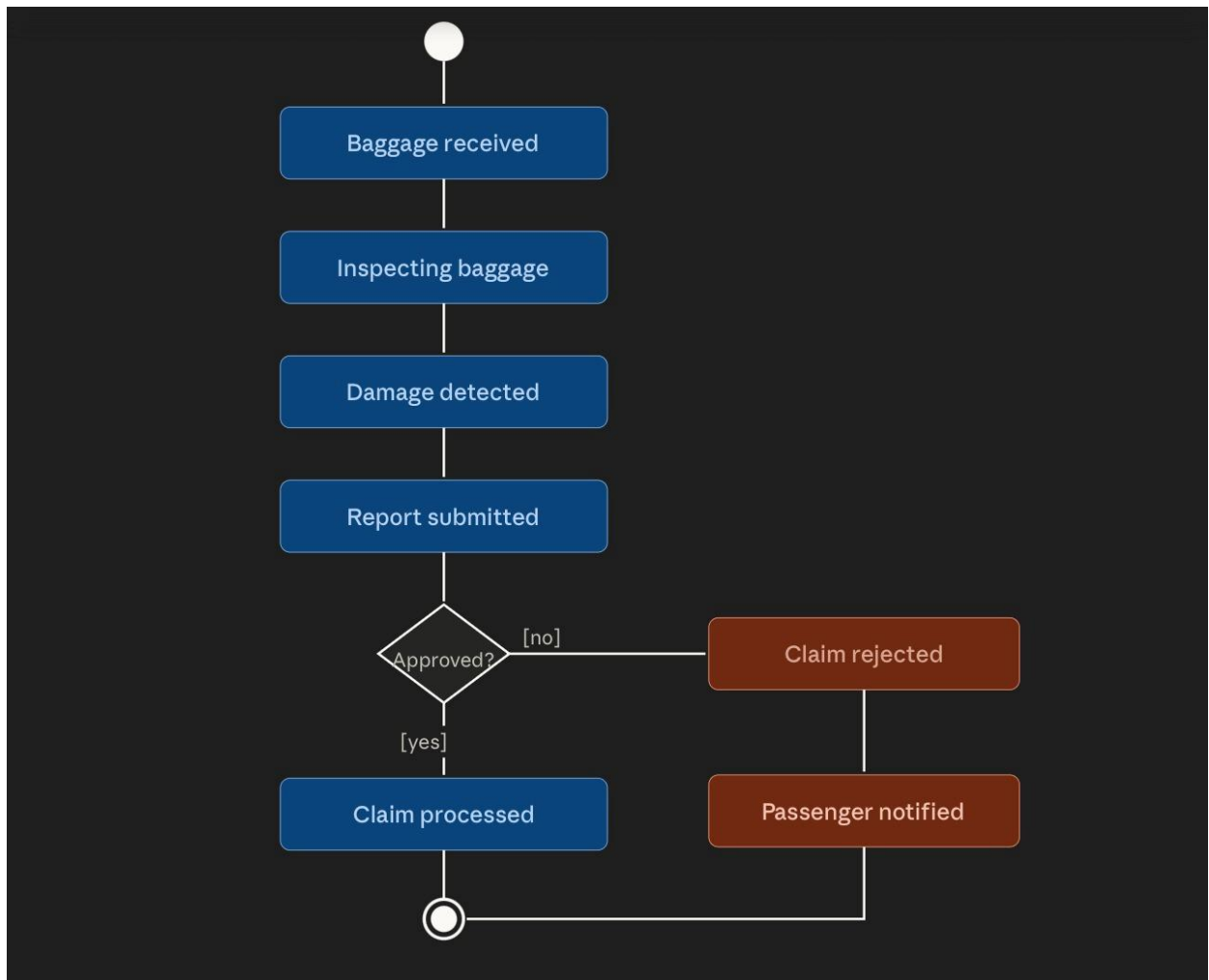


- *Logged In: The passenger accesses the system.*
- *Tracking Requested: The passenger selects tracking.*
- *Fetching Data: The system retrieves baggage location data.*
- *Displaying Location: The system shows the baggage location.*
- *Data Unavailable: Real-time data cannot be retrieved.*
- *Showing Last Location: The system displays last known location.*
- *Retrying: The system attempts to fetch data again.*

UC-10: Report Damaged Baggage

Core Purpose: To model the process of reporting and handling damaged baggage after arrival, ensuring proper verification and claim processing.

Mapped Requirements: UC-10 (Report Damaged Baggage). Ensures damage reports are recorded, verified, and processed efficiently.

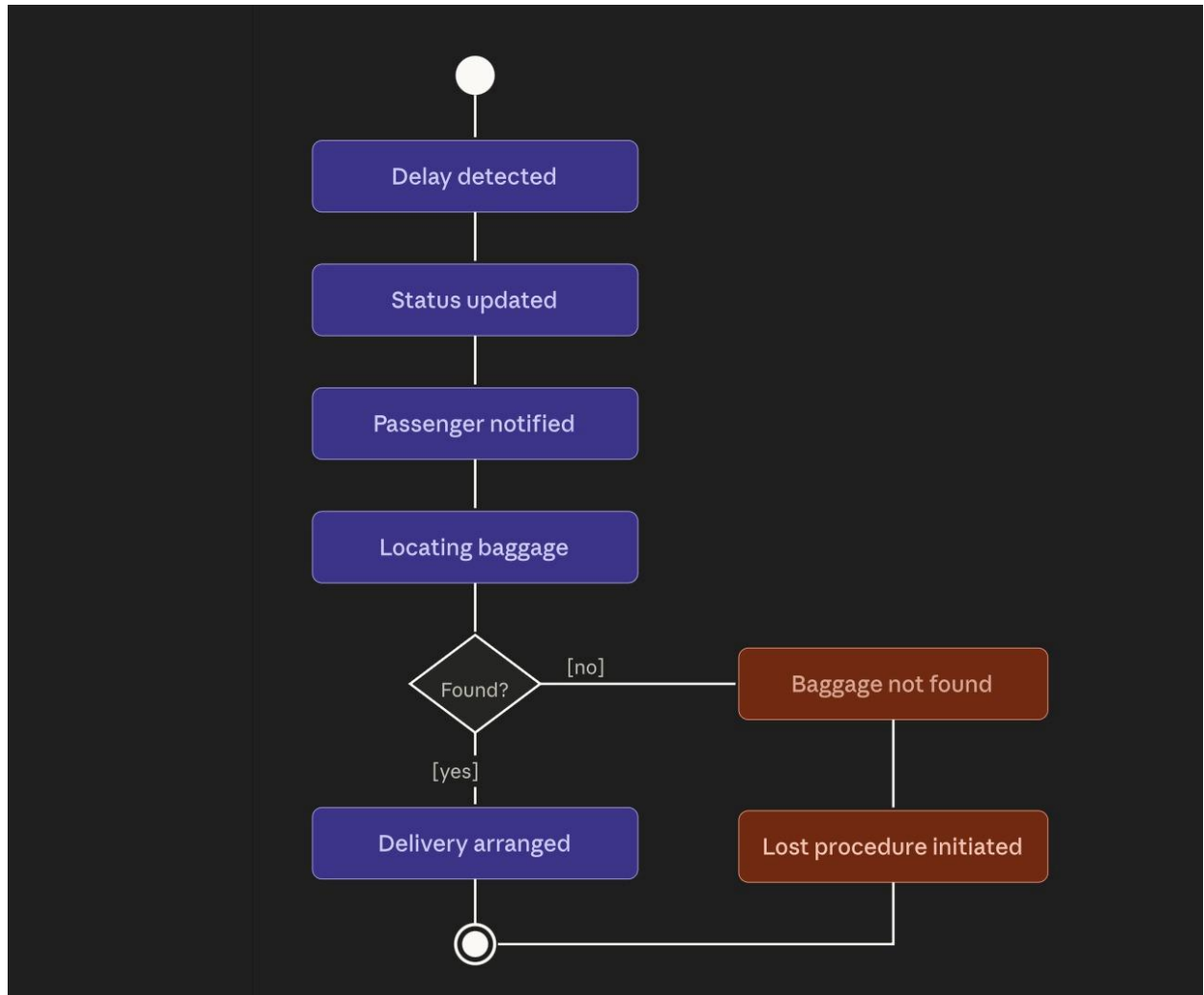


- *Baggage Received:* The passenger has received their baggage.
- *Inspecting Baggage:* The passenger checks baggage condition.
- *Damage Detected:* Damage is identified.
- *Report Submitted:* The passenger submits a damage report.
- *Verification:* Staff verifies the reported damage.
- *Claim Processed:* The claim is approved and processed.
- *Claim Rejected:* The claim is denied due to invalid proof.
- *Passenger Notified:* The passenger is informed of the result.

UC-11: Handle Delayed Baggage

Core Purpose: To model how the system detects, manages, and resolves delayed baggage cases, ensuring timely updates and resolution.

Mapped Requirements: UC-11 (Handle Delayed Baggage). Ensures delayed baggage is tracked, communicated, and resolved efficiently.

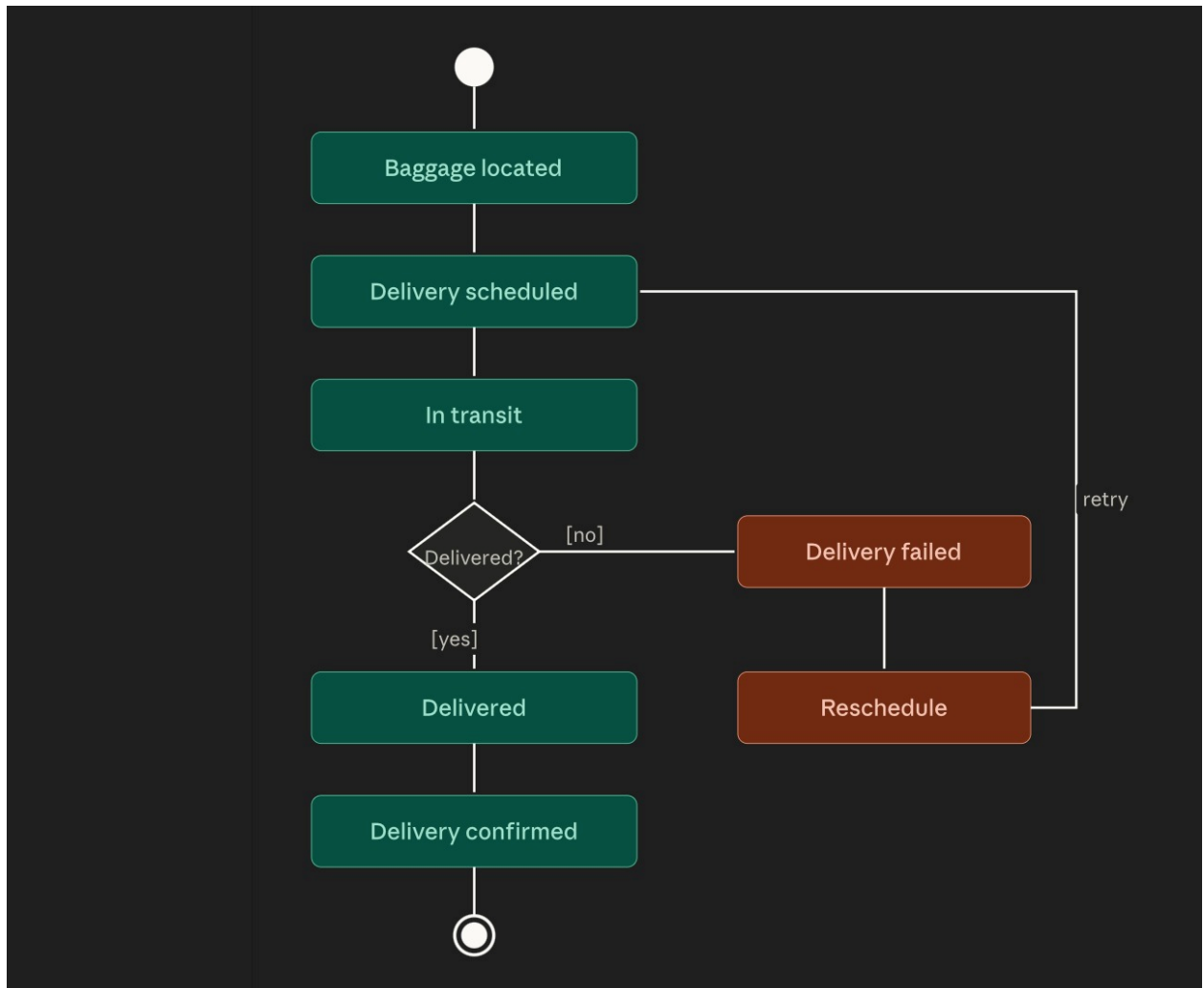


- *Delay Detected: The system identifies baggage delay.*
- *Status Updated: The system updates baggage status.*
- *Passenger Notified: The passenger is informed of the delay.*
- *Locating Baggage: The system attempts to locate baggage.*
- *Delivery Arranged: Delivery is scheduled.*
- *Baggage Not Found: The baggage cannot be located.*
- *Lost Procedure Initiated: Lost baggage handling begins.*

UC-12: Deliver Baggage to Passenger

Core Purpose: To model the process of delivering baggage to passengers after delay or rerouting, ensuring successful and secure delivery.

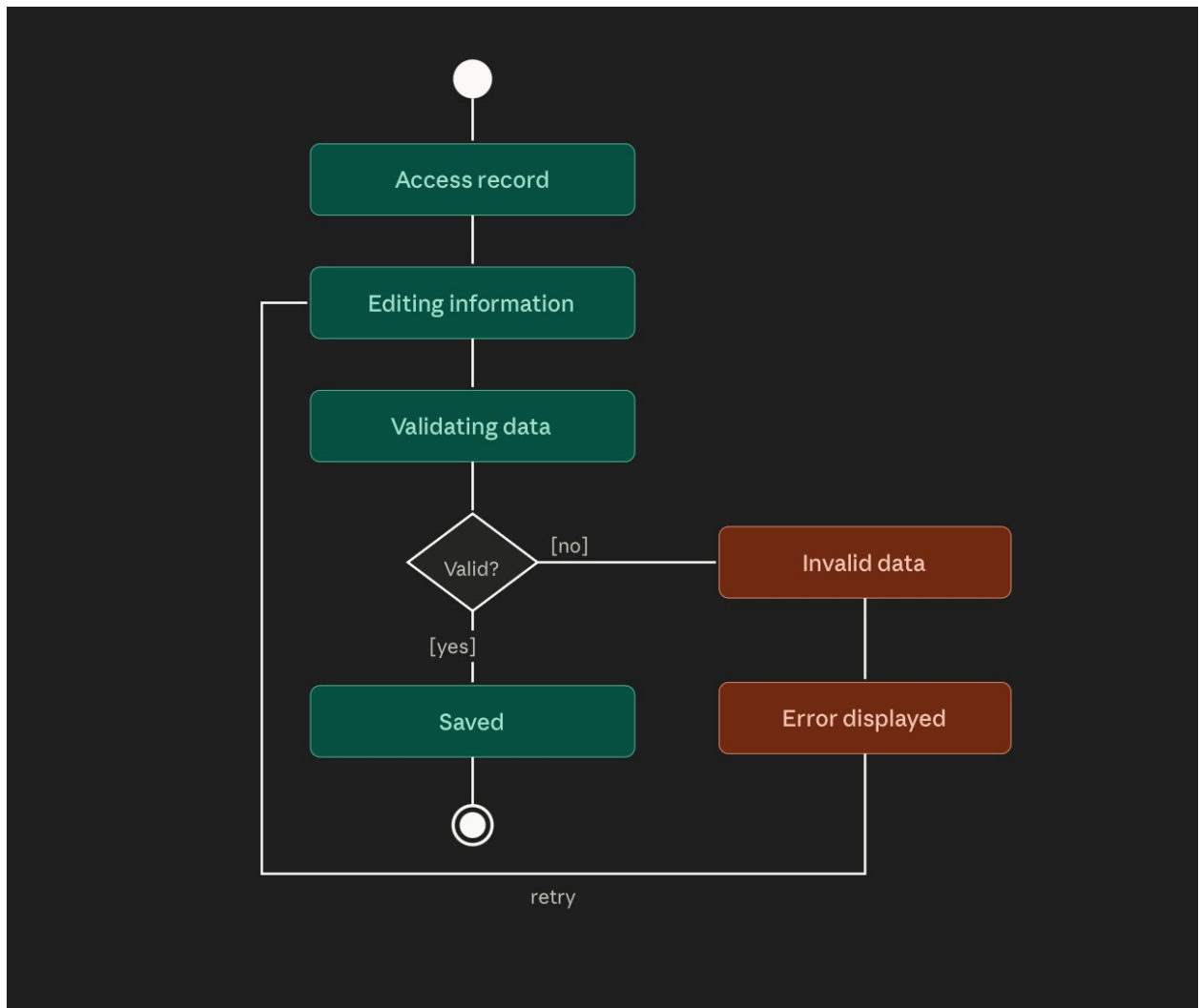
Mapped Requirements: UC-12 (Deliver Baggage to Passenger). Ensures baggage is delivered efficiently and receipt is confirmed.



UC-13: Update Baggage Information

Core Purpose: To model how staff update baggage information in the system, ensuring data accuracy and system consistency.

Mapped Requirements: UC-13 (Update Baggage Information). Ensures baggage data is properly maintained and updated.



- *Access Record: Staff opens baggage record.*
- *Editing Information: Staff updates data.*
- *Validating Data: The system checks data accuracy.*
- *Saved: Changes are successfully stored.*
- *Invalid Data: Incorrect input is detected.*
- *Error Displayed: The system shows an error message.*

Airport Management System

Prepared by: Eneida Balliu

Course: Software Modeling and Design

Document Overview:

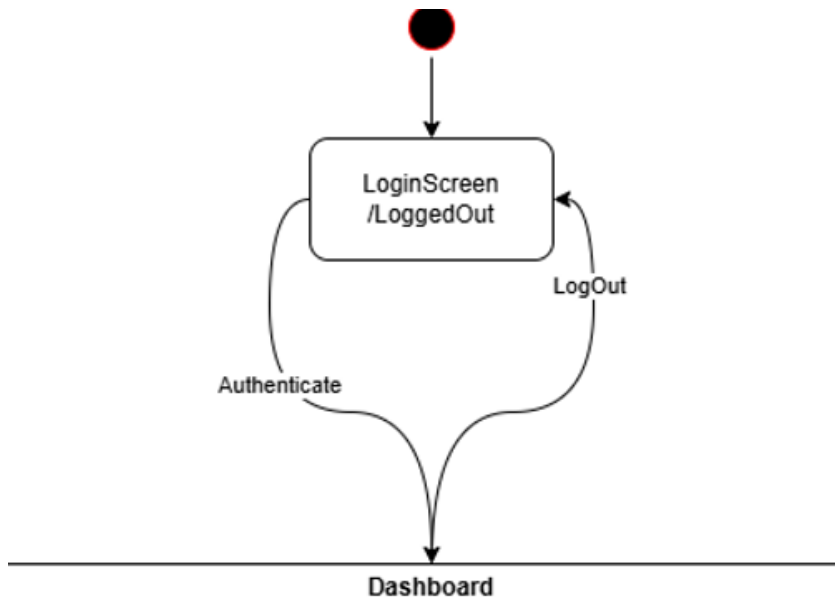
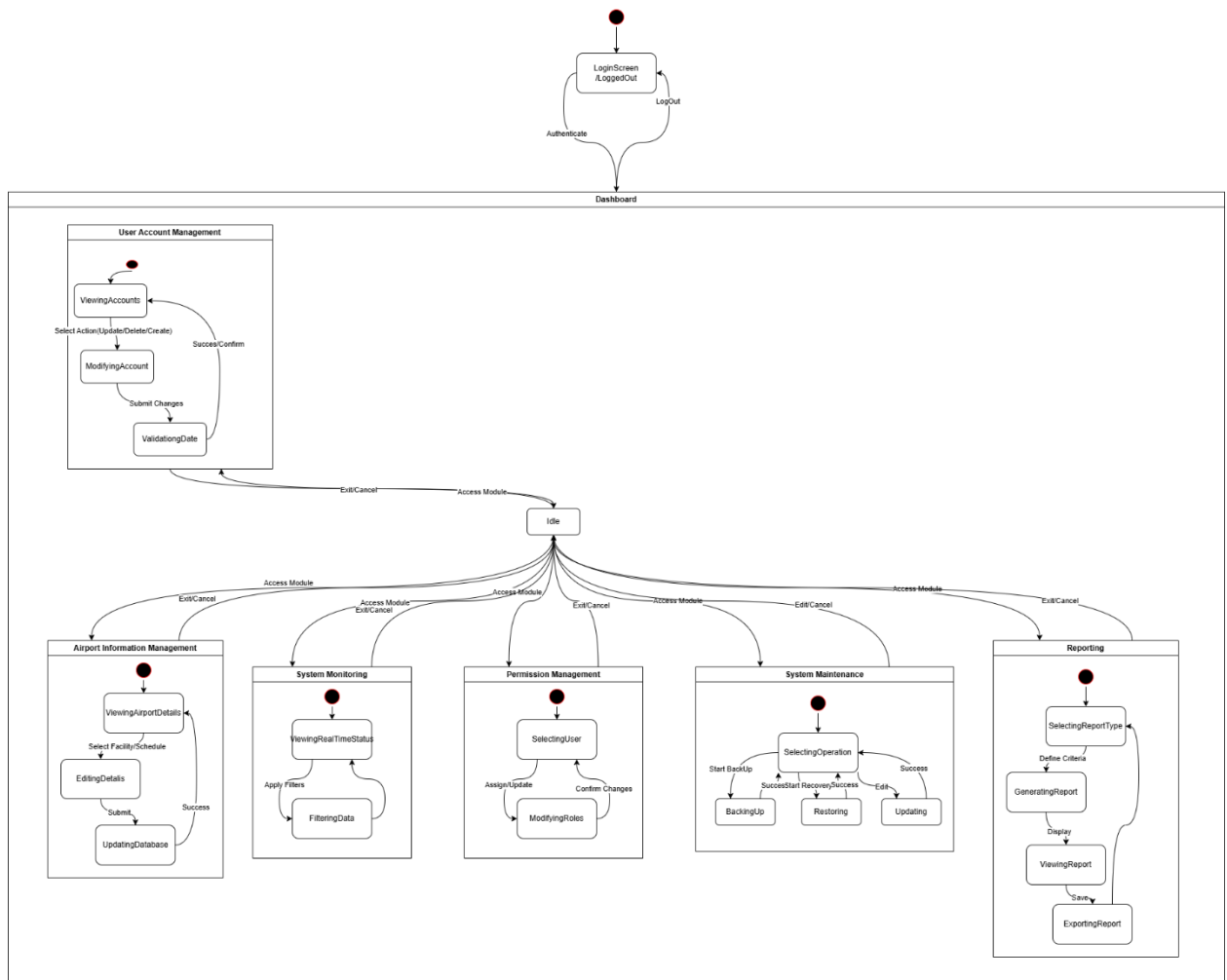
This document presents the state machine models for the Airport Management System (AMS). It illustrates the exact states, events, and transitions of the system's main objects.

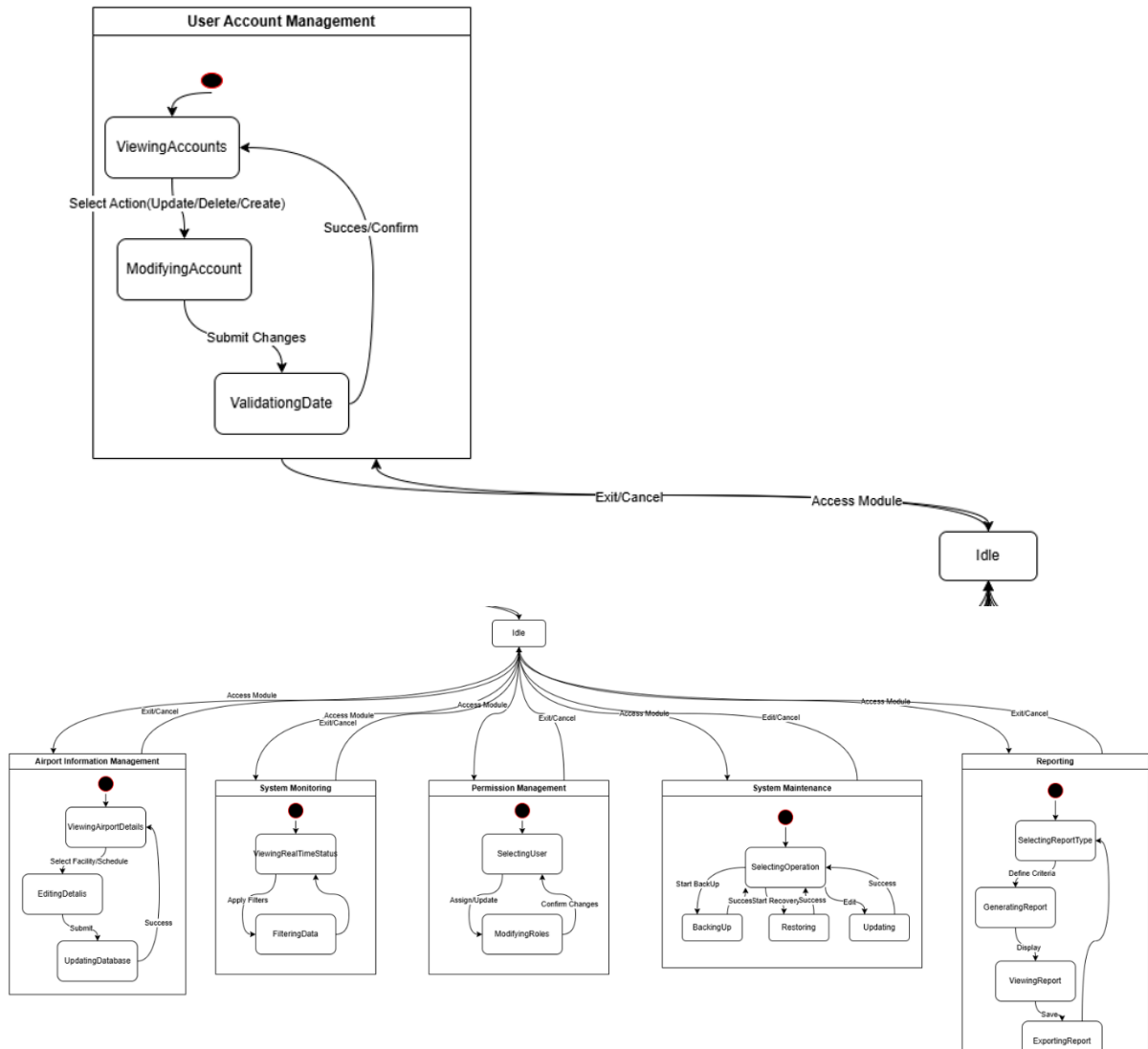
Objective :

The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements and serves as a foundation for implementation.

State Machine Diagrams

This section defines the dynamic behavior and state transitions for the core system objects managed by System Administrator.





This state machine diagram models the dynamic behavior of the Airport Management System from the perspective of the System Administrator, covering the administrative modules defined in the project requirements.

Functional Logic:

1. **Authentication & Session Control:** The system begins in the LoggedOut state. Access to the Dashboard (the main composite state) is only granted after a successful authenticate event. This ensures the security precondition is met for all administrative tasks.
2. **The Idle State (Main Menu):** Once inside the Dashboard, the system resides in the Idle state. This acts as a central hub where the administrator chooses which module to access. Every module includes an Exit/Cancel transition that returns the system to this Idle state without saving changes, as per the alternative sequences.
3. **Module Sub-States:**
 1. **User & Permission Management (UC_40, UC_43):** These states handle the lifecycle of user accounts and access rights. They include a ValidatingData state to ensure information accuracy before the database is updated.
 2. **Airport Info Management (UC_41):** This state allows for the modification of airport facilities and schedules, moving from an editing phase to a database update phase.

3. *Monitoring & Reporting (UC_42, UC_45)*: These states focus on data retrieval. The system transitions through filtering, generating, and finally Exporting reports in formats like PDF or CSV.
4. *System Maintenance (UC_44)*: This complex state manages critical operations including Data Backup, Data Recovery, and Configuration Updates.
4. Error Handling (Alternative Flows): The diagram accounts for "Alternative Sequences" described in the documentation, such as invalid data entries or database failures. In these cases, the system transitions to an error state or requests correction instead of confirming the operation.
5. Post-conditions: Successful transitions out of a functional state result in the database being updated and the activity being recorded in the System Logs for auditing purposes.

Airport Management System

Prepared by: Andri Mucllari

Course: Software Modeling and Design

Document Overview:

This document presents the state machine diagrams for the Airport Management System (AMS). It shows how key system components behave by outlining their states and the transitions between them.

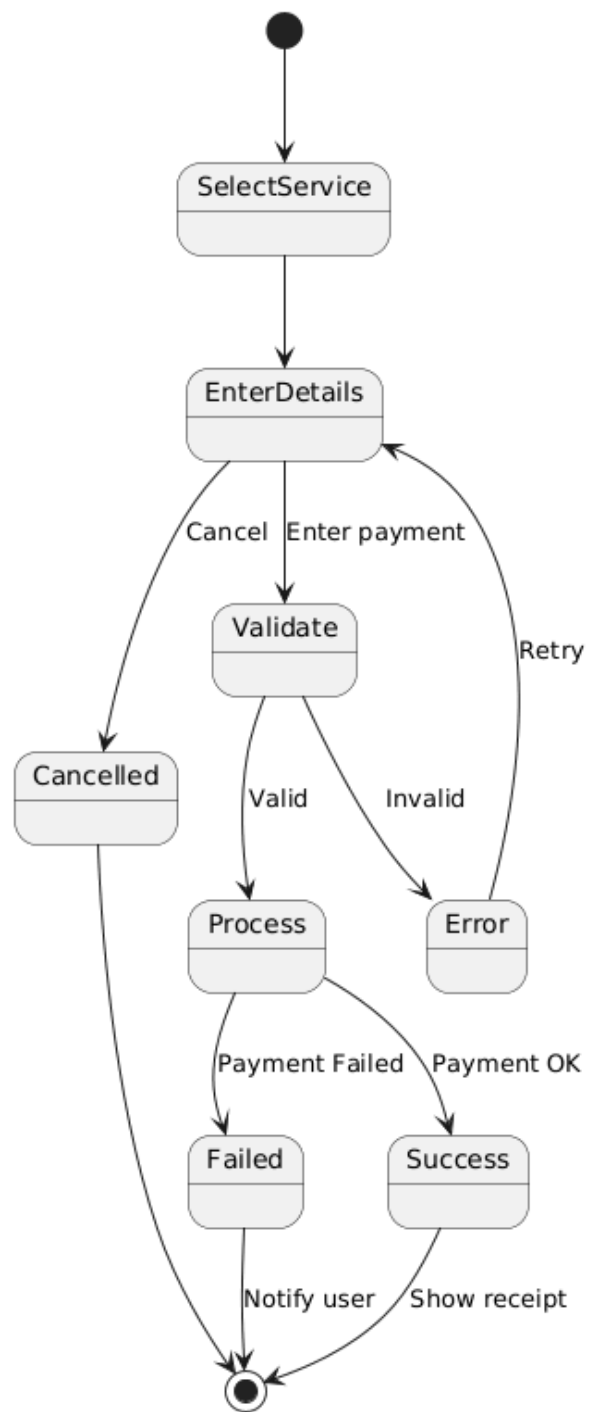
Objective :

The objective of this document is to represent the system's behavior in a clear and structured way. It focuses on modeling workflows and object lifecycles to ensure the system operates correctly and meets its functional requirement.

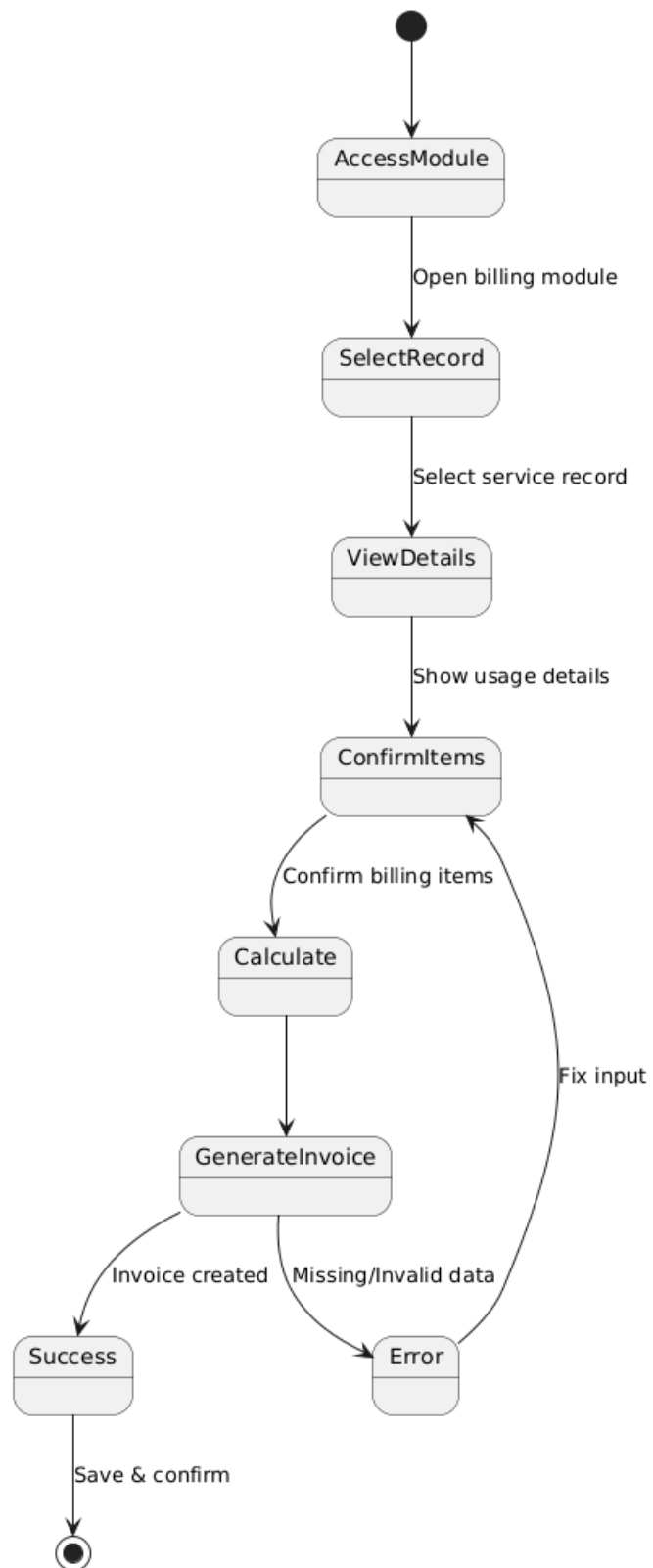
State Machine Diagrams

This section describes the dynamic behavior of the main system entities by illustrating their possible states and transitions. The diagrams focus on core processes within the financial and service management modules of the system.

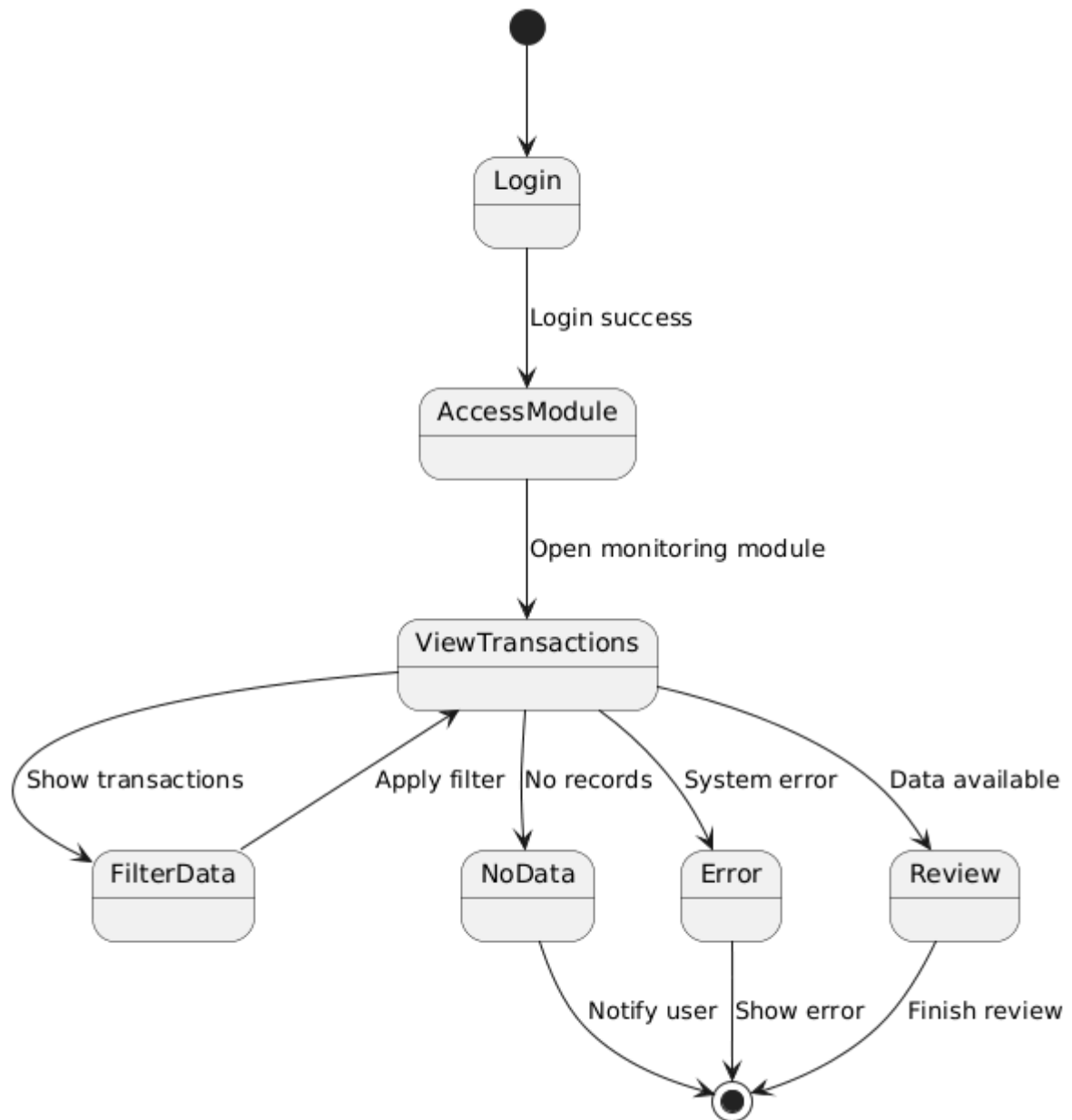
Processing Airport Service Payments (UC_50)



Generating Airport Service Invoices (UC_51)



Monitoring Financial Transactions (UC_52)



Airport Management System

Prepared by: Alvi Elezi

Course:
Software Modeling and Design

Document Overview:

This document presents the state machine models for the Airport Management System (AMS). It illustrates the exact states, events, and transitions of the system's main objects.

Objective:

The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements and serves as a foundation for implementation.

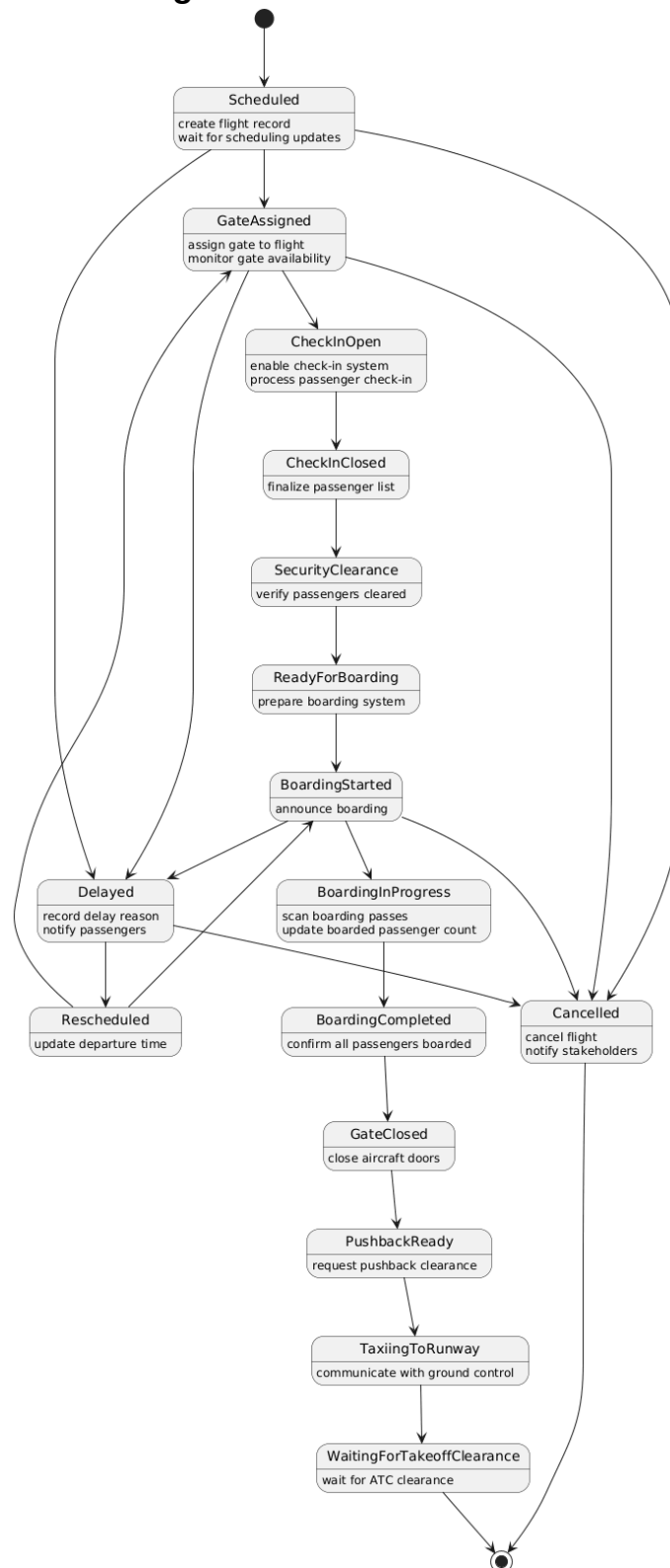
State Machine Diagrams

This section defines the dynamic behavior and state transitions for the core system objects managed by the Flight Operations module.

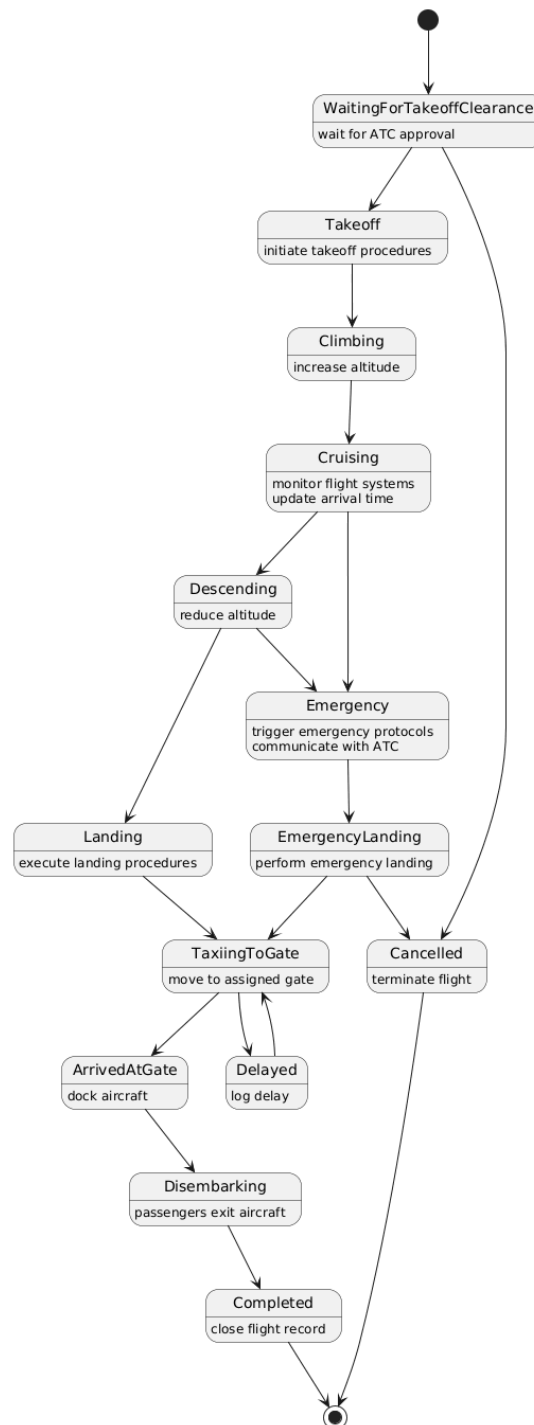
How the module is divided:

The Flight Operations state model is divided into two diagrams to improve readability and maintainability. The first diagram represents pre-flight and ground operations, including scheduling, check-in, boarding, and taxiing. The second diagram represents in-flight operations and arrival processes, including takeoff, cruising, landing, and exception handling such as delays and emergencies.

Pre - Flight operations diagram:



Flight execution diagram:



Airport Management System

Prepared by: Ani Velo

Course: Software Modeling and Design

Document Overview:

*This document presents the state machine models for the **Security Management System (SMS)** within the Airport Management System. It illustrates the exact states, events, and transitions of the system's core security objects—specifically the **Access Request** and the **Security Incident**.*

Objective :

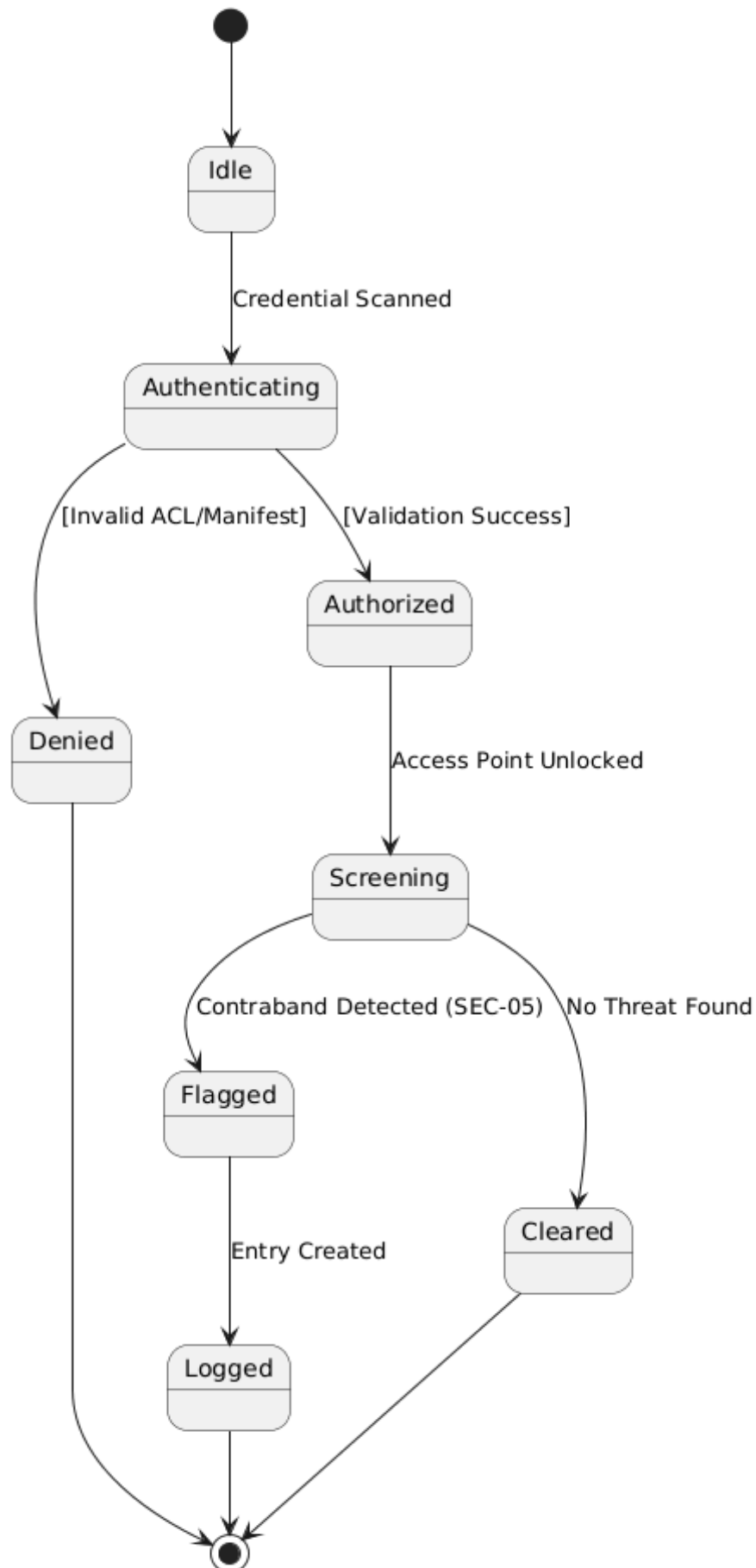
The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements. It focuses on ensuring that security protocols, such as identity validation and emergency lockdowns, are executed in a structured, fail-safe manner that serves as a foundation for implementation.

4.1 Access Control Lifecycle (Passenger & Staff Roles)

- **Core Purpose:** To model the exact states an access request passes through, from initial credential scan by a passenger or staff member to final clearance or denial.
- **Mapped Requirements:** Covers **UC-SEC-01** (Passenger Screening), **UC-SEC-02** (Area Access), and **UC-SEC-05** (Prohibited Item Log).

State Descriptions:

- **Idle:** The system is in a ready state, waiting for a hardware scan at a checkpoint.
- **Authenticating:** The system is actively validating scanned ID/Manifest data against the database.
- **Authorized:** Validation is successful; the system has triggered the physical gate to unlock.
- **Screening:** A physical search is being performed; the system is waiting for the Officer to input results.
- **Flagged:** The system has detected a threat/contraband and has initialized a Seizure Log (UC-SEC-05).
- **Cleared/Denied:** Final terminal states where the access attempt is recorded and closed.



4.2 Security Incident Lifecycle (Officer & Supervisor Roles)

- *Core Purpose:* To define the operational statuses of a security threat to ensure no incident is closed without proper oversight, evidence logging, and 2-Factor Authentication (2FA).
- *Mapped Requirements:* To define the operational statuses of a security threat to ensure no incident is closed without proper oversight, evidence logging, and 2-Factor Authentication (2FA).

State Descriptions:

- **Reported:** A new incident entry has been initialized in the system (UC-SEC-03).
- **Investigating:** An Officer ID has been linked to the ticket, and surveillance feeds are being reviewed.
- **Escalated:** The priority attribute has been set to "Critical," moving the system into high-alert.
- **Lockdown Pending:** The system is issuing a 2FA challenge to the Supervisor to authorize facility-wide locking.
- **Active Lockdown:** The system has successfully verified credentials and transmitted "Lock" signals to all hardware (UC-SEC-06).
- **Resolved/Archived:** The incident is closed with a digital signature and moved to secure 30-day storage.

Airport Management System

Prepared by: Betül Kaan

Course: Software Modeling and Design

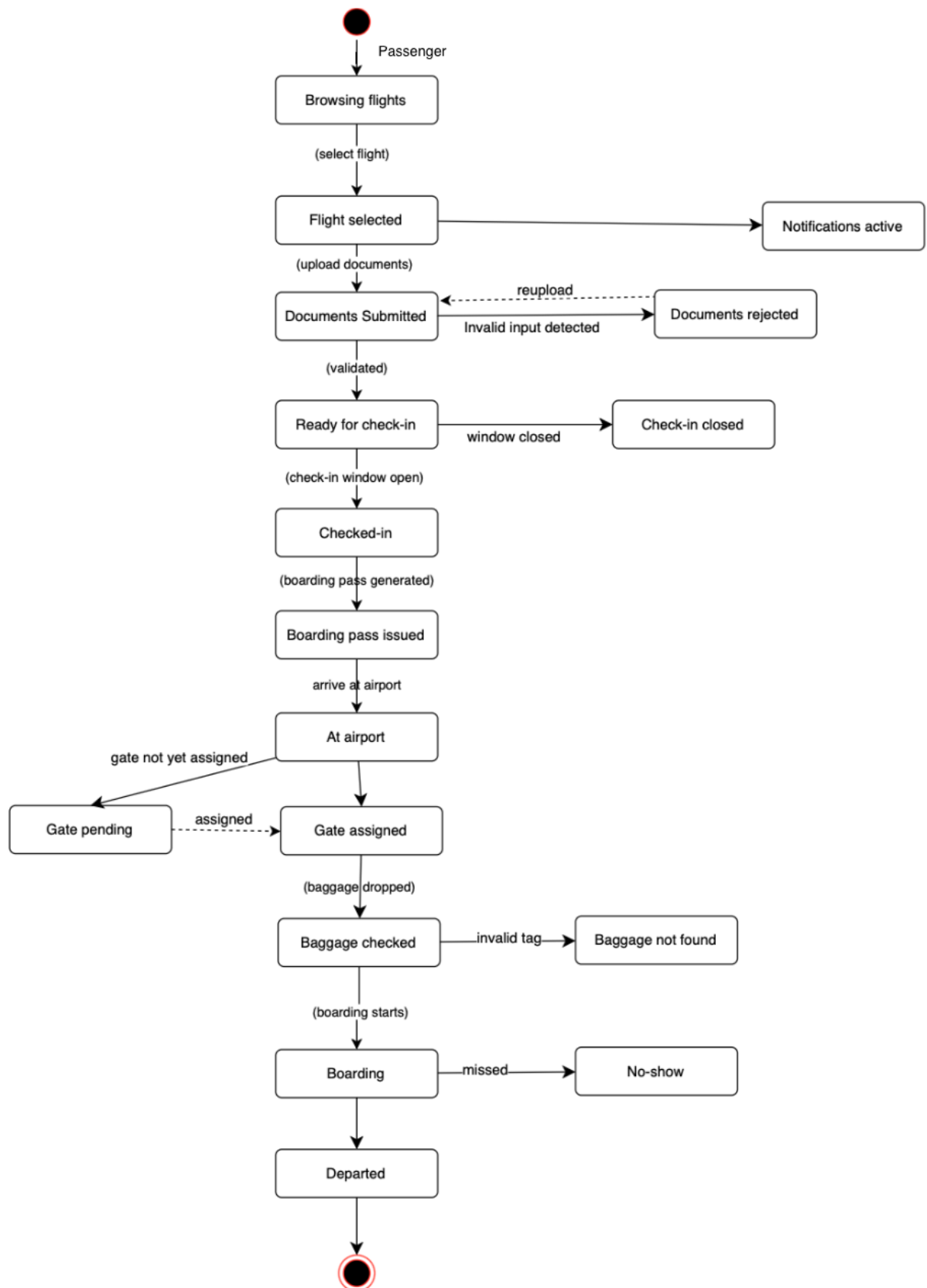
Document Overview:

This document presents the state machine models for the Airport Management System (AMS). It illustrates the exact states, events, and transitions of the system's main objects.

Objective :

The goal is to clearly map the behavioral workflows and object lifecycles to ensure the system design logically fulfills its operational requirements and serves as a foundation for implementation.

This section defines the dynamic behavior and state transitions for the core system objects managed within the System Administration models for the Airport Management System.



5.1 Passenger Service Lifecycle (Passenger Service Module)

- Core Purpose:** To model the exact states a passenger passes through from initial flight browsing to final departure, capturing both the successful journey and all exception flows arising from document validation, check-in timing, baggage handling, and gate assignment.

Mapped Requirements: Ensures a passenger can successfully search flights, submit and validate travel documents, complete online check-in, download a boarding pass, track baggage, check gate information, and board their flight — with alternative paths handled at every critical point.

State Descriptions:

- **Browsing flights:** The passenger has accessed the system and is searching for available flights by date, destination, or flight number (UC-01).
- **Flight selected:** The passenger has chosen a specific flight and holds a valid booking reference.
- **Notifications active:** The passenger has opted in to receive real-time alerts for delays, gate changes, and boarding calls. This state runs in parallel and can be entered as soon as a flight is selected (UC-07).
- **Documents submitted:** The passenger has uploaded required travel documents such as a passport or visa to the system (UC-02).
- **Documents rejected:** The uploaded file was in an unsupported format or failed validation. The passenger must re-upload corrected documents before proceeding.
- **Ready for check-in:** Travel documents have been validated and the passenger is eligible to begin online check-in once the check-in window opens.
- **Check-in closed:** The passenger attempted check-in outside the permitted time window. The passenger is redirected to contact customer support (UC-03 alternative).
- **Checked-in:** The passenger has successfully completed online check-in and their status is confirmed in the system (UC-03).
- **Boarding pass downloaded:** The passenger has downloaded their boarding pass as a PDF or image file, ready for use at the airport (UC-04).
- **At airport:** The passenger has physically arrived at the airport on the day of travel.
- **Gate pending:** A gate has not yet been assigned to the flight. The passenger is advised to check back or rely on notifications (UC-05 alternative).
- **Gate assigned:** The departure gate and terminal details are confirmed and visible to the passenger (UC-05).
- **Baggage checked:** The passenger's luggage has been dropped off and registered in the system by airport staff (UC-06).
- **Baggage not found:** The baggage tag number entered was invalid or unrecognized. The passenger is prompted to contact support (UC-06 alternative).
- **Boarding:** The boarding call has been issued and the passenger is in the process of boarding the aircraft.
- **No-show:** The passenger failed to board before the gate closed, resulting in a terminal exception state.
- **Departed:** The passenger has successfully boarded and the flight has departed. This is the final successful end state.

Airport Management System

Prepared by: Eneida Balliu, Betül Kaan, and Ajla Alcani

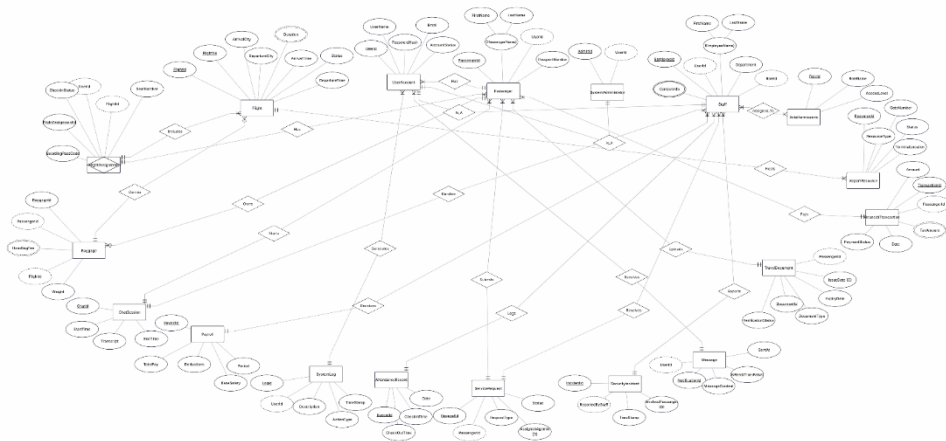
Course: Software Modeling and Design

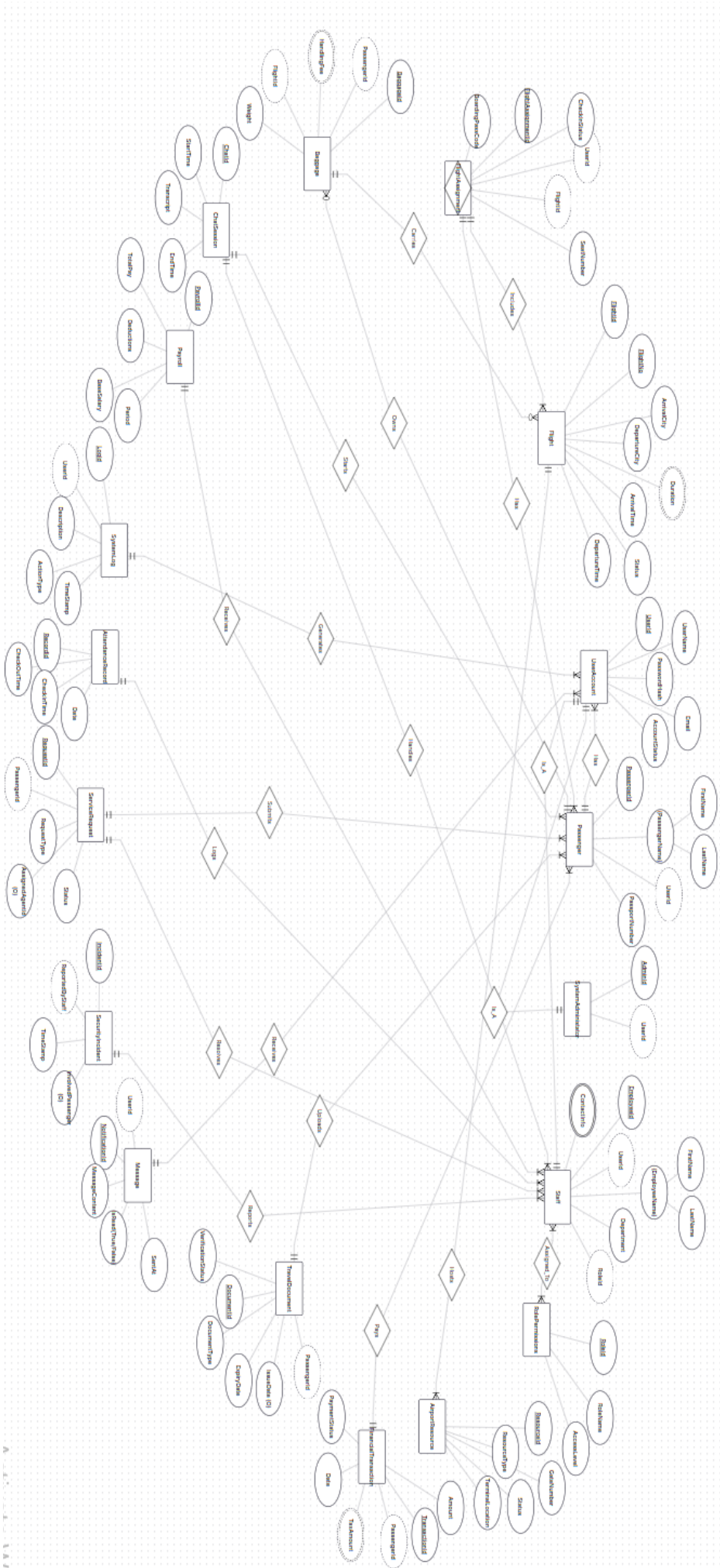
Document Overview

This document presents the state machine models and the comprehensive Entity Relationship Diagram (ERD) for the Airport Management System (AMS). It illustrates the exact states and transitions of the system's main objects, as well as the complete database architecture across all modules.

Objective

The goal is to clearly map the behavioral workflows and structural data models to ensure the system design logically fulfills its operational requirements and serves as a strong foundation for implementation.





Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Eneida Balliu

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the System Administration Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: The System Administrator.
2. Boundary Objects: User Interfaces (UIs) used for data input and display.
3. Control Objects: Controllers that orchestrate business logic and validation.
4. Entity Objects: The database layer including User Accounts, System Logs, and Resources.

System Interaction Standards

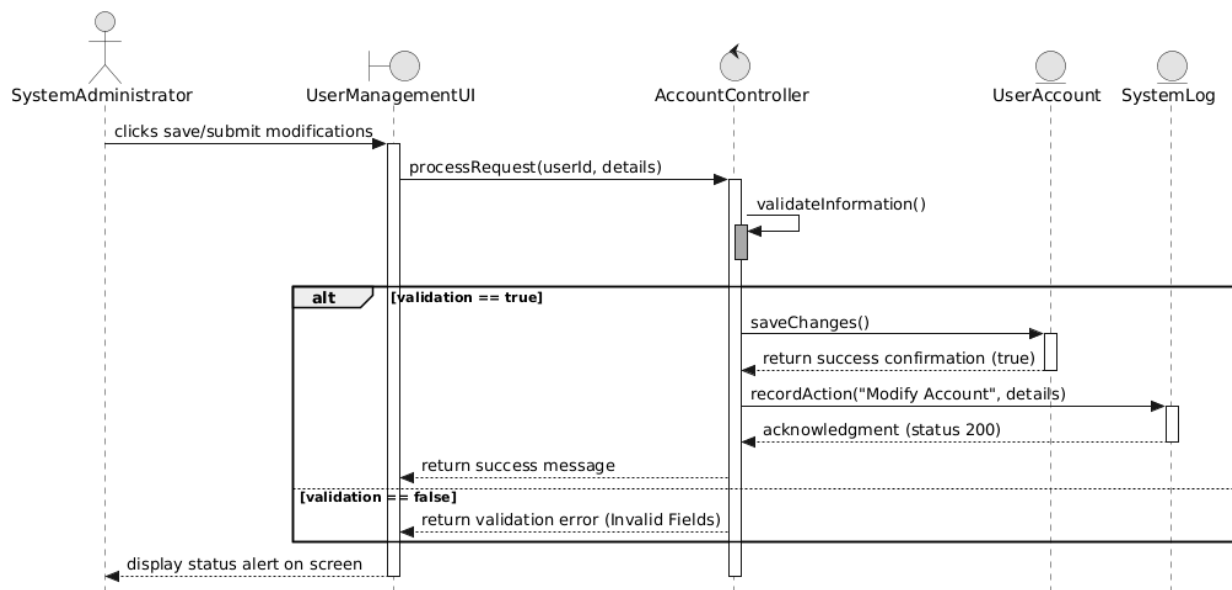
To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

1. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
2. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
3. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
4. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
5. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

UC_40: Manage User Accounts

Focus: Account Lifecycle

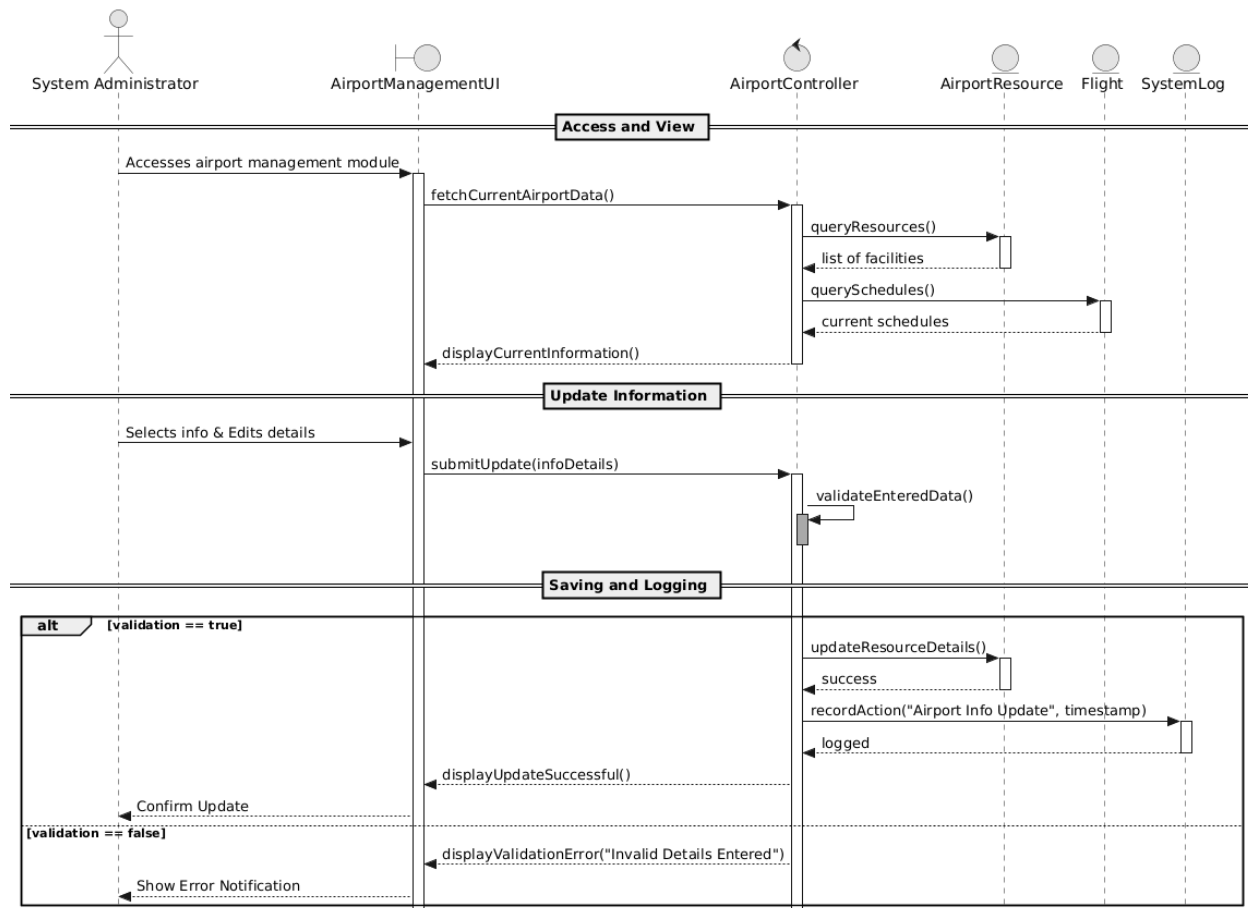
- Description: This diagram illustrates the standard process for creating, updating, or deleting user accounts. It demonstrates the interaction between the Administrator and the core identity management system.
- Key Logic: The AccountController performs a self-validation check on user data before committing any changes to the UserAccount entity.
- Compliance: The flow concludes with a mandatory update to the SystemLog, ensuring all administrative changes are audited.



UC_41: Update Airport Information

Focus: Resource and Schedule Maintenance

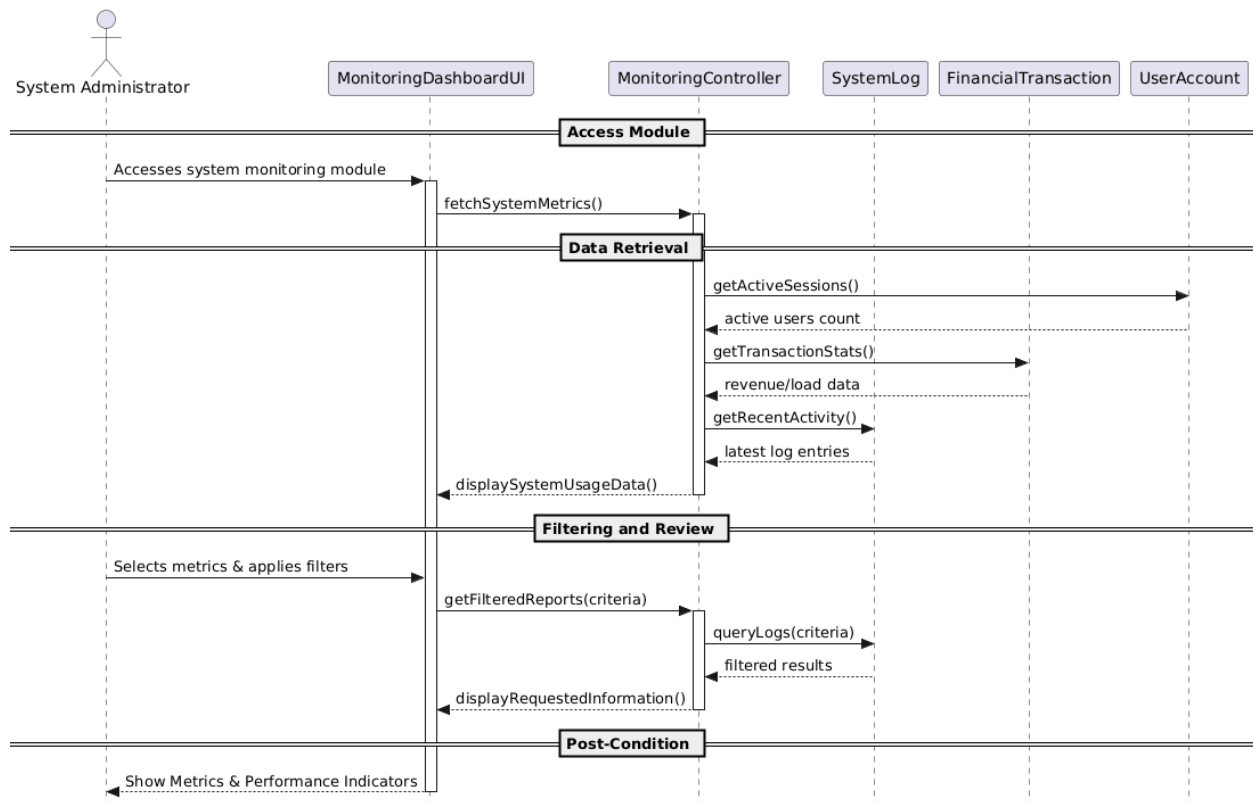
- Description: This sequence shows how the system updates static and dynamic airport data, specifically facilities and flight schedules.
- Key Logic: The diagram highlights the system's ability to interact with multiple entities (AirportResource and Flight) through a single administrative request.
- Compliance: Data integrity is maintained by validating input at the controller level before updating the database.



UC_42: Monitor System Usage

Focus: Real-time Performance Visualization

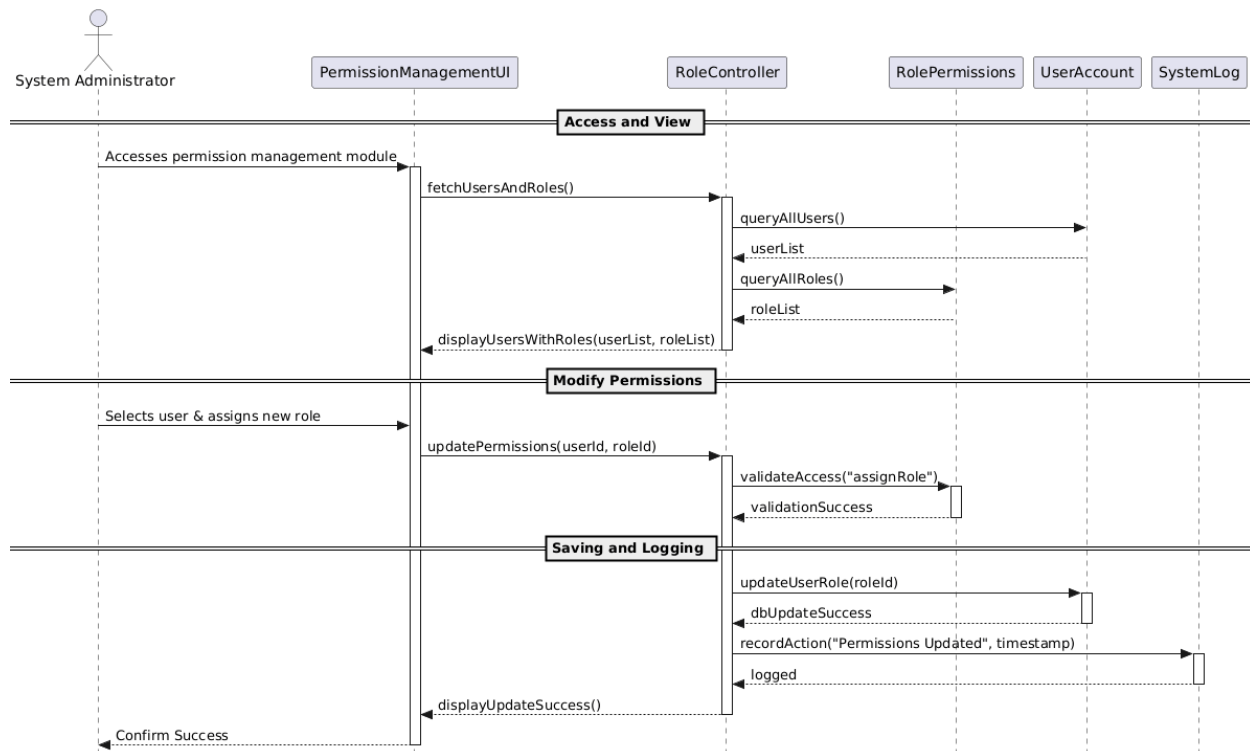
- Description: This diagram represents a "Read-Only" operation where the administrator retrieves live system performance indicators.
- Key Logic: The MonitoringController acts as an aggregator, pulling concurrent data from UserAccount (active users), FinancialTransaction (system load), and SystemLog (activity history).
- Compliance: It fulfills the requirement for high system availability by providing real-time data without modifying the database.



UC_43: Manage System Permissions

Focus: Role-Based Access Control (RBAC)

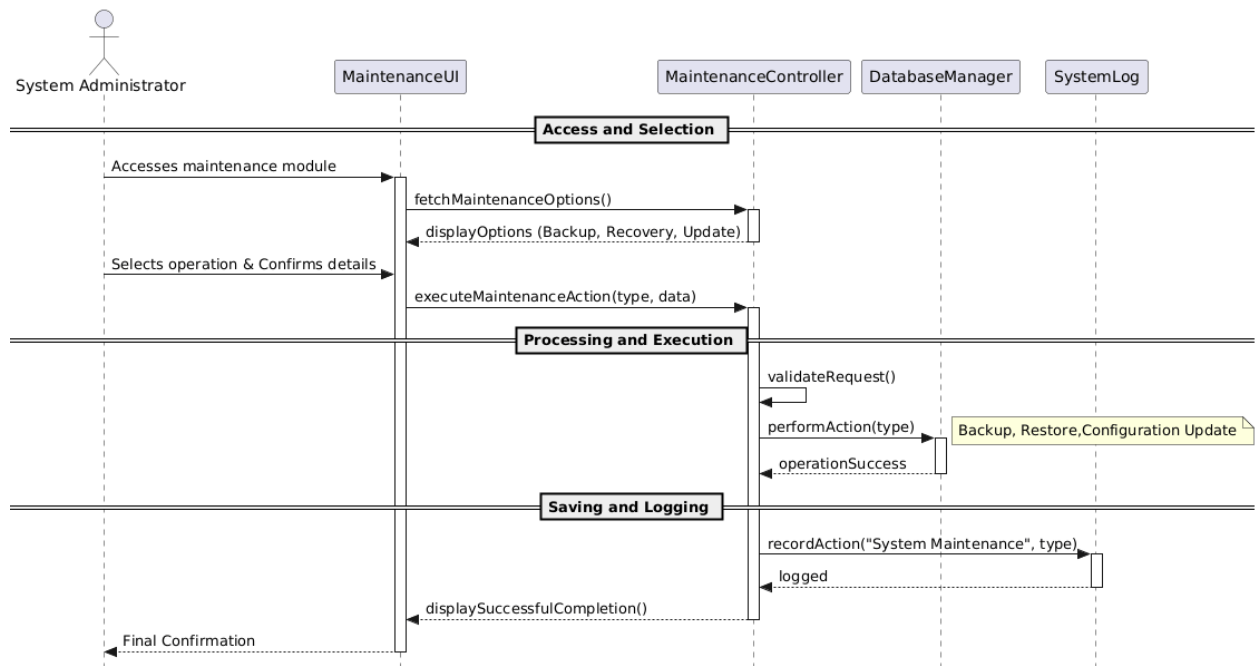
- **Description:** This sequence illustrates the process of assigning or modifying access rights for existing users.
- **Key Logic:** A specific security step is included where the RoleController calls `validateAccess` on the RolePermissions class to ensure the change is authorized and consistent with system rules.
- **Compliance:** Strict adherence to RBAC standards is demonstrated by separating the "Role" logic from the "User" data.



UC_44: Perform System Maintenance

Focus: System Health and Data Safety

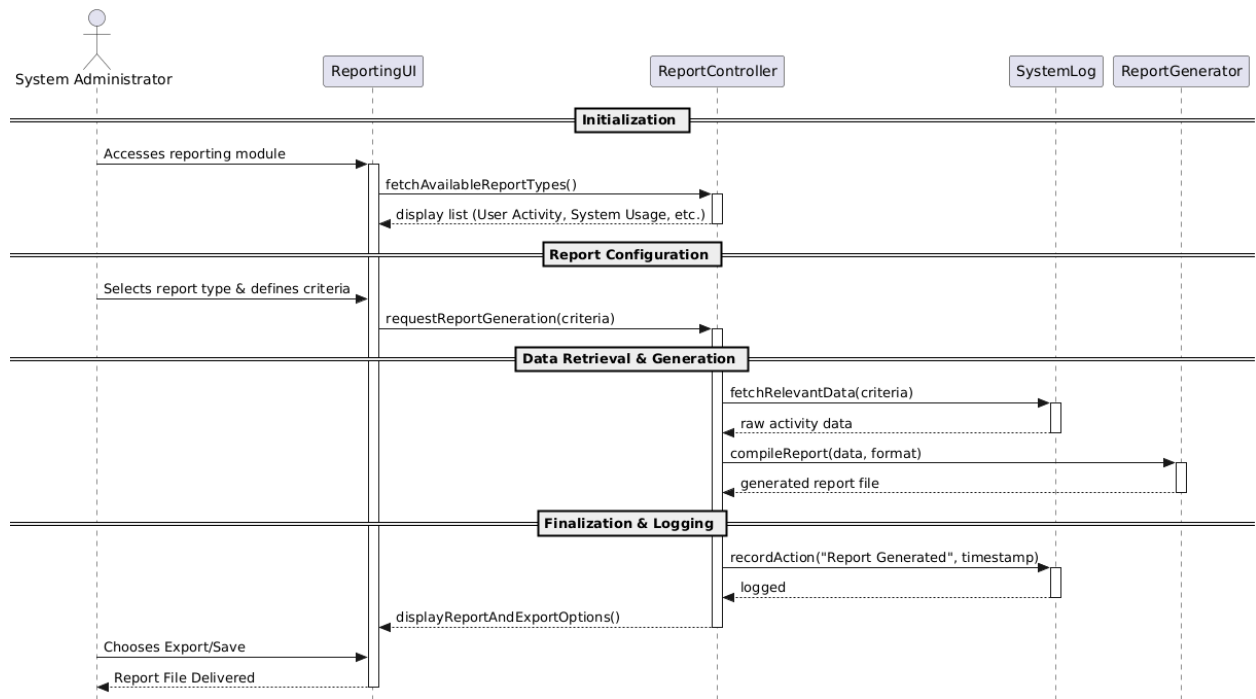
- **Description:** This diagram covers critical maintenance tasks: data backup, recovery, and system configuration updates.
- **Key Logic:** The interaction with a specialized DatabaseManager shows that these operations are performed at a system-wide level rather than a user level.
- **Compliance:** It emphasizes data integrity and disaster recovery protocols, recording each maintenance event in the audit logs.



UC_45: Generate System Activity Reports

Focus: Data Analysis and Exporting

- Description: This final sequence shows how system data is processed into formal reports (PDF/CSV) for administrative review.
- Key Logic: The diagram introduces a ReportGenerator utility class, separating the complex task of document compilation from standard data retrieval.
- Compliance: It satisfies the non-functional requirement for reporting and exporting, providing a documented trail of system activity.



Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Alvi Elezi

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the Flight Operations Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: Airport Staff and Passenger.
2. Boundary Objects: User Interfaces (UIs) used for data input and display.
3. Control Objects: Controllers that orchestrate business logic and validation.
4. Entity Objects: The database layer including User Accounts, System Logs, and Resources.

System Interaction Standards

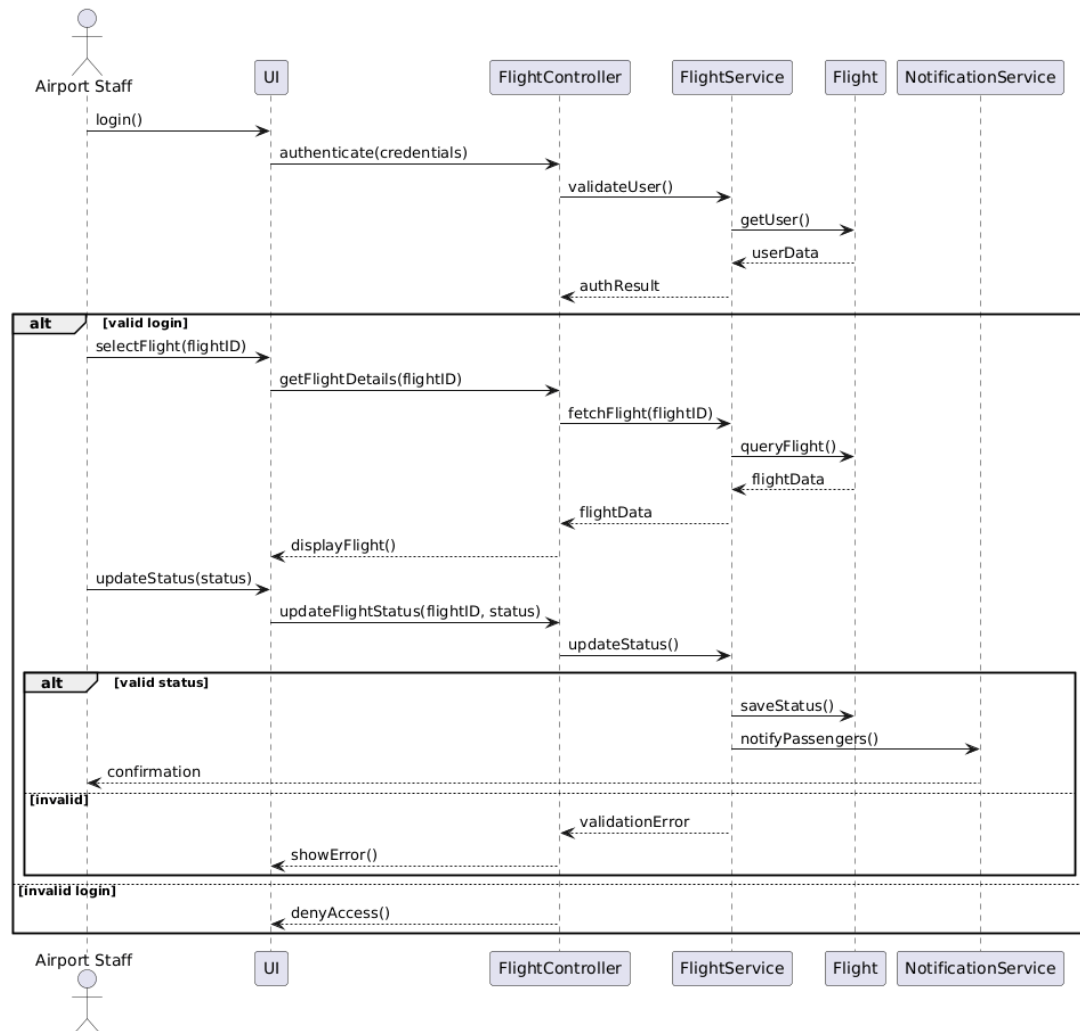
To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

1. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
2. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
3. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
4. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
5. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

1. Update flight Status

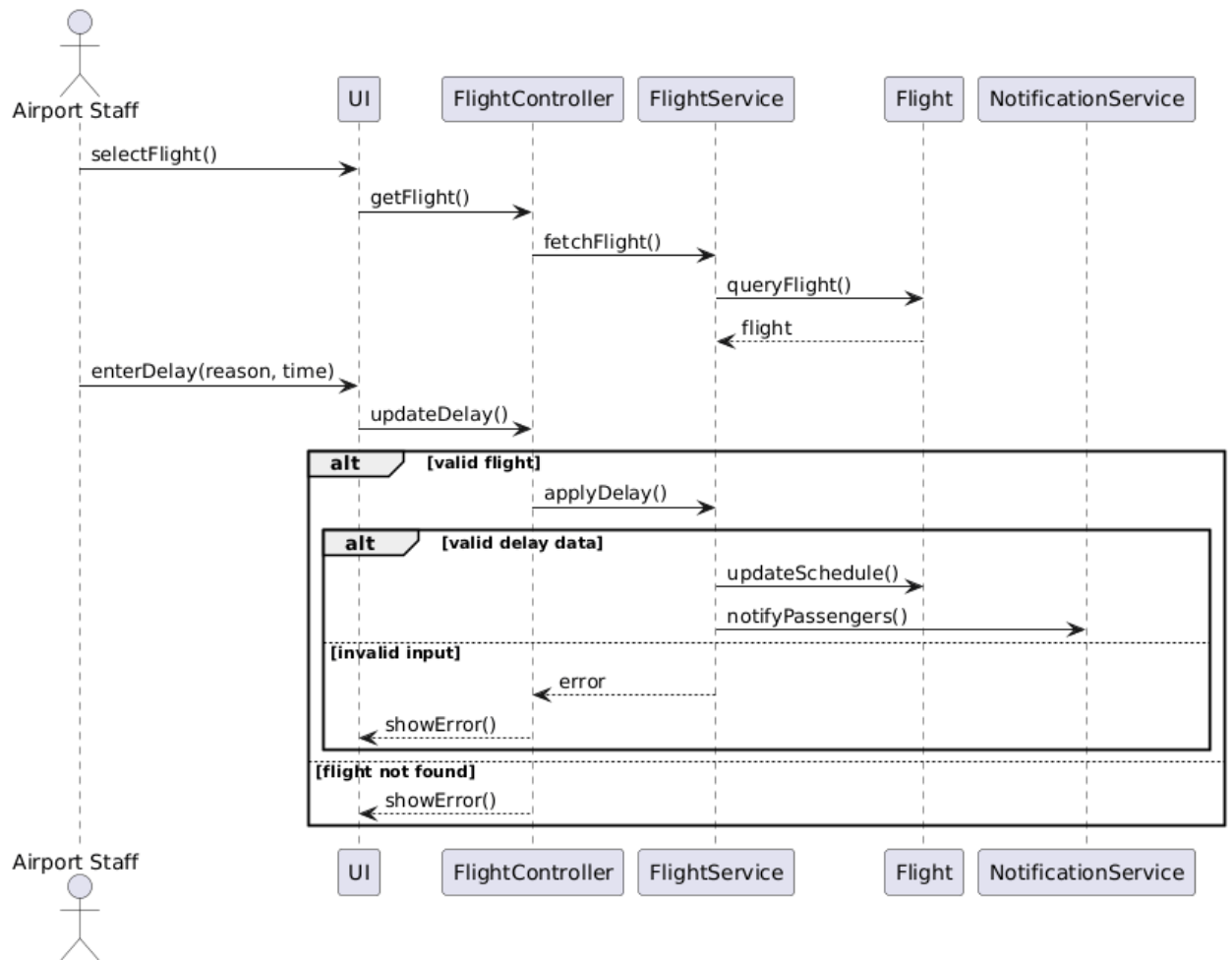
- **Focus:** Flight Status Management
- **Description:** This sequence illustrates how airport staff update the operational status of a flight and propagate the change across the system.
- **Key Logic:** The controller validates the update request, retrieves the flight entity, applies the new status, and triggers notifications to passengers.

- **Compliance:** Data consistency is ensured through validation before persistence, and system-wide synchronization is achieved via notification services.



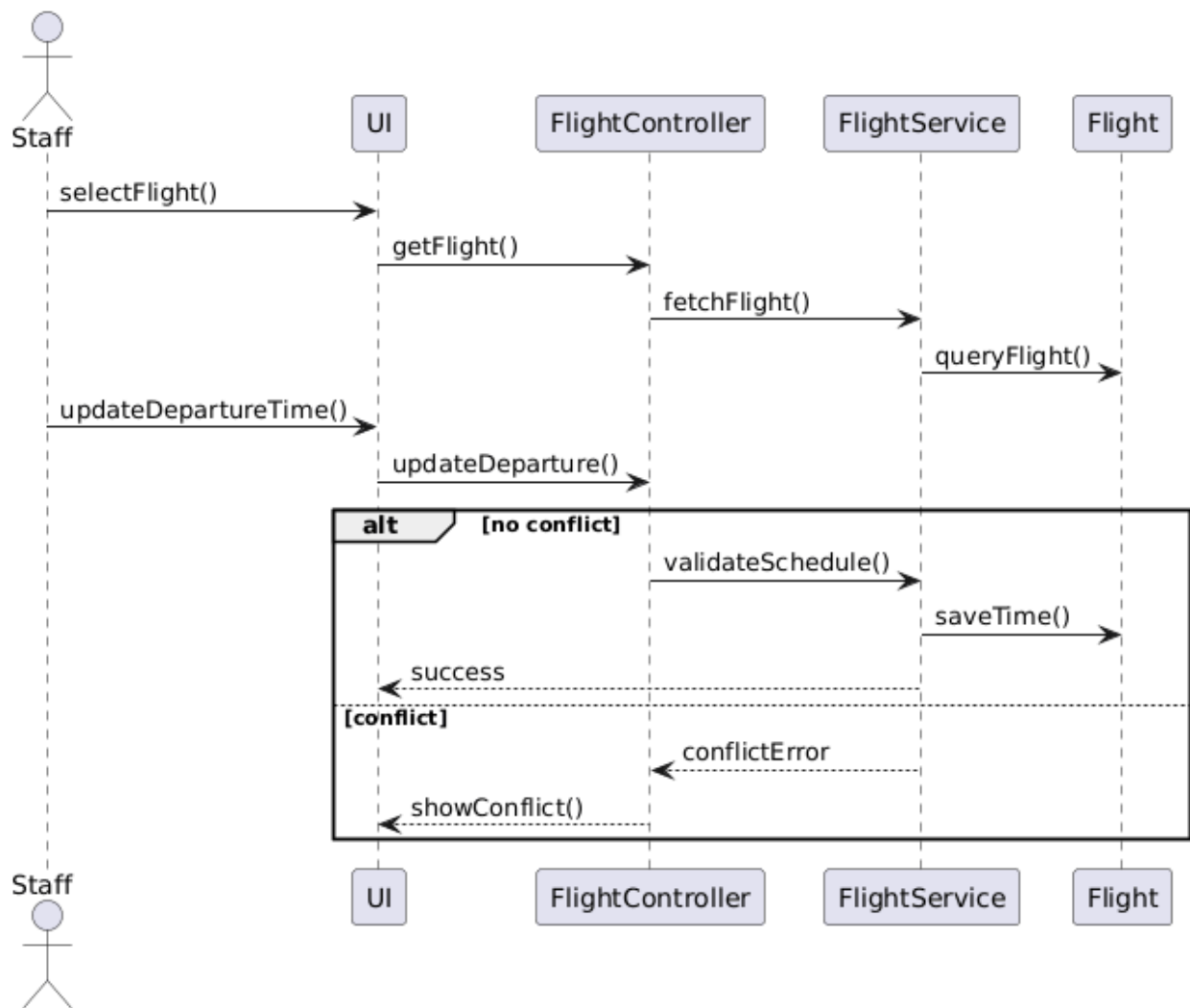
2. Manage Flight Delays

- **Focus:** Delay Handling and Schedule Adjustment
- **Description:** This sequence demonstrates how delays are recorded, processed, and reflected in the flight schedule.
- **Key Logic:** The system validates delay input, updates the flight schedule, and automatically notifies affected passengers.
- **Compliance:** Input validation at the controller level prevents inconsistent scheduling, ensuring reliable operational updates



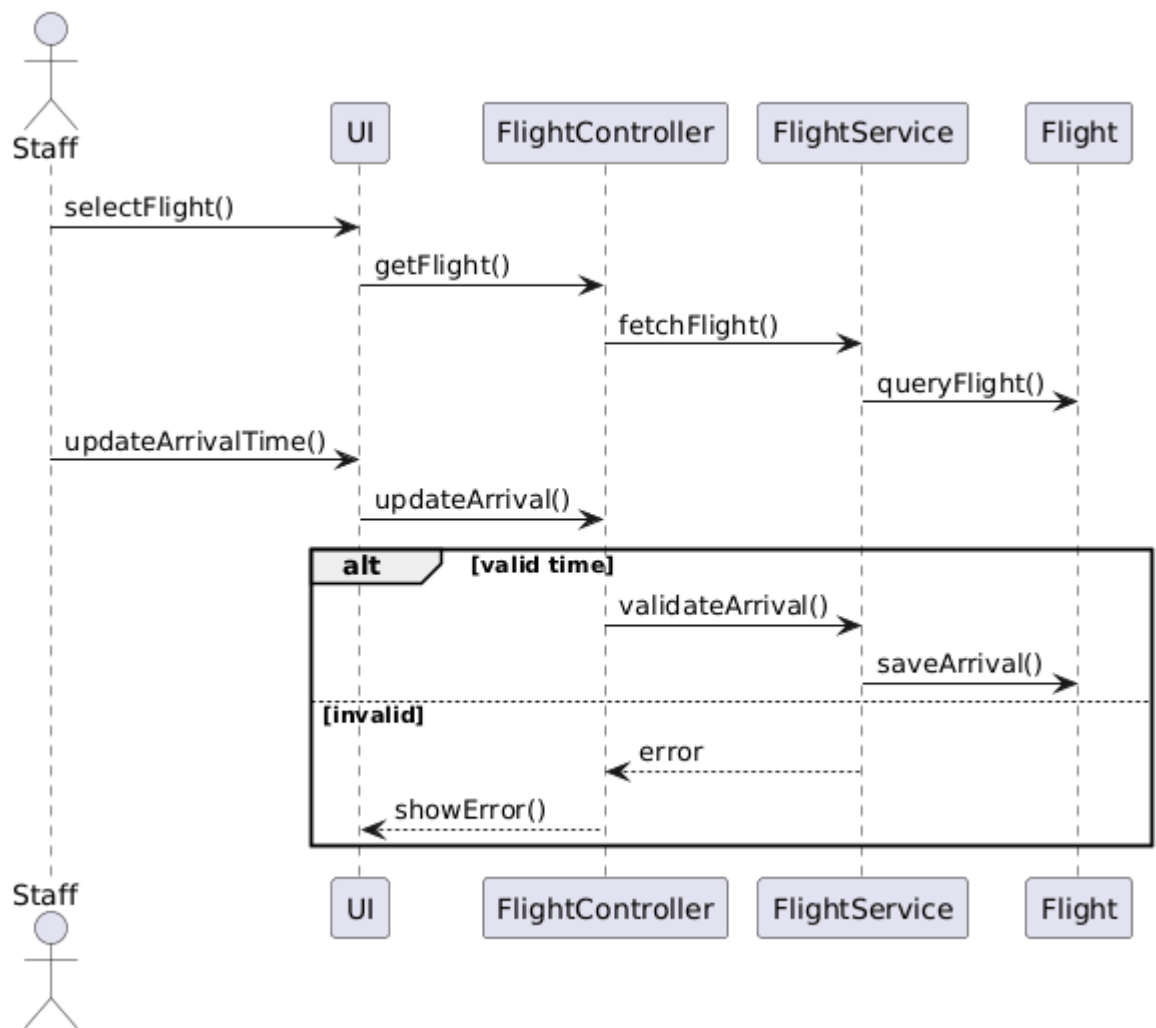
3. Update departure time

- **Focus:** Departure Scheduling
- **Description:** This sequence shows how staff modify the scheduled departure time of a flight.
- **Key Logic:** The controller verifies scheduling constraints (such as runway availability) before committing updates to the system.
- **Compliance:** Conflict detection mechanisms ensure that updated schedules do not violate operational constraints.



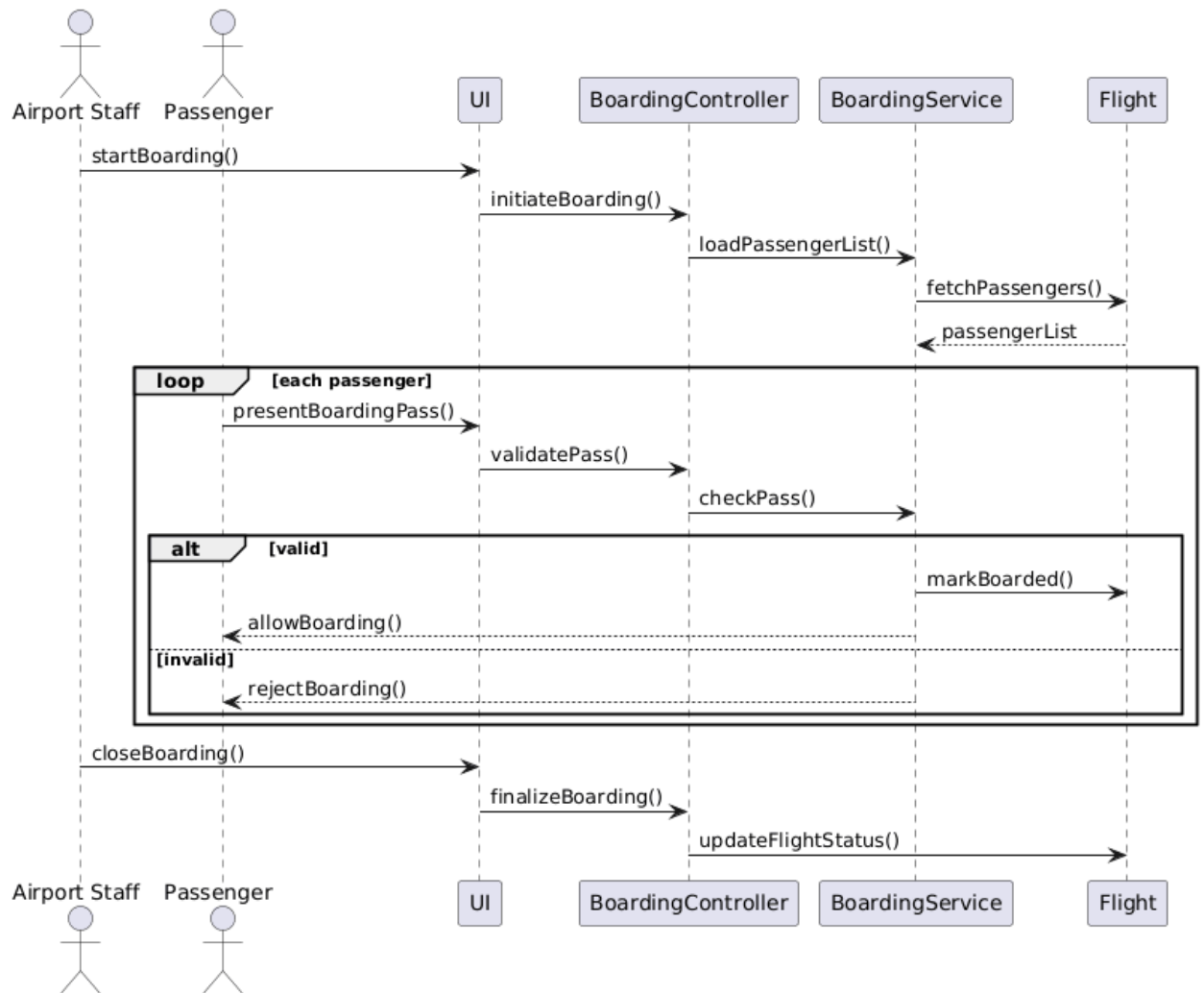
4. Update arrival time

- **Focus:** Arrival Time Synchronization
- **Description:** This sequence represents how arrival time updates are processed and reflected across the system.
- **Key Logic:** The system validates the new arrival time and updates the flight entity while maintaining synchronization with operational displays.
- **Compliance:** Validation ensures logical consistency between departure and arrival times.



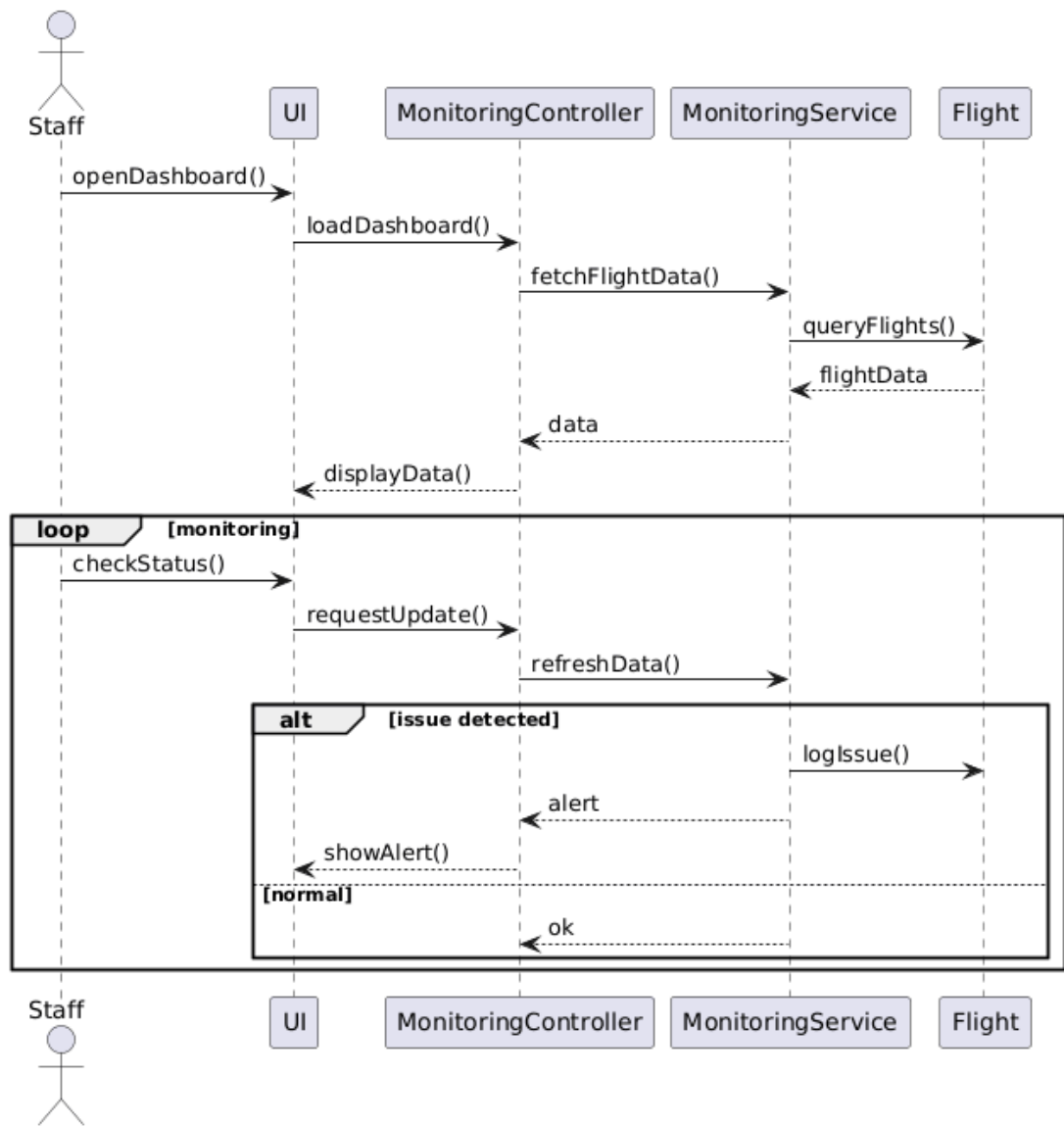
5. Coordinate boarding process

- **Focus:** Passenger Boarding Control
- **Description:** This sequence details how the boarding process is initiated, managed, and completed by airport staff.
- **Key Logic:** The system iteratively validates boarding passes, updates passenger status, and tracks boarding progress in real time.
- **Compliance:** Each boarding action updates the system state, ensuring accurate passenger tracking and operational visibility.



6. Monitor Flight Operations

- **Focus:** Real-Time Operational Monitoring
- **Description:** This sequence shows how airport staff monitor flight operations through a centralized dashboard.
- **Key Logic:** The system continuously retrieves and refreshes flight data, detects anomalies, and generates alerts when issues arise.
- **Compliance:** Continuous data synchronization ensures that operational decisions are based on up-to-date information.



Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Ajla Alcani

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the **Human Resources and Customer Support Modules** through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill system requirements.

Technical Scope

This document focuses on the interaction between:

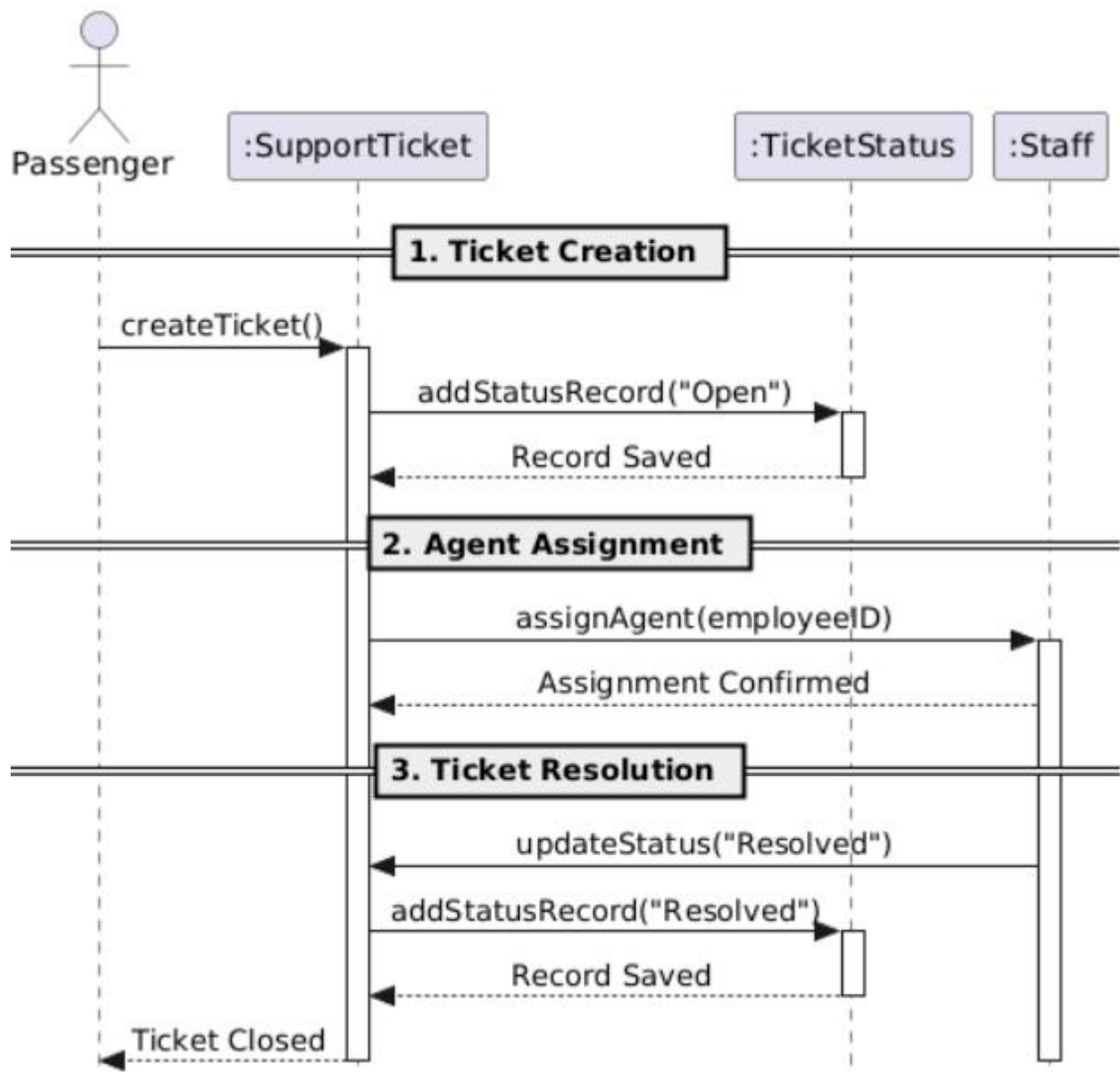
1. **Actors:** HR Admin, Employee (Staff), and Passenger.
2. **Boundary Objects:** User Interfaces (UIs) used for data input and display (e.g., Chat UI, Ticketing Dashboard).
3. **Control Objects:** Controllers that orchestrate business logic and validation for staff and support operations.
4. **Entity Objects:** The database layer including SupportTicket, TicketStatus, Staff, AttendanceRecord, Payroll, and ChatSession.

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

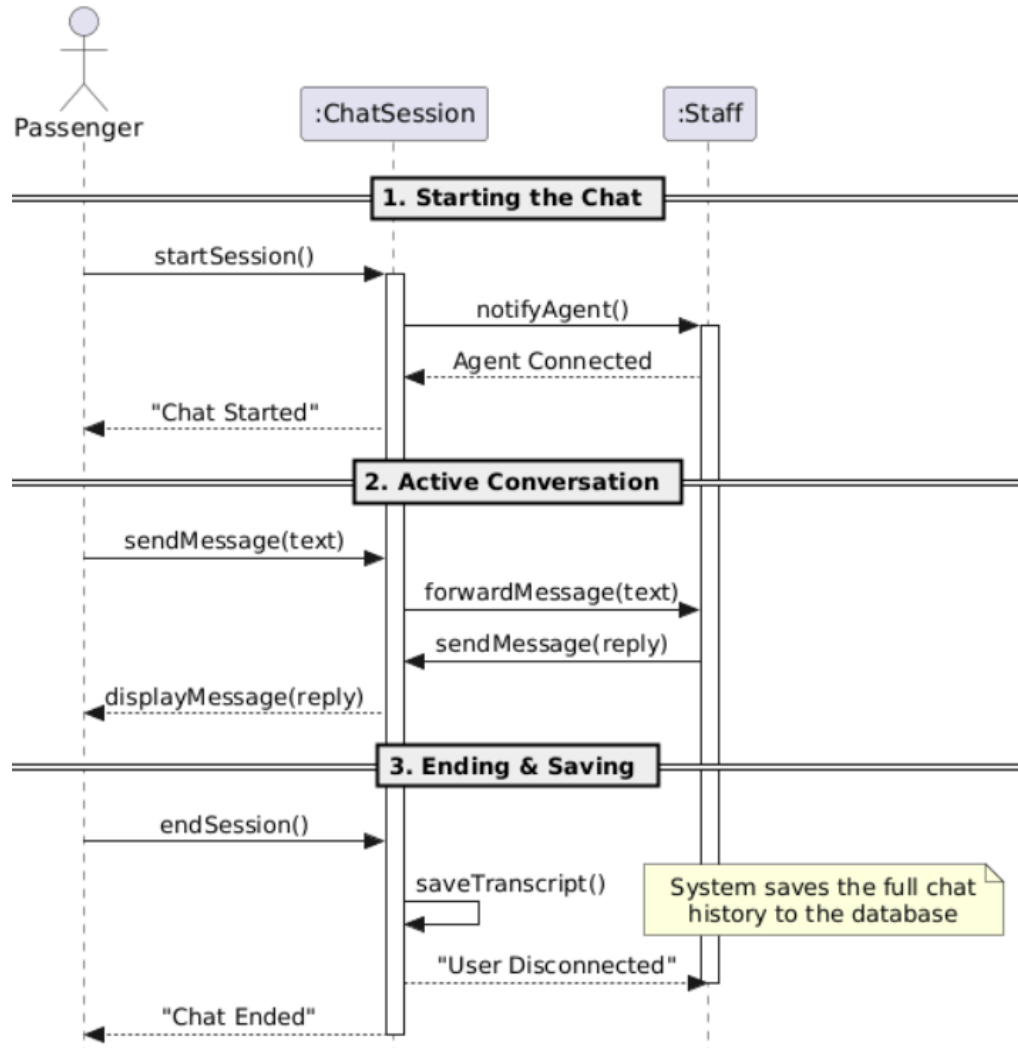
1. **Strict UML Notation:** All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
2. **Controller-Centric Logic:** To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
3. **Entity-Layer Persistence:** Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
4. **Automated Audit Integration:** Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger, satisfying non-functional auditing requirements.
5. **Linear Success Flow:** The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

1. Submit Support Ticket



- **Focus:** Issue Tracking and Resolution
- **Description:** This sequence illustrates how a passenger creates a support ticket, how it is assigned to an agent, and resolved.
- **Key Logic:** The system logs the initial "Open" state in the `TicketStatus` entity, assigns a `Staff` agent, and updates the status to "Resolved" upon completion.
- **Compliance:** All state changes are immediately recorded in the status entity to maintain a complete and accurate audit trail of the ticket's lifecycle.

2. Live Chat Session



Focus: Real-Time Customer Assistance

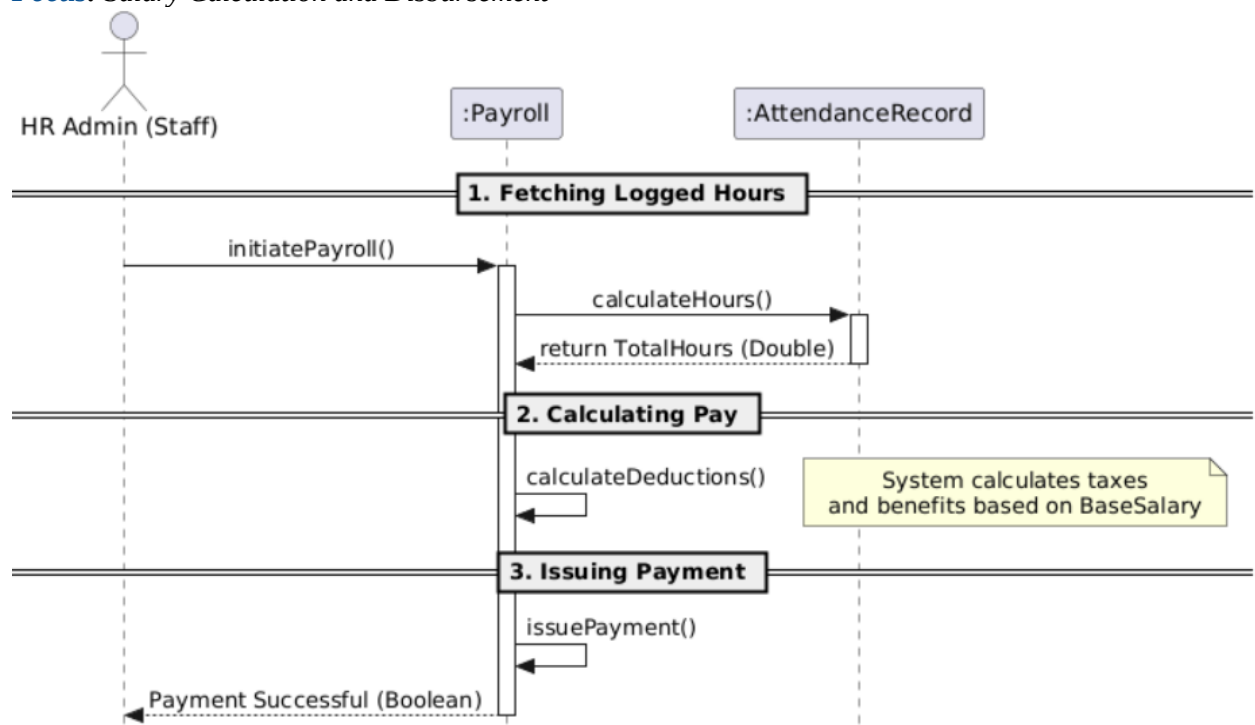
Description: This diagram covers the synchronous, real-time communication between a Passenger and a Staff agent through the system.

Key Logic: The *ChatSession* acts as the intermediary, forwarding messages back and forth. Upon disconnection, it triggers a self-message to save the transcript.

Compliance: The system automatically fulfills data retention requirements by saving the full chat history to the database the moment the session ends.

3. Process Payroll

Focus: Salary Calculation and Disbursement



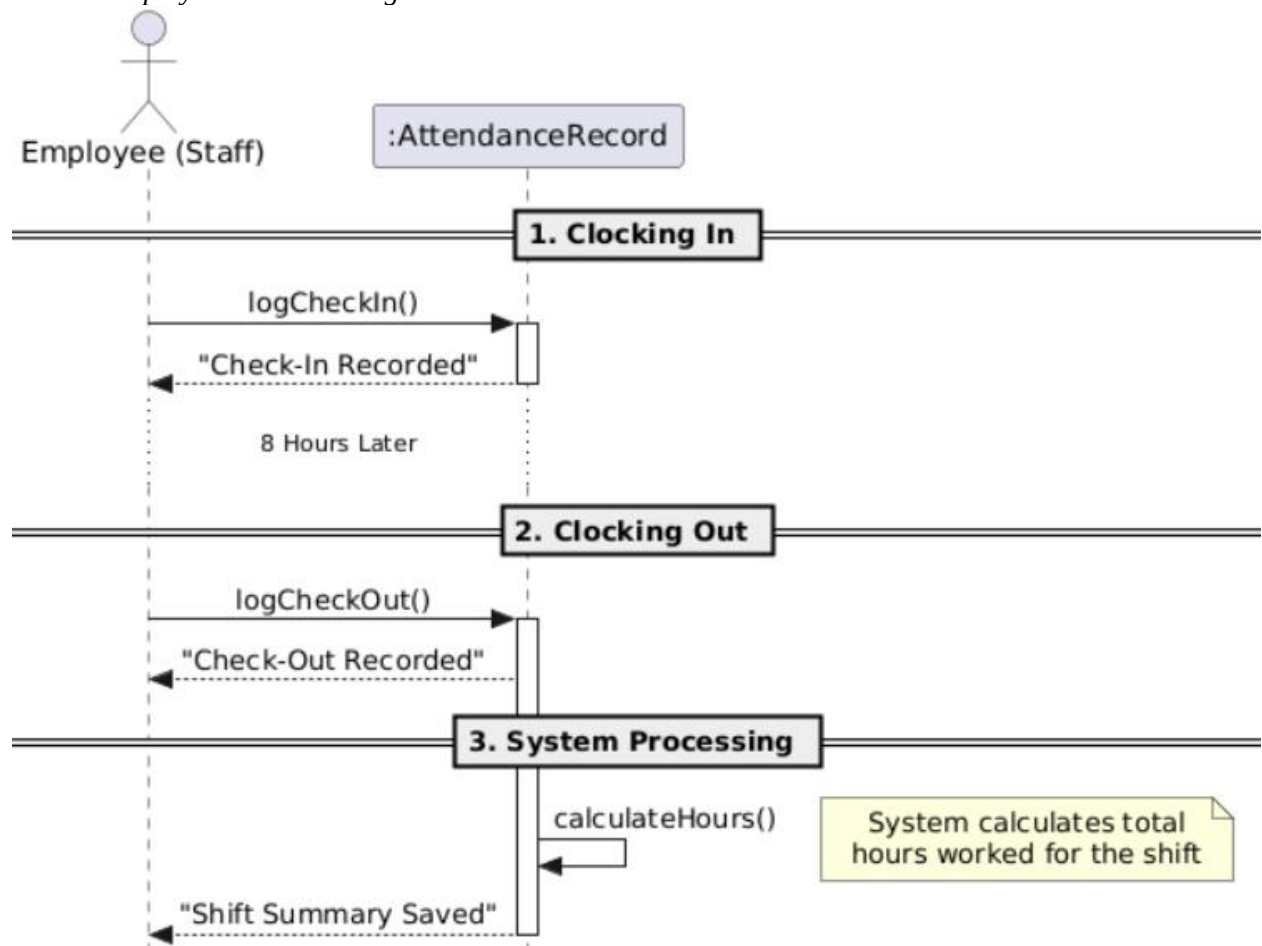
Description: This sequence demonstrates the internal process of calculating an employee's pay based on their logged hours and issuing the final payment.

Key Logic: The *Payroll* entity interacts with the *AttendanceRecord* to fetch total hours, executes an internal calculation for deductions, and issues a success confirmation.

Compliance: Financial calculations and raw working hours are strictly separated into different entities to maintain data integrity before issuing any payments.

4. Log Staff Attendance

Focus: Employee Time Tracking



Description: This sequence shows an employee clocking in at the start of their shift, clocking out, and the system processing their total duration.

Key Logic: The `AttendanceRecord` captures exact timestamps for check-in and check-out, then executes an internal system process to calculate the total hours worked.

Compliance: Automated time calculations at the entity level prevent manual entry errors, ensuring perfectly accurate data is passed to the payroll module.

Airport Management System
Technical Design: Sequence Model Specifications

Prepared by: Andri Mucllari

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

This document presents the **dynamic behavior** of the **Financial and Billing Management Module** through a series of **Sequence Diagrams**. This diagrams illustrate **how objects interact over time** to handle payment processing, invoice generation, and financial record management.

Technical Scope

This document focuses on the interaction between:

1. **Actors:** Finance Staff and Customers.
2. **Boundary Objects:** User Interfaces for payment entry, invoice review, and report generation.
3. **Control Objects:** Controllers that manage transaction validation, invoice computation, and payment confirmation.
4. **Entity Objects:** Database entities including Customer Accounts, Payment Records, Invoices, and System Logs.

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document follows these architectural rules:

1. **Strict UML Notation:** All models utilize standard UML symbols. Solid lines represent synchronous requests, while dashed lines indicate responses or returned data.
2. **Controller-Centric Logic:** To keep the UI lightweight, all business rules, transaction processing, and validation are handled within Controllers.
3. **Entity-Layer Persistence:** Communication with the database occurs only through Entity objects, ensuring a clean separation of logic and data storage.
4. **Automated Audit Integration:** Any action affecting system state (Create, Update, Delete) triggers a corresponding entry in the SystemLog entity to satisfy auditing requirements.

Payment Processing

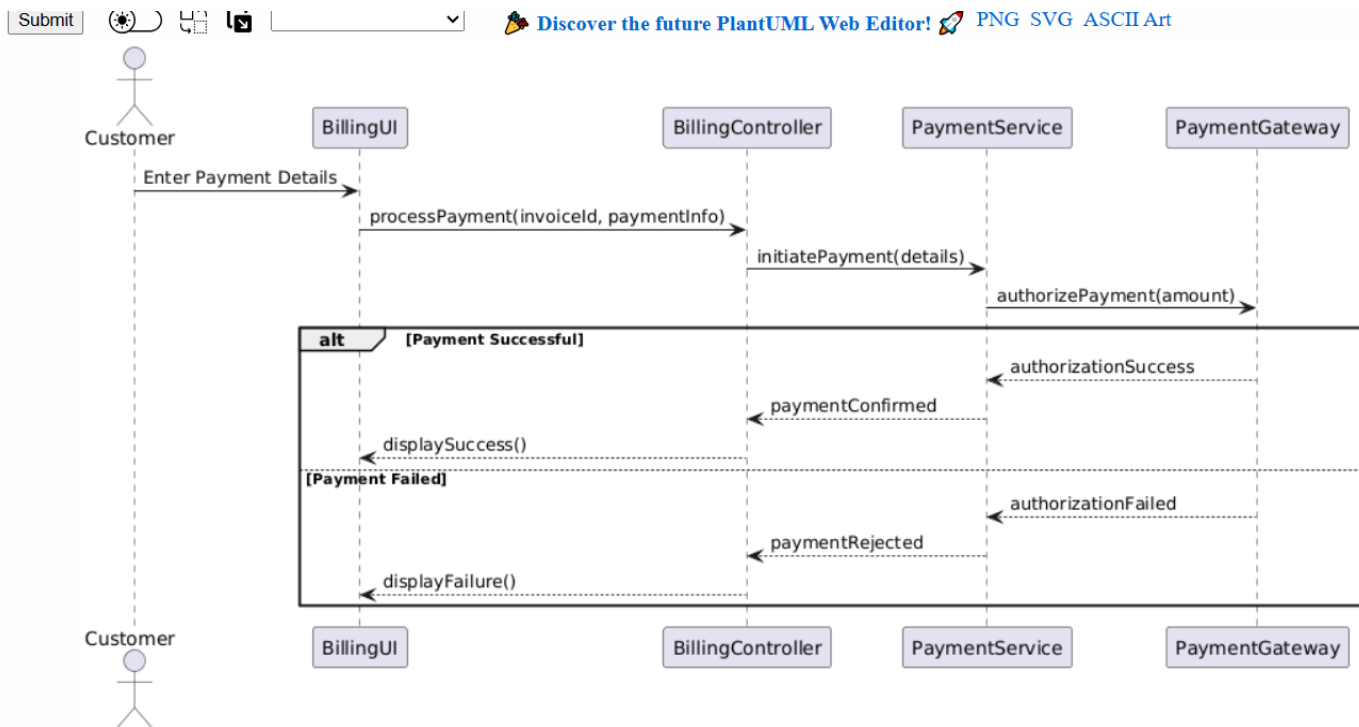
Objective:

To model how an invoice is generated and delivered to a customer

This diagram traces the end-to-end payment journey from a customer entering payment details through to success or failure outcomes. The flow begins with the Customer entering payment details into the billingUI.

The billingController then calls processPayment(invoiced, paymentInfo), which triggers the PaymentService to communicate with the PaymentGateway.

The gateway attempts authorizePayment(amount), returning either authorizationSuccess or authorizationFailed. On success, paymentConfirmed leads to displaySuccess(). On failure, paymentRejected leads to displayFailure.

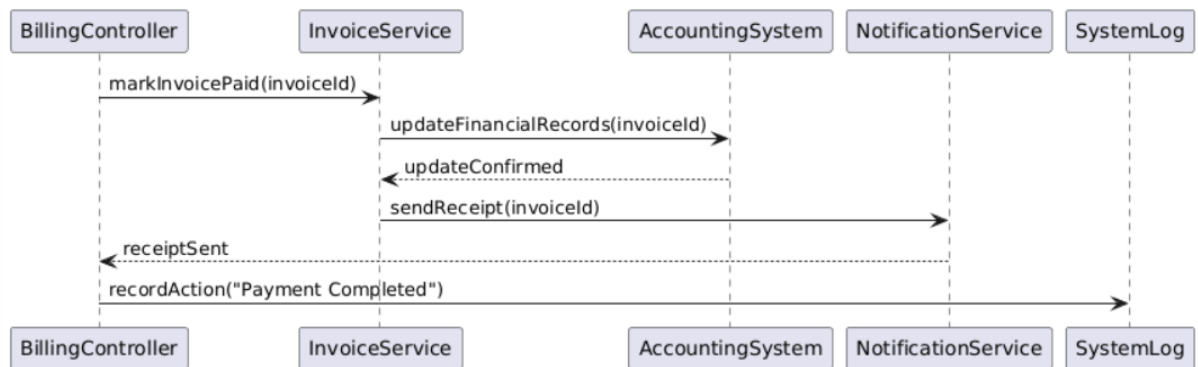


Payment Receipt & Financial Update

Objective:

To represent how the system updates financial records and sends a payment receipt after a successful transaction.

This diagram illustrates the post-payment workflow that occurs after a customer successfully pays. The BillingController coordinates with the system to complete the financial and customer notification.



Technical Design: Sequence Model Specifications

Prepared by: Betül Kaan

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the Passenger Service Management Module through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill administrative requirements.

Technical Scope

This document focuses on the interaction between:

1. Actors: Passenger and AirportStaff (secondary, UC-06 only).
2. Boundary Objects: User interfaces used for data input and display — FlightSearchUI, DocumentUploadUI, CheckInUI, BoardingPassUI, GateInfoUI, BaggageTrackingUI, and NotificationSettingsUI.
3. Control Objects: Controllers that orchestrate business logic and validation for passenger service operations — System(present in all UCs), CheckIn (manages window validation and boarding pass generation in UC-03), and FlightNotification(manages preference registration and alert triggering in UC-07).
4. Entity Objects: The database layer including Flight, Booking, TravelDocument, BoardingPass, BaggageRecord, Gate, and FlightNotification.

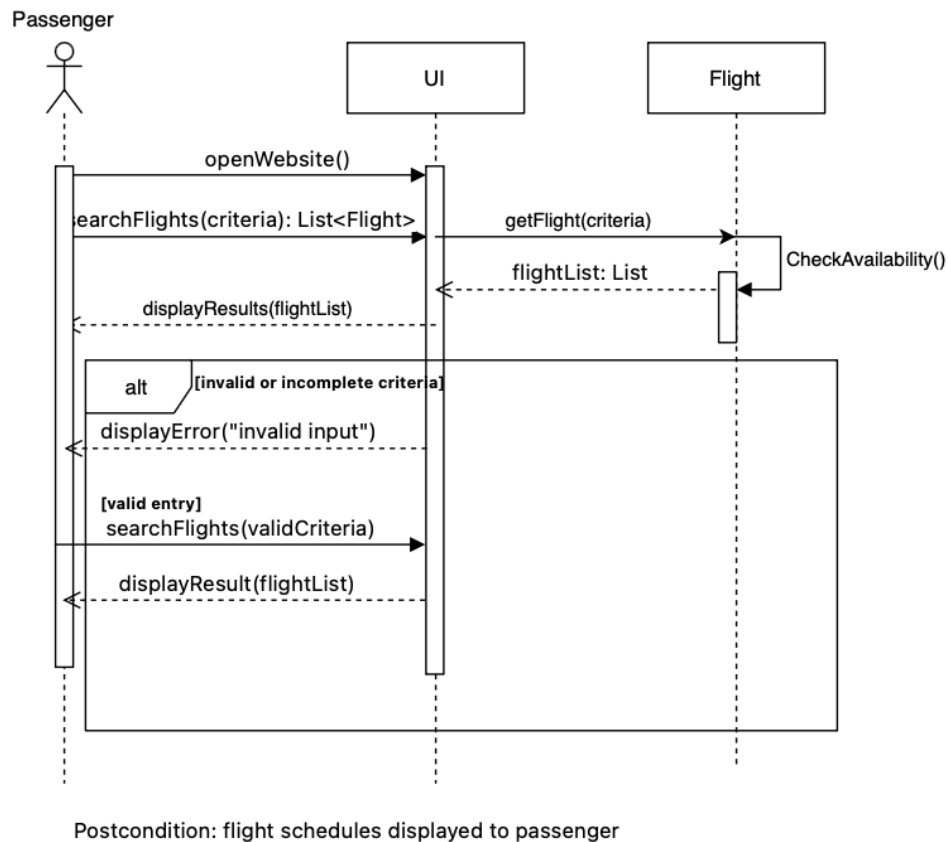
System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

6. Strict UML Notation: All models utilize standardized UML symbols. Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).
7. Controller-Centric Logic: To keep the User Interface (UI) "thin," all business rules, data processing, and validation steps are localized within "Controller" objects.
8. Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects, ensuring a clean separation between the logic and the data.
9. Automated Audit Integration: Every administrative action that changes the system state (Create, Update, Delete) includes an automated trigger to the SystemLog entity, satisfying non-functional auditing requirements.
10. Linear Success Flow: The diagrams focus on the "Main Success Scenario" to provide clear documentation of the primary system logic as defined in the Use Case specifications.

UC-01: View Flight Schedules

Focus: Flight Search and Schedule Retrieval



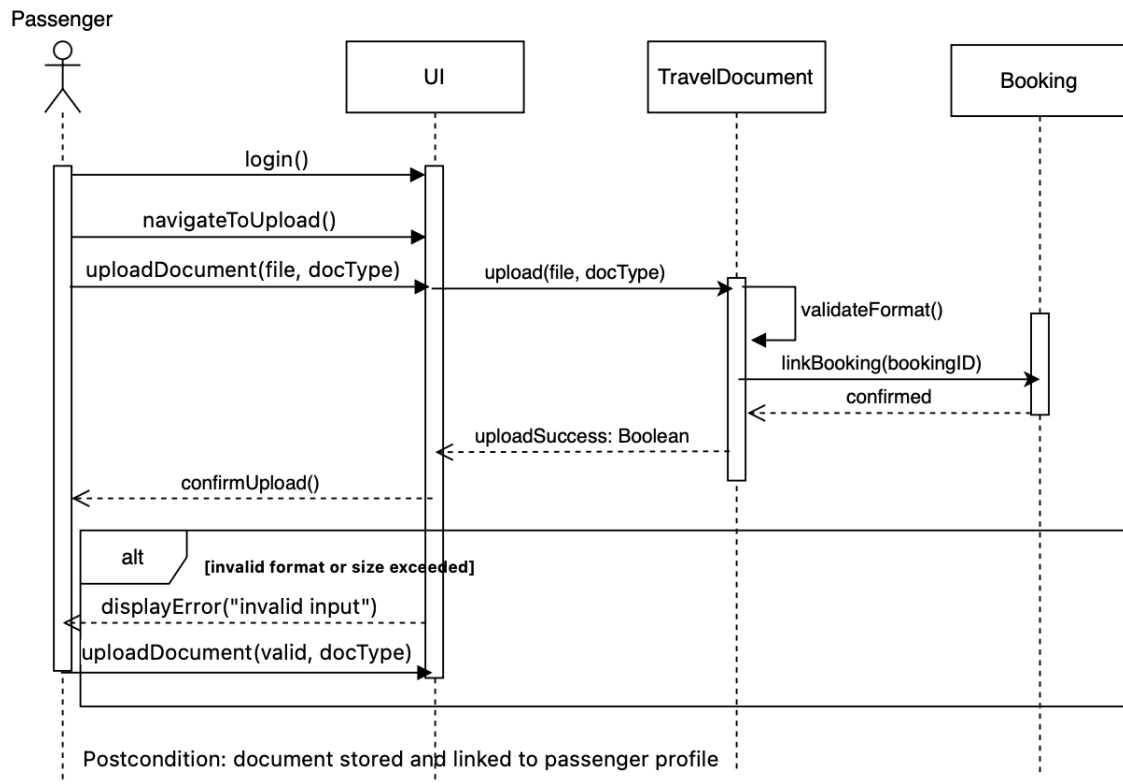
Description: This sequence demonstrates how a passenger searches for available flights and receives matching schedule results from the system.

Key Logic: The system validates the search criteria, queries the *Flight* entity for matching records, and displays the results. An error message is shown if the input is invalid, prompting the passenger to retry.

Compliance: Input validation at the controller level prevents empty or malformed queries from reaching the data layer, ensuring accurate and reliable search results.

UC-02: Upload Travel Documents

Focus: Document Submission and Validation

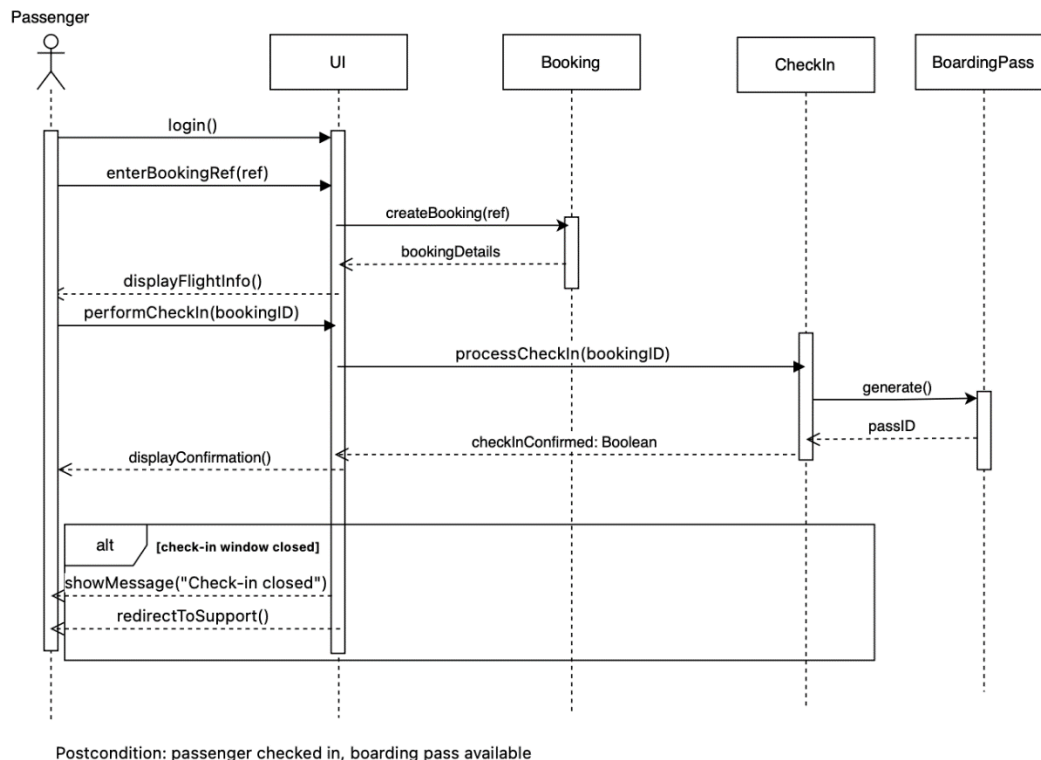


Description: This sequence demonstrates how a passenger uploads travel documents that are validated and securely linked to their booking profile.

Key Logic: The system passes the uploaded file to TravelDocument, which validates the format and size before linking it to the corresponding Booking. A confirmation is returned upon success.

Compliance: Format and size validation at the entity level ensures only compliant documents are stored, protecting data integrity and meeting security requirements.

UC-03: Perform Online Check-In



Focus: Check-In Processing and Boarding Pass Generation

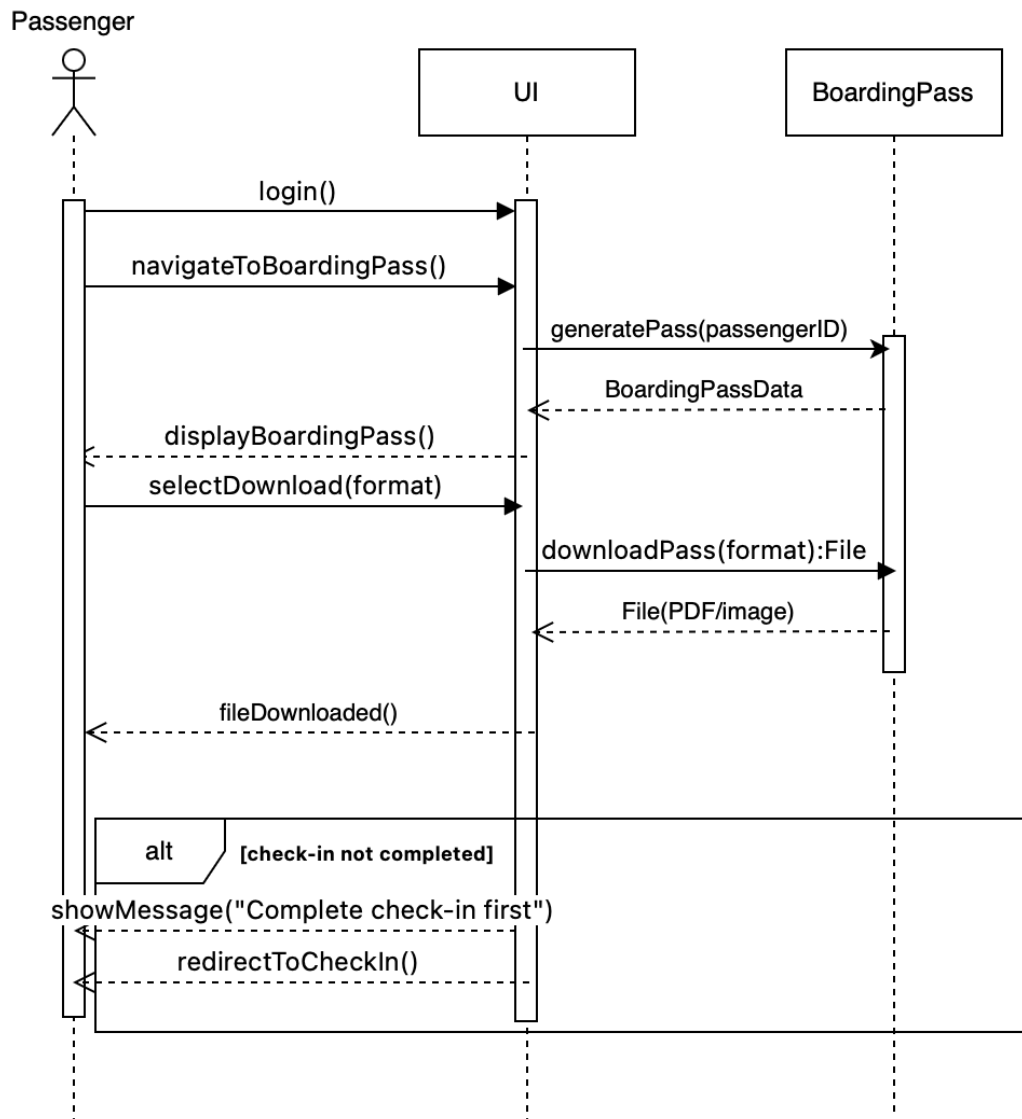
Description: This sequence demonstrates how a passenger completes online check-in by confirming their booking details, triggering the check-in process and boarding pass creation.

Key Logic: The CheckIn controller validates that the check-in window is open via `isWindowOpen()` before processing. Upon success, it calls `generatePass()` on BoardingPass. If the window is closed, the passenger is redirected to support.

Compliance: Window validation at the controller level prevents check-ins outside the allowed time frame, ensuring process integrity and consistent passenger handling.

UC-04: Download Boarding Pass

Focus: Boarding Pass Retrieval and Download



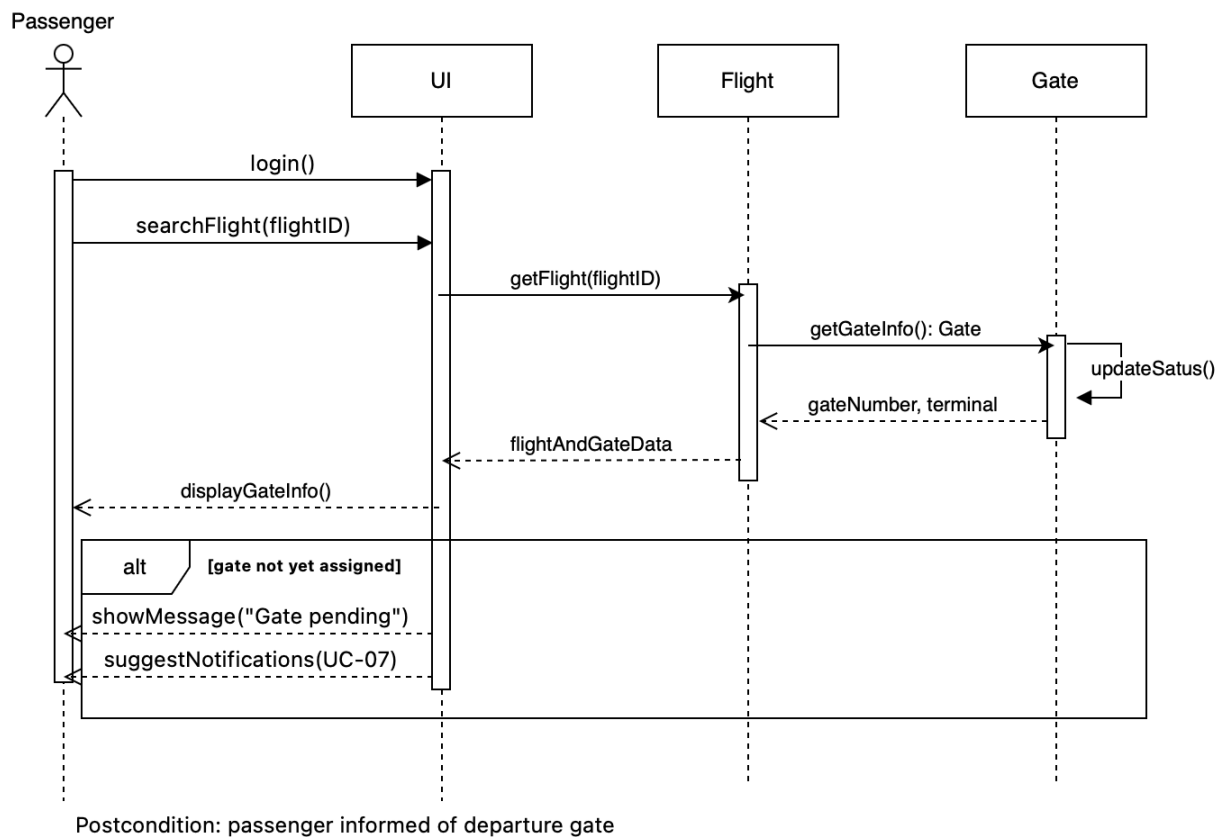
Postcondition: passenger has downloaded boarding pass

Description: This sequence demonstrates how a passenger retrieves and downloads their boarding pass after completing check-in.

Key Logic: The system retrieves the generated BoardingPass and calls `downloadPass(format)` to produce a file. If check-in has not been completed, the passenger is redirected to UC-03.

Compliance: The dependency check on check-in status enforces the correct process order, preventing passengers from downloading a pass before they are formally checked in.

UC-05: Check Assigned Gate Information
Focus: Gate Assignment Retrieval

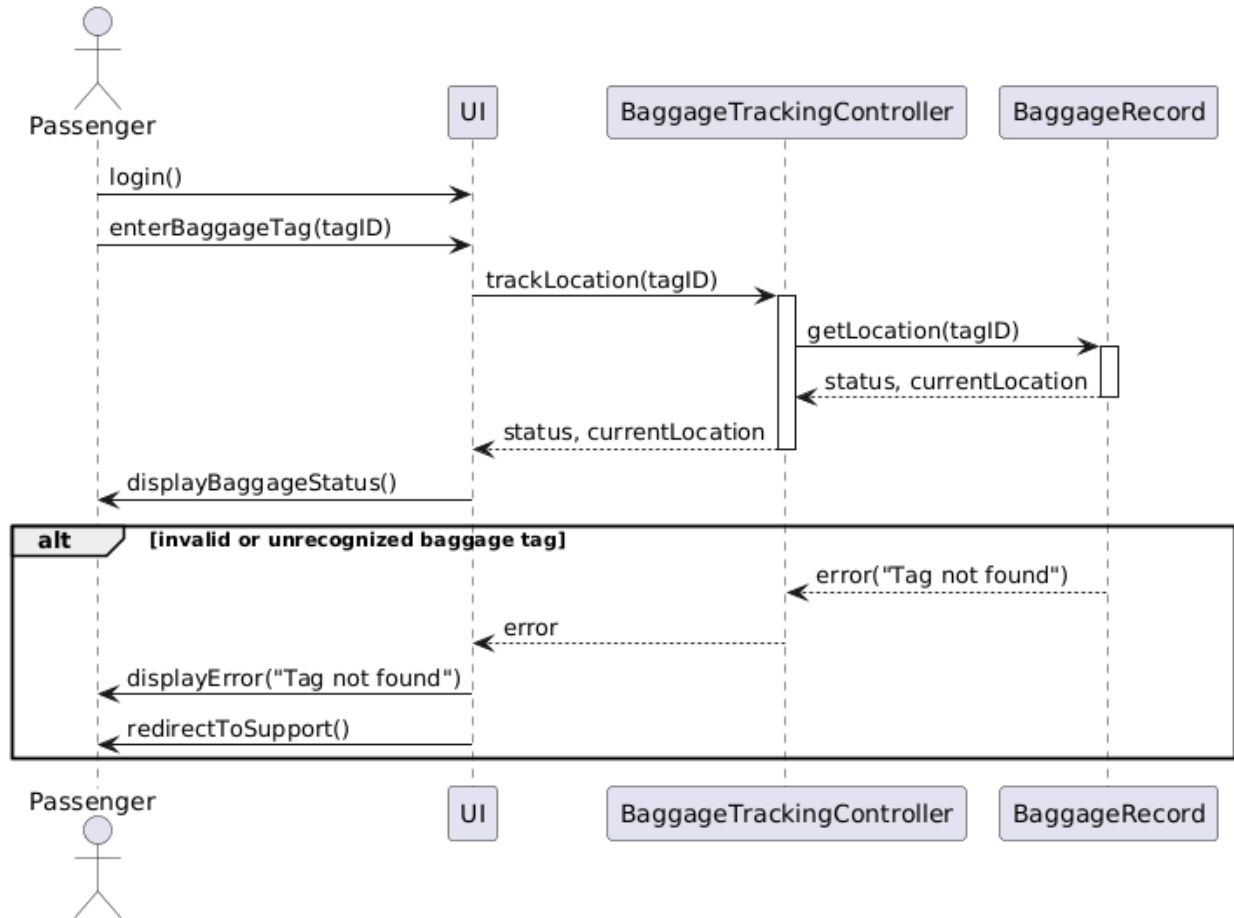


Description: This sequence demonstrates how a passenger looks up their assigned departure gate, with AirportStaff assigning the gate as a precondition before the passenger queries the system.

Key Logic: The system retrieves the Flight record, which in turn calls `getGateInfo()` on Gate to return the gate number and terminal. If no gate has been assigned yet, the passenger is advised to check back or enable notifications.

Compliance: The precondition of staff gate assignment ensures gate data is present before the passenger queries, preventing the display of incomplete or misleading information.

UC-06: Track Baggage Status



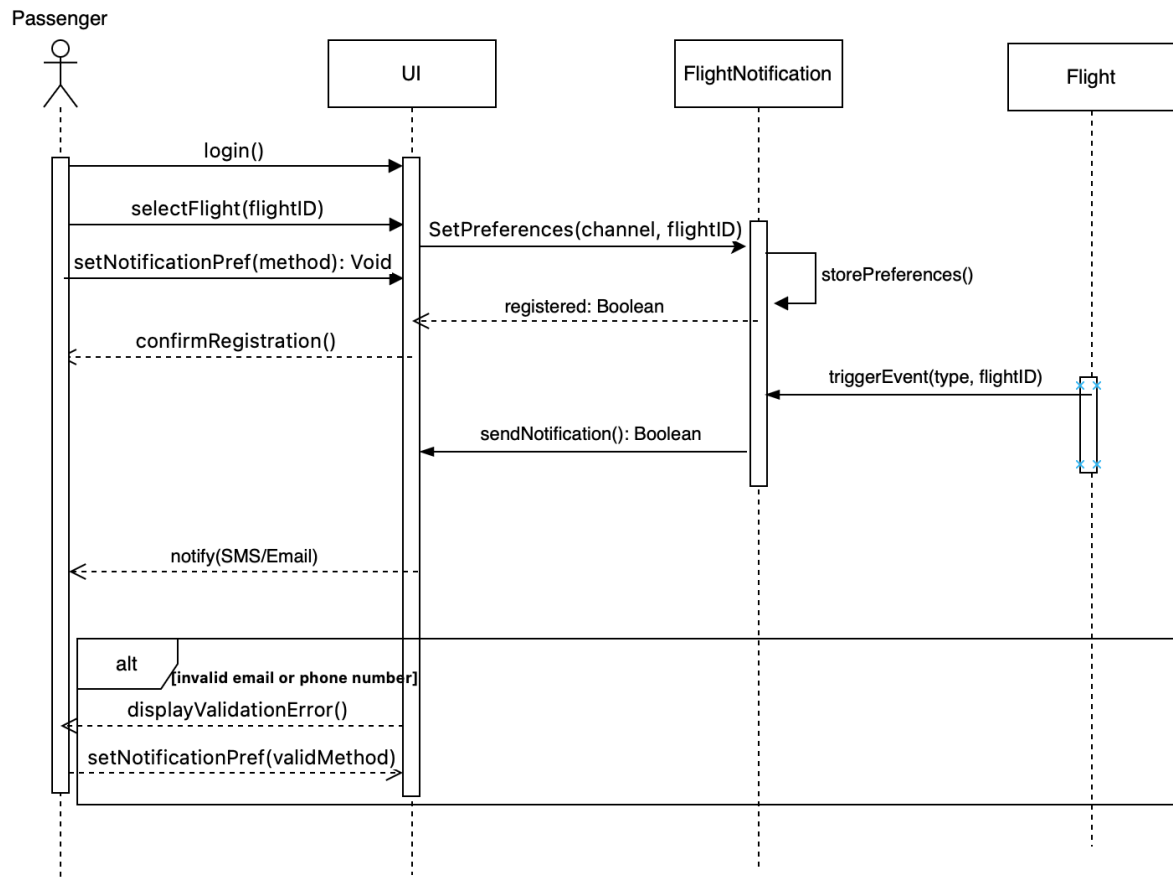
Focus: Real-Time Baggage Tracking

Description: This sequence demonstrates how a passenger tracks the current status and location of their checked baggage.

Key Logic: The system calls `getLocation()` on `BaggageRecord` to retrieve the real-time status and current location. If the baggage tag is invalid or unrecognized, an error is displayed and the passenger is redirected to customer support.

Compliance: Tag validation at the controller level prevents unrecognized queries from returning empty or incorrect results, ensuring passengers receive accurate tracking information.

UC-07: Receive Flight Notifications



Postcondition: passenger receives real-time flight alerts

Focus: Notification Preference Registration and Alert Delivery

Description: This sequence demonstrates how a passenger registers for real-time flight alerts and how the system delivers notifications when a flight event occurs.

Key Logic: The passenger sets their preferred channel via `setNotificationPref()`, which the system stores through `FlightNotification`. When `Flight` triggers an event (delay, gate change, boarding), `FlightNotification` calls `sendNotification()` to deliver the alert via SMS or email.

Compliance: Contact validation ensures notifications are only registered with reachable addresses, and the event-driven trigger model guarantees timely delivery without manual intervention.

Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Ani Velo

Course: Software Modeling and Design

Date: April 10, 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the **Security Management Module** through a series of Sequence Diagrams. While Use Case diagrams define what the system does, these models illustrate how objects interact over time to fulfill security and safety requirements.

Technical Scope

This document focuses on the interaction between:

Actors: Passengers, Airport Staff, and Security Officers.

Boundary Objects: User Interfaces (UIs) such as the Screening Dashboard, Incident Reporting Form, and Lockdown Control Panel.

Control Objects: Controllers that orchestrate business logic, such as the SecurityController and AlertManager

Entity Objects: The database layer including PassengerProfiles, SecurityLogs, ProhibitedItems, and IncidentReports.

System Interaction Standards

To maintain technical consistency, every sequence diagram in this document adheres to the following architectural rules:

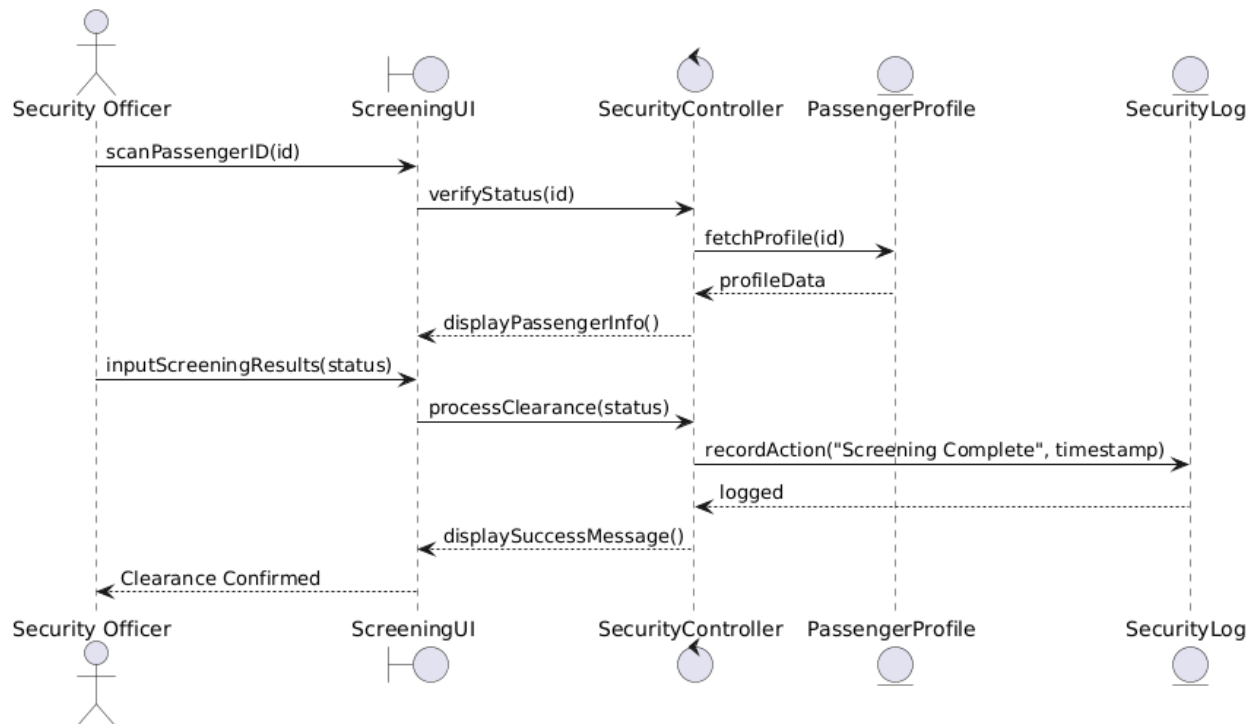
Strict UML Notation: Solid lines indicate synchronous message calls (requests), while dashed lines indicate return messages (data response).

Controller-Centric Logic: To keep the UI "thin," all business rules and validation steps are localized within "Controller" objects.

Entity-Layer Persistence: Direct communication with the database is handled through specific "Entity" objects.

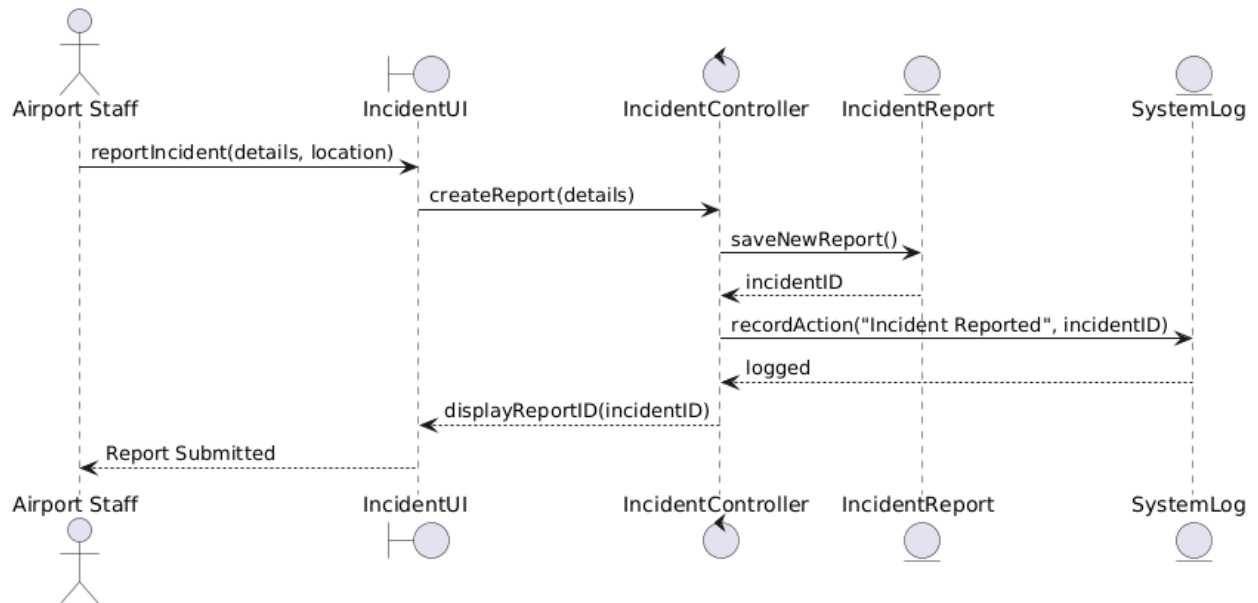
Automated Audit Integration: Every administrative action that changes the system state includes an automated trigger to the SystemLog entity for auditing purposes.

1. UC_SEC_01: Passenger Security Screening



- **Focus:** Screening Workflow and Clearance
- **Description:** This sequence illustrates the process of verifying a passenger's status and logging physical screening results
- **Key Logic:** The SecurityController validates the passenger's ID against the manifest before allowing the officer to enter screening results.
- **Compliance:** The process concludes with a mandatory update to the SecurityLog to ensure all clearances are audited.

2. UC_SEC_02: Report Security Incident



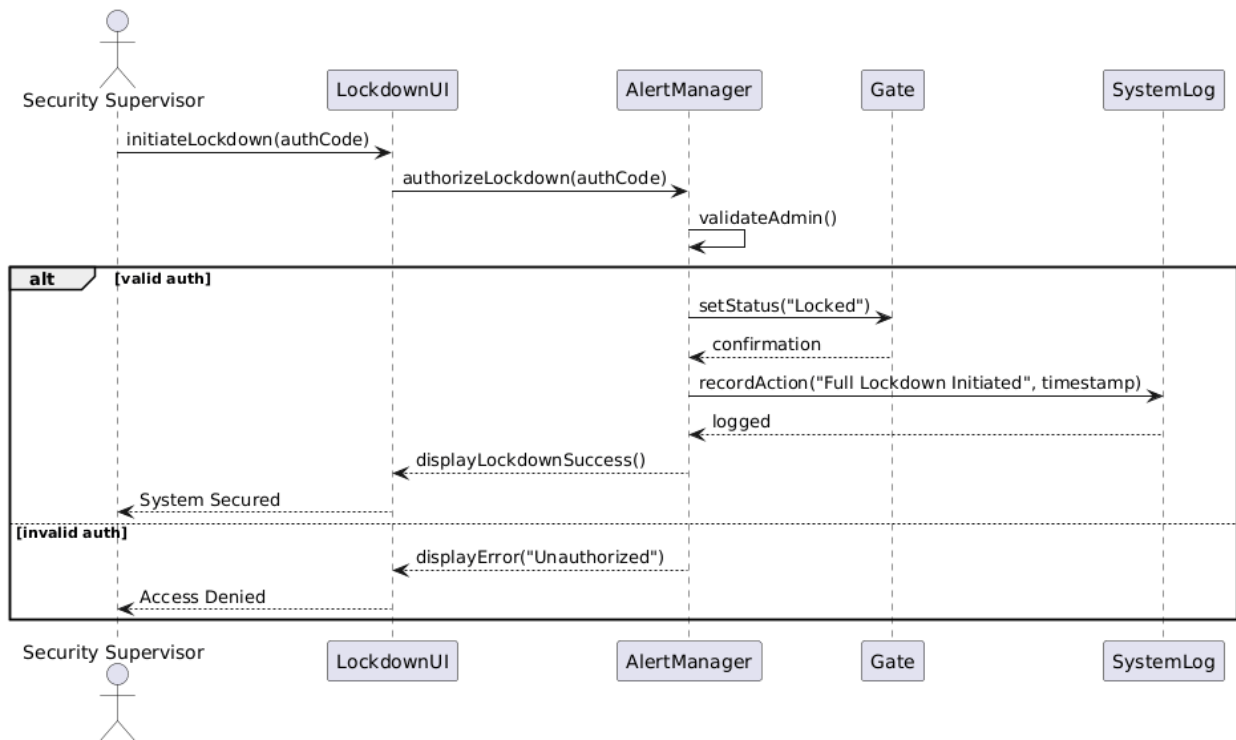
Focus: Incident Logging and Documentation

Description: This sequence demonstrates how a staff member or officer creates a formal security incident report.

Key Logic: The system creates a new *IncidentReport* entity, assigns a unique ID, and updates the status logs.

Compliance: Immediate logging in the *SystemLog* ensures that the discovery of the incident is time-stamped for auditing.

3. UC_SEC_03: Emergency Facility Lockdown



Focus: Critical Safety Response

Description: This sequence represents the high-priority process of initiating a facility-wide lockdown.

Key Logic: The controller verifies administrative permissions and updates the status of physical Gate and Door entities.

Compliance: Every maintenance and emergency event is recorded in the audit logs to maintain data integrity.

Airport Management System

Technical Design: Sequence Model Specifications

Prepared by: Albina Blloshmi

Course: Software Modeling and Design

Date: April 2026

Document Objective

The primary objective of this document is to detail the dynamic behavior of the **Baggage Management Module** through a series of sequence diagrams. While use case diagrams define what the system does, these models illustrate how system components interact over time to support baggage-related operations.

Technical Scope

This document focuses on the interaction between:

1. **Actors:** Passenger, Airport Staff
2. **Boundary Objects:** User interfaces used for baggage registration, tracking, and reporting
3. **Control Objects:** System components responsible for processing requests and managing logic
4. **Entity Objects:** Core data entities such as Baggage, Flight, and SystemLog

System Interaction Standards

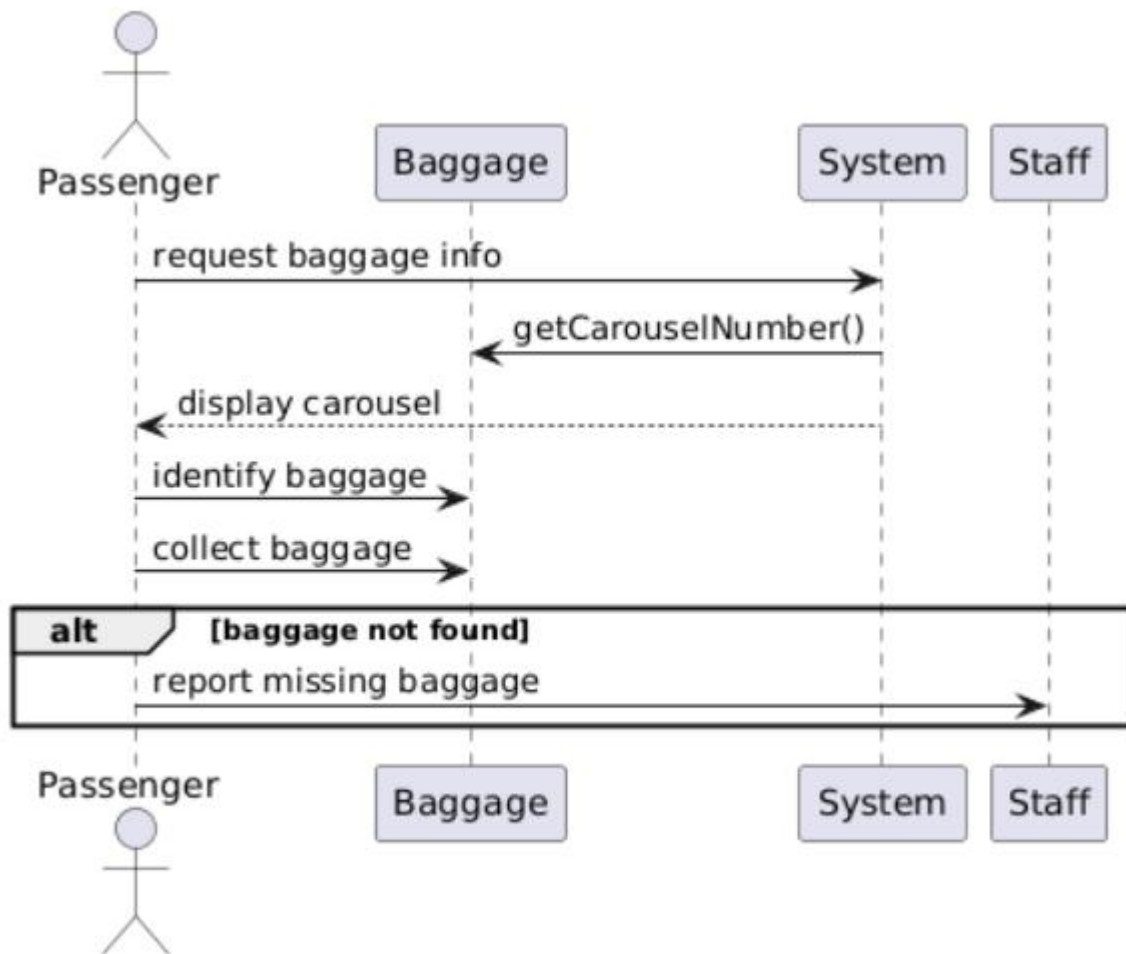
To ensure consistency, all sequence diagrams follow these principles:

1. **Standard UML Notation:** Solid arrows represent requests, dashed arrows represent responses
2. **Controller-Based Logic:** Business logic is handled by system/control components
3. **Entity Separation:** Data handling is performed by entity objects (e.g., Baggage, Flight)
4. **Audit Logging:** All major actions trigger updates to the SystemLog
5. **Main Success Focus:** Diagrams emphasize the primary success scenario

UC-08: Claim Baggage at Destination

Focus: Baggage Retrieval Process

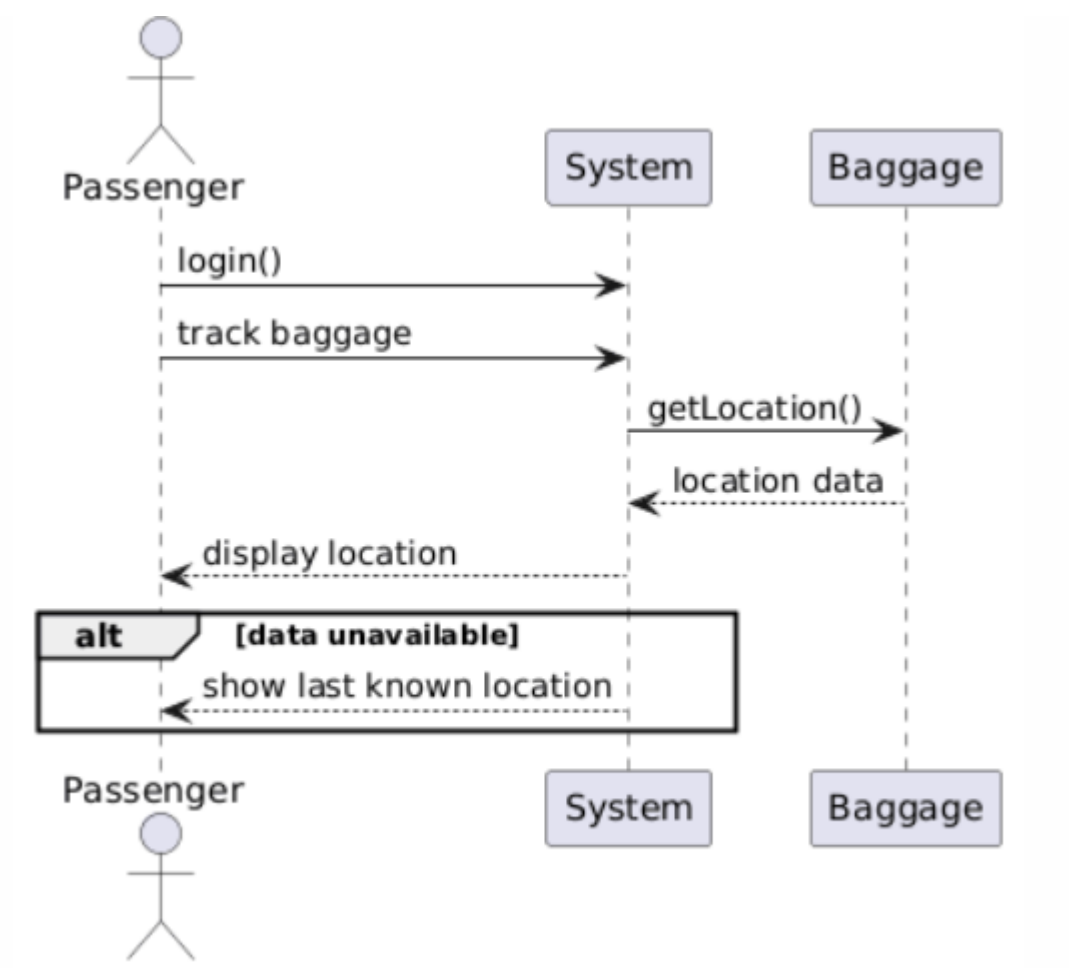
- **Description:**
This sequence illustrates how a passenger retrieves baggage upon arrival, including system assistance in locating the correct carousel.
- **Key Logic:**
The system provides carousel information and allows the passenger to identify and collect baggage efficiently.
- **Compliance:**
Ensures minimal waiting time and clear passenger guidance.



UC-09: Track Baggage in Real-Time

Focus: Real-Time Monitoring

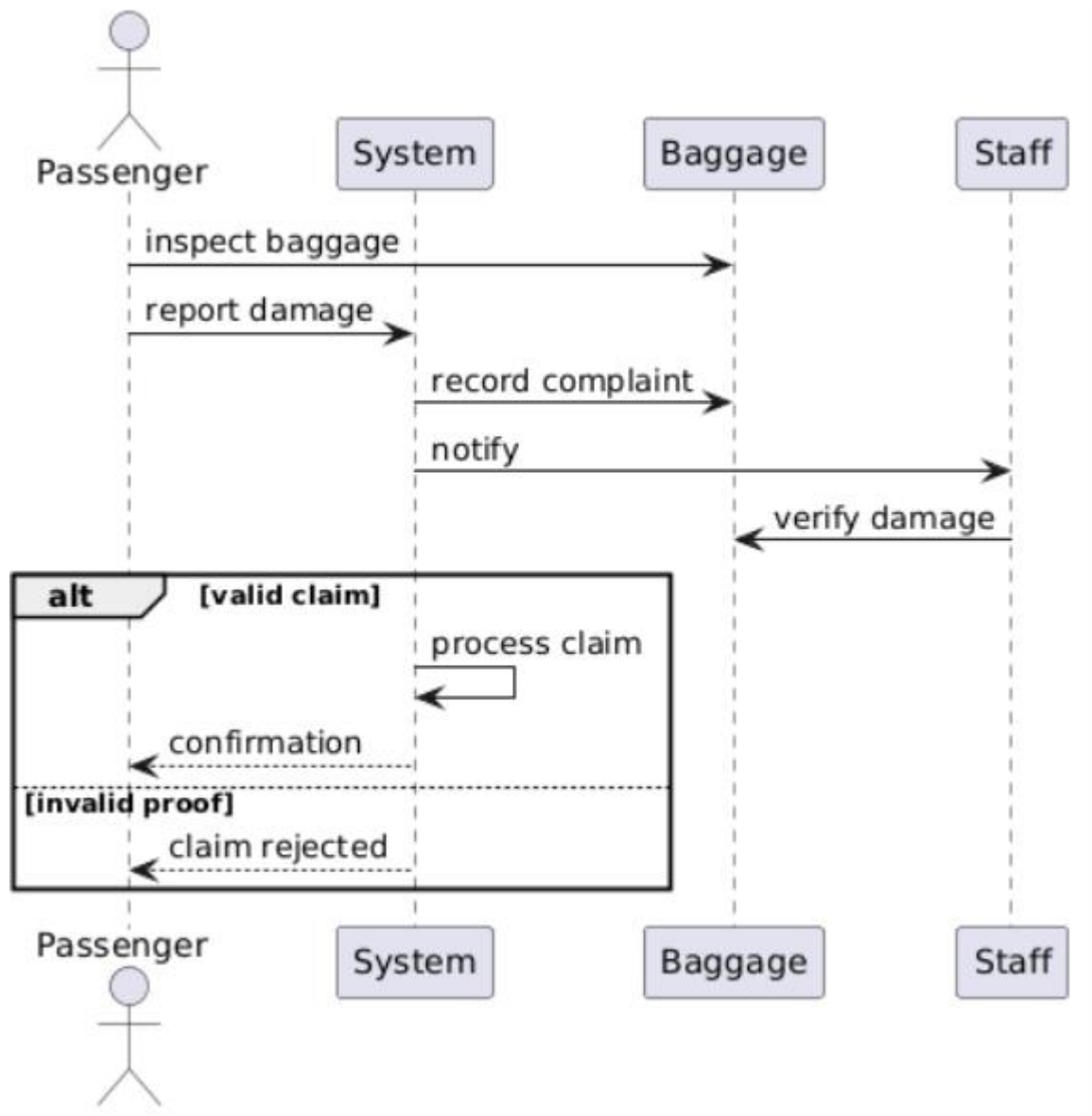
- **Description:**
This sequence shows how passengers track baggage location throughout their journey using real-time system updates.
- **Key Logic:**
The system retrieves and displays current baggage location data.
- **Compliance:**
Ensures high accuracy and real-time responsiveness.



UC-10: Report Damaged Baggage

Focus: Damage Reporting and Claim Handling

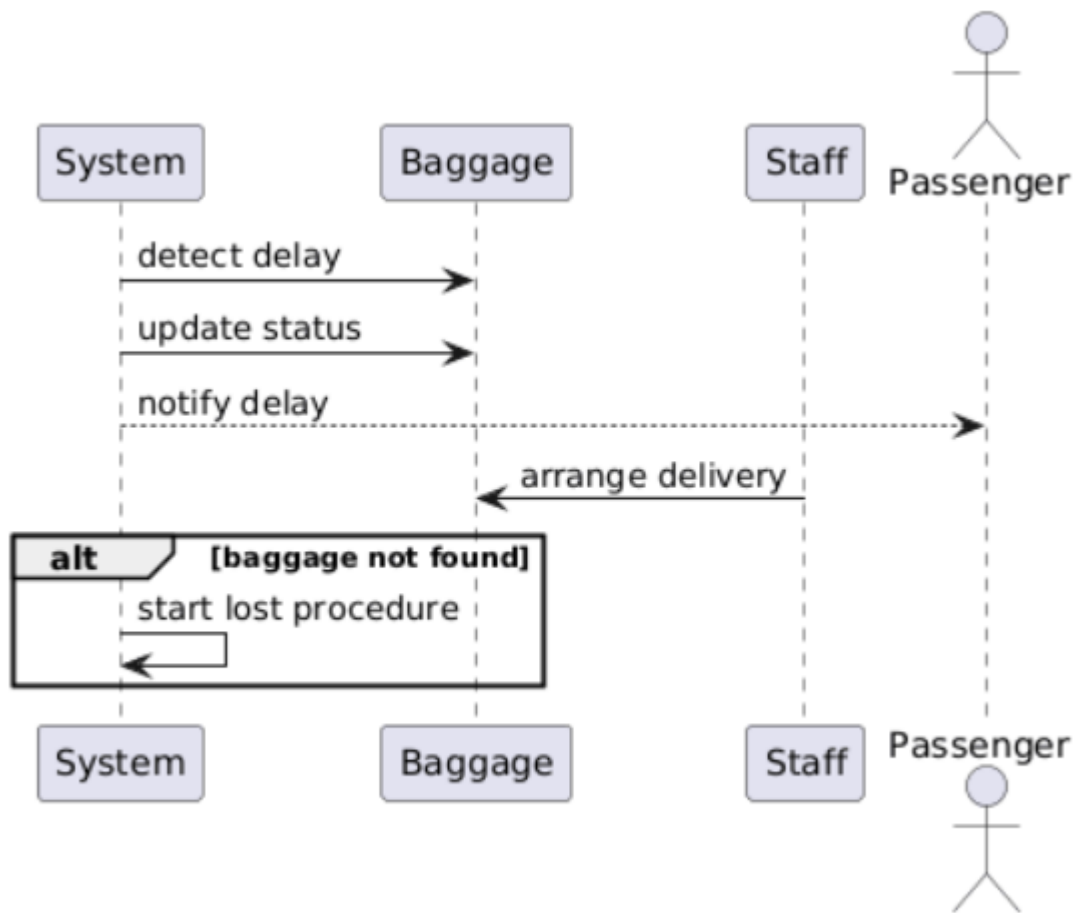
- **Description:**
This sequence represents how passengers report damaged baggage and how the system processes the claim.
- **Key Logic:**
The system records the complaint, staff verifies the damage, and the claim is processed accordingly.
- **Compliance:**
Ensures secure and fair claim handling.



UC-11: Handle Delayed Baggage

Focus: Delay Management

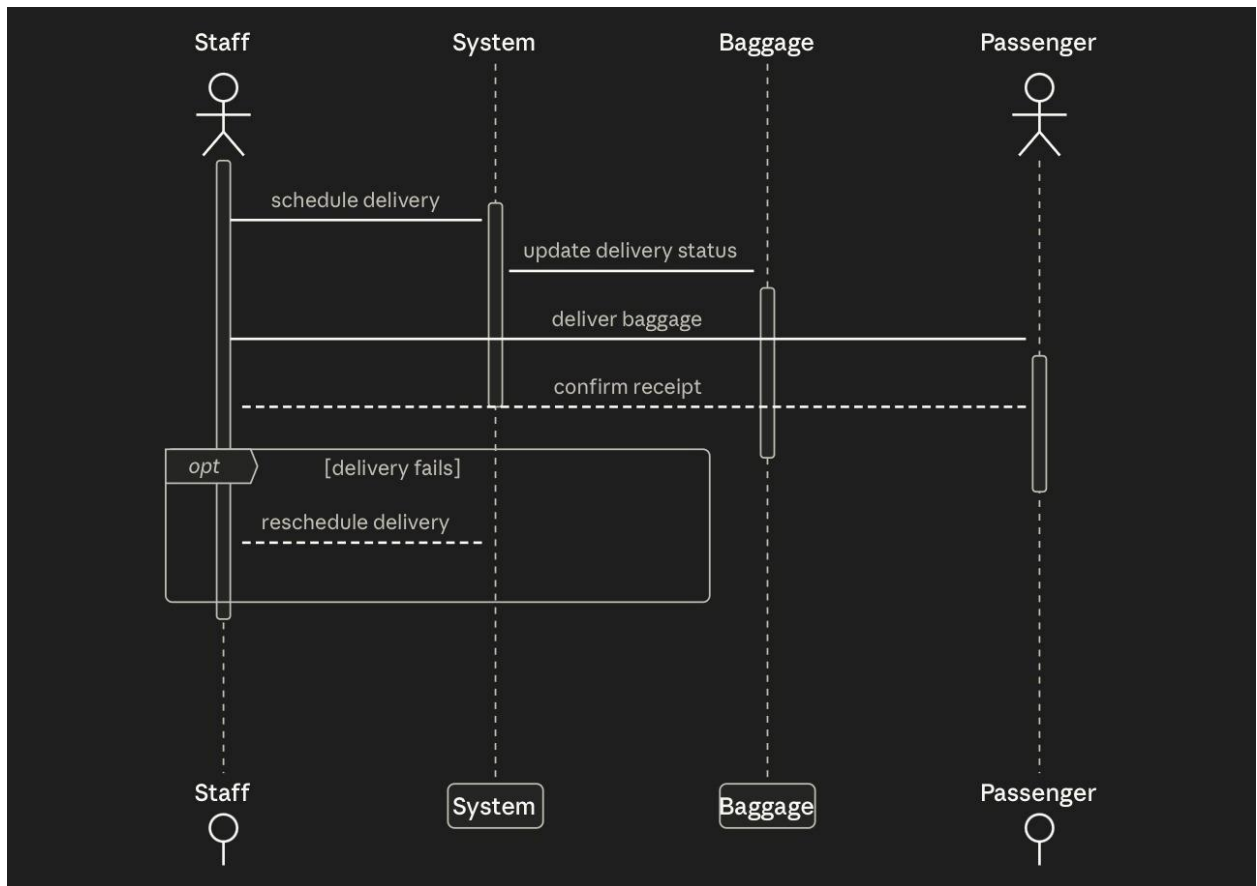
- **Description:**
This sequence shows how the system detects and handles delayed baggage situations.
- **Key Logic:**
The system updates baggage status and notifies passengers, while staff arranges delivery.
- **Compliance:**
Ensures timely communication and reliable tracking.



UC-12: Deliver Baggage to Passenger

Focus: Baggage Delivery

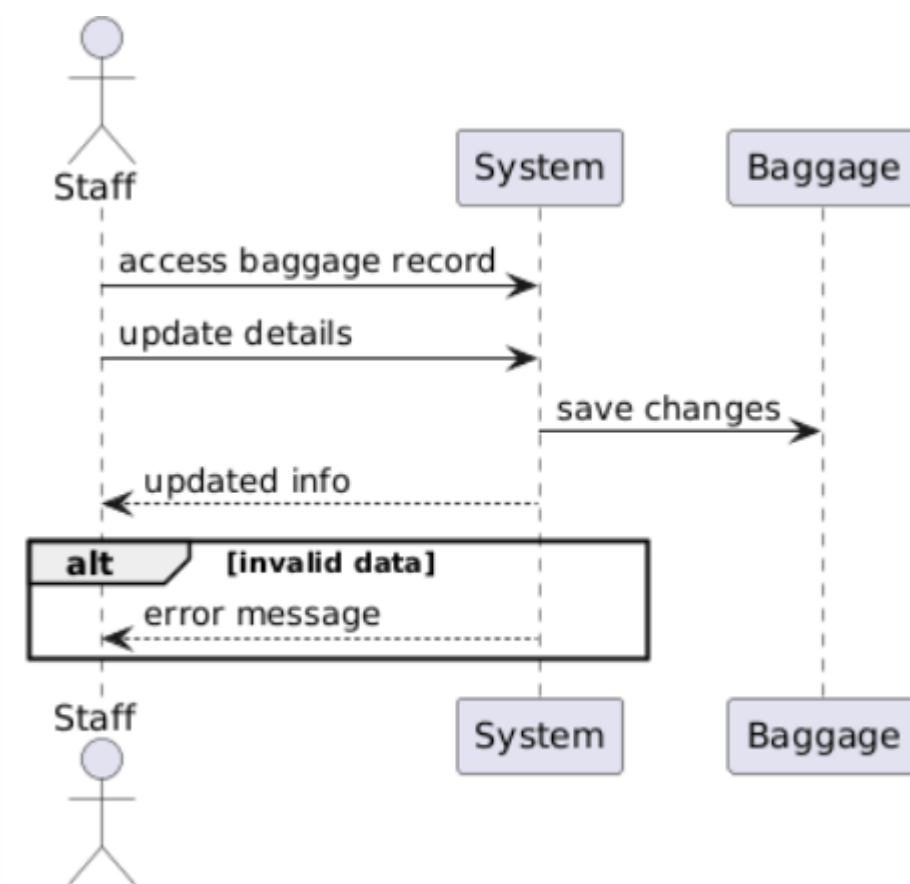
- **Description:**
This sequence illustrates how delayed or rerouted baggage is delivered to the passenger.
- **Key Logic:**
Staff schedules delivery, and the system updates delivery status until confirmation.
- **Compliance:**
Ensures secure and timely delivery.



UC-13: Update Baggage Information

Focus: Data Management

- **Description:**
This sequence shows how airport staff update baggage information in the system.
- **Key Logic:**
The system validates and saves updated data.
- **Compliance:**
Ensures data accuracy and secure access.



AMEO LULAJ:

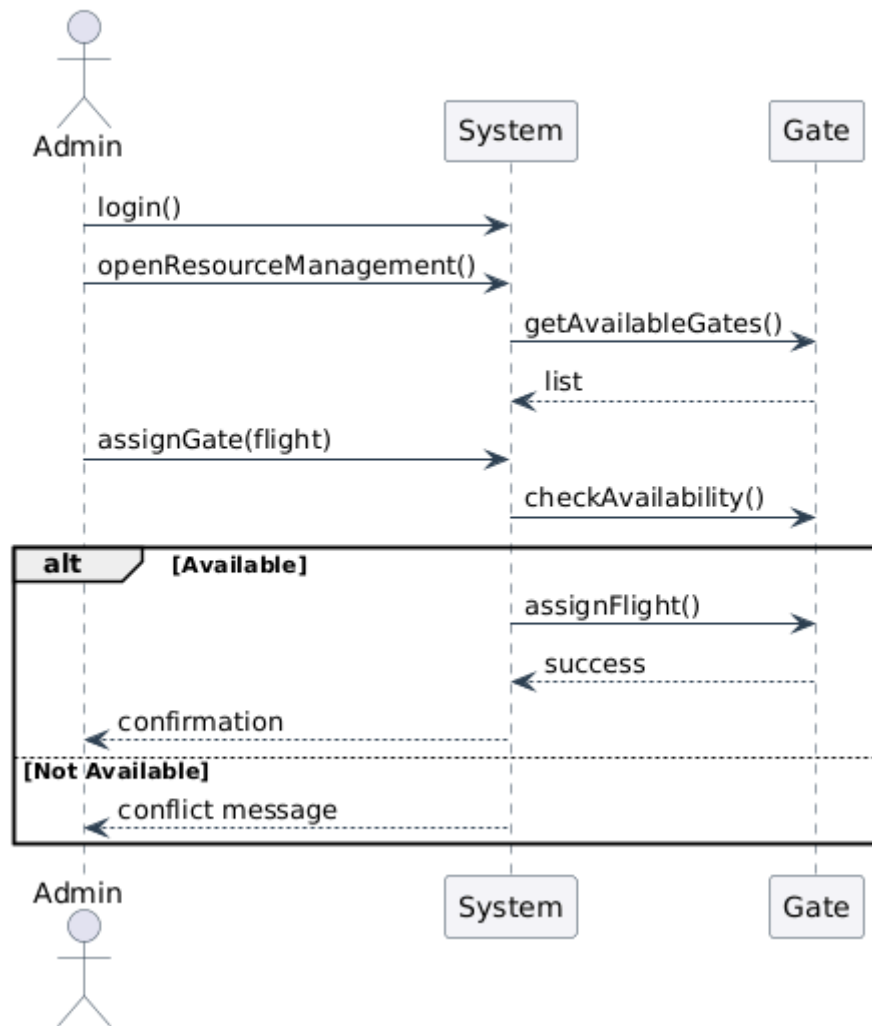
This sequence diagram represents the overall workflow of the Airport Resource Management module by integrating four use cases: Gate Allocation (UC-ARM-01), Runway Scheduling (UC-ARM-02), Staff Allocation (UC-ARM-03), and Equipment Management (UC-ARM-04). The process begins with the System Administrator accessing the resource management system.

Depending on the selected operation, the system interacts with different resources such as Gate, Runway, Staff, or Equipment. Each interaction follows a structured sequence where the system retrieves availability or status information, processes the administrator's request, and performs validation checks.

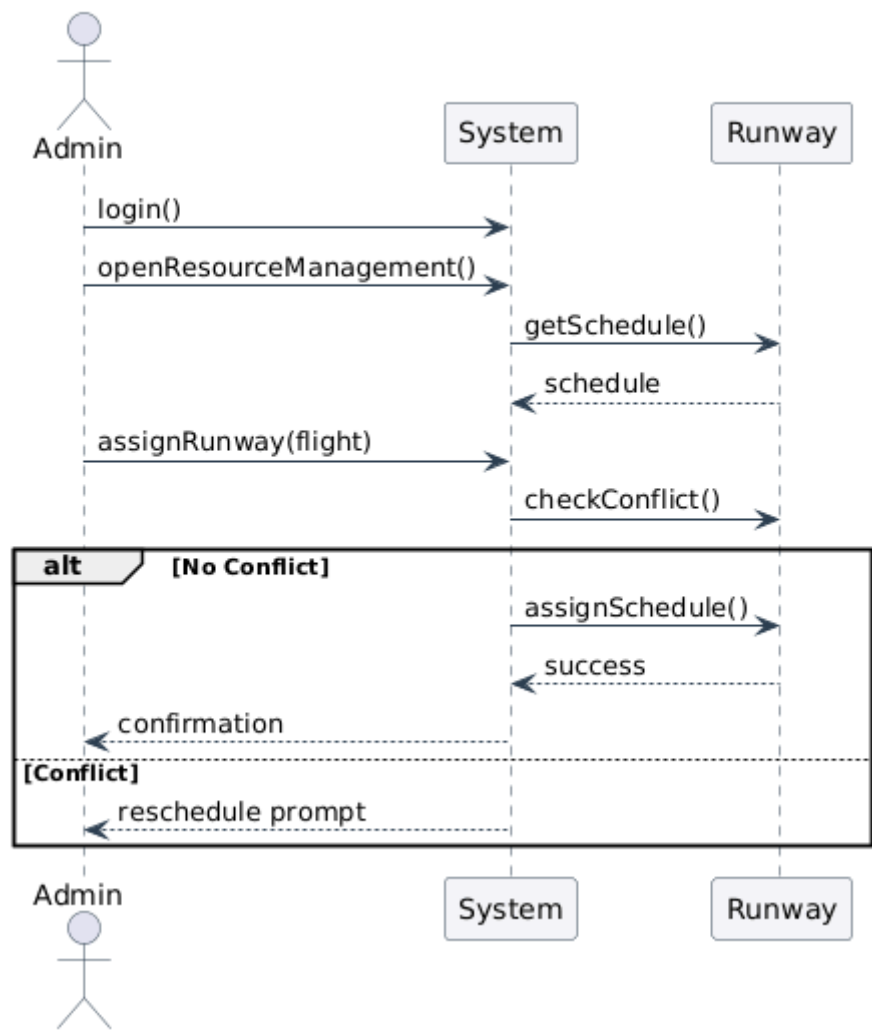
Alternative flows are represented using conditional paths. For example, if a resource is unavailable or a conflict is detected, the system notifies the administrator and suggests corrective actions. Otherwise, the system confirms the successful allocation or update of the resource.

Overall, the diagram demonstrates how the system coordinates multiple resource types while ensuring efficient allocation, conflict handling, and real-time status updates within airport operations.

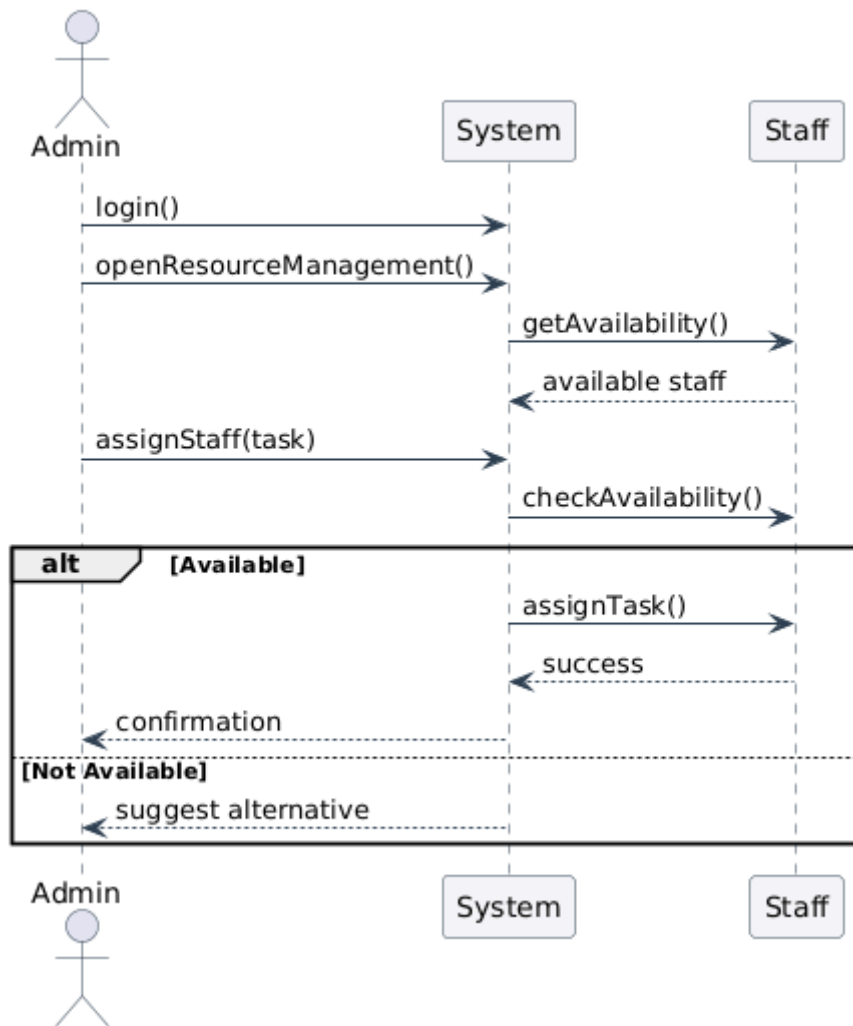
UC-ARM-01: Gate Allocation - Sequence Diagram



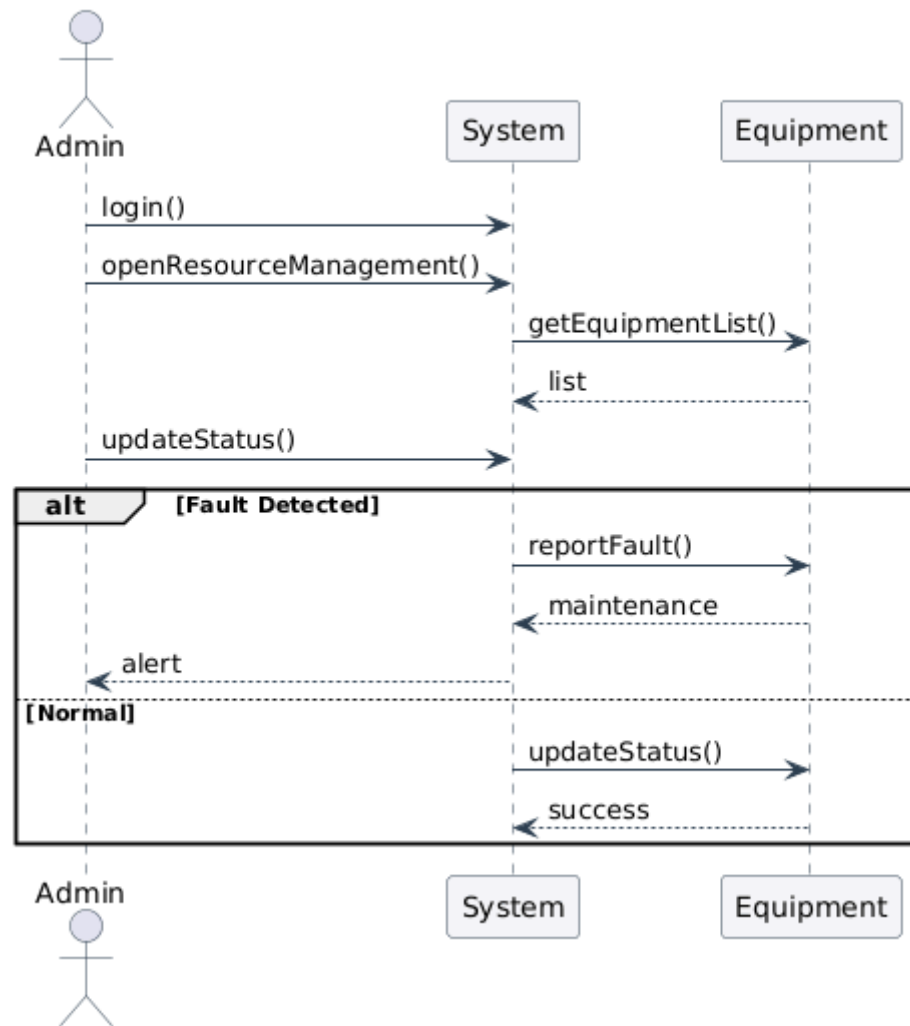
UC-ARM-02: Runway Scheduling - Sequence Diagram



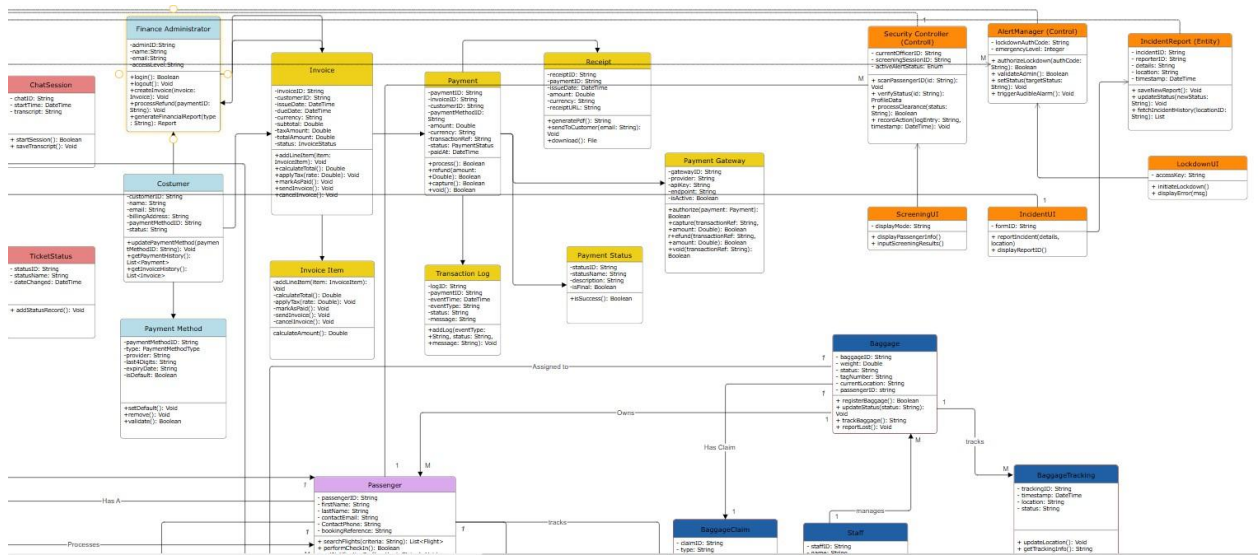
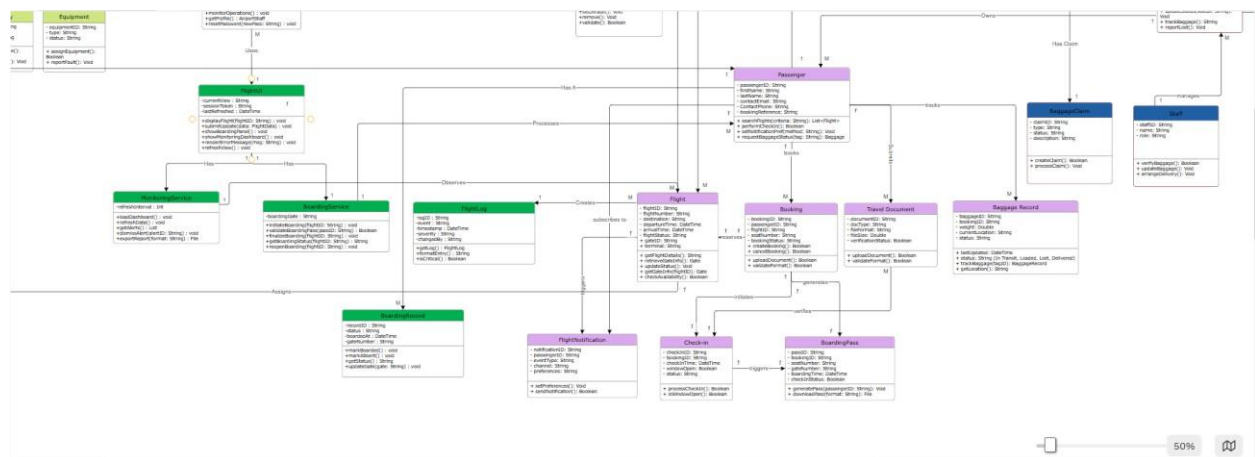
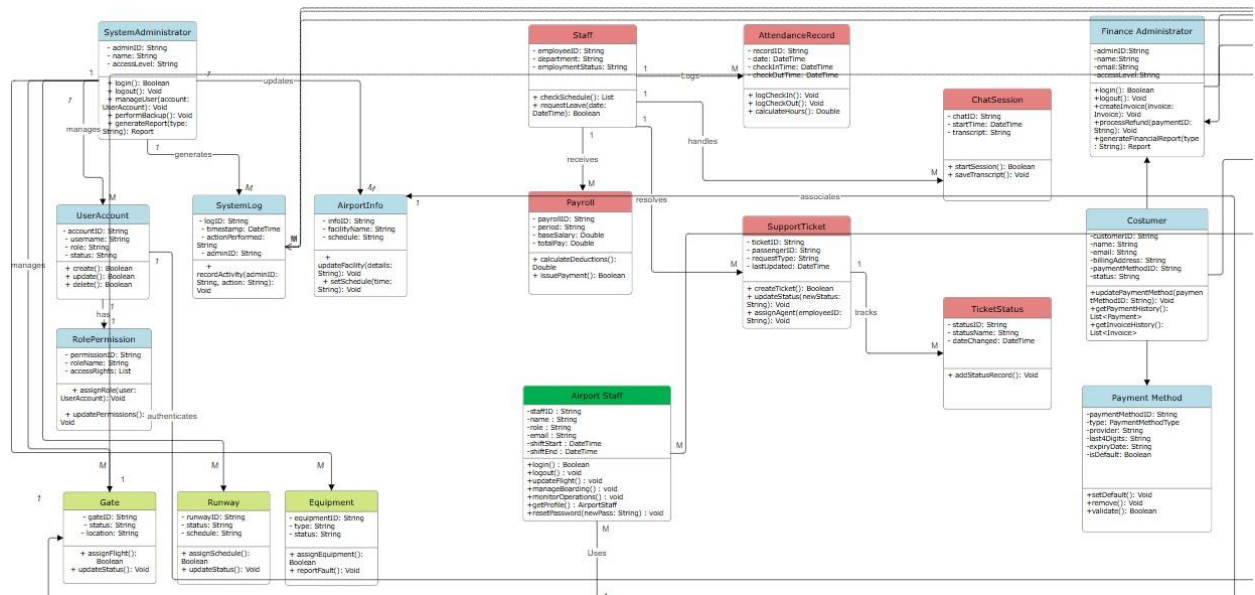
UC-ARM-03: Staff Allocation - Sequence Diagram



UC-ARM-04: Equipment Management - Sequence Diagram



Class Diagram:



Airport Management System

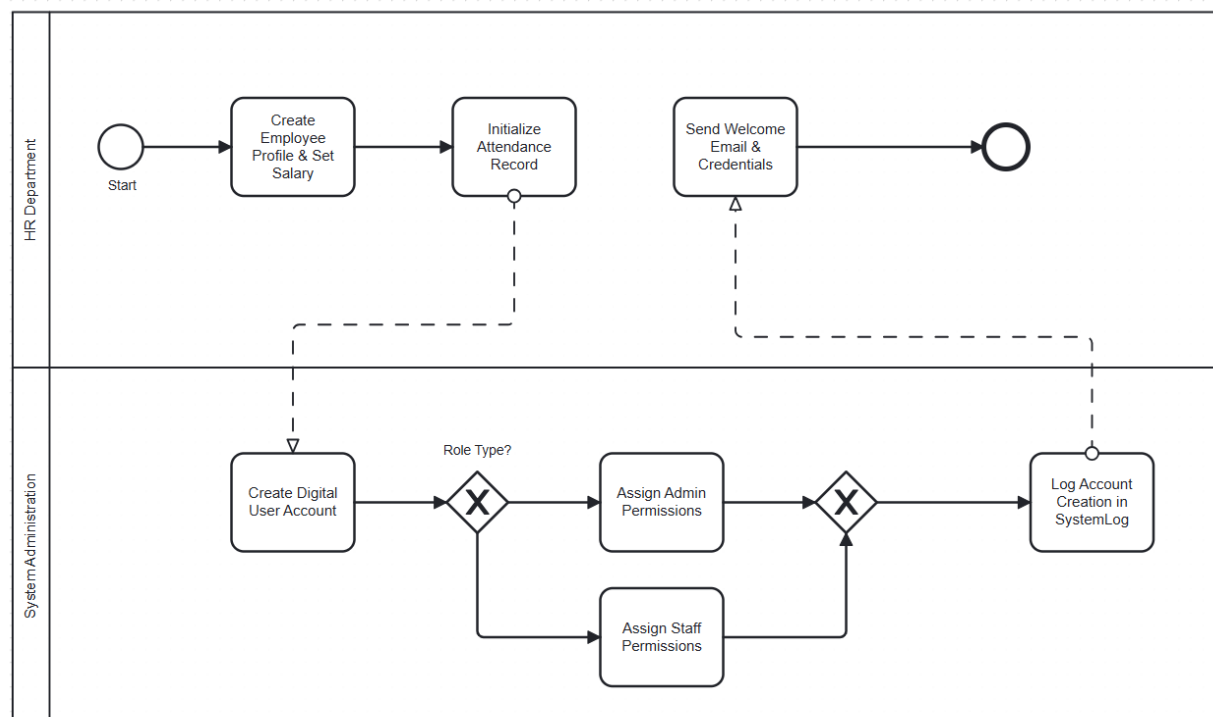
Business Process Modeling (BPMN) Specifications

Prepared by: Ajla Alcani & Eneida Balliu

Course: Software Modeling and Design Date: April 16, 2026

Document Objective

The purpose of this section is to outline the high-level business workflows of the Airport Management System. While our previous UML models focused on software objects and database interactions, these Business Process Model and Notation (BPMN) diagrams map the cross-functional operational procedures and physical handoffs between distinct airport departments. Please note that the following diagram represents the first of three main business processes modeled for this assignment.



Airport Management System

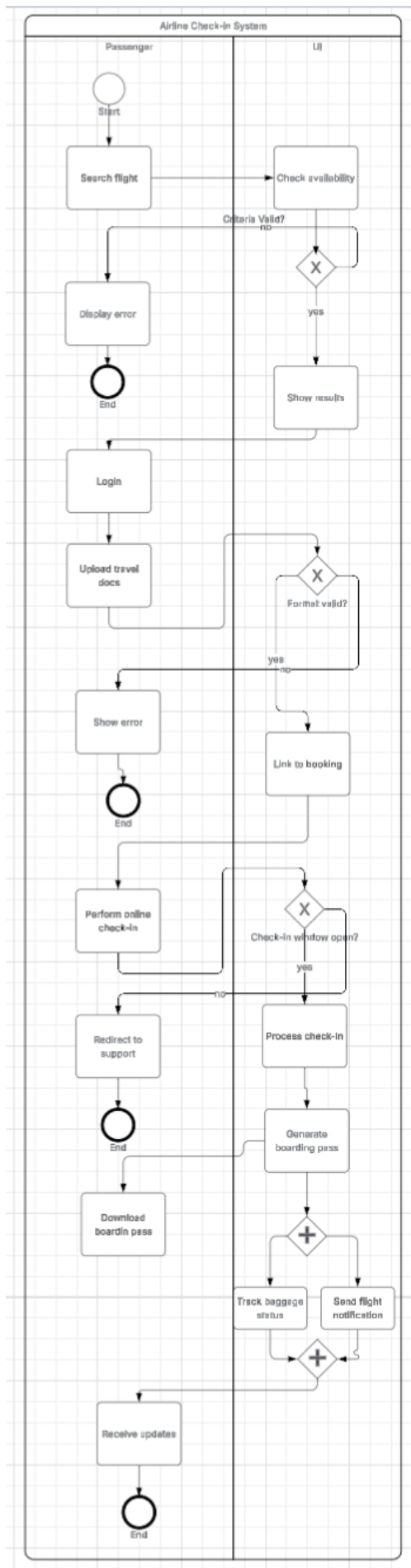
Business Process Modeling (BPMN) Specifications

Prepared by: Betül Kaan

Course: Software Modeling and Design Date: April 16, 2026

Document Objective

The purpose of this section is to outline the high-level business workflows of the Airport Management System. While our previous UML models focused on software objects and database interactions, these Business Process Model and Notation (BPMN) diagrams map the cross-functional operational procedures and physical handoffs between distinct airport departments.



Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: [Ajla Alcani](#)

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the **Human Resources and Customer Support Modules** through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the passage of time, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between objects.

Technical Scope

This document focuses on the interaction network between:

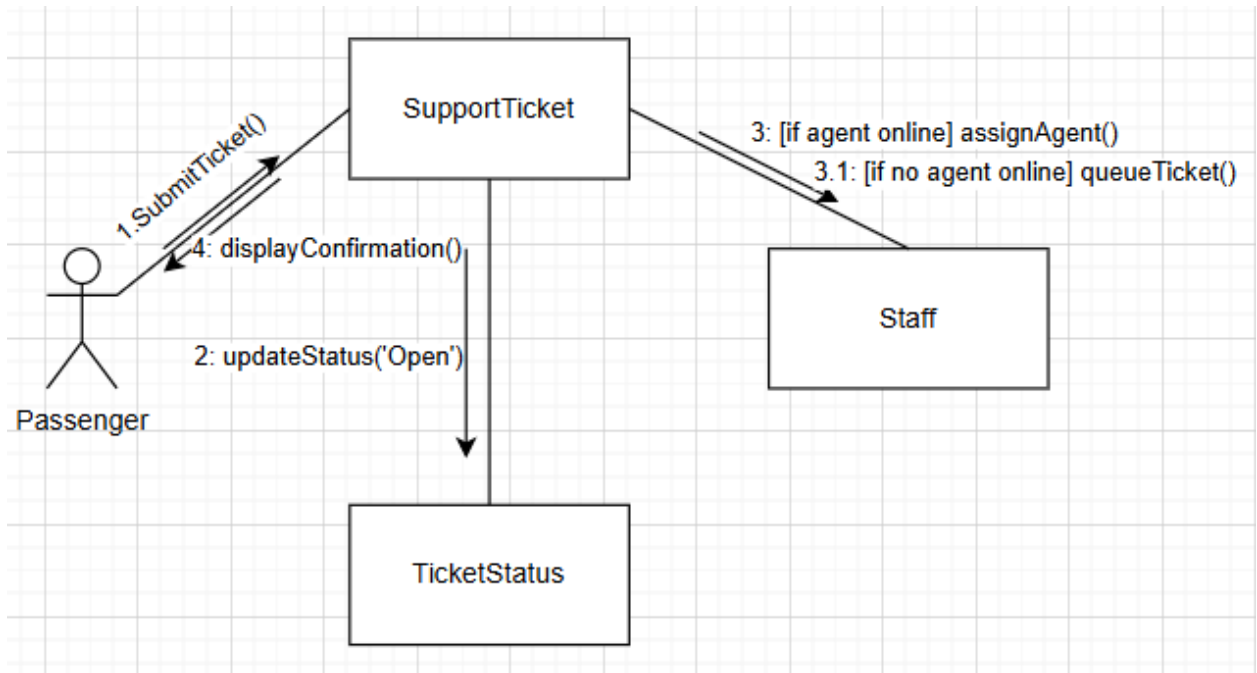
1. **Actors:** HR Admin, Employee (Staff), and Passenger.
2. **Control/Boundary Objects:** System elements responsible for routing requests (e.g., ChatSession, SupportTicket, Payroll).
3. **Entity Objects:** The database layer including TicketStatus, Staff, and AttendanceRecord.

System Interaction Standards

To maintain technical consistency, every collaboration diagram in this document adheres to the following architectural rules:

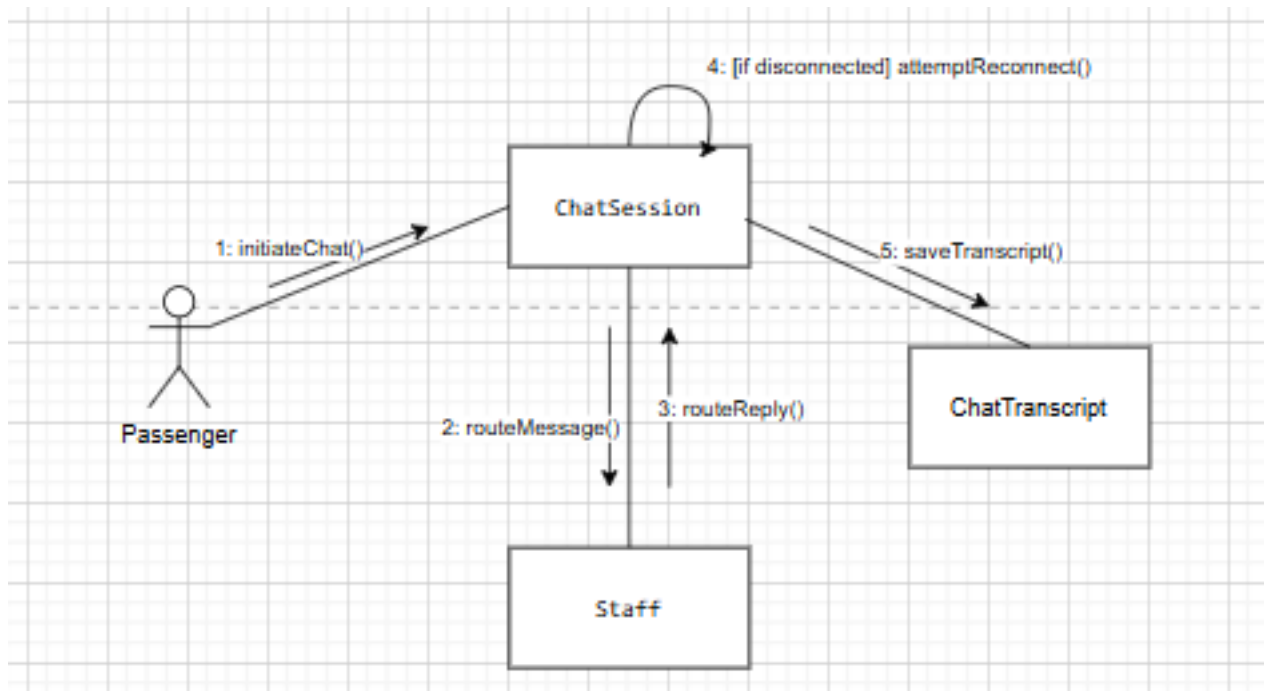
1. **Strict UML Notation:** All models utilize standardized UML collaboration/communication symbols.
2. **Structural Links:** Solid lines represent the structural pathways over which objects can communicate.
3. **Hierarchical Numbering:** Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2, 2.1) to clearly indicate nested operations and sequential logic.
4. **Controller-Centric Routing:** Central objects are used to route messages between actors and data entities, keeping the architecture decoupled and highly cohesive.

1. Submit Support Ticket



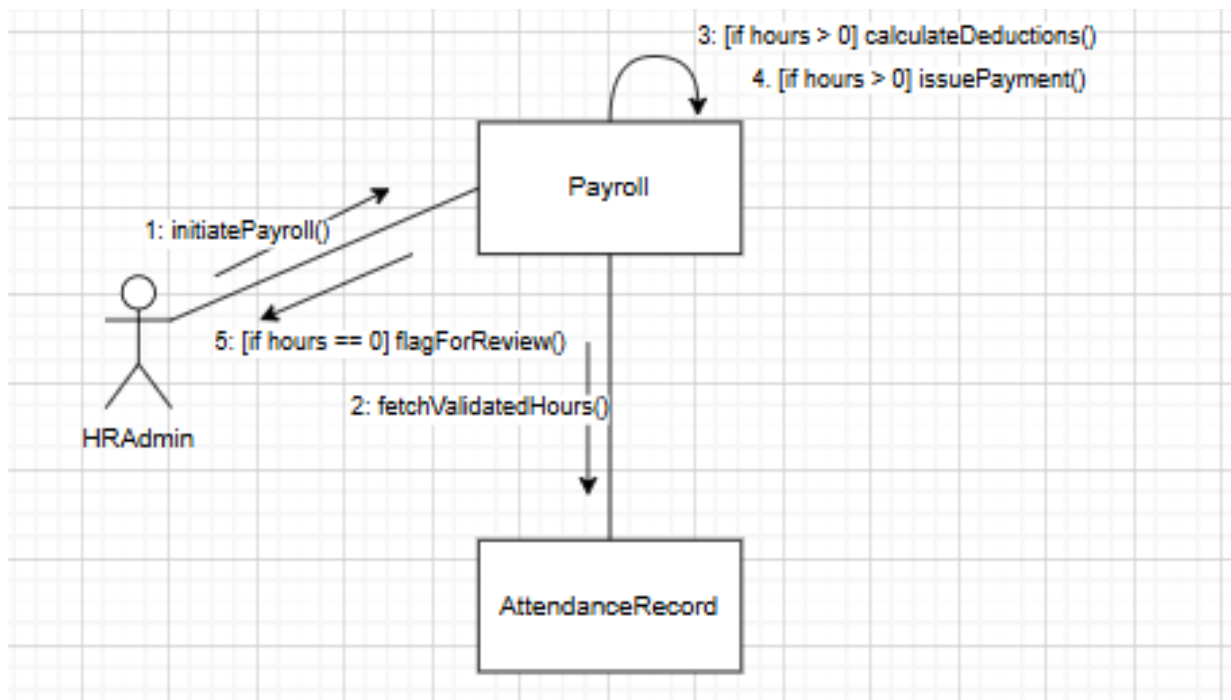
- **Focus:** Structural Message Routing for Ticket Resolution
- **Description:** This collaboration diagram maps the direct structural links between the **Passenger** actor, the core **SupportTicket** object, the assigned **Staff** agent, and the **TicketStatus** history entity.
- **Key Logic:** The numbered messages dictate the strict execution sequence. The **SupportTicket** acts as the central control object. Upon receiving the initial creation trigger (1), it sequentially logs its open state (1.1). It then routes the assignment to an available agent (2). Once the agent resolves the issue (3), the ticket ensures data compliance by logging its final resolved state (3.1).

2. Live Chat Session



- **Focus:** Synchronous Message Routing and Data Preservation
- **Description:** This collaboration diagram illustrates the structural communication pathways required to facilitate real-time chat between a Passenger and a Customer Support Agent (Staff), using the ChatSession as the central mediating object.
- **Key Logic:** The decimal numbering explicitly tracks the synchronous back-and-forth of the conversation. The passenger initiates the connection (1), which immediately prompts the session to notify an agent (1.1). During the active conversation loop, passenger messages (2) are structurally routed to the agent (2.1), and agent replies are routed back (2.2). Finally, when the passenger ends the session (3), the ChatSession object executes a self-message to save the transcript (3.1), fulfilling data retention requirements without needing external triggers.

3. Process Payroll

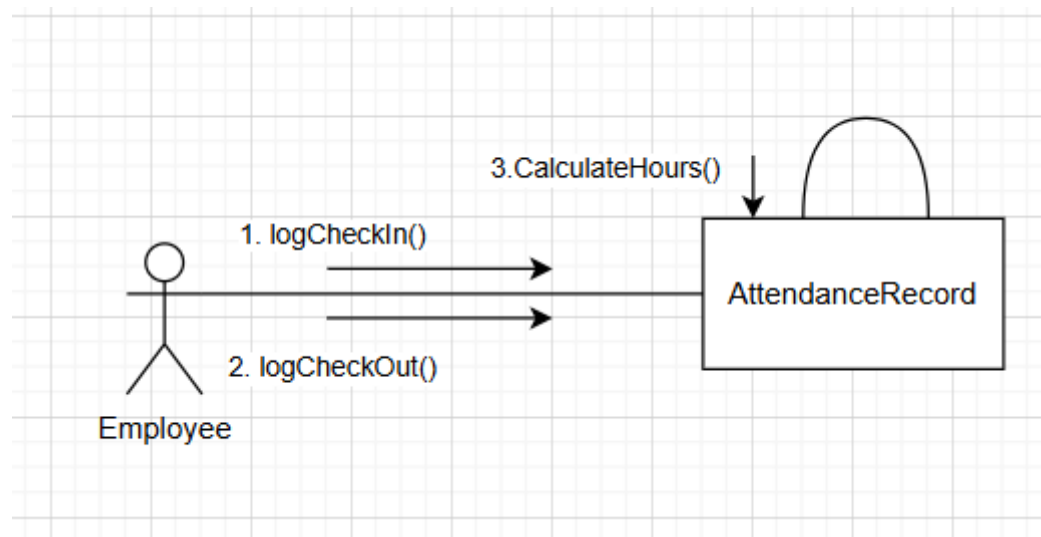


Focus: Internal Data Retrieval and Financial Processing

Description: This collaboration diagram maps the structural relationships required for salary disbursement. It highlights how the core *Payroll* object acts as an autonomous controller, sitting structurally between the HR Admin actor and the raw employee data.

Key Logic: The execution sequence guarantees strict data integrity. When the HR Admin initiates the process (1), the *Payroll* object does not immediately issue payment. Instead, it is structurally forced to route a request to the *AttendanceRecord* to calculate and return the validated hours (1.1). Only after receiving this external data does the *Payroll* object execute its internal self-messages to calculate deductions (2) and safely issue the final payment (3).

4. Log Staff Attendance



Focus: Structural Time Tracking and Automated Processing

Description: This collaboration diagram depicts the direct structural interaction between the Employee actor and the **AttendanceRecord** entity for tracking shift durations.

Key Logic: The interaction is divided into two distinct primary events. The employee initiates the sequence by routing a check-in message to the system (1). At the end of the shift, the employee structurally routes a check-out message (2). This second message instantly triggers a nested, internal system operation within the **AttendanceRecord** to calculate the total hours worked (2.1), ensuring automated and mathematically accurate timekeeping without manual HR intervention.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Eneida Balliu

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the System Administration Module through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the temporal order of events, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between system objects to ensure a robust and scalable management framework.

Technical Scope

This document focuses on the interaction network between:

1. Actor: System Administrator.
2. Control/Boundary Objects: System elements responsible for routing administrative requests (e.g., AccountController, AirportController, MonitoringController).
3. Entity Objects: The persistence layer including UserAccount, AirportResource, Flight, and the SystemLog.

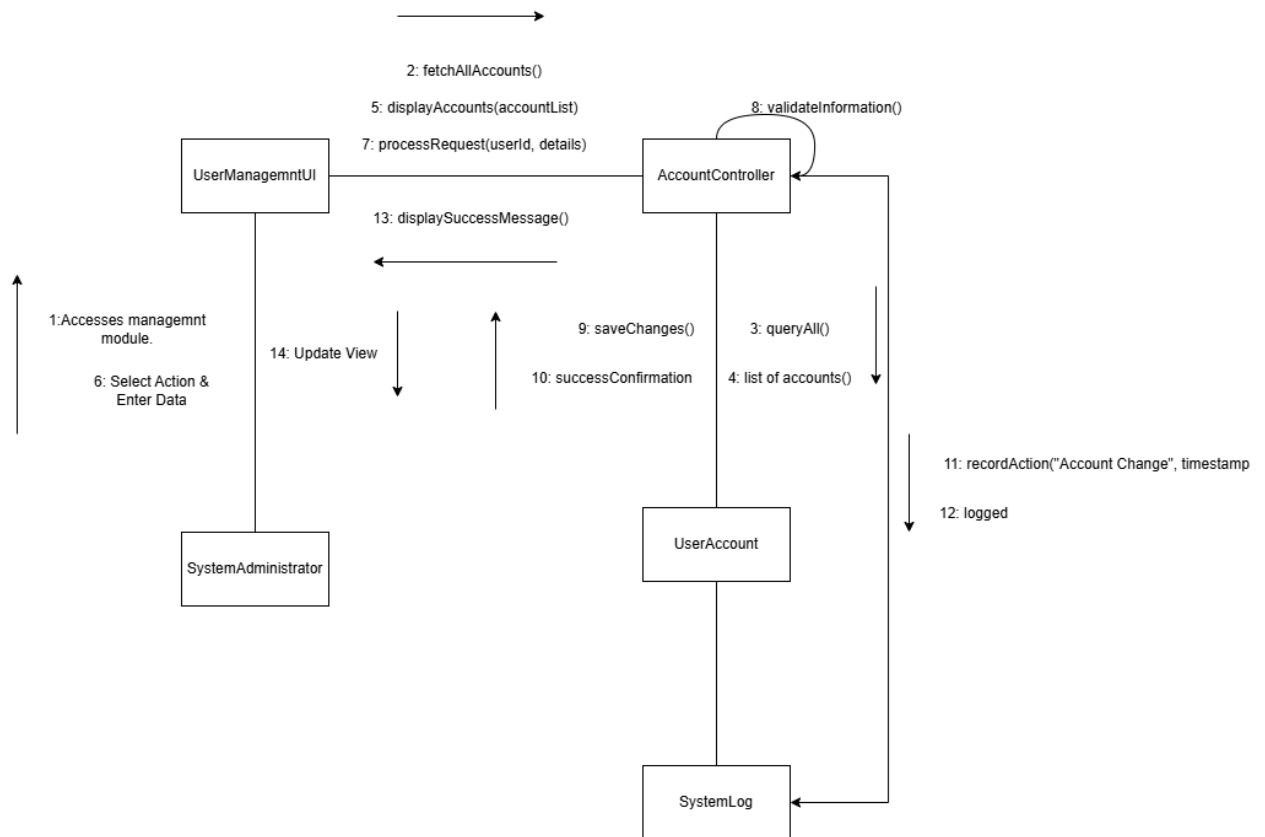
System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this document adheres to the following architectural rules:

1. Strict UML Notation: All models utilize standardized UML collaboration/communication symbols (Object Boxes, Link Lines, and Message Arrows).
2. Structural Links: Solid lines represent the static structural pathways over which objects are permitted to communicate.
3. Hierarchical Numbering: Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.
4. Controller-Centric Routing: Centralized Control objects are used to decouple the User Interface (Boundary) from the Database layer (Entities), ensuring high cohesion and low coupling across the management system.

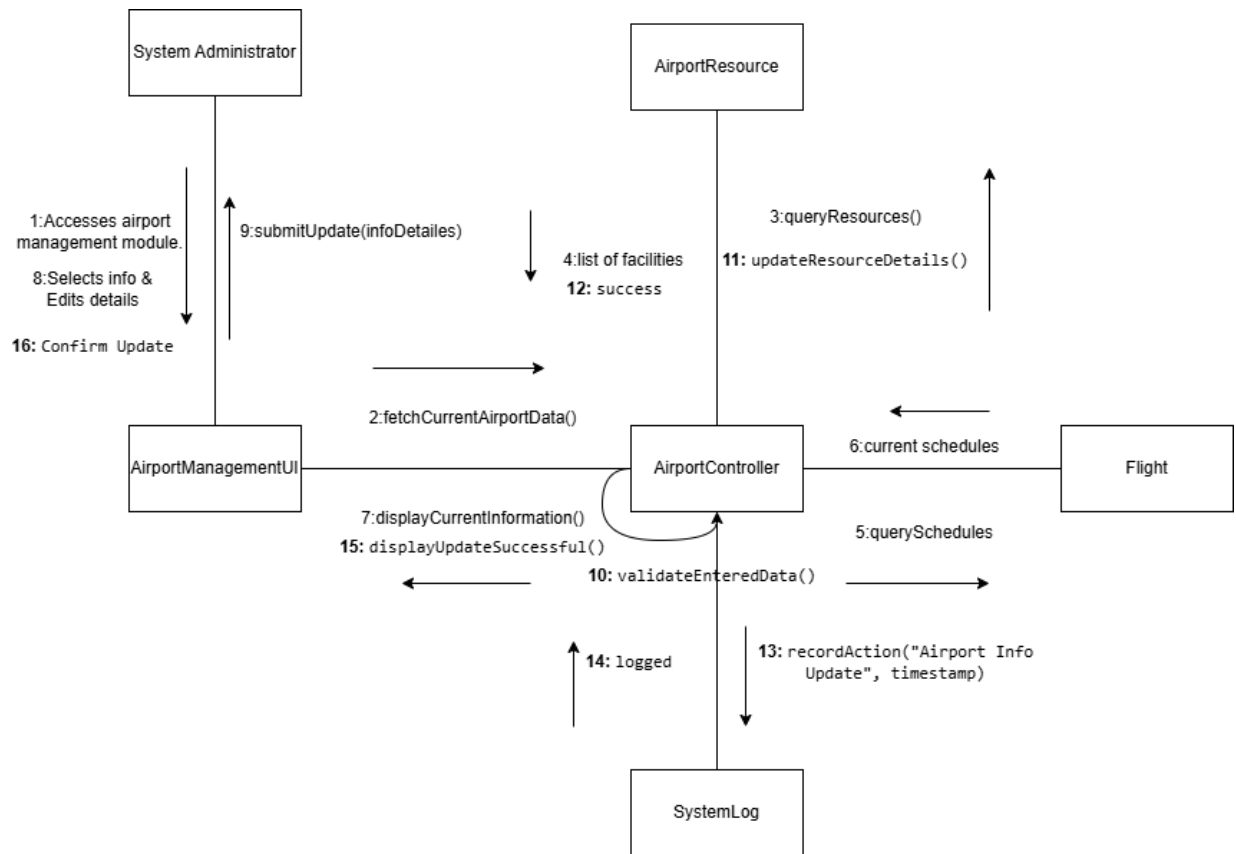
UC_40: Manage User Accounts

- Focus: CRUD operations for administrative security.
- Description: Illustrates the lifecycle of a user account from the UI request through the AccountController to database persistence in the UserAccount entity.
- Key Logic: Centralized validation (7) ensures data integrity before any saveChanges() operation is permitted.



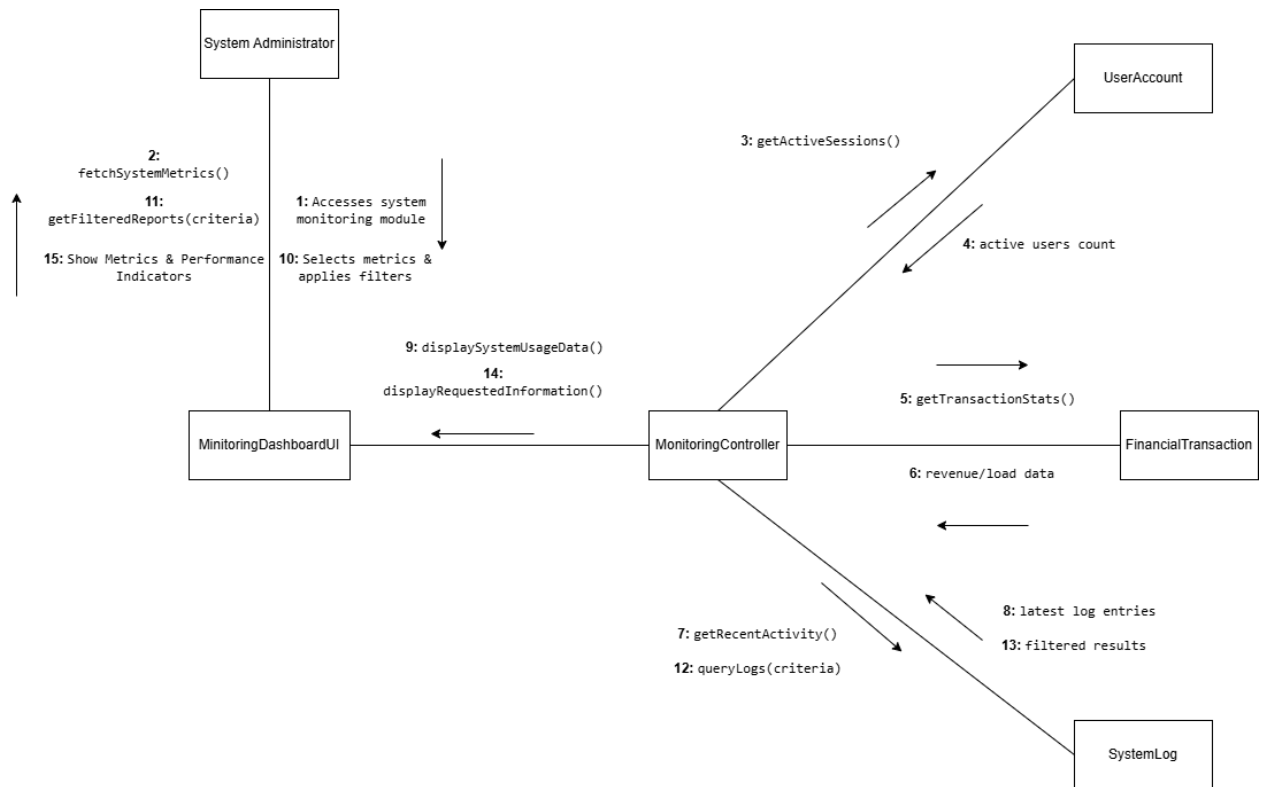
UC_41: Update Airport Information

- *Focus:* Multi-entity synchronization of airport infrastructure.
- *Description:* Models the AirportController as a coordinator that pulls and pushes data across both AirportResource (facilities) and Flight (schedules) objects.
- *Key Logic:* Divergent message routing (3 & 4) allows a single administrative action to update disparate system components simultaneously.



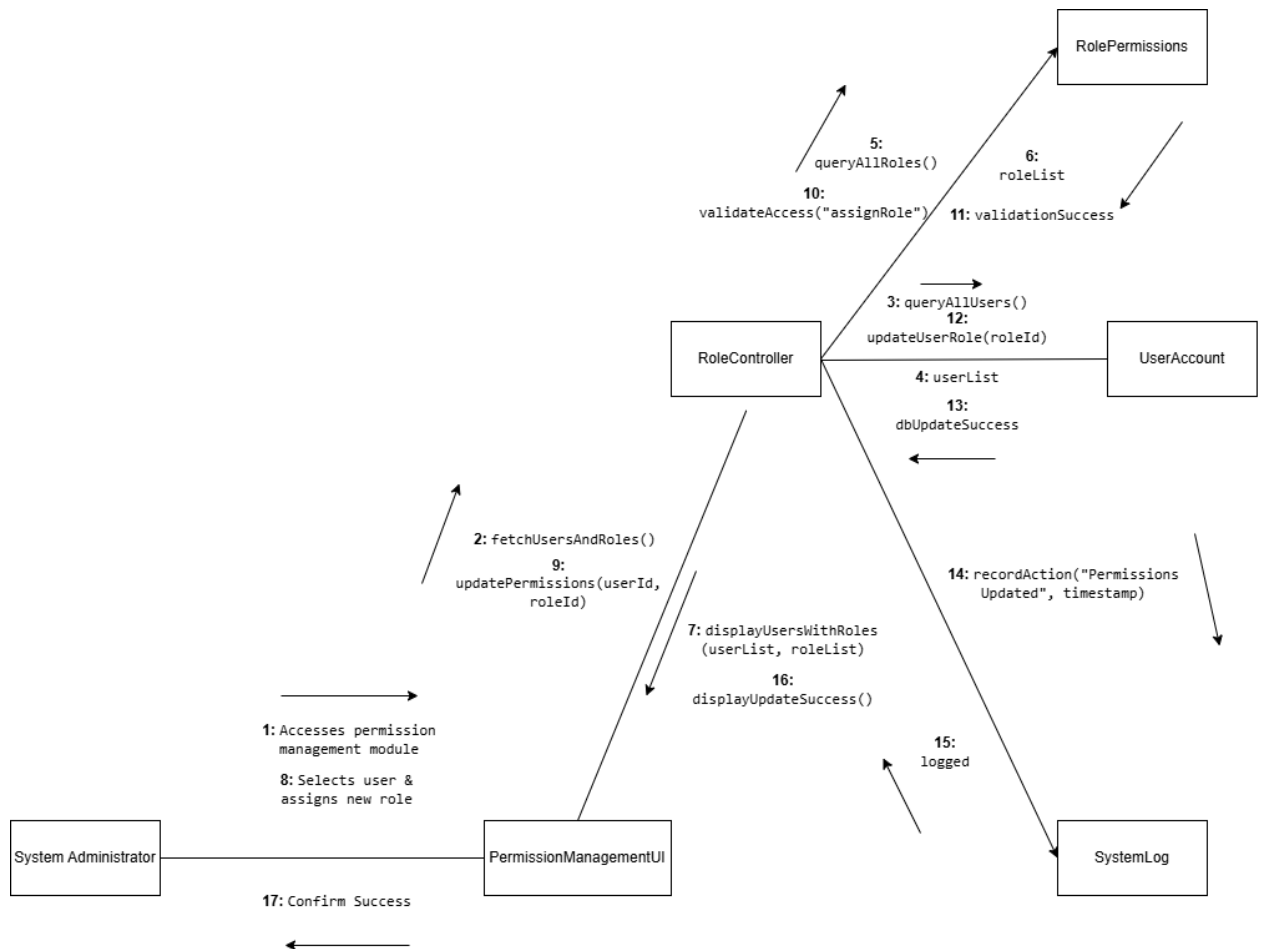
UC_42: Monitor System Usage

- *Focus: Real-time data aggregation and performance visibility.*
- *Description: Demonstrates a high-fan-out retrieval pattern where the MonitoringController queries UserAccount, FinancialTransaction, and SystemLog to populate a unified dashboard.*
- *Key Logic: Read-only data flow (3, 4, 5) optimized for low-latency reporting without altering the state of the entity objects.*



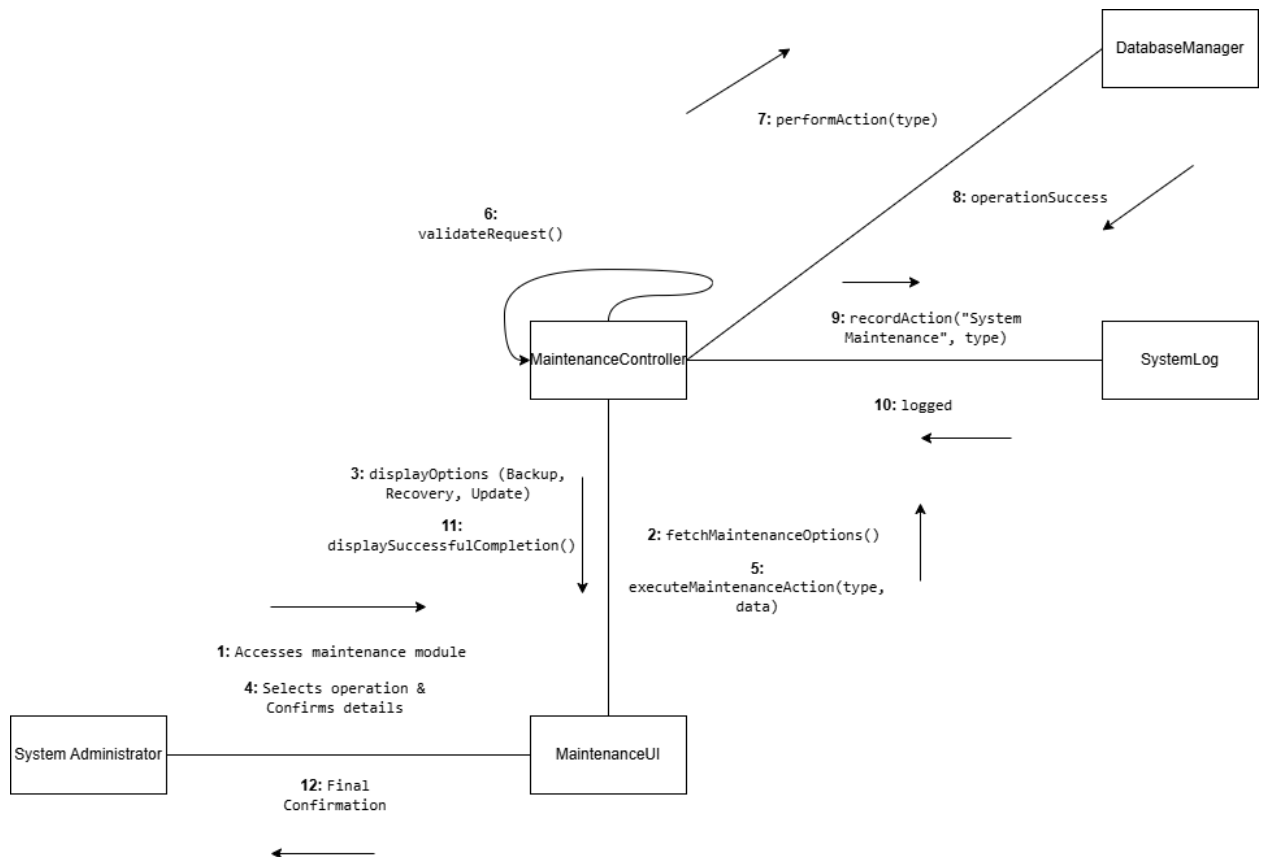
UC_43: Manage System Permissions

- *Focus:* Role-Based Access Control (RBAC) enforcement.
- *Description:* Details the structural path for modifying access rights, requiring coordination between UserAccount profiles and RolePermissions definitions.
- *Key Logic:* Nested validation (10) ensures that permission changes are checked against existing security policies before the user record is updated.



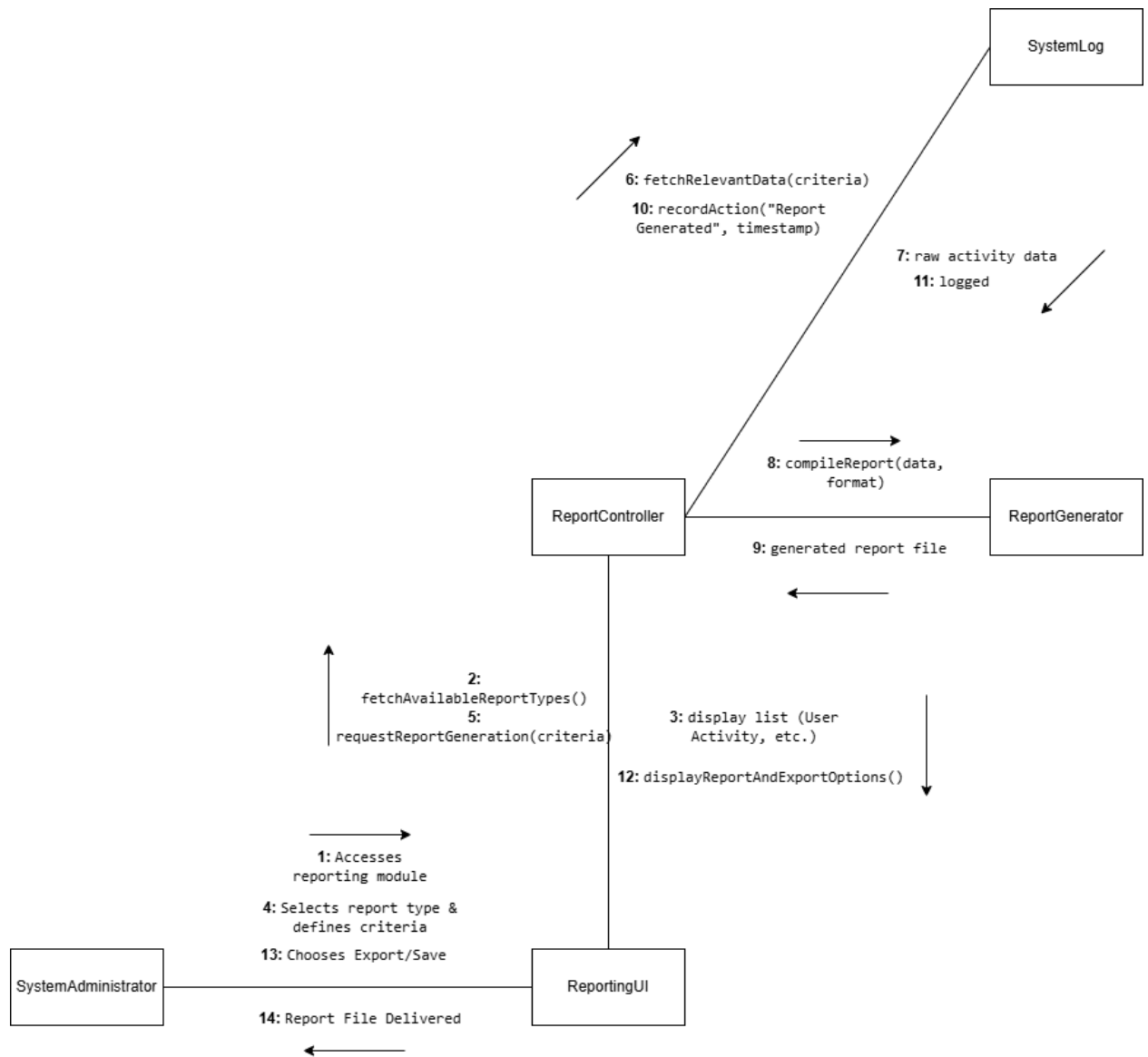
UC_44: Perform System Maintenance

- *Focus:* Database health and disaster recovery operations.
- *Description:* Traces the execution of critical maintenance tasks (backup/restore) handled by a specialized DatabaseManager utility via the MaintenanceController.
- *Key Logic:* Atomic execution flow—the system ensures a successful operationSuccess (8) return from the database before logging the activity.



UC_45: Generate System Activity Reports

- *Focus: Data compilation and professional auditing.*
- *Description: Shows the interaction between the ReportController and a dedicated ReportGenerator utility to transform raw SystemLog data into formatted documents.*
- *Key Logic: Sequential processing where raw data is first fetched (6), then passed to a utility object (8) for transformation, keeping the controller decoupled from formatting logic.*



Airport Management System

Flight Operations Management – Use Case Specifications

Course: Software Modeling and Design

Date: March 20, 2026

Prepared by: Albina Blloshmi

Document Objective

The primary objective of this document is to present a structured analysis of the Baggage Management Module within the Airport Management System through a collection of formal Use Cases.

The document models the interaction between passengers, airport staff, and the system during baggage handling operations, including baggage claiming, tracking, delayed baggage processing, baggage delivery, and baggage information updates.

These Use Cases define the operational workflow, alternative flows, validation procedures, and system requirements necessary to ensure efficient, secure, and reliable baggage management services.

Technical Scope

This document focuses on the operational interaction between:

1. Actors

- *Passenger*
- *Airport Staff*
- *System*

2. Functional Components

- *Baggage Tracking Module*
- *Claim Processing Module*
- *Delivery Management Module*
- *Notification and Monitoring Services*

3. Entity Objects

- *BaggageRecord*
 - *PassengerAccount*
 - *FlightInformation*
 - *DeliveryStatus*
 - *SystemLog*
-

System Interaction Standards

To maintain consistency and academic rigor, every Use Case in this document follows these standards:

1. Structured Use Case Format

Each Use Case contains:

- Summary
- Dependency
- Actors
- Preconditions
- Main Sequence
- Alternative Sequence
- Non-functional Requirements
- Postconditions

2. Sequential Logic

All operational steps are described in chronological order to clearly define actor-system interaction.

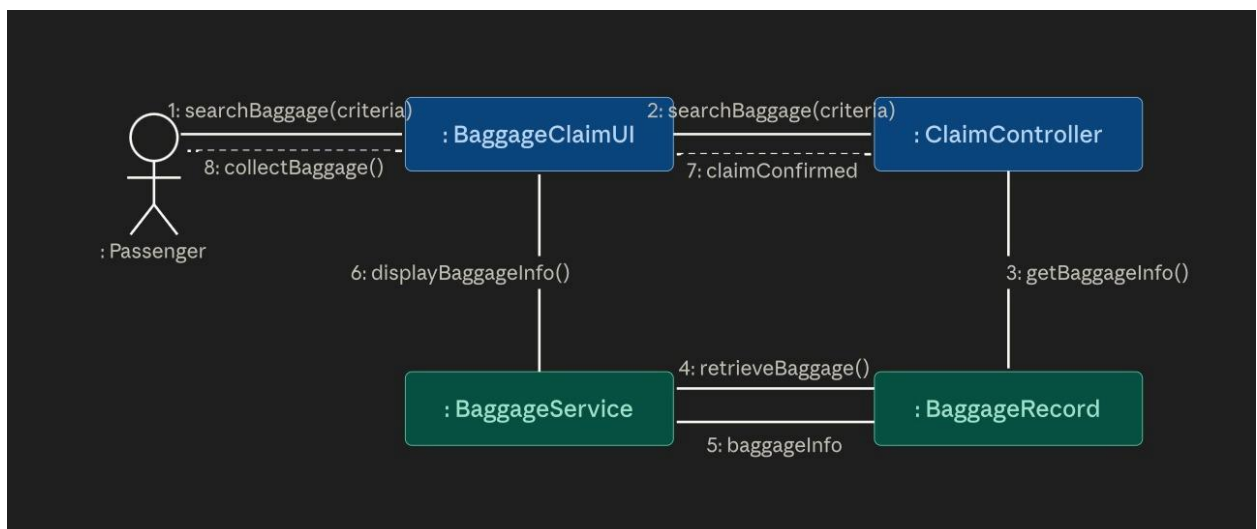
3. Validation Control

System validation ensures data accuracy before processing baggage operations, tracking requests, claims, or updates.

4. Secure System Processing

All baggage operations are executed through authenticated system procedures to maintain security and data integrity.

UC-08: Claim Baggage at Destination



Focus

Passenger baggage retrieval process after flight arrival.

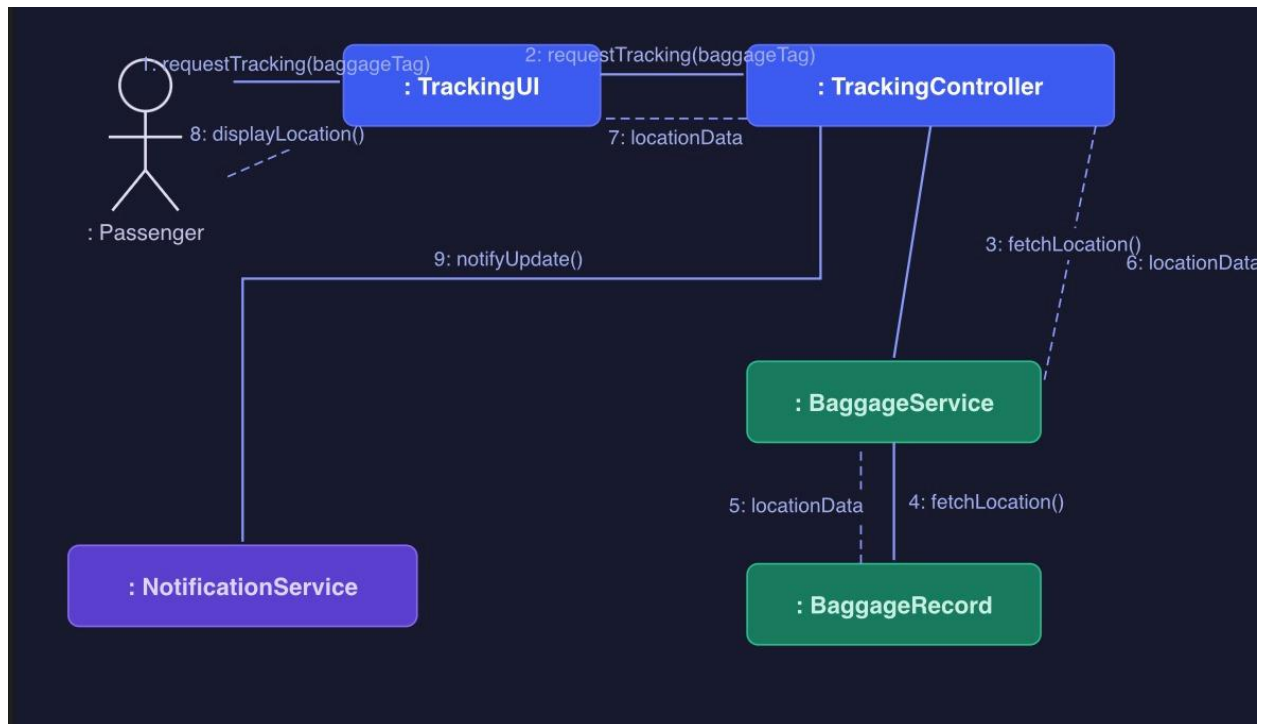
Description

This Use Case models the workflow in which a passenger retrieves baggage from the designated baggage claim area after arriving at the destination airport.

Key Logic

The system coordinates baggage carousel assignment and supports issue reporting if baggage cannot be located.

UC-09: Track Baggage in Real-Time



Focus

Real-time baggage location monitoring.

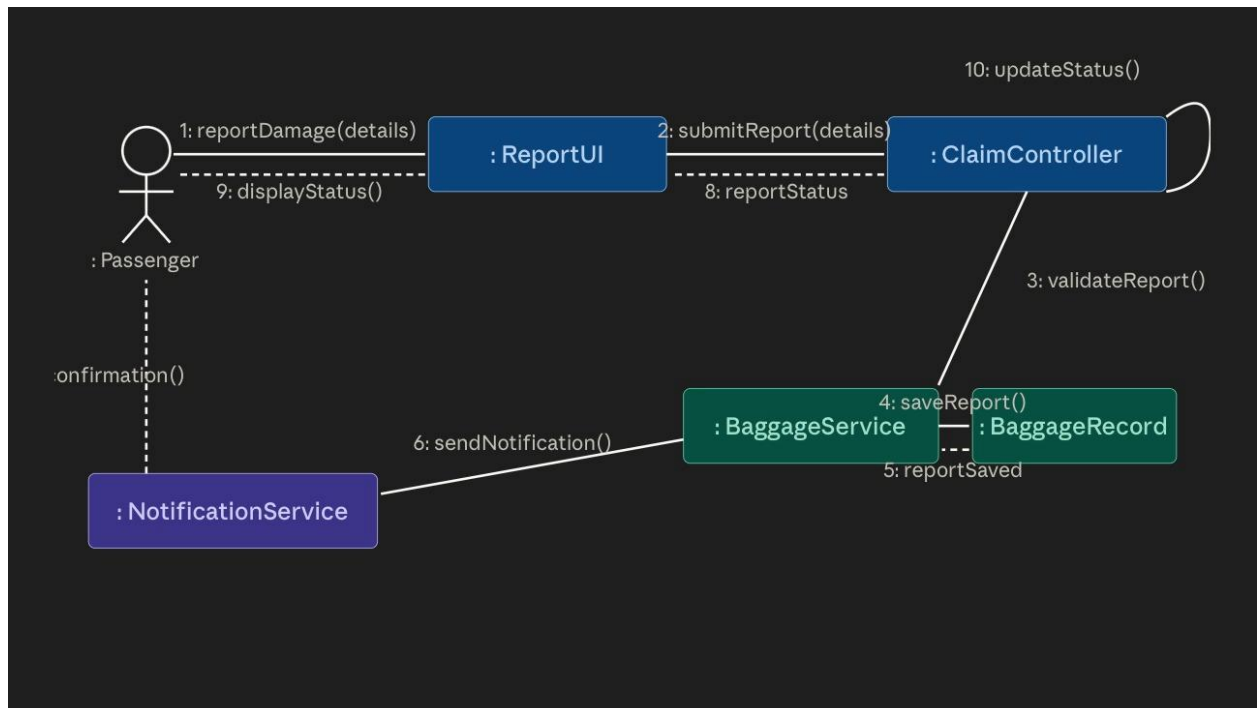
Description

This Use Case illustrates how passengers monitor baggage movement and status updates throughout the journey using the tracking system.

Key Logic

The tracking module continuously retrieves live baggage data and displays the latest known location to the passenger.

UC-10: Report Damaged Baggage



Focus

Damage reporting and claim processing.

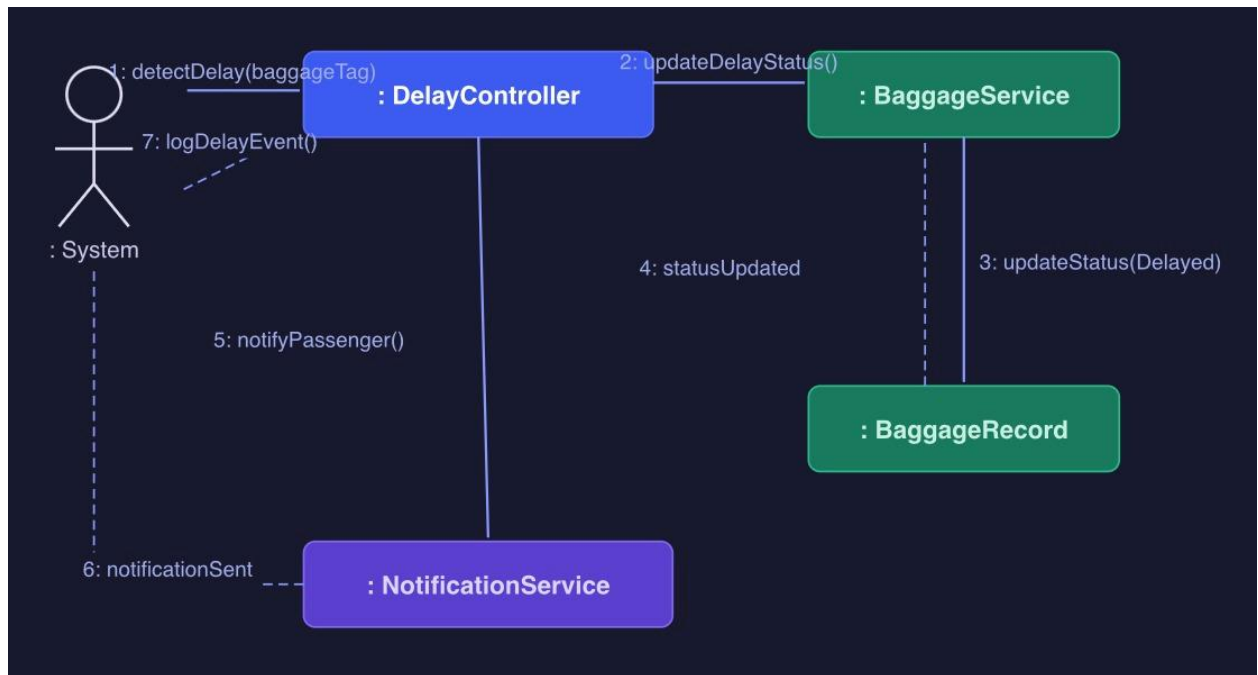
Description

This Use Case models the process where passengers report damaged baggage after retrieval and airport staff verify and process the complaint.

Key Logic

Centralized verification ensures that only valid baggage damage claims are accepted and processed.

UC-11: Handle Delayed Baggage



Focus

Management of delayed baggage situations.

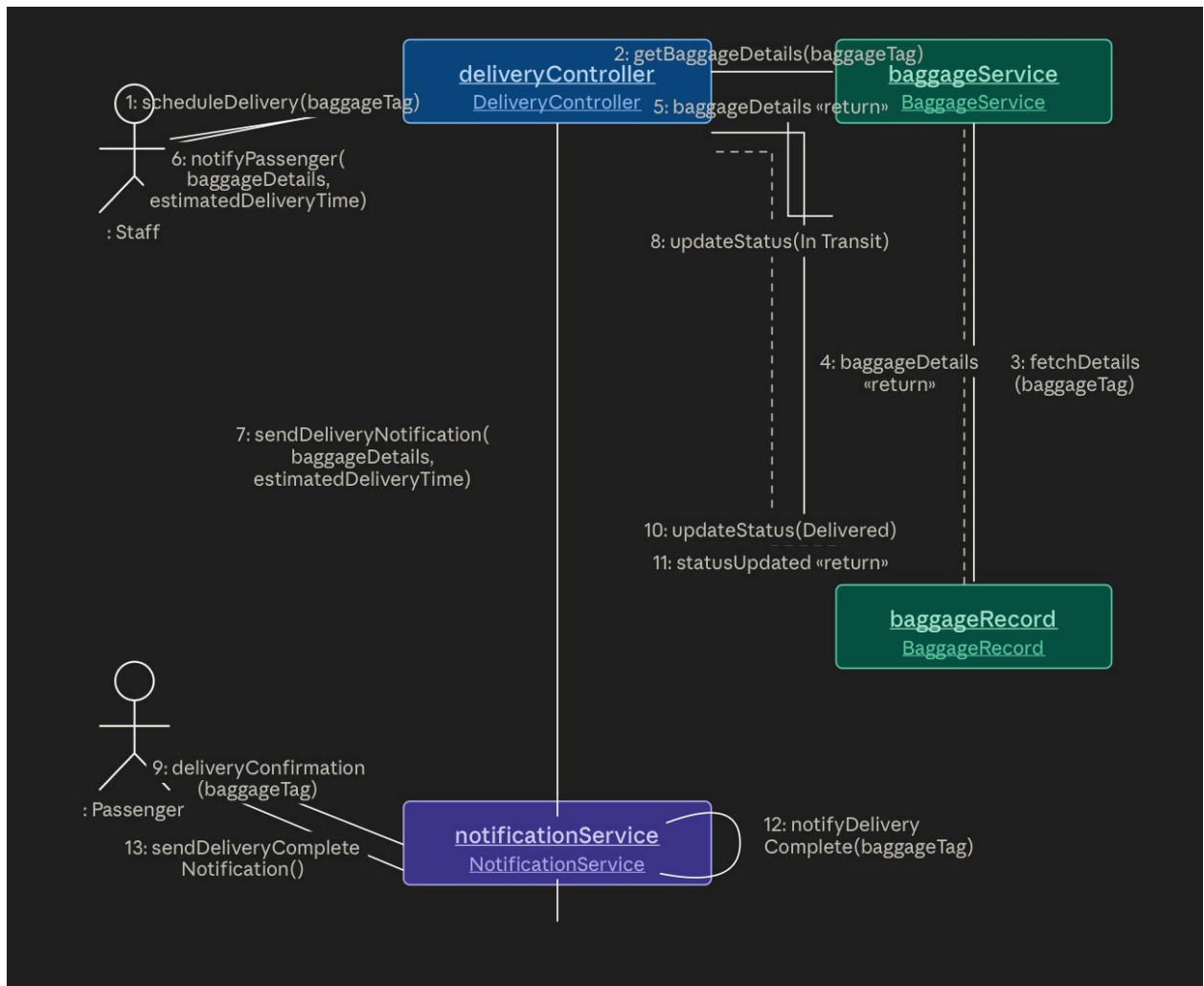
Description

This Use Case describes how the system detects baggage delays, updates baggage status, notifies passengers, and initiates recovery procedures.

Key Logic

Automated notifications and tracking updates minimize passenger uncertainty and improve operational efficiency.

UC-12: Deliver Baggage to Passenger



Focus

Baggage delivery coordination after delay recovery.

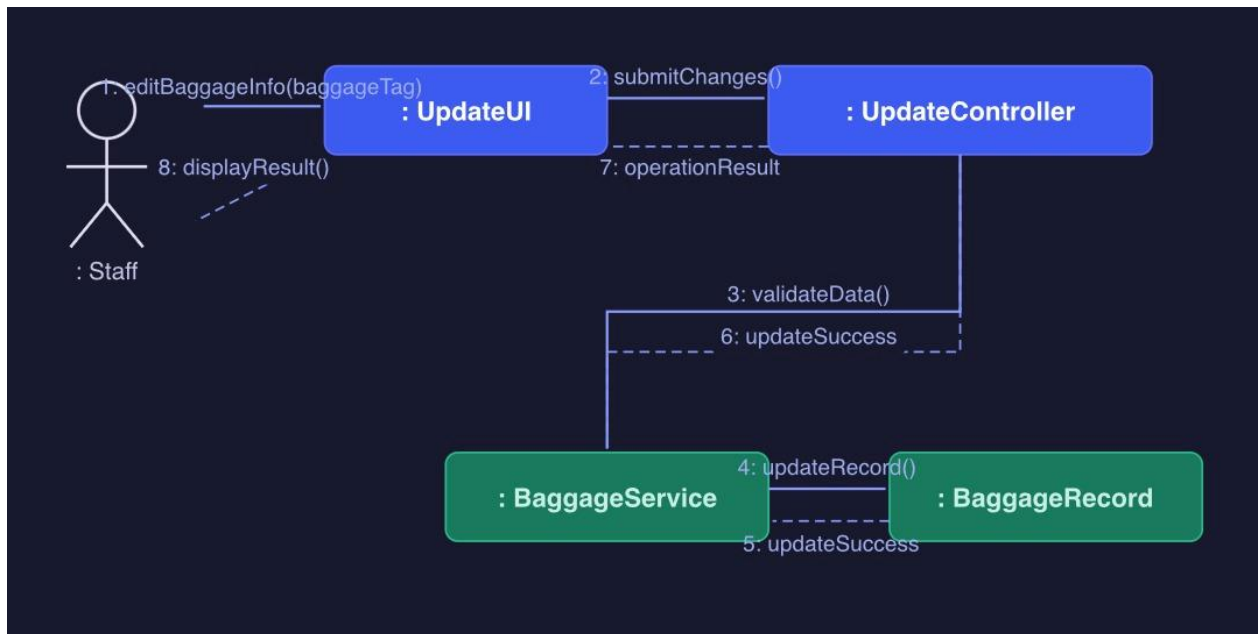
Description

This Use Case models the delivery workflow where located baggage is transported and delivered to the passenger.

Key Logic

The delivery subsystem coordinates scheduling, status updates, and delivery confirmation.

UC-13: Update Baggage Information



Focus

Modification and maintenance of baggage records.

Description

This Use Case illustrates how airport staff access and update baggage information within the system database.

Key Logic

Input validation and secure access control ensure accurate and protected baggage data management.

Conclusion

This document provides a structured representation of baggage management operations within the Airport Management System. The presented Use Cases define the interaction logic, validation flow, and operational responsibilities required to support secure, accurate, and efficient baggage handling services for both passengers and airport staff.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Andri Mucllari

Document Objective

The primary objective of this document is to define the structural organization and message flow of the **Finance and Billing Management subsystem** within the Airport Management System. This is achieved through a set of Collaboration (Communication) Diagrams.

Unlike Sequence Diagrams, which focus on the chronological order of interactions, these Collaboration Diagrams emphasize the architectural relationships between system components and the routing of messages across objects involved in financial operations such as payments, transaction monitoring, and billing processes.

This document focuses on the interaction network between the following components:

Actors:

Finance Staff, Airport Staff, Passenger

Control/Boundary Objects:

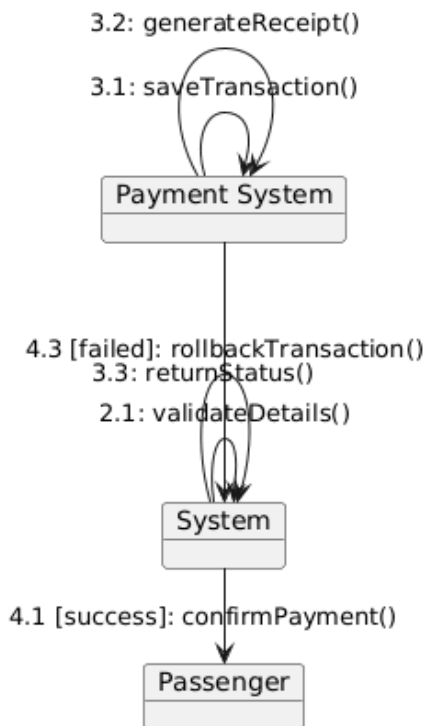
Core system components responsible for handling and routing financial operations, such as PaymentSystem, BillingController, TransactionMonitor, and AttendanceSystem

Entity Objects:

Data-layer components responsible for storing and managing financial and administrative data, including Transaction, Invoice, StaffRecord, and AttendanceRecord

1. **Strict UML Notation:** All diagrams adhere to standard UML communication diagram conventions.
2. **Structural Links:** Solid lines represent communication paths between objects, defining how components are connected within the system.
3. **Message Flow Representation:** Messages indicate interactions between actors, control objects, and entity objects, clearly showing how financial processes are executed

Payment Transaction



- This diagram shows how a passenger interacts with the airport system to complete a service payment. The system validates the payment details, communicates with an external payment system to process the transaction, and then returns the result to the passenger.
- It illustrates the structured communication and message flow between all components involved in the payment process

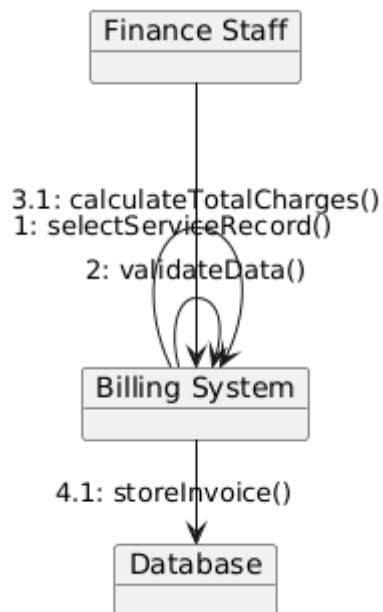
Actors

Passenger – the user selecting a service and providing payment details

System – the main application that handles validation, processing logic, and user interaction

Payment System – the external service responsible for processing the payment and generating transaction records

Airport Billing Process



- This diagram shows how a passenger requests an airport service, triggering the airport service system to communicate with the invoice system to generate an invoice, which is then returned to the passenger.

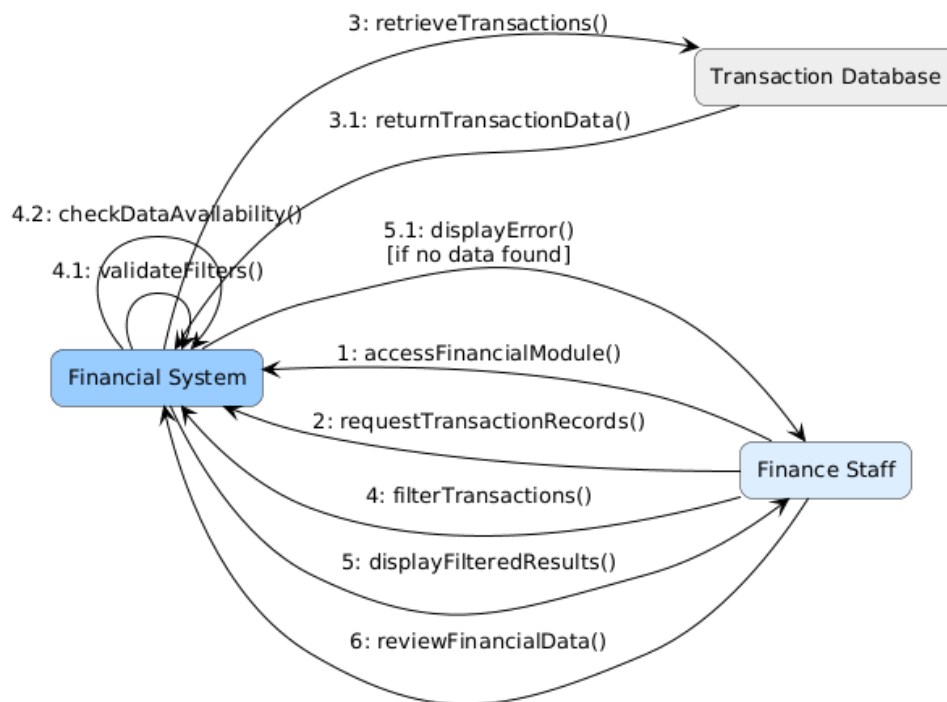
Actors

Finance Staff – the user who initiates invoice generation and reviews billing details

Billing System – the main system that validates data, calculates charges, and manages invoice creation

Database – the storage component that provides usage data and stores generated invoices

Financial Transaction Monitoring



- This diagram shows how finance staff monitor financial transactions by interacting with the financial system, which retrieves transaction records from the database, validates and filters the data, and then displays the financial results or error messages.

Actors

- **Finance Staff** – the user responsible for reviewing and monitoring financial transactions
- **Financial System** – the system that processes requests, filters transactions, validates data, and displays results
- **Transaction Database** – the storage system that contains transaction records and financial data

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Betül Kaan

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the Passenger Service Management through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the temporal order of events, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between system objects to ensure a robust and scalable management framework.

Technical Scope

This document focuses on the interaction network between:

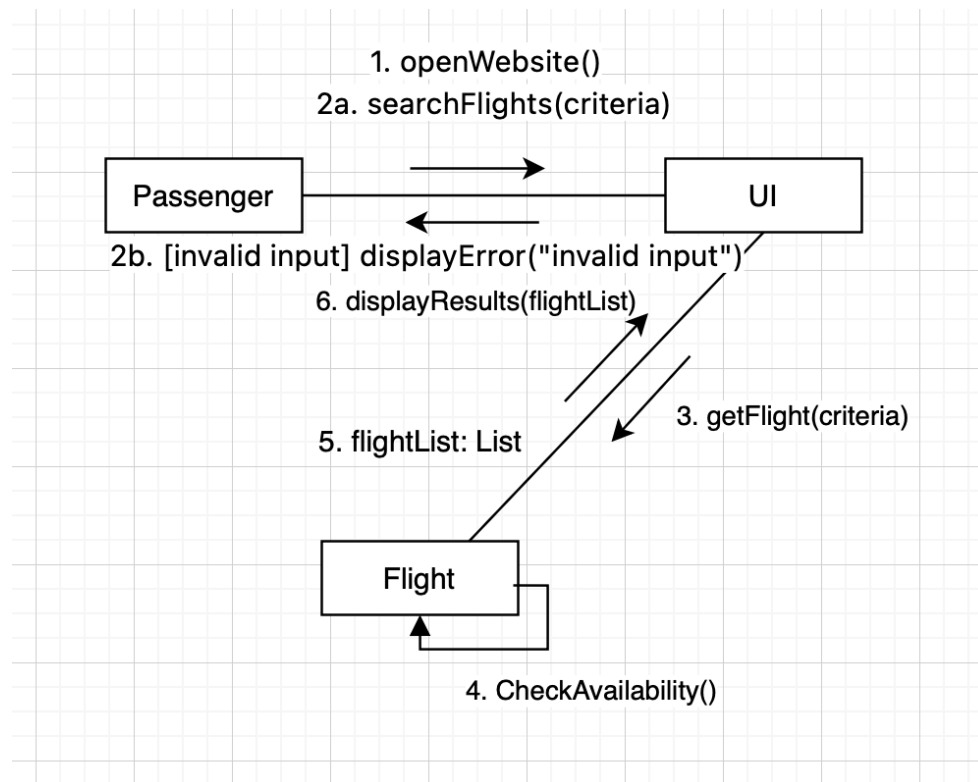
1. *Actor: Passenger*
2. *Boundary Objects: User interfaces used for data input and display — FlightSearchUI, DocumentUploadUI, CheckInUI, BoardingPassUI, GateInfoUI, BaggageTrackingUI, and NotificationSettingsUI.*
3. *Control Objects: Controllers that orchestrate business logic and validation for passenger service operations — UI(present in all UCs), CheckIn (manages window validation and boarding pass generation in UC-03), and FlightNotification(manages preference registration and alert triggering in UC-07).*
4. *Entity Objects: The database layer including Flight, Booking, TravelDocument, BoardingPass, BaggageRecord, BaggageTrackingController, Gate, and FlightNotification.*

System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this document adheres to the following architectural rules:

5. *Strict UML Notation: All models utilize standardized UML collaboration/communication symbols (Object Boxes, Link Lines, and Message Arrows).*
6. *Structural Links: Solid lines represent the static structural pathways over which objects are permitted to communicate.*
7. *Hierarchical Numbering: Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.*
8. *Controller-Centric Routing: Centralized Control objects are used to decouple the User Interface (Boundary) from the Database layer (Entities), ensuring high cohesion and low coupling across the management system.*

UC-01: View Flight Schedules



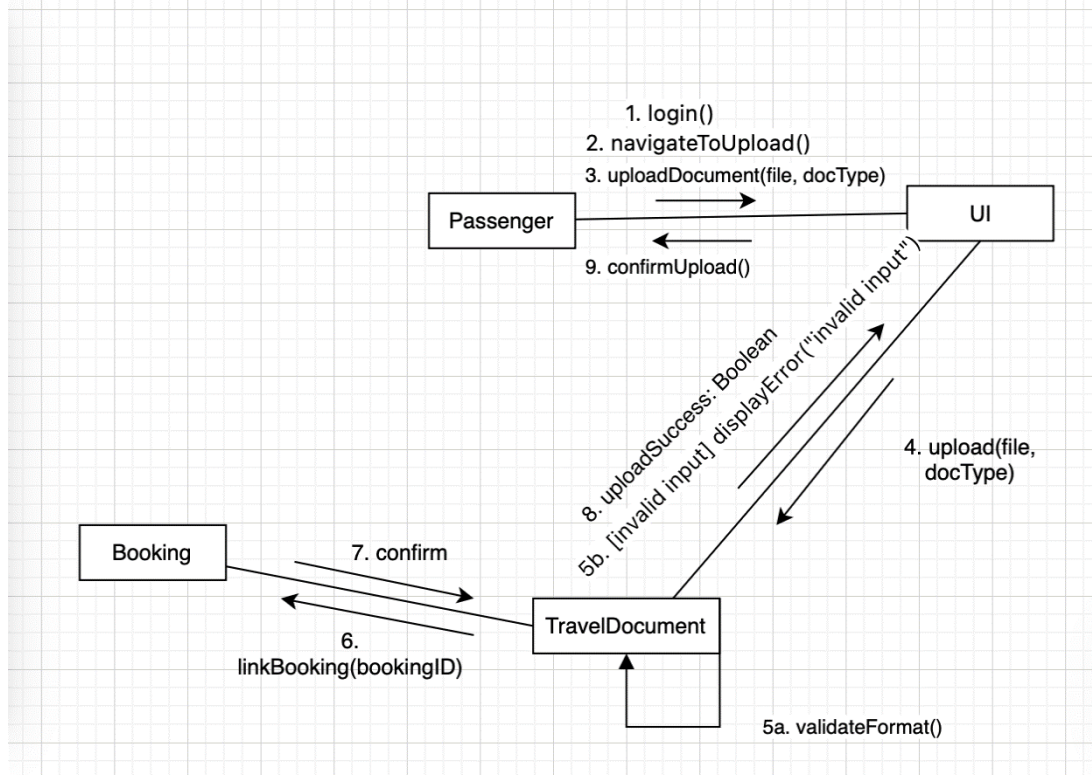
Focus: Flight search and availability retrieval for passengers.

Description: Illustrates the flow of a flight schedule query from the passenger's initial website interaction through the UI layer to the Flight entity, culminating in a filtered list being rendered back to the user.

Key Logic: The UI acts as the central mediator — it receives the search criteria from the Passenger, delegates the lookup via `getFlight(criteria)` (3) to the Flight entity, which internally runs `CheckAvailability()` (4) before returning the `flightList` (5), ensuring only valid and available flights are surfaced in `displayResults()` (6).

UC-02: Upload Travel Documents

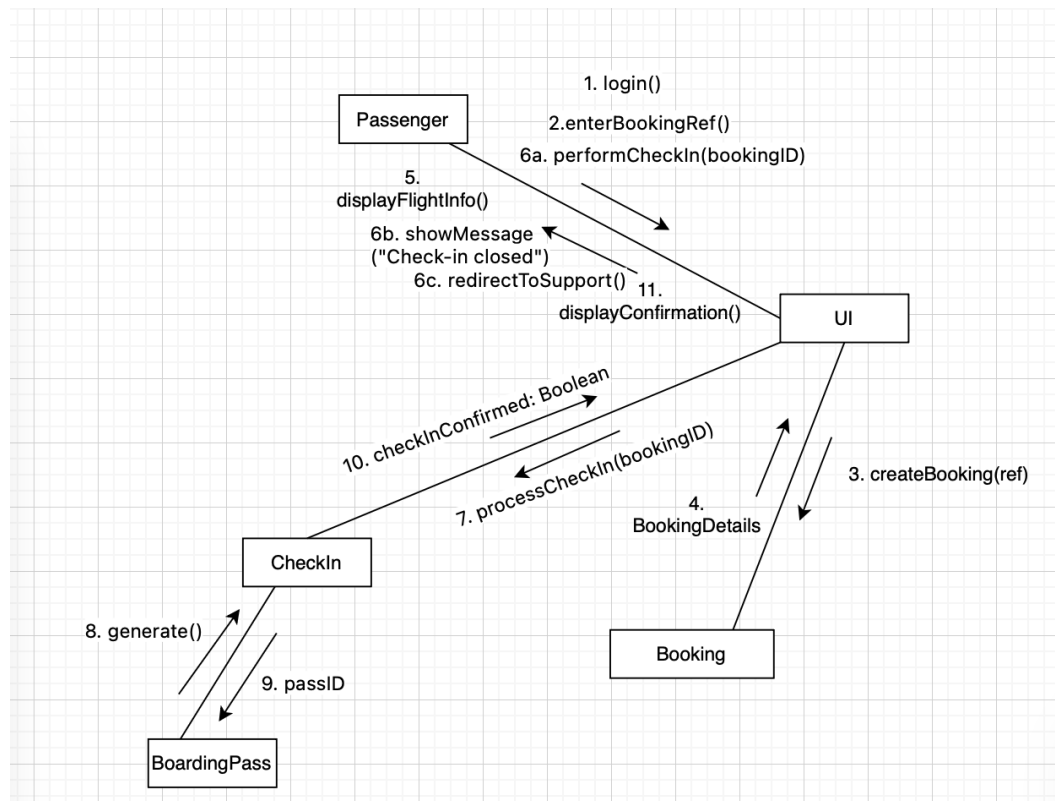
Focus: Secure document submission and booking association for authenticated passengers.



Description: Illustrates the lifecycle of a travel document upload from the passenger's login and navigation through the UI layer to the TravelDocument entity, including format validation, booking linkage via the Booking entity, and a final upload confirmation returned to the passenger.

Key Logic: The TravelDocument entity acts as the processing hub — after receiving the file via `upload(file, docType)` (4), it runs `validateFormat()` (5) before allowing `linkBooking(bookingID)` (6), ensuring documents are only associated with a valid booking once they pass format checks, with the outcome surfaced back to the UI as `uploadSuccess: Boolean` (8).

UC-03: Perform Online Check-In

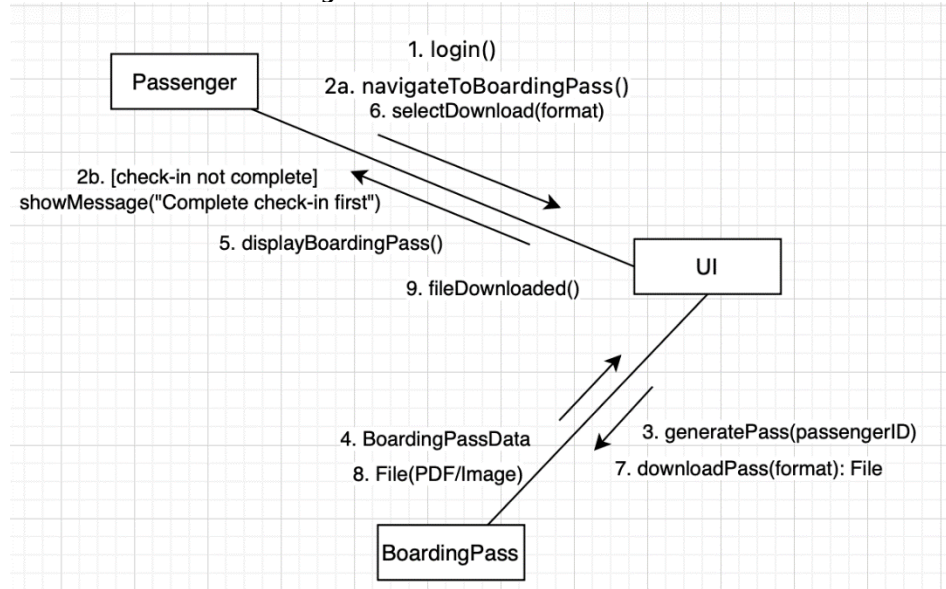


Focus: End-to-end online check-in flow from booking retrieval to boarding pass generation.

Description: Illustrates the multi-step process of a passenger checking in online, beginning with login and booking reference entry through the UI, progressing through booking retrieval, check-in processing, and boarding pass generation, and concluding with a confirmation displayed back to the passenger.

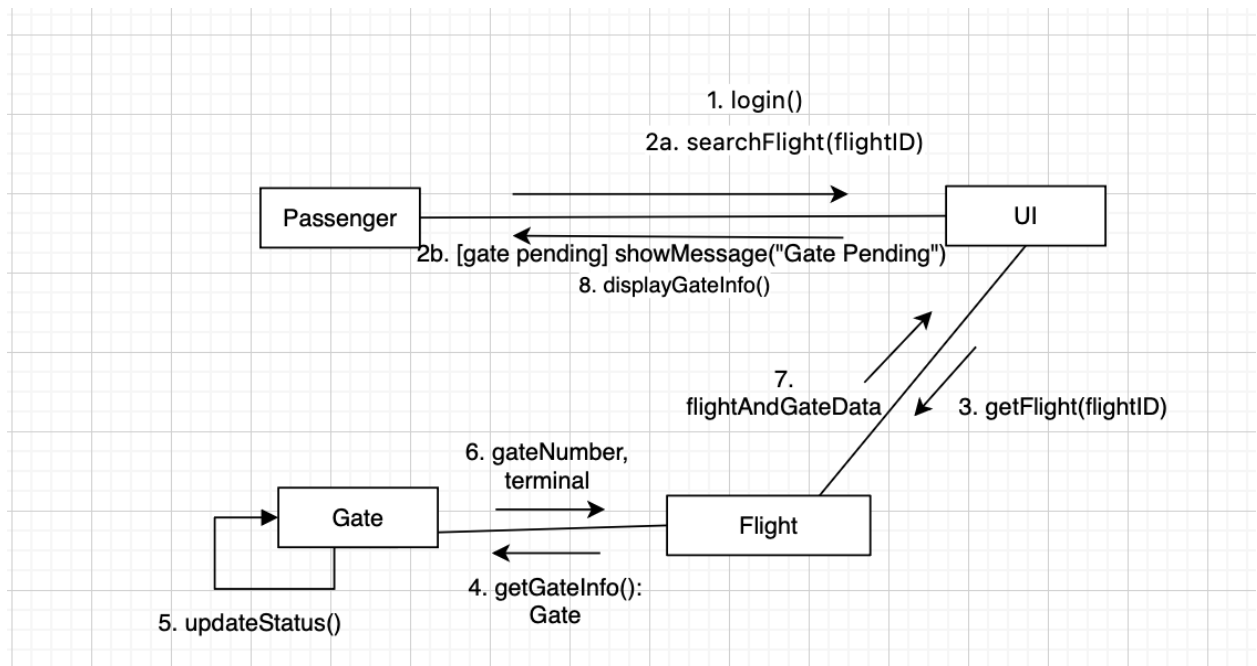
Key Logic: The CheckIn entity serves as the operational core — once the UI triggers processCheckIn(bookingID) (7), the CheckIn entity calls generate() (8) on the BoardingPass entity, which returns a passID (9), enabling the checkInConfirmed: Boolean (10) response to propagate back to the UI. The Booking entity acts as a prerequisite gate, as BookingDetails (4) must be successfully retrieved via createBooking(ref) (3) before the check-in sequence can proceed.

UC-04: Download Boarding Pass



- **Focus:** Boarding pass retrieval and format-specific file delivery to the passenger.
- **Description:** Illustrates the flow of a boarding pass download request from the passenger's login and navigation through the UI layer to the BoardingPass entity, covering pass generation, format selection, and final file delivery back to the passenger.
- **Key Logic:** The BoardingPass entity handles both generation and delivery — the UI first triggers `generatePass(passengerID)` (3), which returns `BoardingPassData` (4) to render the preview via `displayBoardingPass()` (5), and then upon the passenger's `selectDownload(format)` (6), the UI calls `downloadPass(format): File` (7) on the same entity, returning a format-specific `File(PDF/Image)` (8), ensuring the passenger always previews before committing to a download format.

UC-05: Check Assigned Gate Information

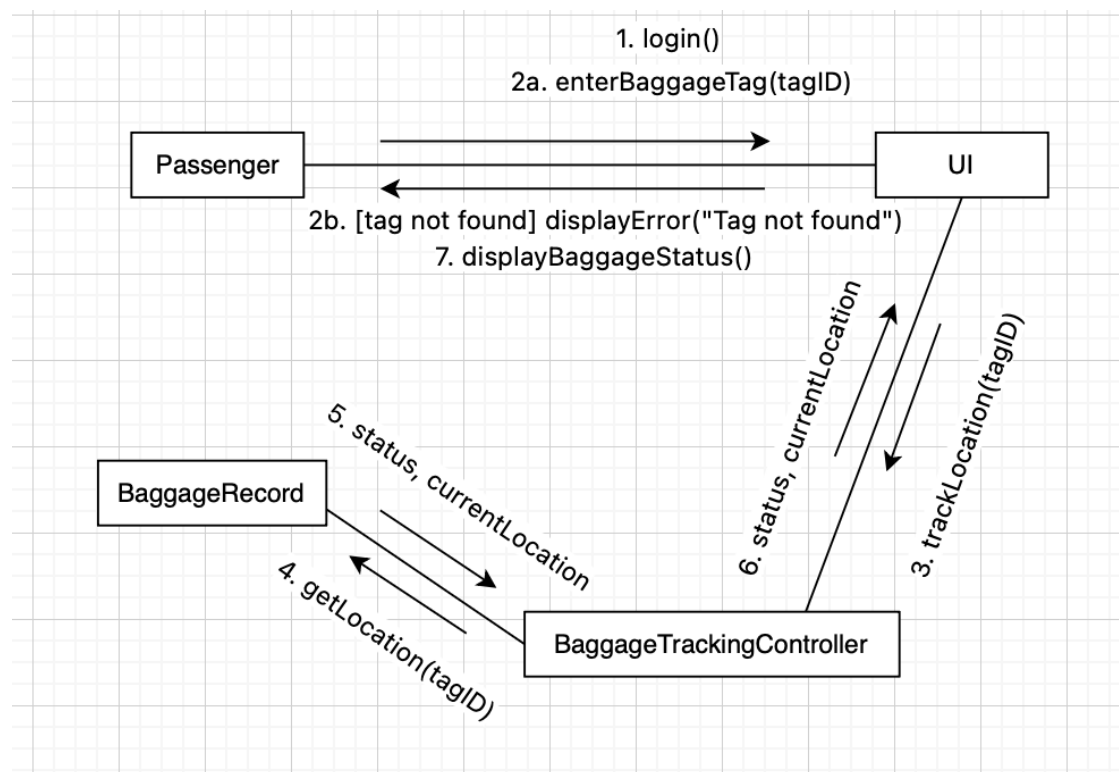


Focus: Real-time gate assignment lookup for a passenger's specific flight.

Description: Illustrates the flow of a gate information query from the passenger's login and flight search through the UI layer to the Flight entity, which delegates to the Gate entity for current assignment data, with the consolidated result returned to the passenger.

Key Logic: The Flight entity acts as the intermediary bridge — after receiving `getFlight(flightID)` (3) from the UI, it calls `getGateInfo(): Gate` (4) on the Gate entity, which first runs `updateStatus()` (5) on itself to ensure current data before returning `gateNumber, terminal` (6). This guarantees that the `flightAndGateData` (7) surfaced to the UI via `displayGateInfo()` (8) always reflects the most up-to-date gate assignment rather than a potentially stale cached value.

UC-06: Track Baggage Status



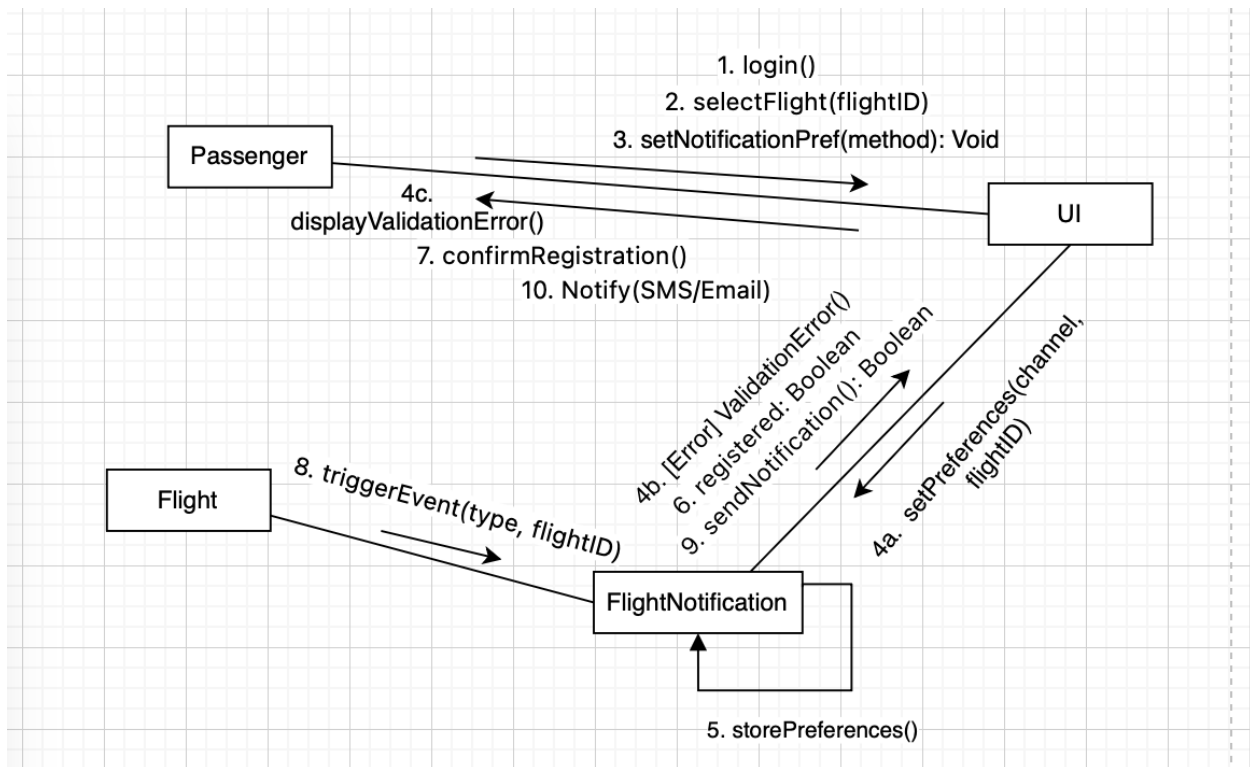
Focus:

Real-time baggage location and status retrieval via tag-based tracking.

Description: Illustrates the flow of a baggage tracking request from the passenger's login and tag entry through the UI to the BaggageTrackingController, which queries the BaggageRecord entity for current location data and propagates the result back to the passenger.

Key Logic: The BaggageTrackingController serves as the dedicated tracking intermediary — upon receiving trackLocation(tagID) (3) from the UI, it calls getLocation(tagID) (4) on the BaggageRecord entity, which returns status, currentLocation (5) sourced directly from the record store. This data is then relayed back to the UI as status, currentLocation (6), ensuring the displayBaggageStatus() (7) presented to the passenger is always fetched live from the BaggageRecord rather than derived from any intermediate cache.

UC-07: Receive Flight Notifications



Focus: Passenger notification preference registration and event-driven alert delivery.

Description: Illustrates the two-phase lifecycle of flight notifications — first, the passenger registers their preferred channel and flight via the UI, and second, the FlightNotification entity autonomously triggers alerts when the Flight entity emits an event, delivering the notification back to the passenger via their chosen method.

Key Logic: The FlightNotification entity drives both phases — during setup, it receives `setPreferences(channel, flightID)` (4) from the UI and internally calls `storePreferences()` (5) before returning `registered: Boolean` (6) to confirm subscription. During the event phase, the Flight entity calls `triggerEvent(type, flightID)` (8) on FlightNotification, which then executes `sendNotification(): Boolean` (9) to dispatch the `Notify(SMS/Email)` (10) to the passenger, ensuring the delivery channel always matches the originally stored preference.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Alvi Elezi

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the **Flight Operations Module** through a series of Collaboration (Communication) Diagrams. While Sequence Diagrams emphasize the passage of time, these Collaboration Diagrams focus on the architectural relationships and the specific routing of messages between objects.

Technical Scope

This document focuses on the interaction network between:

1. **Actors:** Airport Staff.
2. **Control/Boundary Objects:** System elements responsible for flight operations.
3. **Entity Objects:** The database layer including FlightService, FlightController, and NotificationService.

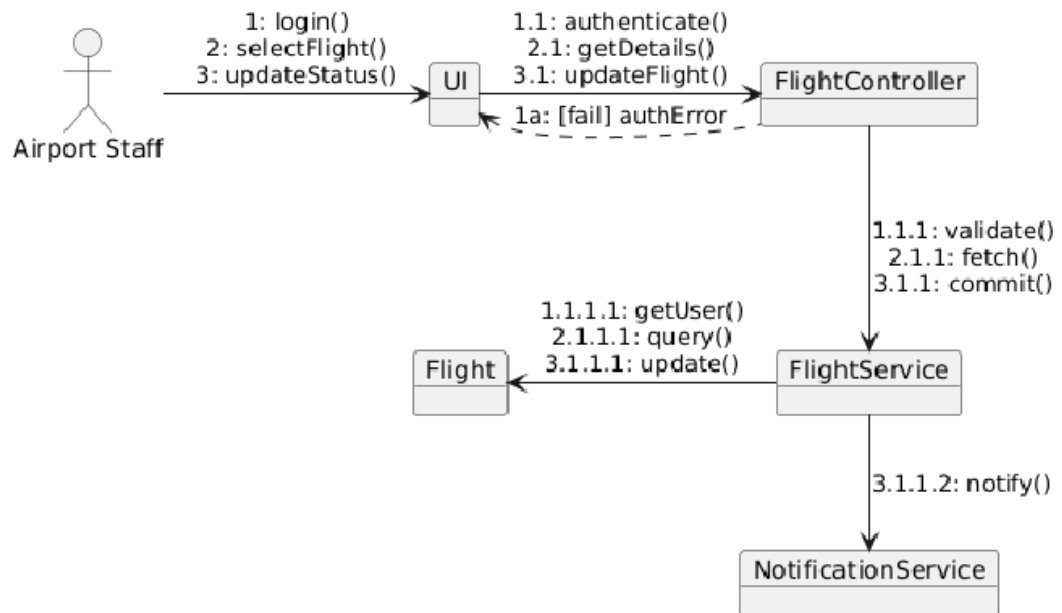
System Interaction Standards

To maintain technical consistency, every collaboration diagram in this document adheres to the following architectural rules:

1. **Strict UML Notation:** All models utilize standardized UML collaboration/communication symbols.
2. **Structural Links:** Solid lines represent the structural pathways over which objects can communicate.
3. **Hierarchical Numbering:** Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2, 2.1) to clearly indicate nested operations and sequential logic.

Controller-Centric Routing: Central objects are used to route messages between actors and data entities, keeping the architecture decoupled and highly cohesive.

1. Update flight status

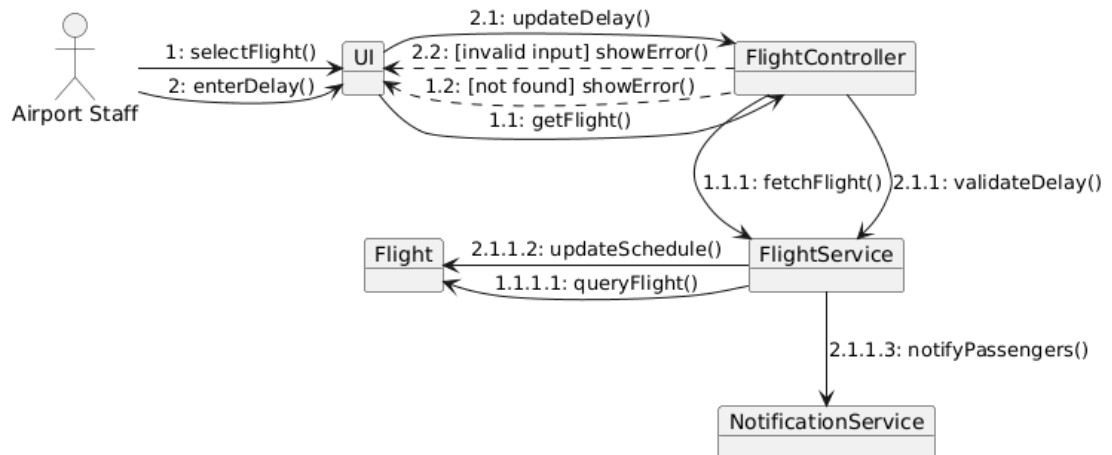


Focus: Flight Status Synchronization

Description: This collaboration diagram illustrates how airport staff initiate and propagate a flight status update across the system. It shows the interaction between the user interface, controller, service layer, and database to ensure the update is consistently applied.

Key Logic: The process is driven by the controller, which validates the request, delegates the update to the service layer, persists the change in the database, and triggers passenger notifications to ensure system-wide consistency.

2. Manage Flight Delays

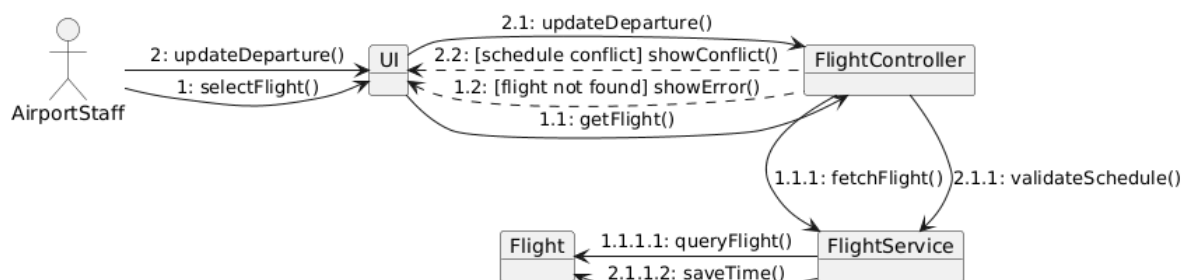


Focus: Delay Processing and Notification

Description: This diagram demonstrates how flight delays are handled within the system, starting from staff input to updating the flight schedule and notifying affected passengers.

Key Logic: The controller coordinates delay validation and delegates schedule updates to the service layer, which ensures that all changes are persisted and communicated through the notification service.

3. Update departure time

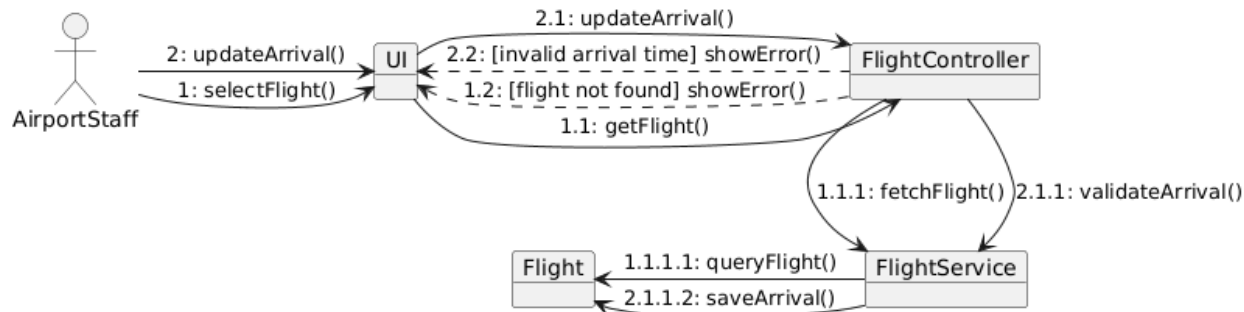


Focus: Departure Time Adjustment

Description: This collaboration diagram represents how modifications to a flight's departure time are processed and stored. It highlights the interaction flow required to retrieve flight data and apply scheduling changes.

Key Logic: The controller enforces scheduling validation rules before the service layer commits the updated departure time to the database, ensuring no operational conflicts occur.

4. Update Arrival Time

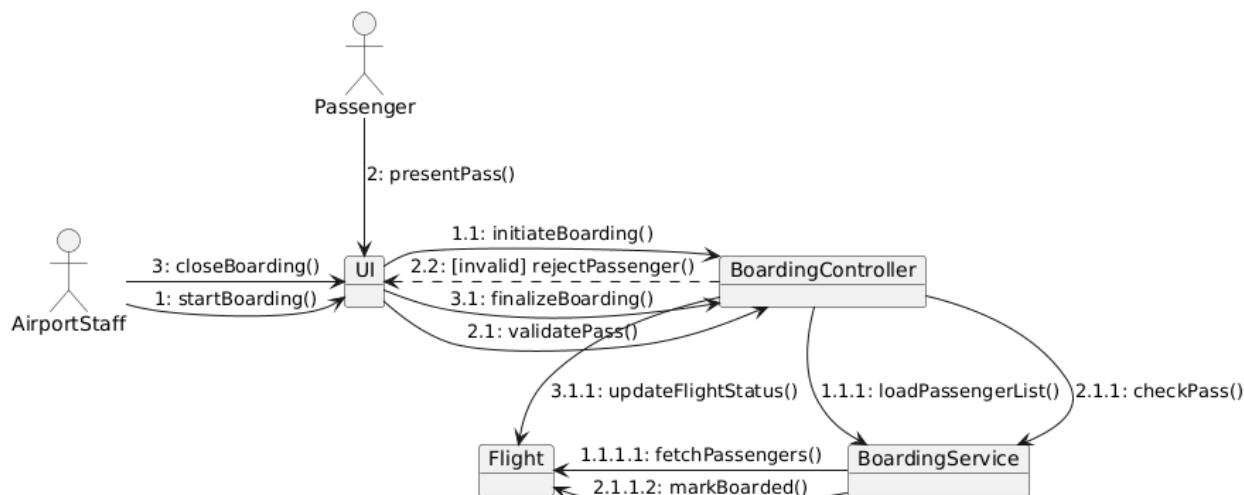


Focus: Arrival Time Management

Description: This diagram shows how updated arrival times are processed within the system, ensuring that flight data remains accurate and synchronized across all components.

Key Logic: The system validates the new arrival time through the controller before updating the flight entity in the database, maintaining consistency with operational constraints.

5. Coordinate Boarding Process

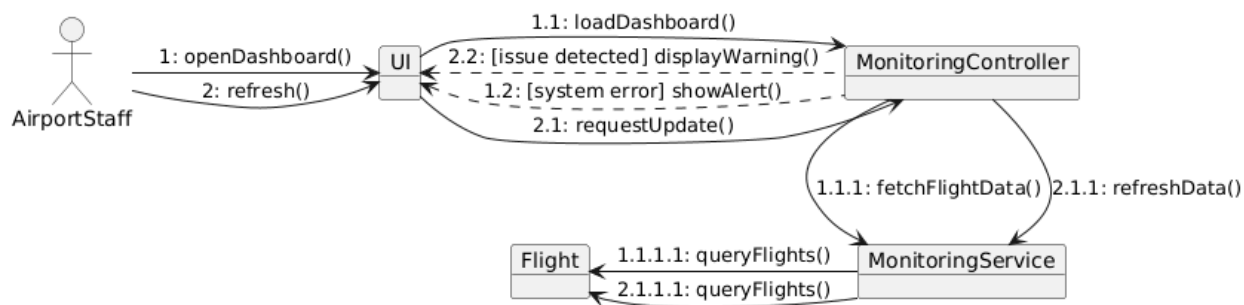


Focus: Boarding Control and Passenger Validation

Description: This collaboration diagram models the boarding process, including passenger validation and status updates as individuals board the aircraft.

Key Logic: The boarding controller manages iterative validation of boarding passes, while the service layer updates passenger records in real time, ensuring accurate tracking of boarding progress.

6. Coordinate Flight Operations



Focus: Real-Time Operational Monitoring

Description: This diagram illustrates how airport staff monitor flight operations through a centralized dashboard, retrieving and refreshing real-time flight data.

Key Logic: The monitoring controller continuously interacts with the service layer to fetch updated flight information from the database, enabling timely detection of operational changes and supporting decision-making.

Airport Management System

Technical Design: Collaboration Model Specifications

Course: Software Modeling and Design

Date: April 16, 2026

Prepared by: Ani Velo

Document Objective

The primary objective of this document is to detail the structural organization and message flow of the **Security Management Module** through a series of Collaboration (Communication) Diagrams. These diagrams emphasize the architectural relationships and the specific routing of messages between objects to ensure a robust security framework .

Technical Scope

This document focuses on the interaction network between:

Actors: Security Officer, Security Supervisor, and Passenger.

Control/Boundary Objects: SecurityController, AlertManager, and IncidentDashboard.

Entity Objects: PassengerProfile, SecurityLog, and IncidentReport.

System Interaction Standards

To maintain technical consistency and academic rigor, every collaboration diagram in this section adheres to the following architectural rules:

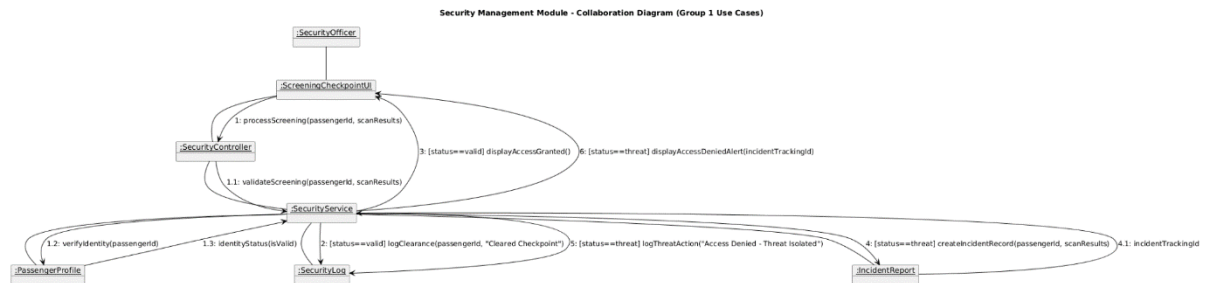
Strict UML Notation: All models utilize standardized UML collaboration and communication symbols, including Object Boxes, Link Lines, and Message Arrows.

Structural Links: Solid lines represent the static structural pathways over which objects are permitted to communicate within the security framework.

Hierarchical Numbering: Message execution order is strictly defined using decimal numbering (e.g., 1, 1.1, 2) to clearly indicate nested operations, return flows, and sequential logic.

Controller-Centric Routing: Centralized Control objects (such as the **SecurityController** and **AlertManager**) are used to decouple the User Interface from the Database layer, ensuring high cohesion and low coupling.

1. UC_SEC_01,02,05 : Process Passenger Security Screening

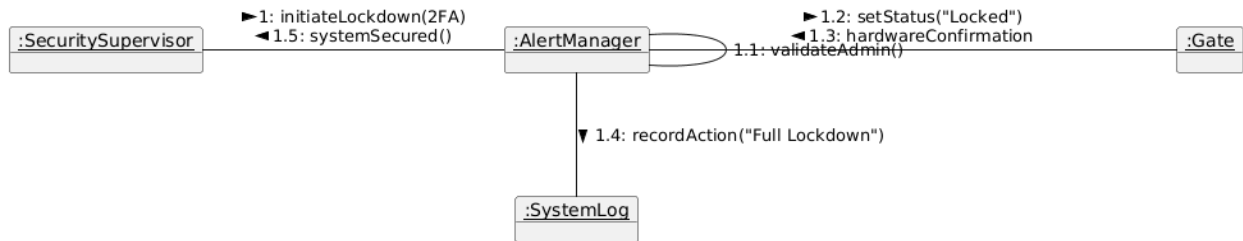


Focus: Passenger Screening and Incident Verification.

Description: This collaboration diagram illustrates how security personnel initiate, verify, and execute a screening workflow across checkpoint boundaries. It shows the unified message routing between the interface UI, security controller, intermediate security service layer, and database entities when a threat or prohibited item is isolated.

Key Logic: The process is driven by the controller, which captures the screening metrics, validates passenger credentials against the profile record, delegates conditional persistence rules to the service layer, and routes automated logs or incident triggers to ensure system-wide security synchronization.

2. UC_SEC_03,04,06: Emergency Facility Lockdown



Focus: High-Priority Command Propagation

Description: Highlights the structural relationships between the Security Supervisor, the AlertManager, and the hardware Gate entities during a critical threat.

Key Logic:

Hierarchical Validation: Nested validation (1.1) ensures that the supervisor's administrative permissions are checked before any hardware command is issued.

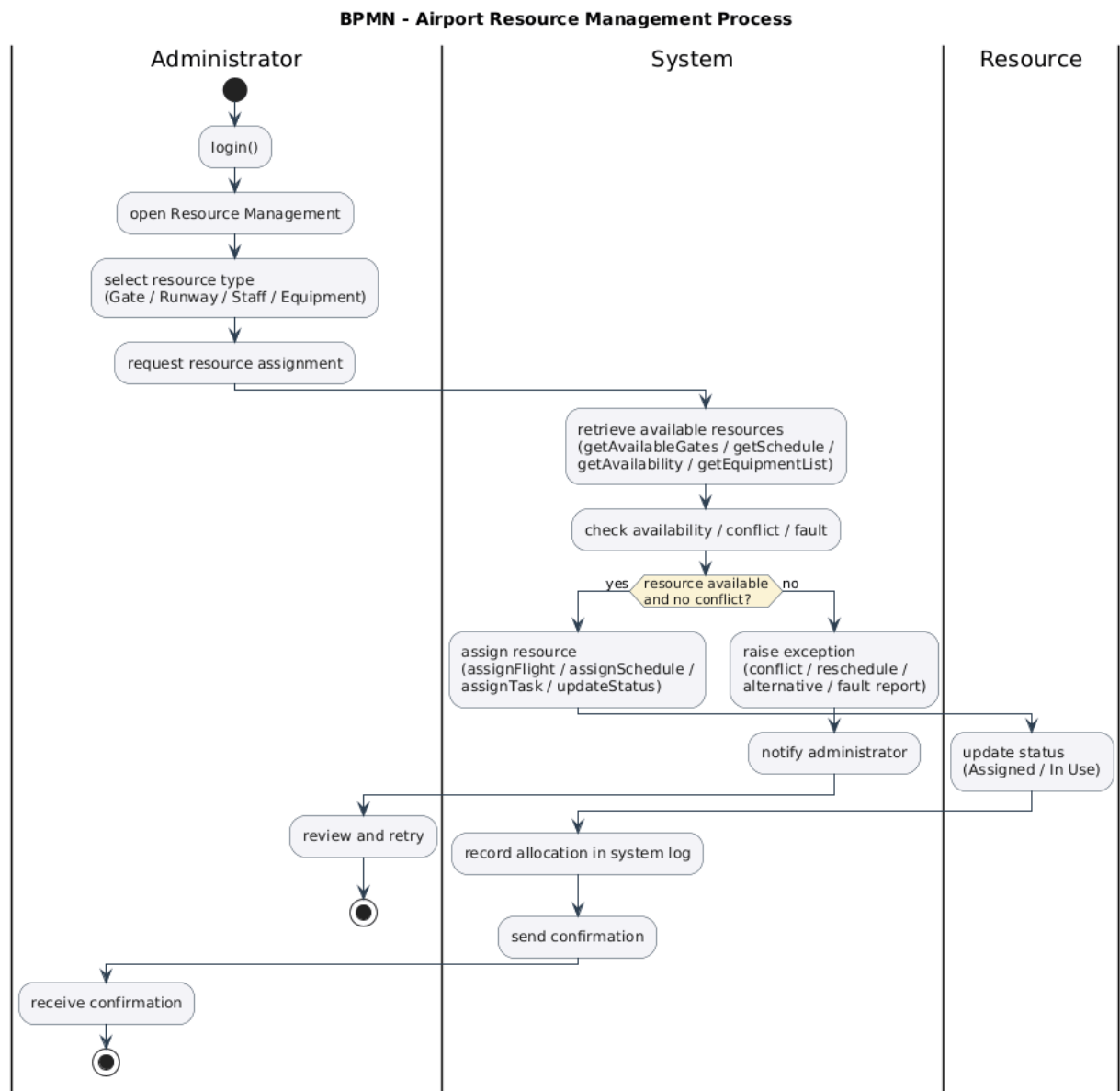
Command Propagation: The *AlertManager* handles divergent message routing, transmitting "Lock" signals to physical *Gate* entities while simultaneously triggering global alerts.

Atomic Confirmation: The logic requires a successful *hardwareConfirmation* return from the entity layer before the system state is logged and a success message is rendered.

AMEO LULAJ:

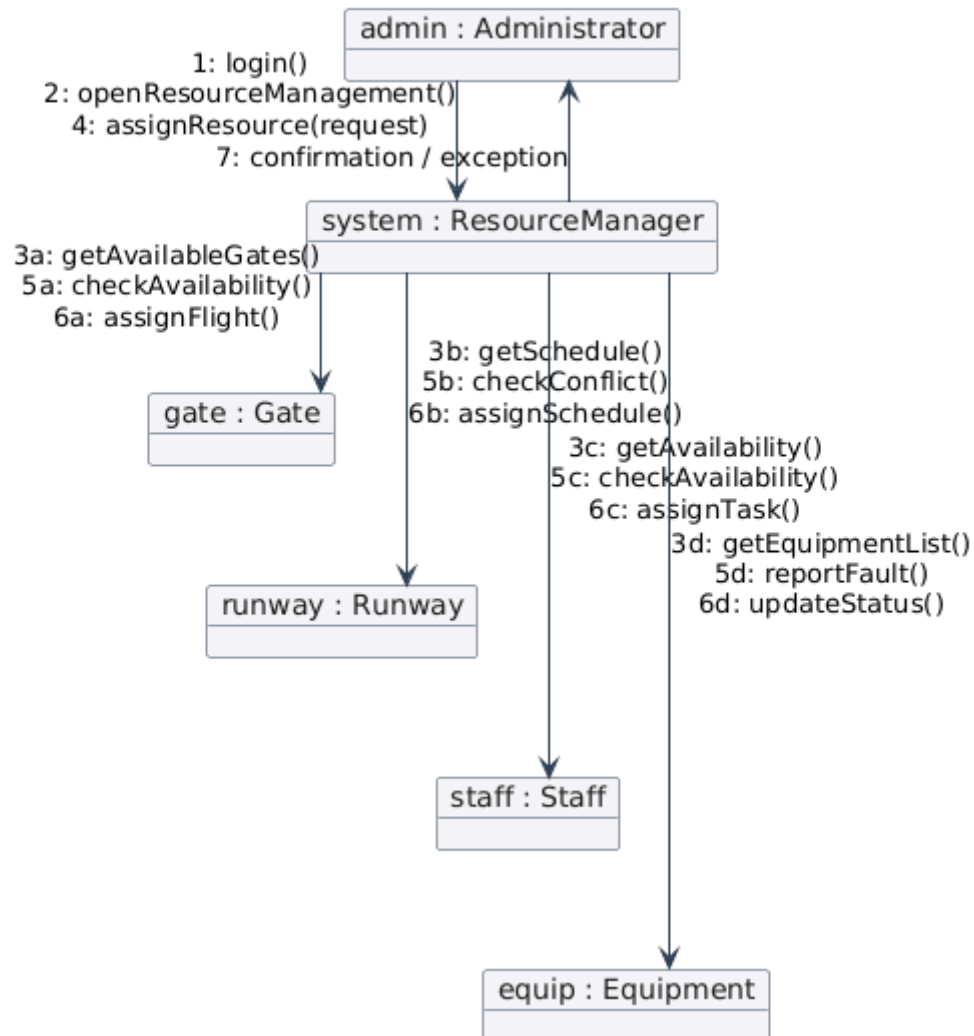
Since all four resources (Gate, Runway, Staff, Equipment) follow the identical interaction pattern, I modeled one generic process and showed the resource-specific operation names per branch.

- **BPMN Diagram – Airport Resource Management**



- *Collaboration Diagram – Airport Resource Management*

Collaboration (Communication) Diagram - Airport Resource Management



Technical Specification: Interaction Overview Diagram

Module: System Integration & Administrative Control

Prepared by: Eneida Balliu & Ajla Alcani

Requirement Reference: UC_40 (Integration Layer)

Document Objective

The purpose of this diagram is to provide a macro-perspective of the system's behavior by coordinating multiple interaction models into a single unified workflow. While Collaboration Diagrams show detailed messaging, the Interaction Overview Diagram focuses on the "Big Picture" logic—specifically the hand-off between the Human Resources (HR) module and the System Administration module.

Technical Scope

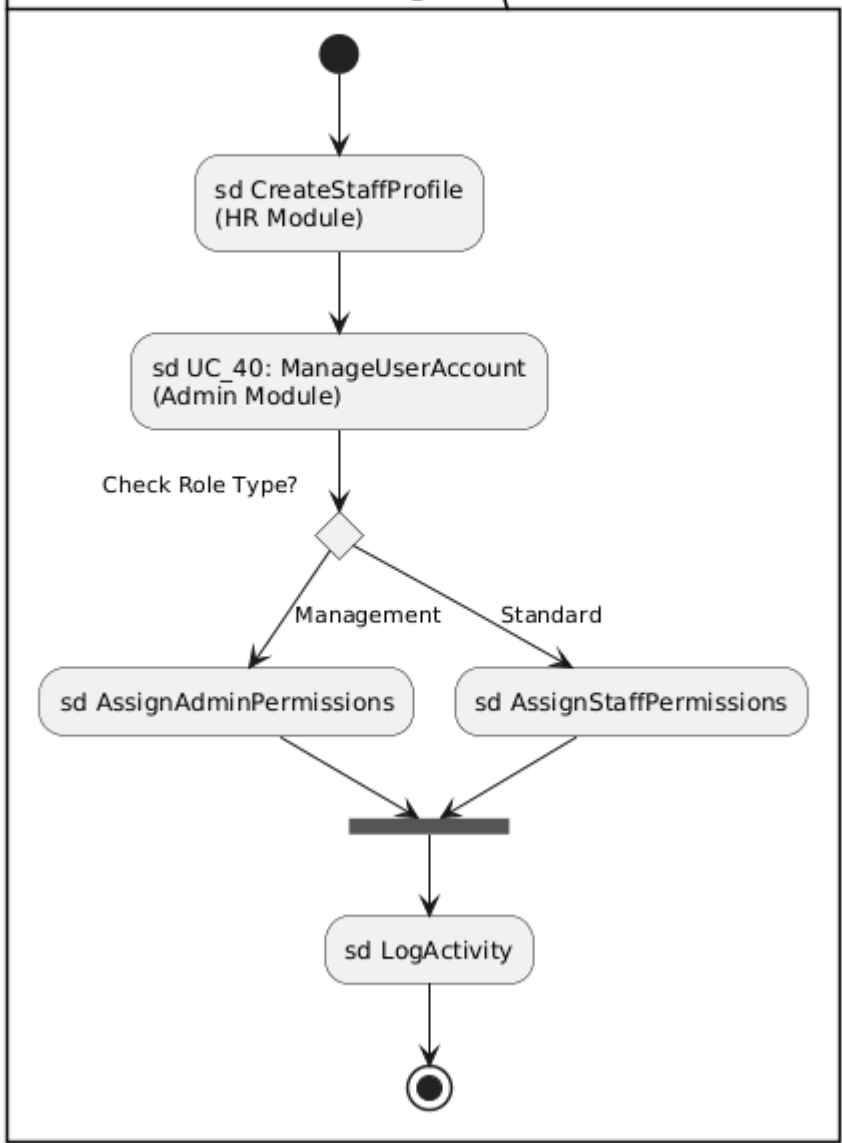
This model encapsulates the following system transitions:

- *Inter-Module Hand-off: The transition from HR profile creation to Digital Account management.*
- *Interaction Occurrences: The use of `sd` (Sequence/Interaction) frames to reference externalized logic without cluttering the main flow.*
- *Conditional Synchronization: The routing of security permissions based on user role guards.*

Interaction Overview: Staff Onboarding Process (HR to Admin)

Please use '!option handwritten true' to enable handwritten

Interaction Overview Diagram



Technical Specification: Interaction Overview Model (Staff Onboarding)

Objective: This diagram illustrates the high-level orchestration of the Staff Onboarding Process, specifically mapping the integration between the Human Resources (HR) and System Administration modules.

Core Components & Logic:

- Interaction References (sd Blocks): Each rectangular box (e.g., sd UC_40: ManageUserAccount) represents an encapsulated Interaction Use. This indicates that the diagram references complex message flows (Sequence or Collaboration diagrams) previously defined, maintaining modularity and a clean architectural overview.

- **Decision and Merge Nodes:** The model utilizes a Decision Diamond to branch the workflow based on user roles (Guard Conditions: [Management] vs. [Standard]).
- **Logical Synchronization:** A Merge Node is implemented following the permission-assignment phase. This ensures that regardless of the chosen path, the control flow is synchronized to perform the final LogActivity interaction.
- **Final State Reachability:** By closing the "Management" branch and merging it back into the main flow, the diagram satisfies the UML requirement for Reachability, ensuring every possible scenario concludes at the Final Node.

Architectural Value: This model provides traceability across system boundaries. It proves that the administrative account management (UC_40) is a prerequisite for permission assignment and that all actions are subject to a unified system logging procedure for security compliance.

Technical Specification: Interaction Overview Diagram

Module: System Integration & Administrative Control

Prepared by: Albina Blloshmi and Betül Kaan

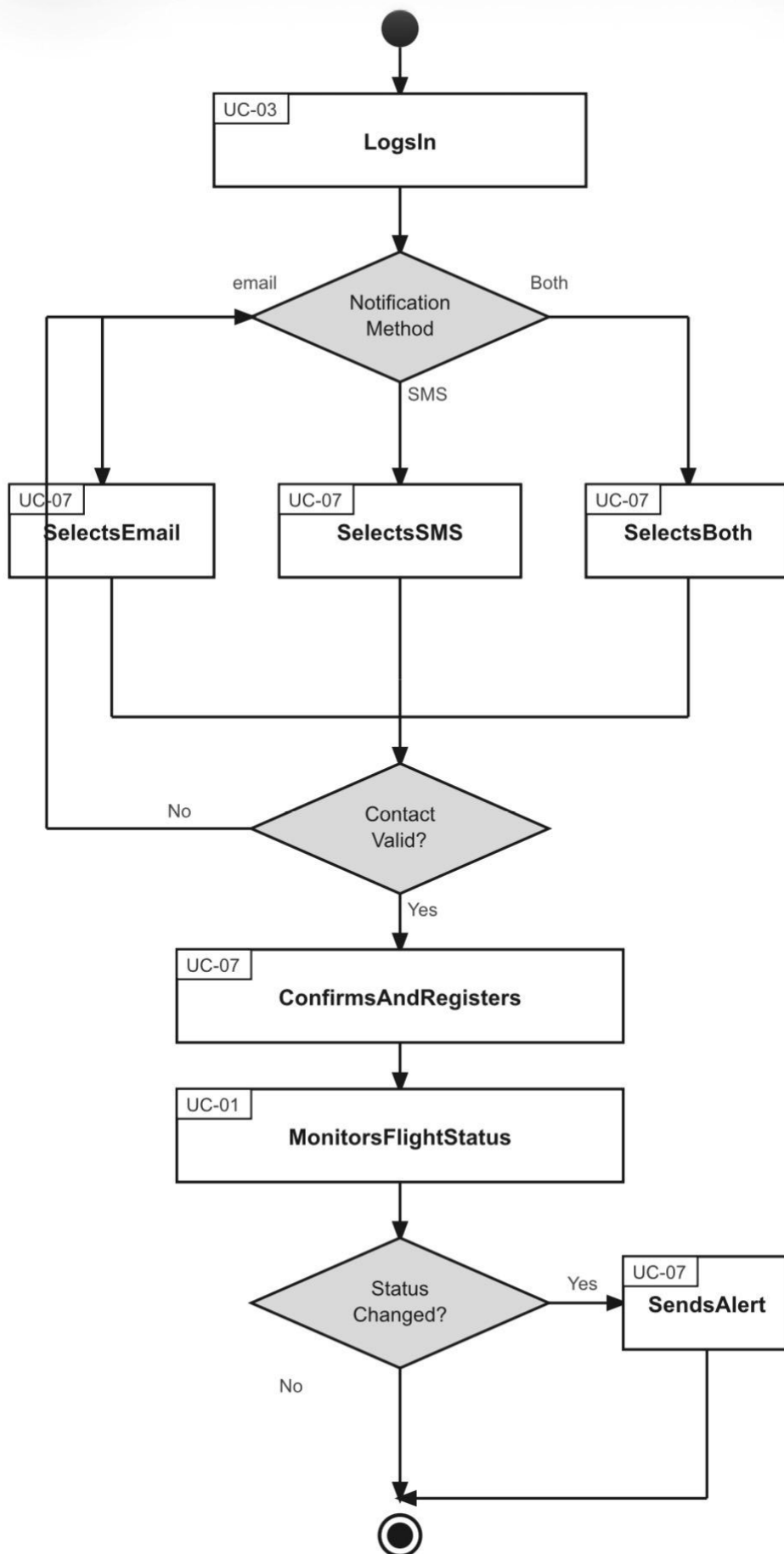
Requirement Reference: UC_07 (Flight Notification)

Document Objective

The purpose of this diagram is to provide a macro-perspective of the system's behavior by coordinating multiple interaction models into a single unified workflow. While Collaboration Diagrams show detailed messaging, the Interaction Overview Diagram focuses on the "Big Picture" logic—specifically the hand-off between the Human Resources (HR) module and the System Administration module.

Technical Scope

- **Passenger-to-System Hand-off:** The transition from passenger-initiated actions (authentication, flight selection, and contact method preference) to system-owned processes (contact validation, alert registration, and real-time flight monitoring).
- **Interaction Occurrences:** The use of ref frames to reference externalized notification logic — such as SelectsEmail, SelectsSMS, and SelectsBoth — without cluttering the main orchestration flow, preserving modularity across subordinate sequence diagrams.
- **Conditional Branching on Notification Method:** The routing of the subscription flow across three mutually exclusive paths based on the passenger's chosen delivery channel, governed by guard conditions [EMAIL], [SMS], and [BOTH].
- **Contact Validation Gate:** The enforcement of a loopback mechanism via the ContactValid? decision node, ensuring that malformed contact details are intercepted before registration is committed to the system.
- **Monitoring and Alert Dispatch:** The transition from a one-time registration event to a continuous monitoring state, with conditional alert delivery governed by the StatusChanged? guard, ensuring notifications are only dispatched upon a verified flight status update.



Process Explanation

1. Start
 - * The process begins when the user enters the system.
2. Login
 - * The user logs into the application or platform.
3. Choose Notification Method
 - * The system asks the user to select how they want to receive notifications.
 - * The user can choose:
 - * Email
 - * SMS
 - * Both Email and SMS
4. Notification Selection
 - * Depending on the selected option:
 - * SelectsEmail → notifications will be sent through email.
 - * SelectsSMS → notifications will be sent through SMS.
 - * SelectsBoth → notifications will be sent through both channels.
5. Contact Validation
 - * The system checks whether the provided contact information is valid.
 - * Examples:
 - * valid email format
 - * active phone number
 - * If the contact information is invalid, the process loops back so the user can choose or re-enter the notification method/details again.
 - * If the contact information is valid, the process continues.
6. Confirms and Registers
 - * The user confirms their information.
 - * The system registers the user successfully for flight notifications.
7. Monitor Flight Status
 - * The system continuously monitors the status of the selected flight.
8. Check if Status Changed
 - * The system checks for any changes in the flight status, such as:
 - * delays
 - * cancellations
 - * gate changes
 - * departure time updates
9. Send Alert
 - * If the flight status changes, the system sends an alert to the user using the selected notification method(s).
10. End Process
 - * If there are no status changes, or after alerts are sent, the workflow reaches the end state.

Overall Purpose

The diagram represents a notification management system that allows users to receive real-time flight updates through email, SMS, or both communication methods.

Technical Specification: Interaction Overview Diagram

Module: Flight Operations & Security

Prepared by: Alvi Elezi & Ani Velo

Requirement Reference: UC_SEC_01-03; UC_01, 02, 05, 06

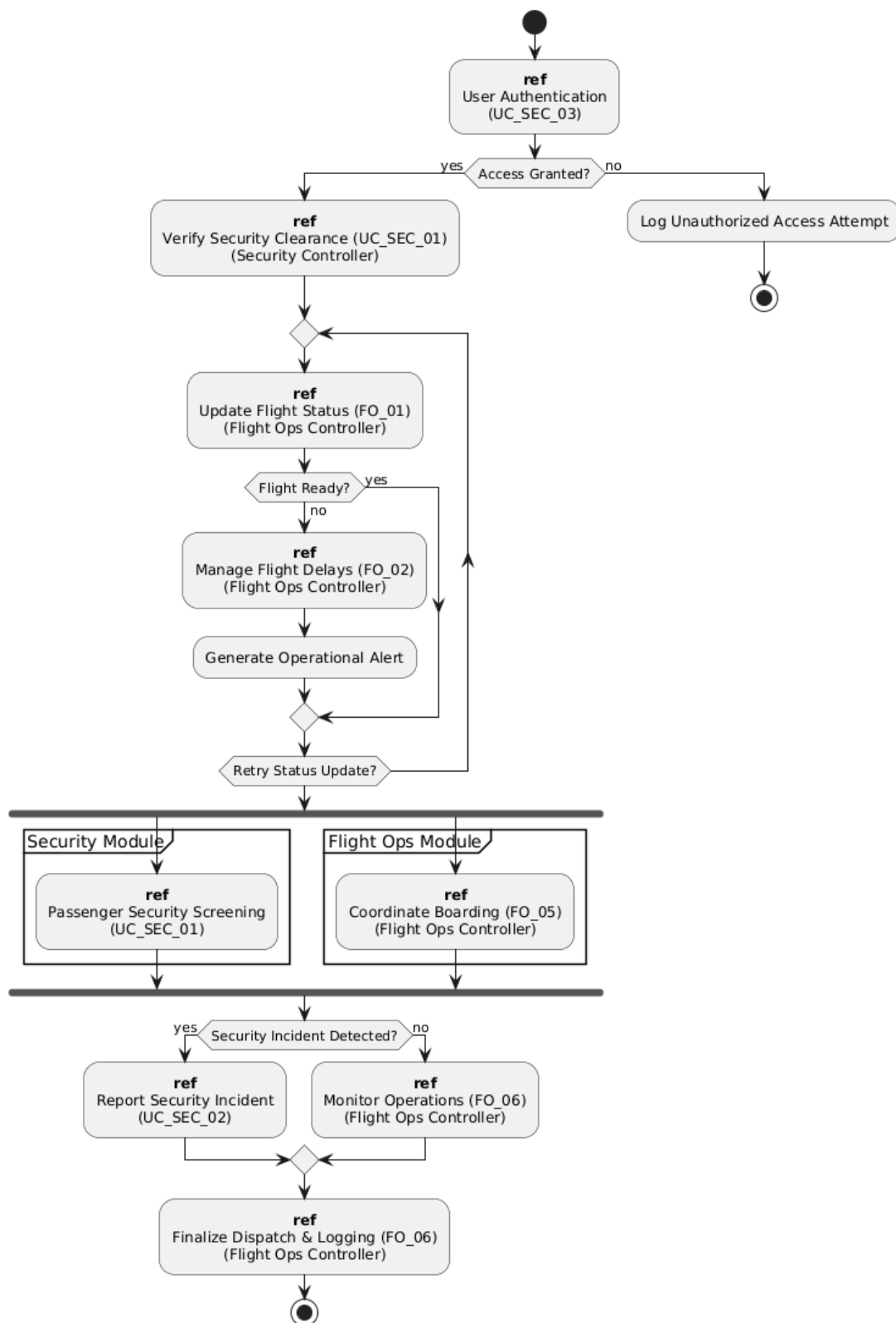
Document Objective

This diagram visualizes the high-level control flow between the Security and Flight Operations modules, focusing on the orchestration of multiple use cases. It establishes the logical dependencies and safety protocols required to clear a flight for dispatch, bridging the gap between individual sequence models and the complete operational lifecycle.

Technical Scope

The specification covers the coordination between the Security and Flight Operations Controllers, specifically addressing authentication, clearance verification, and iterative delay management. It details concurrent processes like parallel passenger screening and boarding, while ensuring that all security incidents and final dispatch actions are recorded within the entity layer for compliance.

Interaction overview diagram:



Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Ajla Alcani & Eneida Balliu

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architectures used to solve specific structural problems. It specifically details the Observer Pattern applied to bridge the Human Resources and System Administration modules.

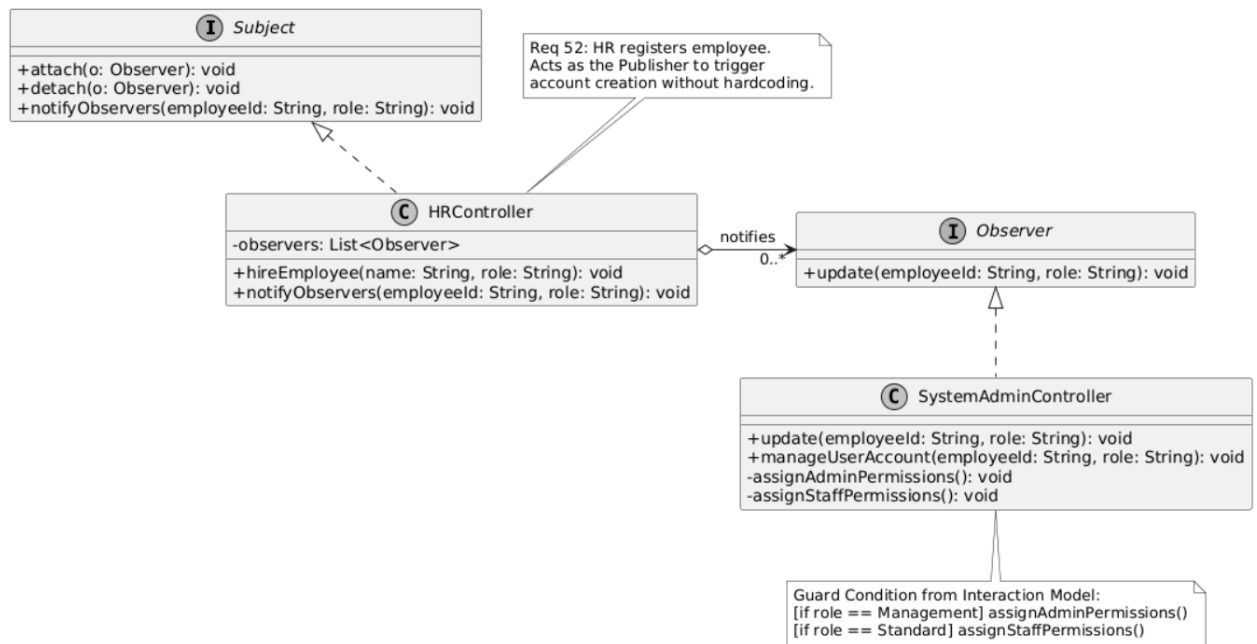
Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows and preventing tight coupling between independent subsystems.

Design Pattern 1: Observer Pattern (Module Integration: HR & System Admin)

- **Requirement Mapped:** Req 52 (HR registers employee records) and System Admin Account Creation.
- **Problem:** In our interaction models, when the Human Resources module (Ajla) registers a new employee, it needs to trigger the System Administration module (Eneida) to automatically generate a digital user account. If we hardcoded the System Admin classes directly into the HR code, the two modules would be tightly coupled, making the system fragile if administrative rules ever change.
- **Solution:** We applied the Behavioral **Observer Pattern**. The **HRController** acts as the Subject (or Publisher). When an employee is hired, it broadcasts a notification. The **SystemAdminController** acts as the Observer (or Subscriber), listening for that event to create the account.
- **Justification:** This pattern is ideal because it allows us to establish relations between objects at runtime. The HR backend does not need to know the internal implementation of the Admin backend. It also securely manages our conditional logic: when the Admin observer receives the notification, it uses our predefined guard conditions `[if role == Management]` or `[if role == Standard]` to assign the correct permissions.

2. The Class Diagram



Behavioral Design Patterns Documentation

Prepared by: Alvi Elezi

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architectures used to solve specific structural problems. It specifically details the Facade Pattern used in the Flight Operations Module.

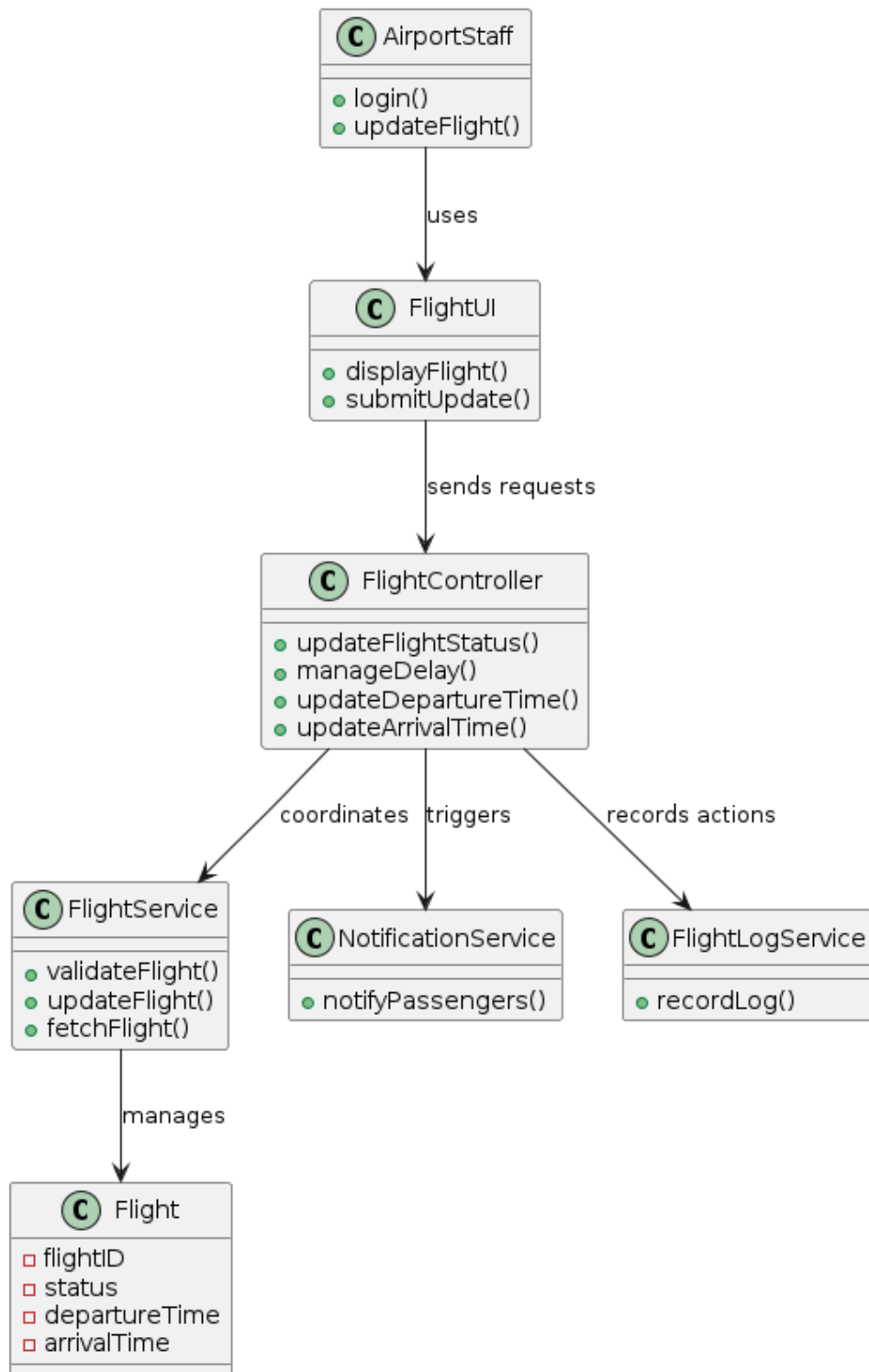
Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows and preventing tight coupling between independent subsystems.

Design Pattern 2: Facade Pattern (Module Integration: Airport Staff)

- **Requirements Mapped:**
 - Flight status management
 - Flight delay handling
 - Departure and arrival time updates
 - Passenger notification handling
 - Operational logging and database persistence
 - Centralized coordination of flight operations
- **Problem:** The Flight Operations Module contains multiple interconnected subsystems such as flight validation, scheduling, notifications, logging, and database management. If the user interface interacted directly with all these components, the system would become highly coupled, difficult to maintain, and more complex to extend. Any modification to one subsystem could require changes across several parts of the application.
- **Solution:** The system implements the **Facade Pattern** through the **FlightController** class. The controller acts as a centralized access point between the user interface and the internal operational subsystems. Instead of directly communicating with services such as **NotificationService** and **FlightLogService** the UI sends all operational requests to the **FlightController**, which coordinates the required actions internally.
- **Justification:** The Facade Pattern is appropriate for the Flight Operations Module because it simplifies communication with complex operational subsystems while reducing coupling between layers. The pattern improves maintainability by isolating subsystem changes from the user interface and centralizing operational workflows within the controller layer. It also enhances scalability, as additional operational services can be integrated behind the facade without requiring modifications to external components.

2. The Class Diagram



Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Albina Blloshmi and Betül Kaan

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of the Behavioral Chain of Responsibility Design Pattern within the Airport Management System (AMS). While previous system models focused on operational workflows and subsystem interactions, this document formally defines the request-handling architecture used to manage passenger service processes. It specifically details how passenger requests are sequentially processed through multiple handlers, including check-in, baggage registration, baggage tracking, and baggage claim services, without creating tight dependencies between system components.

Objective

The primary goal of this document is to justify the architectural decisions behind the implementation of the Chain of Responsibility Pattern based on established software design principles. It provides UML Class Diagrams to demonstrate how the system efficiently processes passenger-related requests through independent handlers while ensuring flexibility, scalability, and maintainability. The approach prevents tight coupling between modules and supports future expansion of airport service functionalities.

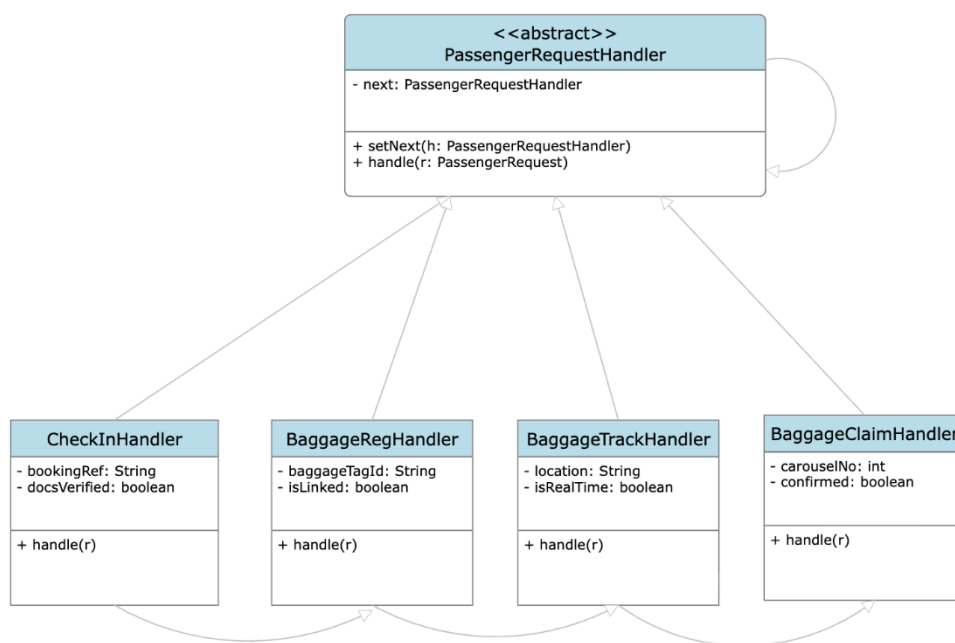
Design Pattern 3: Chain of Responsibility Pattern (Passenger Request Handling)

- **Requirement Mapped:** Passenger service request processing within the Airport Management System, including check-in, baggage registration, baggage tracking, and baggage claim handling.
- **Problem:** In our Airport Management System, passenger requests can involve multiple services depending on the situation. For example, a passenger may first require check-in validation, then baggage registration, baggage tracking, or baggage claim confirmation. If all these responsibilities were hardcoded into one centralized controller, the system would become tightly coupled and difficult to maintain. Any modification in one process (e.g., baggage tracking rules) could negatively affect the entire request-handling workflow.
- **Solution:** We applied the Behavioral Chain of Responsibility Pattern. The abstract PassengerRequestHandler acts as the base handler and defines the request-processing chain through

the `setNext()` and `handle()` methods. Each concrete handler (`CheckInHandler`, `BaggageRegHandler`, `BaggageTrackHandler`, and `BaggageClaimHandler`) is responsible for processing a specific passenger-related request. If a handler cannot fully process the request, it forwards it to the next handler in the sequence. This approach enables requests to move dynamically through the chain without hardcoding dependencies between modules.

- **Justification:** This pattern is ideal because it promotes loose coupling between request senders and handlers. Each class has a single responsibility, making the system easier to extend and maintain. For example, new passenger service handlers can be added without changing the existing logic. Additionally, the sequential handling process ensures that airport operations such as check-in, baggage registration, tracking, and claim management are processed in an organized and scalable manner.

2. The Class Diagram



Behavioral Design Patterns Documentation

Prepared by: Ani Velo

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the **Security Management Module** of the Airport Management System (AMS). While previous models focused on dynamic message sequencing, this document formally defines the static class architectures used to solve specific security and validation problems. It specifically details the **Proxy Pattern** as they apply to passenger screening and emergency protocols.

Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the security module fulfills its functional requirements while maintaining "thin" user interfaces and automated audit compliance.

Design Pattern 1: Proxy Pattern (Divergent Checkpoint Verification Pipeline)

Requirements Mapped (Group 1):

- UC_SEC_01: Passenger Security Screening
- UC_SEC_02: Restricted Area Access Control
- UC_SEC_05: Prohibited Item Logging

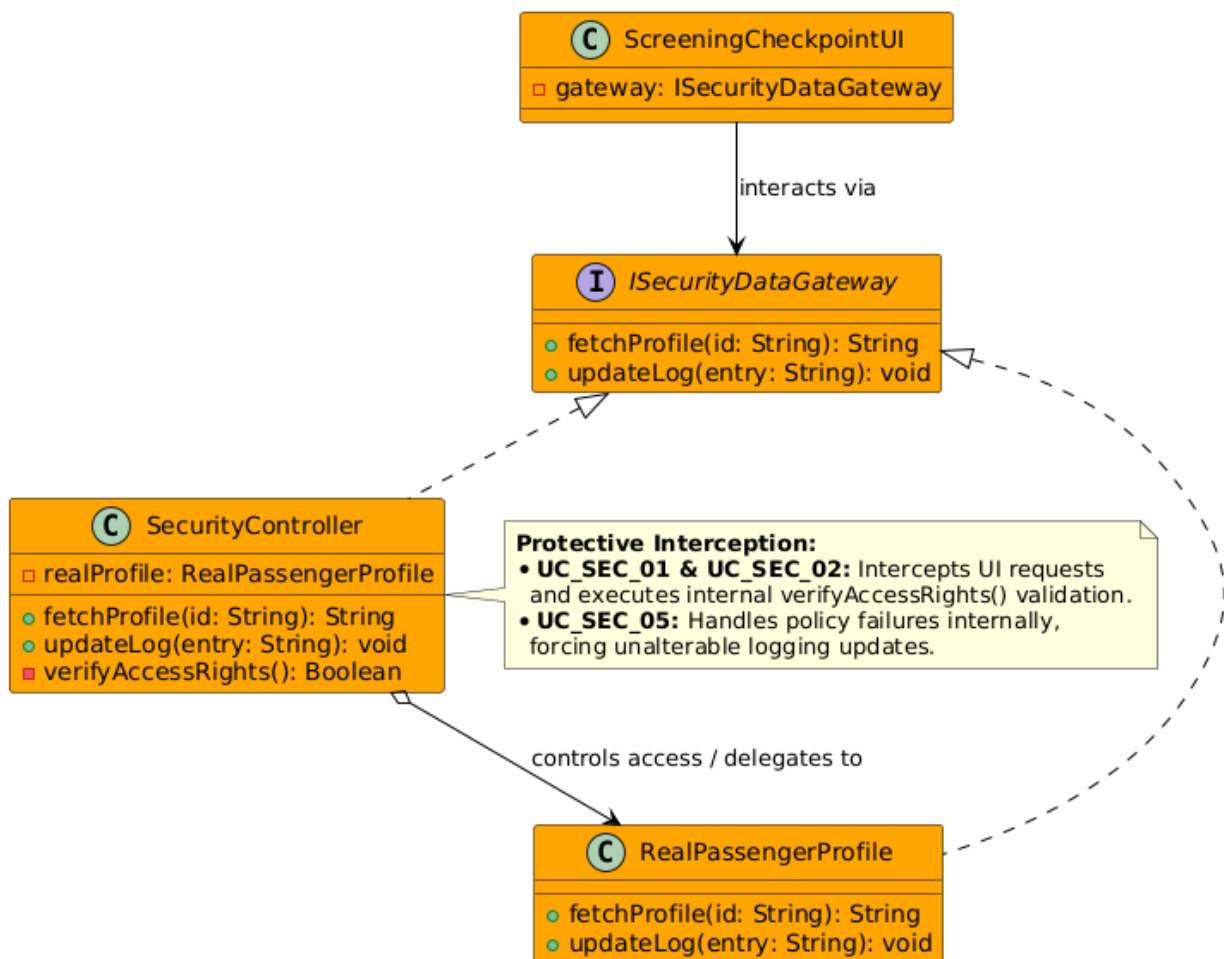
Problem: During passenger security screening (UC_SEC_01) and access authorization checks (UC_SEC_02), system operators and checkpoint interfaces need to fetch highly sensitive passenger profile data and log system-level access events. Directly allowing a user interface component (**ScreeningCheckpointUI**) to call, mutate, or read data from the underlying data entities (**PassengerProfile**) introduces severe security vulnerabilities, data misuse risks, and tight architectural coupling. If the interface layer is compromised, the data persistence tier is completely exposed.

Solution: We applied the Structural **Proxy Pattern**. We established a common interface wrapper contract, **ISecurityDataGateway**, which exposes operations like **fetchProfile(id: String)** and **updateLog(entry: String)**. The actual database connection logic is isolated inside a concrete **RealPassengerProfile** class. We then introduce the **SecurityController** to act as the protective **Proxy** object implementing that same interface. The **ScreeningCheckpointUI** only interacts with the proxy controller. When a method is invoked, the proxy executes an internal **verifyAccessRights()** interceptor routine. It only forwards the request to the real profile object if security criteria are satisfied.

Justification: This pattern creates a controlled boundary around your sensitive persistence layer data. It allows the system to seamlessly perform logging audits, parameter sanitization, and access token validation without complicating the core business logic of the real entity data classes. If a screening reveals a weapon or illicit substance (UC_SEC_05), the proxy controller intercepts the failure, automatically flags the profile status, and routes a protected transaction write to initialize a contraband entry, ensuring no unauthenticated interface can manipulate or erase active security violation registries.

2. The Class Diagram

Security Management Module - Proxy Pattern Class Architecture (Group 1)



Airport Management System

Behavioral Design Patterns Documentation

Prepared by: Aneo Lulaj

Course: Software Modeling and Design

Date: May 14, 2026

Document Overview

This document presents the implementation and structural mapping of Behavioral Design Patterns within the Airport Management System (AMS). While previous models focused on dynamic state transitions and message sequencing, this document formally defines the static class architecture used to solve specific behavioral problems. It specifically details the State Pattern applied to manage the lifecycle of the shared airport resources handled by the Resource Management module - gates, runways, staff and equipment.

Objective

The primary goal of this document is to justify our architectural choices based on established software design principles. It provides strict UML Class Diagrams to demonstrate how the system fulfills its functional requirements while safely handling alternative flows (conflicts, faults, unavailability) and preventing tight coupling between a resource and its many lifecycle behaviours.

State Pattern (Behavioral)

Intent: Allow a resource object to alter its behaviour when its internal state changes, so the object appears to change its class.

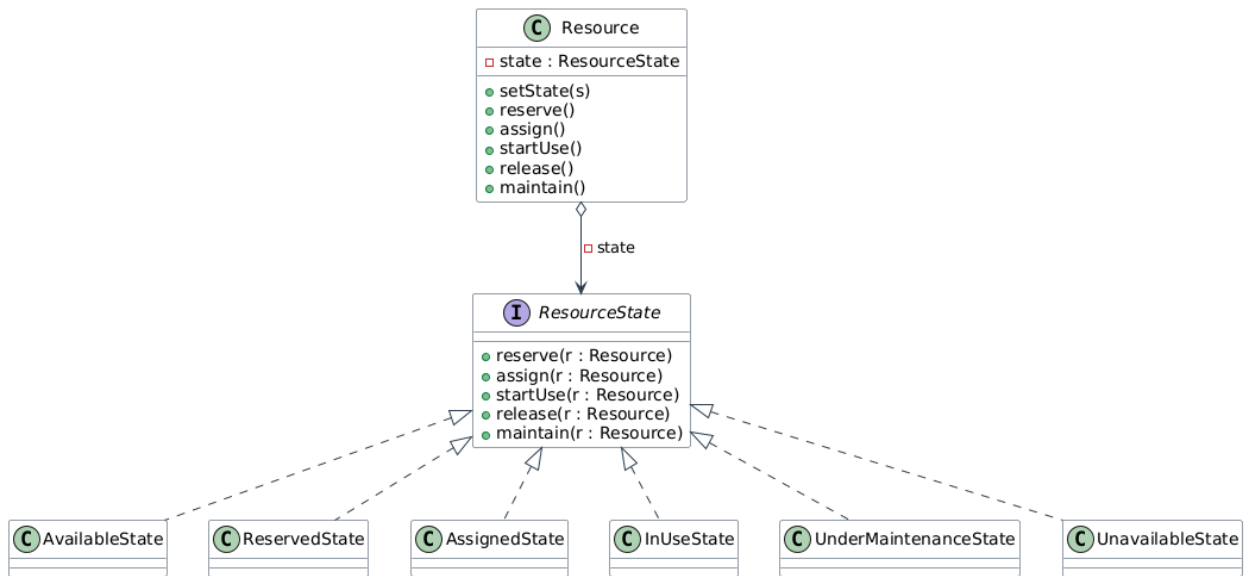
Problem in AMS: Every shared resource moves through a defined lifecycle - Available, Reserved, Assigned, In Use, Under Maintenance and Unavailable. Implementing this with large if/switch blocks on a single status flag scatters the transition rules, makes invalid transitions easy to introduce, and violates the Open/Closed Principle whenever a new state is required.

Solution: The Resource (context) delegates all behaviour to a ResourceState object. Each concrete state class (AvailableState, ReservedState, AssignedState, InUseState, UnderMaintenanceState, UnavailableState) implements only the transitions that are legal from that state and rejects the rest, then swaps the context to the next state. This is the class-level realisation of the UC-ARM state machine diagrams.

SOLID justification: OCP - a new lifecycle state is added by writing one new state class, with no change to existing states or to the Resource. SRP - each state class is solely responsible for the rules of exactly one lifecycle stage.

Mapped requirements: UC-ARM-01 (Gate Allocation), UC-ARM-02 (Runway Scheduling), UC-ARM-03 (Staff Allocation), UC-ARM-04 (Equipment Management) - directly implements their state diagrams.

State Pattern - Resource Lifecycle



Airport Management System

Software Design Patterns Documentation

Prepared by: Ajla Alcani

Document Overview

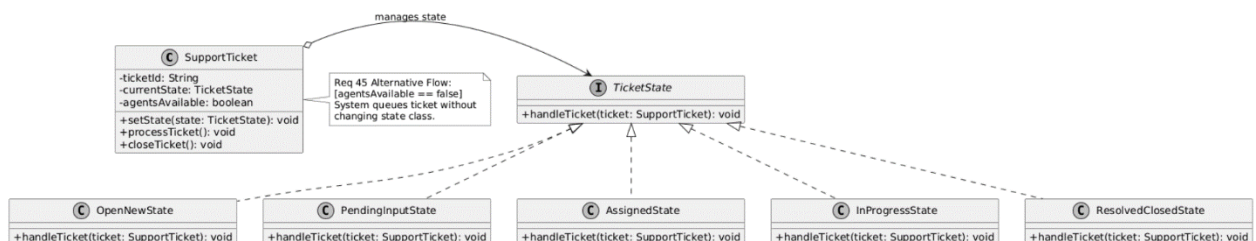
This document presents the implementation and structural mapping of five foundational Software Design Patterns within the Airport Management System. It expands upon our previous behavioral and interaction models by formally defining the static class architectures used to solve specific structural, behavioral, and creational problems across multiple system modules.

Objective

The primary goal of this report is to justify these architectural choices based on established software engineering principles, specifically the Single Responsibility and Open/Closed Principles. Through strict UML Class Diagrams, this document demonstrates how the system satisfies its core functional requirements while securely managing alternative workflows, handling exceptions gracefully, and preventing tight coupling between independent subsystems.

Design Pattern 1: State Pattern

- **Requirement Mapped:** Req 45 (Track service requests).
- **Problem:** A support ticket has a strict lifecycle: Open, Pending, Assigned, In Progress, and Resolved. Managing these transitions using large **if/else** conditional blocks within the **SupportController** would make the code fragile and prone to illegal state transitions.
- **Solution:** We applied the **State Pattern**. The **SupportTicket** object delegates its state-specific behavior to dedicated classes that implement the **TicketState** interface.
- **Justification:** This pattern simplifies the code of the context by eliminating bulky state machine conditionals. It provides a secure structure to handle our alternative flows; for example, the **OpenState** class specifically enforces the **[if no agent online]** **queueTicket()** guard condition.



Airport Management System

Software Design Patterns Documentation

Prepared by: Eneida Balliu

Document Objective

The objective of this section is to identify, specify, and justify the architectural design patterns incorporated into the Airport Management System. While our previous business process models (BPMN) and use case narratives outlined operational requirements, this section transitions those requirements into decoupled, scalable, and reusable object-oriented code structures. Implementing these industry-standard design patterns directly addresses our non-functional requirements—specifically system security compliance, audit traceability, and overall architectural extensibility.

Document Overview

This technical specification details a selected suite of 4–5 core software design patterns tailored to optimize different sub-systems within the airport platform. Each design pattern presentation includes:

- **Pattern Classification & Identification:** Defining the structural, creational, or behavioral nature of the selected pattern.
- **Contextual Project Justification:** A formal description explaining exactly which administration use cases (system maintenance or reporting modules) benefit from the pattern's application.
- **Structural Class Diagrams:** UML-compliant class models detailing interfaces, concrete execution implementations, invoker responsibilities, and data delegation logic.

Design Pattern 2: Command Pattern (System Administration Operations)

Definition: The Command Design Pattern is a behavioral pattern that encapsulates a request or operation as a standalone object. This decouples the interface object triggering the action from the service layer components that contain the actual execution logic.

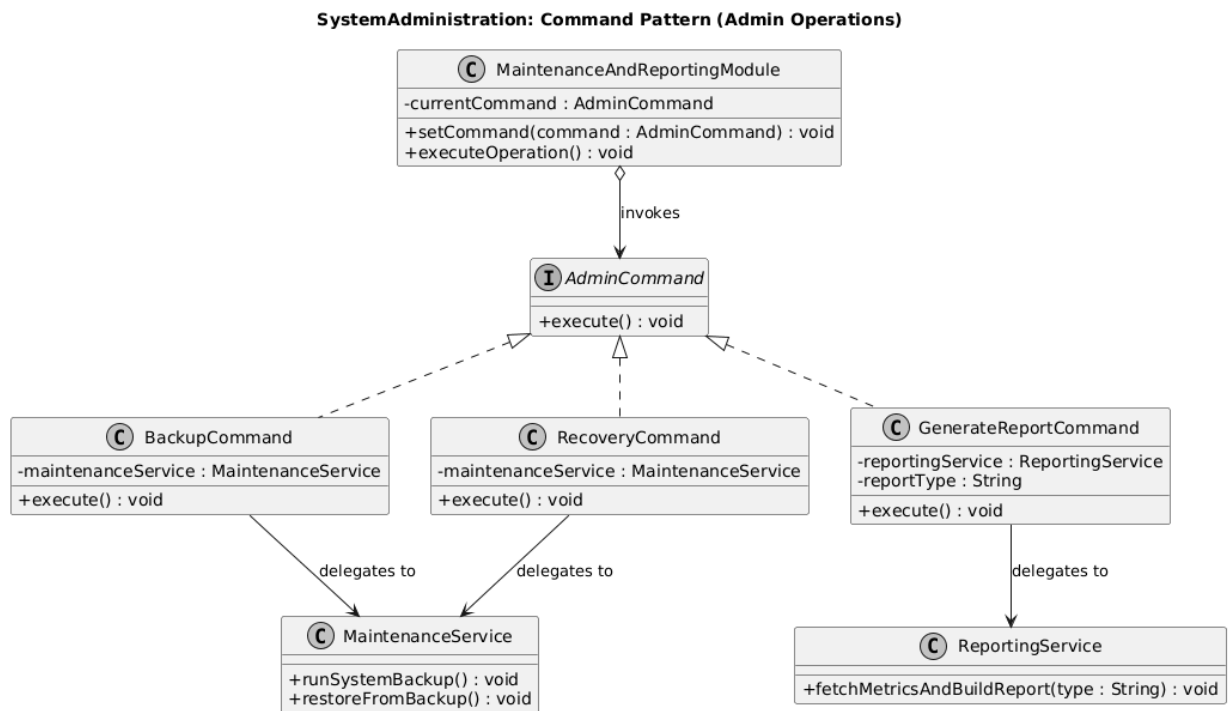
Key Components:

- **The Invoker (MaintenanceAndReportingModule):** The user interface trigger that calls `execute()` on a command without knowing its underlying database logic.
- **The Command Interface (AdminCommand):** Defines the universal execution contract (`execute()`).
- **Concrete Commands (BackupCommand, GenerateReportCommand):** Implement `execute()` by packaging necessary parameters and routing the request to the correct system service.
- **The Receivers (MaintenanceService, ReportingService):** Background domain services containing the actual business logic to run backups, perform data recoveries, or gather system statistics.

Architectural Benefits:

1. **Extensibility:** New administration features can be added as new concrete command classes without modifying existing user interface or controller code.

2. **Graceful Cancellation:** Converting requests into objects makes it straightforward to intercept, halt, or cancel tasks safely before changes alter database states.
3. **Audit Compliance:** Since every administrative action is a distinct object, commands can be sequentially passed to a log manager to record a precise history within the SystemLog.



Airport Management System

Software Design Patterns Documentation

Prepared by: Betül Kaan

Document Overview

This document presents the implementation and structural mapping of five foundational Software Design Patterns within the Airport Management System. It expands upon our previous behavioral and interaction models by formally defining the static class architectures used to solve specific structural, behavioral, and creational problems across multiple system modules.

Objective

The primary goal of this report is to justify these architectural choices based on established software engineering principles, specifically the Single Responsibility and Open/Closed Principles. Through strict UML Class Diagrams, this document demonstrates how the system satisfies its core functional requirements while securely managing alternative workflows, handling exceptions gracefully, and preventing tight coupling between independent subsystems.

Design Pattern 3: Observer Pattern (Flight Notification System)

Definition: The Observer Design Pattern is a behavioral pattern that establishes a one-to-many dependency between objects, so that when one object changes state, all its dependents are notified and updated automatically. This decouples the flight data management layer from the notification delivery mechanisms, ensuring that status changes propagate to all registered passengers without the core flight system requiring any knowledge of how those notifications are delivered.

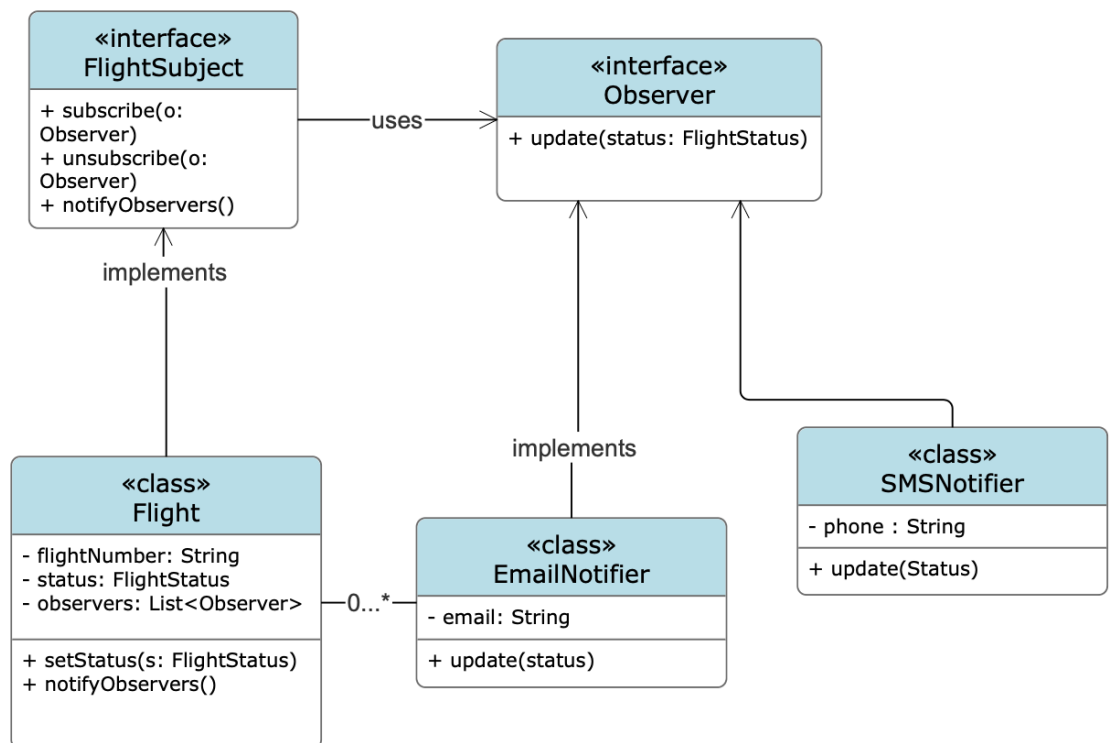
Key Components:

- **The Subject Interface (FlightSubject):** Defines the universal subscription contract `subscribe()`, `unsubscribe()`, and `notifyObservers()` that any observable flight entity must fulfill, allowing observers to register and deregister dynamically at runtime.
- **The Concrete Subject (Flight):** Maintains the authoritative flight state (flight number, current status) alongside an internal list of registered observers. Upon any status change triggered via `setStatus()`, it automatically invokes `notifyObservers()` to propagate the update across all subscribers.

- **The Observer Interface (Observer):** Defines the single-method contract `update(FlightStatus)` that all notification handlers must implement, ensuring the `Flight` class can communicate uniformly with any type of notifier without coupling to its internal logic.
- **Concrete Observers (EmailNotifier, SMSNotifier):** Implement `update()` by translating the received `FlightStatus` into a channel-appropriate alert, a formatted email body or a concise SMS string and delegating delivery to the corresponding service layer (`EmailService`, `SMSService`).

Architectural Benefits:

- **Loose Coupling:** The `Flight` class holds no reference to any specific notifier type. It interacts exclusively through the `Observer` interface, meaning notification channels can be added, removed, or replaced without modifying any flight management code.
- **Runtime Flexibility:** Passengers can subscribe and unsubscribe dynamically during the application lifecycle, reflecting real-world scenarios such as opting in after booking, changing contact preferences, or automatically unsubscribing after a flight completes.
- **Extensibility:** Introducing a new notification channel, such as `PushNotifier` or `WhatsAppNotifier` requires only implementing the `Observer` interface and registering the new instance; no existing classes are modified, fully satisfying the Open/Closed Principle.



Airport Management System

Software Design Patterns Documentation

Prepared by: Ani Velo

Module: Security Management Module

Course: Software Modeling and Design

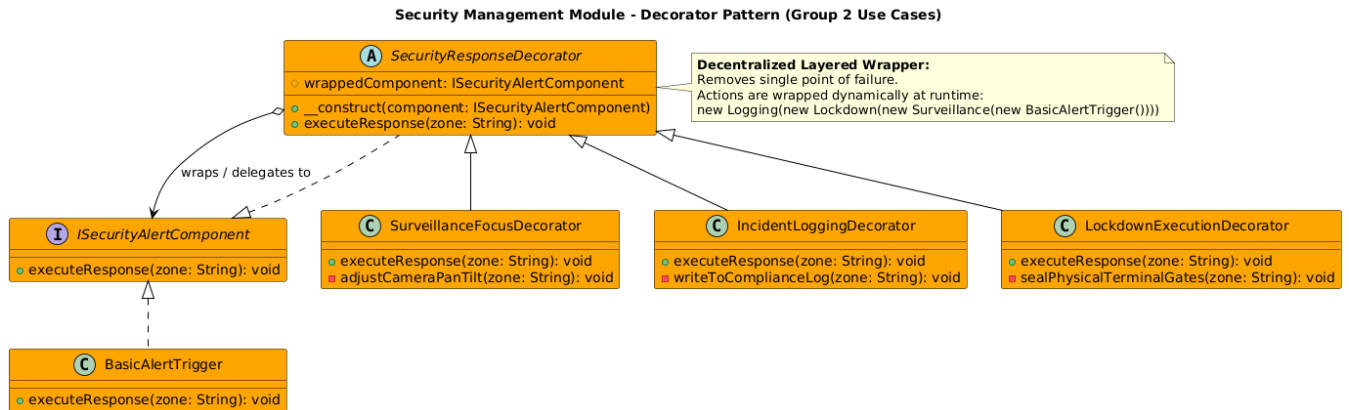
Document Overview

This document presents the implementation and structural mapping of two foundational software design patterns within the **Security Management Module** of the Airport Management System (AMS). By defining static class architectures, this report demonstrates how the security subsystem handles complex conditional screening processes and high-stakes emergency workflows while maintaining loose coupling, single responsibilities, and automated audit trails.

Design Pattern 2: Decorator Pattern (Decentralized Component Wrapper)

- **Requirement Mapped: Requirements Mapped (Group 2):**
 1. Security Incident Reporting (UC_SEC_03)
 2. Surveillance Monitoring (UC_SEC_04)
 3. Emergency Lockdown Procedure (UC_SEC_06)
- **Problem:** When a critical security event is triggered, the system must perform multiple layers of operational safety steps—adjusting camera feeds (UC_04), writing compliance records (UC_03), and sealing physical gates (UC_06). Entrusting this sequence to a single, centralized controller creates a fragile architecture with a dangerous "single point of failure." If that controller component fails, the entire emergency loop crashes. Conversely, hardcoding these functions sequentially makes the system rigid. We need a completely decentralized way to dynamically attach these security actions to an event thread at runtime without causing tight coupling.
- **Solution:** We applied the Structural **Decorator Pattern**. We established a lightweight component interface, `ISecurityAlertComponent`, which defines a standard operational contract: `executeResponse(zone: String)`. We implement a core `BasicAlertTrigger` class that performs the foundational system alert propagation. We then create an abstract `SecurityResponseDecorator` that implements the same interface and holds a structural reference to another component instance. Concrete decorator wrappers—`SurveillanceFocusDecorator`, `IncidentLoggingDecorator`, and `LockdownExecutionDecorator`—extend this abstract decorator to add their specialized use case behaviors before or after delegating to the wrapped component instance.

- Justification:** This architecture completely eliminates any single point of failure. Because every security behavior is isolated inside its own independent wrapper class, the behaviors are linked together modularly at runtime (e.g., wrapping a lockdown inside a logger, inside a camera adjuster). This provides total flexibility: if the airport requires a standard report without a lockdown, the system simply omits the lockdown decorator wrapper. It satisfies the **Open/Closed Principle**, allowing developers to create and attach new security wrappers (such as an automated local police dispatch alert) without altering any existing codebase.



Airport Management System

Software Design Patterns Documentation

Prepared by: Alvi Elezi

Document Objective

The objective of this section is to identify, specify, and justify the architectural design patterns incorporated into the Airport Management System. While our previous business process models (BPMN) and use case narratives outlined operational requirements, this section transitions those requirements into decoupled, scalable, and reusable object-oriented code structures. Implementing these industry-standard design patterns directly addresses our non-functional requirements, specifically system security compliance, audit traceability, and overall architectural extensibility.

Document Overview

This technical specification details a selected suite of 4–5 core software design patterns tailored to optimize different sub-systems within the airport platform. Each design pattern presentation includes:

- **Pattern Classification & Identification:** Defining the structural, creational, or behavioral nature of the selected pattern.
- **Contextual Project Justification:** A formal description explaining exactly which administration use cases (system maintenance or reporting modules) benefit from the pattern's application.
- **Structural Class Diagrams:** UML-compliant class models detailing interfaces, concrete execution implementations, invoker responsibilities, and data delegation logic.

Design Pattern 5: Strategy Pattern (Flight Operations Module)

Definition: The Strategy Design Pattern is a behavioral software design pattern that defines a family of algorithms, encapsulates each one in its own class, and makes them interchangeable at runtime. This allows for switching an object's behavior without changing the code that uses it.

Requirement Mapped: FO_02 Manage Flight Delays

Key Components:

1. Context (FlightController)

The FlightController acts as the context class responsible for selecting and executing the appropriate delay-handling strategy. It does not implement the operational logic itself; instead, it delegates the behavior to a selected strategy object at runtime.

2. Strategy Interface (DelayStrategy)

The DelayStrategy interface defines the common operational method: handleDelay(). All delay-management strategies must implement this interface, ensuring consistency across different operational procedures.

3. Concrete Strategies

These classes implement specialized operational behavior for different delay scenarios: WeatherDelayStrategy, TechnicalDelayStrategy, SecurityDelayStrategy and CrewDelayStrategy. Each strategy contains unique processing logic, scheduling adjustments, and notification procedures.

Architectural Benefits:

1. Better Separation of Concerns

Each strategy encapsulates a specific operational algorithm independently from the controller and other strategies. This improves modularity and keeps responsibilities clearly separated.

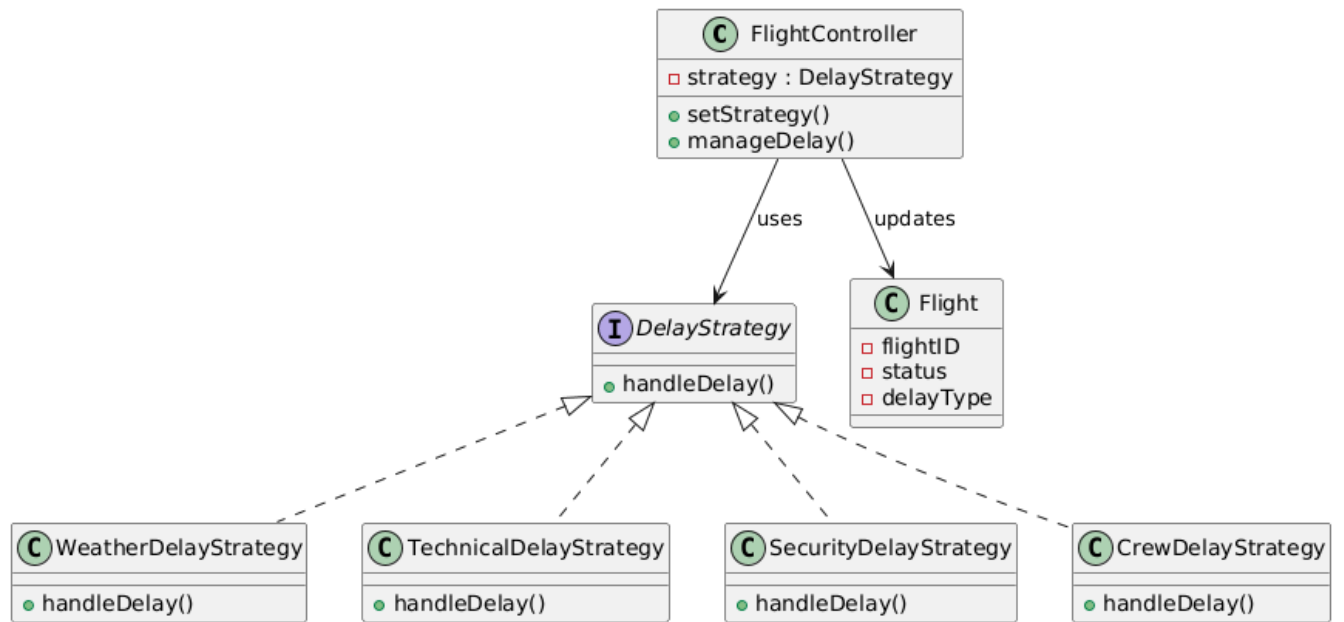
2. Improved Scalability

New delay-handling procedures can be introduced without modifying existing system logic. For example, adding an EmergencyDelayStrategy only requires creating a new strategy implementation.

3. Enhanced Maintainability

Operational rules for each delay category are isolated within their respective classes, making updates, debugging, and testing significantly easier.

Class Diagram:



Airport Management System (AMS)
Software Design Pattern Documentation
Module: Airport Resource Management (UC-ARM-01 .. UC-ARM-04)
Prepared by: Aneo Lulaj
Course: Software Modeling and Design

Document Overview

This document presents the structural mapping of a foundational software design pattern within the Airport Resource Management module of the AMS. It builds on the previous behavioural and interaction models (state, sequence, BPMN and collaboration diagrams) by defining the static class architecture used to allocate and manage the four shared airport resources: gates, runways, staff and equipment.

Objective

The goal is to justify the architectural choice using the Single Responsibility Principle (SRP) and the Open/Closed Principle (OCP), showing how the module satisfies its functional requirements while keeping the four resource subsystems loosely coupled.

Facade Pattern (Structural)

Intent: Provide a single, unified entry point to the four resource subsystems so the Administrator never communicates with them directly.

Problem in AMS: The Admin must allocate gates, runways, staff and equipment. Wiring the Admin UI directly to four separate controllers creates tight coupling - any change inside a subsystem ripples back into the client.

Solution: ResourceManagementFacade exposes simple operations (allocateGate, scheduleRunway, assignStaff, updateEquipment) and delegates each call to the matching subsystem. The Admin depends only on the facade, not on the subsystems behind it.

SOLID justification: OCP - new resource subsystems can be added behind the facade without changing client code. SRP - each subsystem retains its own single responsibility while the facade only coordinates.

Mapped requirements: UC-ARM-01 (Gate Allocation), UC-ARM-02 (Runway Scheduling), UC-ARM-03 (Staff Allocation), UC-ARM-04 (Equipment Management).

